

Rust Book

Indice

0 Note sul corso 3

I Fondamenta 4

1 Primi passi in Rust 5
- Rustup e gli strumenti 5
- Hello, world! 5
  - Compilazione ed esecuzione 5
- Cargo 6
  - Creazione di un progetto 6
  - Comandi principali 6
  - Profili di build 6

2 Il gioco dell'indovina 7
- Variabili e input dell'utente 7
- Gestire il possibile fallimento con `Result` 7
- Placeholder di `println!` 7
- Crate e dipendenze 8
- Numeri casuali 8
- Confronto e `match` 8
- Cicli e input non valido 9

3 Concetti comuni di programmazione 10
- Variabili e mutabilità 10
  - Costanti 10
  - Shadowing 10
- Tipi di dato 11
  - Tipi scalari 11
  - Tipi composti 12
- Funzioni 13
  - Parametri 13
  - Statement ed espressioni 13
  - Valori di ritorno 13
- Commenti 14
- Controllo di flusso 14
  - Espressioni `if` 14
  - Cicli 14

II Ownership e tipi strutturati 16

4 Ownership 17
- Sicurezza e comportamento indefinito 17

Le variabili vivono nello stack 17
I box vivono nell'heap 18
La deallocazione e l'ownership 18
Le collezioni usano i box 18
Le variabili mosse non sono più usabili 18
Il cloning evita i move 19
Riferimenti e borrowing 19
I riferimenti sono puntatori non posseduti 19
Il dereferenziamento 19
Aliasing e mutazione 19
I permessi sulle place 20
Il borrow checker 20
I riferimenti mutabili 20
I lifetime 20
I dati devono sopravvivere ai propri riferimenti 21
Correggere gli errori di ownership 21
Un programma insicuro con riferimento allo stack 21
Permessi insufficienti 21
Aliasing e mutazione di una struttura dati 21
Copiare o muovere da una collezione 22
Programmi sicuri che vengono rifiutati 22
Le slice 22
Il problema degli indici 23
Le string slice 23
La sintassi dei range 23
I letterali stringa sono slice 23
Le slice come parametri 23
Altri tipi di slice 23
Ricapitolazione 24
Ownership e garbage collection 24
Ownership a runtime 24
Ownership a compile time 24
Il collegamento tra permessi e comportamento indefinito 24

5 Struct 26

Definire e istanziare struct 26
Field init shorthand 26
Struct update syntax 26
Tuple struct 27
Unit-like struct 27
Ownership dei dati dello struct 27
Borrow dei campi 27
Un programma di esempio con le struct 27
Il refactoring con le tuple 27
Il refactoring con le struct 28
Stampare le struct e i trait derivati 28
I metodi 28
La sintassi dei metodi 28
Le funzioni associate 29
Le chiamate ai metodi sono zucchero sintattico 29
Metodi e ownership 29
Ricapitolazione 29

6 Enum e pattern matching 30

Definire un enum 30
I valori di un enum 30
L'enum `Option` 30
L'espressione `match` 31
I pattern che legano valori 31
Il pattern `match` su `Option<T>` 32

L'esaurienza dei match 32
Pattern catch-all e _ 32
Match e ownership 32
Flusso di controllo conciso con `if let` e `let...else` 32
`let...else` 33
Ricapitolazione 33

III Modularità, collezioni e gestione degli errori 34

7 Pacchetti, crate e moduli 35
Pacchetti e crate 35
Moduli e controllo di scope e privacy 35
Riepilogo rapido dei moduli 36
Percorsi per riferire un item nell'albero dei moduli 36
Privacy di default e la keyword `pub` 36
Percorsi relativi con `super` 37
Struct ed enum pubblici 37
Portare i percorsi nello scope con `use` 37
Percorsi `use` idiomatici 37
Re-export con `pub use` 38
Pacchetti esterni 38
Percorsi annidati e operatore glob 38
Separare i moduli in file diversi 38
Ricapitolazione 39

8 Collezioni comuni 40
Vettori 40
Creare un nuovo vettore 40
Aggiornare un vettore 40
Leggere gli elementi di un vettore 40
Iterare sui valori di un vettore 41
Usare gli iteratori in sicurezza 41
Usare un enum per memorizzare più tipi 41
Droppare un vettore droppa i suoi elementi 41
Stringhe 41
Definire le stringhe 41
Creare una nuova stringa 41
Aggiornare una stringa 42
L'indicizzazione delle stringhe 42
Byte, valori scalari e cluster di grafemi 42
Slicing di stringhe 43
Iterare sulle stringhe 43
Gestire la complessità delle stringhe 43
Hash map 43
Creare una nuova hash map 43
Accedere ai valori in una hash map 43
Ownership nelle hash map 44
Aggiornare una hash map 44
Funzioni di hash 44
Ricapitolazione 44

9 Gestione degli errori 45
Errori non riparabili con `panic!` 45
Unwinding o abort 45
Il backtrace di un panic 45

Errori riparabili con `Result` 45
Match su errori diversi 46
Scorciatoie per il panic in caso di errore 46
Propagare gli errori 46
L'operatore `?` 47
Dove usare l'operatore `?` 47
Quando usare `panic!` e quando `Result` 47
Quando si ha più informazione del compilatore 47
Linee guida per la gestione degli errori 48
Tipi personalizzati per la validazione 48
Ricapitolazione 48

IV Astrazioni e test 50

10 Tipi generici, trait e lifetime 51

Dalla duplicazione all'astrazione 51
Tipi generici 51
Nelle definizioni di funzione 51
Nelle definizioni di struct 52
Nelle definizioni di enum 52
Nelle definizioni di metodo 52
Le prestazioni del codice generico 52
Definire comportamento condiviso con i trait 53
Implementare un trait su un tipo 53
Implementazioni di default 53
Usare i trait come parametri 54
Ritornare tipi che implementano trait 54
Implementare metodi condizionalmente con i trait bound 54
Validare le reference con i lifetime 54
Reference dangling 55
Il borrow checker 55
Lifetime generici nelle funzioni 55
Nelle firme di funzione 55
Le relazioni tra lifetime 56
Nelle definizioni di struct 56
Lifetime elision 56
Nelle definizioni di metodo 57
Il lifetime statico 57
Tipi generici, trait bound e lifetime insieme 57

11 Test automatizzati 59

Come scrivere i test 59
La struttura delle funzioni di test 59
Verificare i risultati con assert 59
Verificare i panic con should_panic 60
Usare Result T E nei test 60
Controllare l'esecuzione dei test 61
Test in parallelo o in sequenza 61
Mostrare l'output delle funzioni 61
Eseguire un sottoinsieme di test per nome 61
Ignorare test a meno che non richiesti 61
Organizzazione dei test 62
Unit test 62
Integration test 62
Sintesi della lezione 63

12 Un progetto di I/O 64

Accettare argomenti da riga di comando 64
Leggere un file 64
Rifattorizzare per modularità e gestione degli errori 64
Separazione delle responsabilità nei progetti binari 65
Raggruppare i valori di configurazione 65
Un costruttore per Config 65
Estrarre la logica in run e restituire Result 66
Dividere il codice in una crate di libreria 66
Aggiungere funzionalità con lo sviluppo guidato dai test 66
Lavorare con le variabili d'ambiente 67
Reindirizzare gli errori sullo standard error 67
Sintesi della lezione 67

13 Closure e iteratori 68

Closure 68
Catturare l'ambiente 68
Inferire e annotare i tipi delle closure 68
Catturare reference o trasferire ownership 69
Spostare valori catturati fuori dalle closure 69
Elaborare serie di elementi con gli iteratori 70
Il trait Iterator e il metodo next 70
Metodi che consumano l'iteratore 70
Metodi che producono altri iteratori 70
Closure che catturano il loro ambiente 71
Migliorare il progetto di I/O 71
Rimuovere un clone con un iteratore 71
Chiarire il codice con gli iterator adapter 71
Scegliere tra cicli e iteratori 71
Prestazioni nei cicli vs negli iteratori 72

V Ecosistema e smart pointers 73

14 Di più su Cargo e crates.io 74

Personalizzare la compilazione con i release profile 74
Pubblicare una crate su crates.io 74
Commenti di documentazione utili 74
Esportare un'API pubblica conveniente 75
Configurare un account crates.io 75
Aggiungere metadati a una nuova crate 75
Pubblicare su crates.io 76
Deprecare versioni con cargo yank 76
Workspace con Cargo 76
Dipendenze tra i membri e da pacchetti esterni 76
Installare binari con cargo install 77
Estendere Cargo con comandi personalizzati 77
Sintesi della lezione 77

15 Smart pointers 78

Usare Box T per puntare a dati sullo heap 78
Abilitare i tipi ricorsivi con i box 78
Trattare gli smart pointer come reference normali col trait Deref 79
Definire il proprio smart pointer 79
Usare la deref coercion in funzioni e metodi 79
La deref coercion con reference mutabili 80

Eseguire codice di pulizia col trait Drop 80
Rc T lo smart pointer a reference counting 80
Condividere dati 81
Clonare per aumentare il contatore 81
RefCell T e il pattern dell'interior mutability 81
Far rispettare le regole di borrowing a runtime 81
Usare l'interior mutability nei mock object 82
Tenere traccia dei borrow a runtime 82
Owner multipli di dati mutabili con Rc e RefCell insieme 82
I cicli di reference possono causare memory leak 83
Creare un ciclo 83
Prevenire i cicli con Weak T 83
Sintesi della lezione 83

VI Concorrenza e async 85

16 Concorrenza senza paura 86
Usare i thread per eseguire codice simultaneamente 86
Creare un nuovo thread con spawn 86
Aspettare che tutti i thread finiscano 87
Closure move con i thread 87
Trasferire dati tra thread con lo scambio di messaggi 87
Il ruolo dell'ownership nel messaggio 88
Concorrenza a stato condiviso 88
Controllare l'accesso con i mutex 88
L'API di Mutex T 89
Accesso condiviso a Mutex T tra più thread 89
Il confronto tra RefCell Rc e Mutex Arc 89
Concorrenza estendibile con i trait Sync e Send 89
Implementare Send e Sync manualmente è unsafe 90
Sintesi della lezione 90

17 Programmazione asincrona 91
Parallelismo e concorrenza 91
Le due dinamiche di lavoro 91
Futures e sintassi async 92
La pigrizia delle futures 92
Eseguire codice async con un runtime 92
La macchina a stati invisibile 93
Far competere due futures con select 93
Applicare la concorrenza con async 93
Creare un nuovo task con spawn_task 93
Scambio di messaggi tra task 94
Cedere il controllo al runtime 94
Costruire astrazioni async proprie 95
Stream, futures in sequenza 95
Uno sguardo più da vicino ai trait per async 95
Il trait Future 95
Il tipo Pin e il trait Unpin 96
Il trait Stream 97
Futures task e thread insieme 97
Sintesi della lezione 98

VII OOP, PATTERN E FEATURE AVANZATE 99

18 Rust e la programmazione orientata agli oggetti 100

Caratteristiche dei linguaggi orientati agli oggetti 100
- Gli oggetti contengono dati e comportamento 100
- Incapsulamento che nasconde i dettagli implementativi 100
- Ereditarietà come sistema di tipi e come condivisione di codice 100

Trait objects per astrarre su comportamenti condivisi 101
- Trait objects o generics con trait bound 101
- Dynamic dispatch 102

Implementare un pattern di design orientato agli oggetti 102
- L'implementazione in stile tradizionale 102
- I trade-off del state pattern 103
- Codificare gli stati e il comportamento come tipi 104

I trade-off di design 104

Sintesi della lezione 104

19 Pattern e matching 105

Tutti i posti dove si possono usare i pattern 105
- I bracci di match 105
- Le istruzioni let 105
- Le espressioni condizionali if let 105
- I cicli condizionali while let 106
- I cicli for 106
- I parametri delle funzioni 106

La refutabilità 106

La sintassi dei pattern 107
- Letterali e variabili nominate 107
- Pattern multipli con la barra verticale 107
- Intervalli di valori con ..= 107
- Destrukturare i valori 107
- Ignorare i valori in un pattern 108
- Condizioni aggiuntive con i match guard 108
- Binding con la chiocciola 108

Sintesi della lezione 109

20 Feature avanzate 110

Rust unsafe 110
- Perché esiste Rust unsafe 110
- Le cinque azioni unsafe 110
- I raw pointer 110
- Funzioni e metodi unsafe 111
- `extern` e le FFI 111
- Variabili statiche mutabili 111
- Trait unsafe e union 111
- Miri 112

Trait avanzati 112
- Tipi associati 112
- Parametri di tipo di default e overloading degli operatori 112
- Disambiguare metodi con lo stesso nome 113
- Supertrait 113
- Il pattern newtype per la orphan rule 113

Tipi avanzati 113
- Il newtype per sicurezza e astrazione 113
- Alias di tipo 113
- Il never type 114
- Tipi a dimensione dinamica e il trait `Sized` 114

Funzioni e closure avanzate 114
Ritornare closure 115
Macro 115
Macro e funzioni a confronto 115
Macro dichiarative con `macro_rules!` 115
Macro procedurali 115
Un esempio di `derive` e l'ecosistema 116
Sintesi del capitolo 116

21 Un web server multithread 117

I protocolli 117
Il server single-threaded 117
Ascoltare la connessione TCP 117
Leggere la richiesta 117
Il formato di richiesta e risposta 118
I limiti del modello a thread singolo 118
Il thread pool 118
Sviluppo guidato dal compilatore 118
La firma di `execute` 118
I Worker 119
Il canale come coda di lavori 119
Perché `let` e non `while let` 119
Spegnimento pulito 119
`Drop` e `Vec::drain` 120
Chiudere il canale 120
Sintesi del capitolo 120

Capitolo 0

Note sul corso

Questi appunti sono nati dalla curiosità di imparare Rust. Derivano dal sito stesso di Rust: https://rust-book.cs.brown.edu/

Parte I

Fondamenta

Capitolo 1

Primi passi in Rust

Rustup e gli strumenti

Il toolchain di Rust si installa e si gestisce tramite `rustup`, uno strumento da riga di comando che gestisce le versioni di Rust e degli strumenti associati e installa l'ultima versione stabile del compilatore. I comandi principali sono:

- **`rustup update`**: aggiorna Rust alla versione appena rilasciata.
- **`rustup self uninstall`**: disinstalla Rust e `rustup`.
- **`rustup doc`**: apre la copia locale della documentazione, consultabile anche offline; la documentazione API descrive tipi e funzioni della libreria standard.
- **`rustc --version`**: verifica l'installazione, mostrando numero di versione, hash del commit e data.

Rust fornisce garanzie di stabilità: gli esempi che compilano continuano a compilare con le versioni più recenti (l'output può differire leggermente per messaggi di errore e avviso migliorati).

La compilazione richiede un **linker**, programma che unisce gli output compilati in un solo file: in genere è già presente, altrimenti lo fornisce un compilatore C, necessario anche perché alcuni pacchetti Rust comuni dipendono da codice C.

Info

Strumenti di supporto

`rustfmt` è il formattatore automatico incluso nella distribuzione standard e impone uno stile uniforme al codice; **`rust-analyzer`** abilita il supporto nei vari IDE.

Hello, world!

I file Rust terminano con l'estensione `.rs`; per i nomi multi-parola la convenzione è il trattino basso, quindi `hello_world.rs` e non `helloworld.rs`. Il programma minimo è:

```rust
fn main() {
    println!("Hello, world!");
}
```

- **`fn main`**: dichiara la funzione `main`, che è sempre il primo codice eseguito in ogni programma Rust eseguibile; eventuali parametri vanno tra parentesi tonde e il corpo va tra parentesi graffe, con la graffa di apertura sulla stessa riga della dichiarazione, separata da uno spazio.
- **`println!`**: è una **macro**, come indica il `!` che la distingue da una funzione; le macro sono codice che genera altro codice per estendere la sintassi di Rust e non sempre seguono le stesse regole delle funzioni; qui riceve la stringa da stampare come argomento.
- **Punto e virgola**: conclude l'espressione e indica che la successiva può iniziare; termina la maggior parte delle righe di codice Rust.

Compilazione ed esecuzione

La compilazione e l'esecuzione sono fasi separate.

- **`rustc main.rs`**: compila il sorgente e produce un eseguibile binario, analogamente a `gcc` o `clang`.
- **Ahead-of-time compiled**: Rust è un linguaggio compilato prima dell'esecuzione, quindi l'eseguibile può essere distribuito e girare anche su macchine senza Rust installato, a differenza dei linguaggi dinamici come Ruby, Python e JavaScript che richiedono l'interprete.

Cargo

Definition

Cargo

Build system e package manager di Rust: gestisce la compilazione del codice, il download delle librerie da cui il progetto dipende (le **dependencies**) e la loro compilazione. È installato insieme a Rust tramite gli installer ufficiali.

Creazione di un progetto

Cargo genera e organizza i file di un nuovo progetto.

- **`cargo new nome`**: crea la directory del progetto con un file `Cargo.toml` e la directory `src/` contenente `main.rs`; inizializza anche un repository Git con un `.gitignore`, a meno di trovarsi già dentro un repository esistente (il comportamento si controlla con il flag `--vcs`).
- **`cargo init`**: crea il solo `Cargo.toml` in un progetto esistente.

`Cargo.toml` è il file di configurazione in formato **TOML** (Tom's Obvious, Minimal Language):

```toml
[package]
name = "hello_cargo"
version = "0.1.0"
edition = "2024"

[dependencies]
```

- **`[package]`**: intestazione di sezione; le righe seguenti configurano il pacchetto.
- **`name`, `version`, `edition`**: nome, versione ed edizione di Rust con cui compilare il programma.
- **`[dependencies]`**: sezione in cui si elencano le dipendenze del progetto; i pacchetti di codice in Rust si chiamano **crates**.
- **Convenzione `src/`**: Cargo si aspetta il codice sorgente nella directory `src/`, mentre la directory di primo livello è riservata a README, licenze, file di configurazione e tutto ciò che non è codice.

Comandi principali

I comandi principali coprono l'intero ciclo di build.

- **`cargo build`**: compila il progetto e produce l'eseguibile in `target/debug/`; al primo build crea anche `Cargo.lock`, che registra le versioni esatte delle dipendenze e non va mai modificato a mano.
- **`cargo run`**: compila il progetto (solo se i file sono cambiati) e poi esegue l'eseguibile in un solo comando.
- **`cargo check`**: verifica che il codice compili senza produrre un eseguibile; è più veloce di `cargo build` e si usa periodicamente durante la scrittura del codice.

Profili di build

Cargo distingue due profili di compilazione, ciascuno con una propria directory di destinazione.

- **Build di sviluppo** (default): senza ottimizzazioni, in `target/debug`; pensata per ricompilare rapidamente e spesso.
- **`cargo build --release`**: con ottimizzazioni, in `target/release`; la compilazione è più lenta ma l'esecuzione più rapida, ed è l'eseguibile da usare per i benchmark.

I comandi di Cargo sono identici su ogni sistema operativo.

Capitolo 2

Il gioco dell'indovina

Il programma genera un intero casuale tra 1 e 100, chiede un tentativo all'utente e indica se è troppo basso o troppo alto, finché l'utente non indovina. Il tutorial introduce i concetti fondamentali del linguaggio.

Variabili e input dell'utente

Definition

Prelude

Insieme di elementi che la libreria standard porta automaticamente nello scope di ogni programma; un tipo non presente nel prelude va importato esplicitamente con un'istruzione `use`, come `use std::io;` per l'I/O.

- **`let`**: crea una variabile e vi vincola un valore; le variabili sono immutabili per default, quindi il valore non può cambiare dopo il vincolo.
- **`mut`**: dichiarato prima del nome, rende la variabile mutabile.

```rust
let apples = 5;      // immutabile
let mut bananas = 5; // mutabile
```

- **Funzione associata**: funzione implementata su un tipo, richiamata con la sintassi `::`, come `String::new` che restituisce una nuova istanza vuota di `String`; `String` è il tipo della libreria standard per testo estendibile, codificato in UTF-8.
- **`io::stdin()`**: restituisce un handle allo standard input del terminale; il metodo `read_line(&mut guess)` prende in input l'utente e **aggiunge** ciò che viene digitato alla stringa passata come argomento, senza sovrascriverne il contenuto, che quindi dev'essere mutabile.
- **Riferimento `&`**: modo per far accedere più parti del codice a uno stesso dato senza copiarlo; come le variabili, i riferimenti sono immutabili per default, quindi serve `&mut guess` e non `&guess`.

Gestire il possibile fallimento con `Result`

Definition

Enum

Tipo che può trovarsi in uno di più stati possibili, ciascuno dei quali è detto **variant**.

Definition

Result

Enum della libreria standard che codifica la gestione degli errori: la variante `Ok` indica successo e contiene il valore prodotto, la variante `Err` indica fallimento e contiene informazioni su come o perché l'operazione è fallita.

- **`expect`**: metodo di `Result` che, su `Err`, fa terminare il programma con un crash mostrando il messaggio passato come argomento; su `Ok` estrae e restituisce il valore contenuto (qui il numero di byte letti).
- **Warning del compilatore**: ignorare il `Result` restituito da `read_line` produce un avviso, perché segnala un possibile errore non gestito.

Placeholder di `println!`

La stringa di formato di `println!` accetta segnaposto per i valori.

- **`{}`**: segnaposto che tiene il posto di un valore nella stringa di formato; il nome della variabile può stare dentro le graffe, oppure si usano graffe vuote seguite dall'elenco delle espressioni da stampare, in ordine.

```rust
let x = 5;
let y = 10;

println!("x = {x} and y + 2 = {}", y + 2);
```

Crate e dipendenze

Definition

Crate

Insieme di file di codice sorgente Rust: un crate binario è un eseguibile, mentre un crate libreria contiene codice destinato ad altri programmi e non eseguibile da solo.

- `[dependencies]`: sezione di `Cargo.toml` in cui si dichiara ogni crate esterno e la versione richiesta, come `rand = "0.8.5"`.
- **Semantic Versioning (SemVer)**: standard per scrivere i numeri di versione; lo specificatore `0.8.5` è abbreviazione di `^0.8.5`, cioè ogni versione almeno 0.8.5 ma sotto 0.9.0, considerate compatibili nella API pubblica.
- **Registry**: copia dei dati di crates.io, il sito dove si pubblicano i progetti Rust open source; Cargo scarica da lì il crate richiesto e anche i crate da cui dipende, poi compila tutto.
- `Cargo.lock`: creato al primo `cargo build`, registra le versioni esatte scelte per ogni dipendenza e garantisce **build riproducibili**: si ricostruisce sempre lo stesso artefatto finché non si aggiornano le dipendenze esplicitamente; per questo si controlla nel version control.
- `cargo update`: ignora `Cargo.lock`, ricalcola le versioni più recenti ammesse da `Cargo.toml` e le riscrive in `Cargo.lock`.
- `cargo doc --open`: costruisce localmente la documentazione di tutte le dipendenze del progetto e la apre nel browser.

Numeri casuali

La generazione di numeri casuali passa dal crate esterno `rand`.

- `use rand::Rng;`: porta nello scope il trait `Rng`, che definisce i metodi implementati dai generatori di numeri casuali; il trait dev'essere in scope per poterne usare i metodi.
- `thread_rng`: funzione di `rand` che restituisce il generatore locale al thread corrente di esecuzione, con seed fornito dal sistema operativo.
- `gen_range`: metodo del trait `Rng` che genera un numero casuale nell'intervallo ricevuto come argomento; l'espressione di intervallo `start..=end` è inclusiva su entrambi gli estremi, quindi per un numero tra 1 e 100 si scrive `1..=100`.

Confronto e `match`

Il confronto tra due valori produce un esito tipizzato, gestito da `match`.

- `Ordering`: enum della libreria standard con le varianti `Less`, `Greater` ed `Equal`, i tre esiti possibili del confronto tra due valori.
- `cmp`: metodo che confronta due valori e restituisce una variante di `Ordering`; riceve un riferimento al valore con cui confrontare.

Definition

match e armi

Espressione composta da armi (*arm*): ciascuna arm ha un pattern da confrontare col valore e il codice da eseguire in caso di corrispondenza. Rust esamina i pattern delle armi in ordine e termina alla prima corrispondenza.

- **Sistema di tipi**: forte e statico, con **inferenza di tipo**; quando più tipi sono possibili il default per gli interi è `i32`; confrontare una stringa con un numero produce un errore di *mismatched types*.
- **Shadowing**: dichiarare con `let` una nuova variabile con lo stesso nome di una precedente oscura la prima, riusando il nome invece di crearne due come `guess_str` e `guess`; è tipico delle conversioni di tipo.

```rust
let guess: u32 = guess.trim().parse().expect("Please type a number!");
```

- **`trim`**: metodo di `String` che elimina gli spazi all'inizio e alla fine, inclusi il newline `\n` (o `\r\n` su Windows) aggiunti premendo invio.
- **`parse`**: metodo delle stringhe che converte in un altro tipo; può fallire, quindi restituisce un `Result`; l'annotazione `let guess: u32` dice a Rust quale tipo numerico ottenere e fa inferire lo stesso tipo al numero con cui si confronta.

Cicli e input non valido

Il programma completo gestisce più tentativi con un ciclo esplicito.

- **`loop`**: parola chiave che crea un ciclo infinito, interrotto esplicitamente.
- **`break`**: esce dal ciclo (se il ciclo è l'ultima parte di `main`, il programma termina).
- **`continue`**: passa subito alla prossima iterazione del ciclo.
- **`_`**: valore *catch-all* che combacia con ogni possibile contenuto, qui usato nel pattern `Err(_)` per gestire tutti gli errori di `parse` senza esaminarli.

```rust
let guess: u32 = match guess.trim().parse() {
    Ok(num) => num,
    Err(_) => continue,
};
```

Sostituire `expect` con un'espressione `match` sul `Result` di `parse` sposta il comportamento dal crash all'ignorare l'input non numerico, chiedendo un nuovo tentativo all'utente.

Capitolo 3

Concetti comuni di programmazione

Questo capitolo tratta i concetti presenti in quasi ogni linguaggio (variabili, tipi base, funzioni, commenti, controllo di flusso) spiegandoli nel contesto di Rust e delle sue convenzioni.

Info

Keyword

Parole riservate usate dal solo linguaggio, non utilizzabili come nomi di variabili o funzioni; alcune non hanno funzionalità attuali ma sono riservate a possibili estensioni future.

Variabili e mutabilità

Le variabili sono immutabili per default: riassegnare una variabile non mutabile produce un errore a compile time (`cannot assign twice to immutable variable`). L'immutabilità è una spinta verso la sicurezza: se una parte del codice assume che un valore non cambi e un'altra lo cambia, nascono bug difficili da rintracciare; il compilatore garantisce che un valore dichiarato immutabile non cambi davvero, e il codice è più facile da ragionare.

- `mut`: aggiunto prima del nome, rende la variabile mutabile e comunica ai lettori futuri che altre parti del codice ne cambieranno il valore.

Costanti

Definition

Costante

Valore vincolato a un nome che non può mai cambiare, dichiarato con `const` invece di `let` e con il tipo obbligatoriamente annotato.

Le costanti si distinguono dalle variabili immutabili per tre proprietà.

- **Immutabili sempre**: non ammettono `mut`, non solo per default.
- **Scope qualsiasi**: dichiarabili in ogni scope, incluso quello globale.
- **Solo espressioni costanti**: il valore dev'essere valutabile a compile time, mai il risultato di un calcolo runtime.

```rust
const THREE_HOURS_IN_SECONDS: u32 = 60 * 60 * 3;
```

- **Convenzione dei nomi**: maiuscolo con trattini bassi (`THREE_HOURS_IN_SECONDS`); il compilatore valuta un insieme limitato di operazioni a compile time, permettendo di scrivere il valore in forma verificabile.
- **Durata**: una costante è valida per tutto il tempo di esecuzione del programma, nel proprio scope, il che le rende utili per valori del dominio noti a più parti del codice.

Shadowing

Definition

Shadowing

Dichiarazione di una nuova variabile con lo stesso nome di una precedente tramite una nuova `let`: la seconda oscura la prima e prende ogni uso del nome finché non viene a sua volta oscurata o lo scope termina.

Lo shadowing si distingue da `mut` per due aspetti.

- **Trasformazioni immutabili**: senza `let` il riassegnamento è un errore di compilazione; con `let` si possono trasformare i valori e la variabile resta comunque immutabile.
- **Cambio di tipo**: si crea di fatto una nuova variabile, quindi il tipo può cambiare riusando lo stesso nome (una `spaces` stringa e poi numerica); con `mut` il cambio di tipo è invece un errore di compilazione.

Tipi di dato

Ogni valore in Rust ha un **tipo di dato** che dice al compilatore come trattarlo. Rust è un linguaggio **staticamente tipizzato**: i tipi di tutte le variabili devono essere noti a compile time; il compilatore di solito li inferisce dal valore e dall'uso, ma quando più tipi sono possibili serve un'annotazione, come `let guess: u32 = "42".parse().expect("Not a number!");`.

Tipi scalari

Un tipo **scalare** rappresenta un singolo valore: Rust ha quattro tipi scalari primari (interi, numeri in virgola mobile, booleani, caratteri).

Interi

Un intero è un numero senza parte frazionaria; ogni variante ha dimensione esplicita e può essere con segno o senza.

Lunghezza	Con segno	Senza segno
8 bit	`i8`	`u8`
16 bit	`i16`	`u16`
32 bit	`i32`	`u32`
64 bit	`i64`	`u64`
128 bit	`i128`	`u128`
Dipende dall'architettura	`isize`	`usize`

- **Con segno / senza segno**: indica se il numero può essere negativo; i signed sono memorizzati in **complemento a due**.
- **Intervalli**: con segno da $-(2^{n-1})$ a $2^{n-1}-1$ (per `i8`: da -128 a 127); senza segno da 0 a 2^n-1 (per `u8`: da 0 a 255), dove n è il numero di bit.
- **`isize` e `usize`**: dipendono dall'architettura della macchina (64 bit su architettura a 64 bit); l'uso principale è l'indicizzazione di collezioni.
- **Default**: `i32` è generalmente un buon punto di partenza.

I letterali interi ammettono forme diverse, un suffisso di tipo (come `57u8`) e `_` come separatore visivo.

Letterale	Esempio
Decimale	`98_222`
Esadecimale	`0xff`
Ottale	`0o77`
Binario	`0b1111_0000`
Byte (solo `u8`)	`b'A'`

Warning

Integer overflow

Assegnare un valore fuori dall'intervallo del tipo causa un **overflow**: in debug il programma va in **panic** a runtime, mentre in release (flag `--release`) non c'è controllo e il valore esegue il *wrapping* in complemento a due (per `u8`, 256 diventa 0, 257 diventa 1) senza panic; affidarsi al wrapping è considerato un errore.

Per gestire esplicitamente l'overflow la libreria standard offre quattro famiglie di metodi sui tipi numerici primitivi.

- **`wrapping_*`** (es. `wrapping_add`): esegue il wrapping in ogni modalità.
- **`checked_*`**: restituisce `None` in caso di overflow.
- **`overflowing_*`**: restituisce il valore e un booleano che indica se c'è stato overflow.
- **`saturating_*`**: satura al minimo o al massimo del tipo.

Numeri in virgola mobile

Rust fornisce due tipi primitivi per i numeri con virgola.

- **`f32` e `f64`**: i due tipi primitivi per numeri con virgola, rispettivamente a 32 e 64 bit; il default è `f64` perché sulle CPU moderne è grosso modo veloce come `f32` ma con più precisione.
- **IEEE-754**: lo standard con cui sono rappresentati; tutti i tipi in virgola mobile sono con segno.

Operazioni numeriche

Tutti i tipi numerici condividono gli operatori aritmetici fondamentali.

- **Operatori**: addizione, sottrazione, moltiplicazione, divisione e resto per tutti i tipi numerici; la **divisione intera** tronca verso zero al numero intero più vicino.

Booleani

Il tipo booleano ha due soli valori.

- `bool`: tipo con i due valori `true` e `false`, grande un byte; l'uso principale è le condizioni di `if`.

Caratteri

Il tipo alfabetico di Rust lavora su Unicode, non solo su ASCII.

- `char`: tipo alfabetico più primitivo, grande 4 byte; i letterali usano gli apici singoli, a differenza delle stringhe.
- **Unicode scalar value**: `char` rappresenta molto più dell'ASCII (lettere accentate, ideogrammi CJK, emoji, spazi a larghezza zero), con intervalli da `U+0000` a `U+D7FF` e da `U+E000` a `U+10FFFF` inclusi.
- **Attenzione**: "character" non è davvero un concetto di Unicode, quindi l'intuizione su cosa sia un carattere può non coincidere con `char`.

Tipi composti

I tipi **composti** raggruppano più valori in un solo tipo; Rust ha due tipi composti primitivi, tuple e array.

Tuple

La tupla raggruppa valori di tipi diversi con lunghezza fissa.

- **Tupla**: modo generale di raggruppare valori di tipi anche diversi in un tipo composto, con lunghezza fissa: una volta dichiarata non può crescere né rimpicciolirsi.
- **Destrutturazione**: i valori si estraggono con il pattern matching in una `let`, che spezza la tupla singola in più variabili.
- **Accesso diretto**: si usa il punto seguito dall'indice, come `x.0` e `x.1`; il primo indice è 0.
- **Elementi mutabili**: in una tupla mutabile si possono modificare i singoli elementi.

```rust
let mut x: (i32, i32) = (1, 2);
x.0 = 0;
x.1 += 5;
```

Definition

Unit

Tupla vuota e relativo tipo, scritti `()`: rappresenta un valore vuoto o un tipo di ritorno vuoto; le espressioni che non restituiscono altro valore restituiscono implicitamente il unit.

Array

L'array contiene un numero fisso di elementi dello stesso tipo.

- **Array**: collezione di valori tutti dello stesso tipo, con lunghezza fissa, scritti come elenco separato da virgole tra parentesi quadre.
- **Allocazione**: a differenza del **vector** (la collezione equivalente ma estendibile, che vive sull'heap), l'array è un blocco singolo di memoria di dimensione nota allocato sullo **stack**; se si è indecisi, conviene il vector.
- **Tipo**: si scrive `[tipo; lunghezza]`, come `let a: [i32; 5] = [1, 2, 3, 4, 5];`.

```rust
let a = [3; 5]; // uguale a [3, 3, 3, 3, 3]
```

- **Inizializzazione ripetuta**: `[valore; lunghezza]` crea l'array con tutti gli elementi pari al valore iniziale.
- **Indicizzazione**: si accede con `[i]`, partendo da 0.
- **Accesso invalido**: un indice oltre la fine fa scattare un controllo che Rust esegue a runtime: se l'indice è maggiore o uguale alla lunghezza il programma va in **panic**, esce con un errore e non accede alla memoria invalida; il controllo dev'essere runtime perché il compilatore non può conoscere l'input dell'utente; è un esempio dei principi di memory safety del linguaggio.

Funzioni

Le funzioni organizzano il codice in unità riusabili.

- `fn`: parola chiave che dichiara una nuova funzione, seguita dal nome, dalle parentesi dei parametri e dal corpo tra graffe; la posizione della definizione nel sorgente non conta, basta che sia visibile allo scope del chiamante.
- **Snake case**: lo stile convenzionale per nomi di funzioni e variabili: tutto minuscolo con trattini bassi a separare le parole.

Parametri

I parametri della firma seguono una terminologia precisa.

- **Parametro**: variabile speciale che fa parte della firma della funzione; il valore concreto passato alla chiamata è tecnicamente l'**argomento**, anche nell'uso colloquiale i due termini si confondono.
- **Tipi obbligatori**: nella firma va dichiarato il tipo di ogni parametro; è una scelta deliberata: rende il compilatore capace di non richiedere annotazioni quasi mai altrove e di dare messaggi di errore più utili; parametri multipli si separano con virgole.

Statement ed espressioni

Definition

Statement

Istruzione che esegue un'azione e non restituisce alcun valore.

Definition

Expression

Costrutto che produce un valore risultante; Rust è un linguaggio basato su espressioni, e la distinzione è importante per i corpi delle funzioni.

- **Statement**: `let y = 6;` è uno statement, e anche la definizione di una funzione; chiamare una funzione invece non è uno statement.
- **Niente valore dagli statement**: non si può assegnare una `let` a un'altra variabile; a differenza di C e Ruby, in Rust l'assegnamento non restituisce il valore, quindi `x = y = 6` non è valido.
- **Espressioni**: le operazioni matematiche (`5 + 6` vale 11), le chiamate di funzione, le chiamate di macro e i blocchi creati con le graffe.
- **Blocchi come espressioni**: un blocco valuta l'ultima espressione contenuta; il valore può essere vincolato in una `let`.

```rust
let y = {
    let x = 3;
    x + 1
}; // y vale 4
```

- **Punto e virgola**: le espressioni non terminano con `;`; aggiungerlo trasforma l'espressione in uno statement, che quindi non restituisce valore.

Valori di ritorno

Il valore di ritorno si dichiara nella firma della funzione.

- **Firma**: il tipo di ritorno si dichiara dopo una freccia `->`, senza nome per il valore.
- **Ultima espressione**: il valore di ritorno è sinonimo dell'ultima espressione del corpo della funzione; la keyword `return` esce in anticipo con un valore, ma la maggior parte delle funzioni restituisce implicitamente l'ultima espressione.

```rust
fn five() -> i32 {
    5
}
```

- **Errore col punto e virgola**: aggiungere `;` all'ultima espressione la trasforma in uno statement che restituisce `()`, il tipo unit, contraddicendo il tipo dichiarato e producendo un errore *mismatched types*.

Commenti

Rust supporta due sintassi per i commenti.

- `//`: lo stile idiomatico; il commento dura fino a fine riga e per commenti lunghi va ripetuto su ogni riga.
- `/* ... */`: sintassi alternativa per commenti su più righe.
- **Posizione**: a fine riga dopo il codice, oppure sulla riga sopra il codice annotato.
- **Documentation comment**: altro tipo di commento dedicato alla documentazione.

Controllo di flusso

I costrutti principali per controllare il flusso di esecuzione sono le espressioni `if` e i cicli.

Espressioni `if`

Il costrutto condizionale fondamentale è l'espressione `if`.

- `if`: keyword seguita da una condizione e dal blocco tra graffe da eseguire se la condizione è vera; i blocchi legati alle condizioni si chiamano **armi**, come nel `match`.
- `else`: espressione opzionale che offre il blocco alternativo quando la condizione è falsa; senza `else` il programma salta il blocco e prosegue.

Error

Condizione non booleana

La condizione dev'essere un `bool`: Rust non converte automaticamente gli altri tipi in booleano, a differenza di Ruby e JavaScript; occorre essere espliciti, scrivendo per esempio `if number != 0` e non `if number`.

`else if`

Più condizioni si concatenano con `else if`.

- `else if`: combina più condizioni; il programma le esamina a turno ed esegue il primo corpo la cui condizione è vera, ignorando completamente le successive.
- **Troppi rami**: un `else if` eccessivo intralcia il codice; il costrutto `match` è l'alternativa potente per questi casi.

`if` in un'istruzione `let`

Poiché `if` è un'espressione, può stare sul lato destro di una `let` per assegnare il proprio valore a una variabile.

```rust
let condition = true;
let number = if condition { 5 } else { 6 };
```

- **Armi dello stesso tipo**: i potenziali risultati di ogni arm devono avere lo stesso tipo, perché ogni variabile ha un unico tipo che il compilatore deve conoscere a compile time per verificarne gli usi; un tipo deciso solo a runtime renderebbe il compilatore più complesso e con meno garanzie.

Cicli

Rust ha tre tipi di ciclo: `loop`, `while` e `for`; un ciclo esegue il corpo fino alla fine e riparte subito dall'inizio.

`loop`

Il ciclo più primitivo ripete il blocco per sempre.

- `loop`: dice a Rust di eseguire il blocco per sempre, finché non si ferma esplicitamente.
- `break`: dice al programma di uscire dal ciclo.
- `continue`: salta il codice rimanente dell'iterazione e passa alla successiva.
- **Valore di ritorno**: si può aggiungere un valore dopo `break` (come `break counter * 2;`) e quel valore è restituito dall'espressione del ciclo; `break` e `return` sono simili: entrambi prendono un'espressione opzionale, cambiano il flusso, il codice che segue non viene mai eseguito e il loro valore è l'unit `()`; il punto e virgola dopo `break` è tecnicamente opzionale.
- **Etichette di ciclo**: con cicli annidati `break` e `continue` si applicano al ciclo più interno; una **loop label** dichiarata sul ciclo e richiamata come `break 'etichetta` o `continue 'etichetta` indica quale ciclo interessa; le etichette iniziano con un apostrofo.

`while`

Il ciclo condizionale abbrevia il pattern `loop` più condizione.

- **`while`**: ciclo condizionale che esegue il corpo finché la condizione è vera; condensa il pattern altrimenti espresso combinando `loop`, `if`, `else` e `break`.

`for`

Il ciclo più usato itera direttamente sugli elementi.

- **`for`**: esegue del codice per ogni elemento di una collezione, in alternativa al `while` con indice.
- **Limiti del `while` con indice**: è error-prone (una condizione sbagliata causa panic o elementi saltati) e più lento, perché il compilatore aggiunge codice runtime per controllare i limiti dell'array a ogni iterazione; il codice macchina generato dai `for` è più efficiente.
- **Conciso e sicuro**: la sicurezza e la concisione rendono `for` il costrutto di ciclo più usato in Rust, anche per ripetere un'azione un numero fisso di volte.
- **`Range`**: tipo della libreria standard che genera la sequenza di numeri da un estremo incluso fino a quello successivo escluso; il metodo **`rev`** inverte il range.

```rust
for number in (1..4).rev() {
    println!("{number}!");
}
```

Parte II

Ownership e tipi strutturati

Capitolo 4

Ownership

L'ownership è la caratteristica più unica di Rust e ha implicazioni profonde sul resto del linguaggio: permette a Rust di garantire la memoria-sicurezza senza bisogno di un garbage collector. In questa lezione si tratta l'ownership e diversi concetti ad essa collegati: il borrowing, le slice e il modo in cui Rust dispone i dati in memoria.

Sicurezza e comportamento indefinito

L'ownership è una disciplina per garantire la **sicurezza** dei programmi Rust; per capirla occorre prima capire quando un programma è sicuro o insicuro. Consideriamo una funzione `read(y: bool)` chiamata con una variabile `x` definita dopo la chiamata: `read` si aspetta che `x` abbia un valore di tipo `bool`, ma `x` non ha ancora un valore, quindi il programma è insicuro.

```rust
fn main() {
    read(x); // errore: x non è ancora definito!
    let x = true;
}
```

Un interprete come Python o JavaScript rileverebbe l'errore a runtime con un'eccezione (`NameError` o `ReferenceError`), ma ogni controllo a runtime ha un costo. Rust vuole compilare binari efficienti con il minor numero di controlli a runtime possibile: la variabile viene quindi verificata a **compile-time**, non a runtime.

Definition

Comportamento indefinito

Un comportamento è indefinito quando Rust non specifica cosa accade dopo l'esecuzione di un'istruzione, ad esempio `if y { .. }` con `y` che non è né `true` né `false`. Un goal fondativo di Rust è che i programmi non abbiano mai comportamento indefinito: questo è il significato di "sicurezza".

Se il programma insicuro fosse compilato, il registro `edi` conterrebbe un valore qualsiasi (`2`, `100`, `0x1337BEEF`) invece di `0` o `1`. Le conseguenze possibili del comportamento indefinito sono diverse.

- Il codice **esegue senza crash** e nessuno si accorge del problema.
- Il codice **crasha** immediatamente per un segmentation fault o un altro errore del sistema operativo.
- Il codice **esegue senza crash** finché un attaccante non crea l'input giusto per cancellare il database di produzione, sovrascrivere i backup e rubare i dati.

Il comportamento indefinito è particolarmente pericoloso per i programmi low-level con accesso diretto alla memoria: circa il 70% delle vulnerabilità di sicurezza segnalate nei sistemi low-level deriva da corruzione della memoria, una forma di comportamento indefinito. Prevenirlo a compile-time invece che a runtime ha due motivazioni.

- Catturare i bug a compile-time **evita quei bug in produzione**, migliorando l'affidabilità del software.
- Catturare i bug a compile-time **richiede meno controlli a runtime**, migliorando le prestazioni del software.

Le variabili vivono nello stack

Le variabili vivono nei **frame**. Un frame è una mappa da variabili a valori all'interno di un singolo scope, come una funzione; ad esempio il frame di `main` contiene `n = 5`, mentre il frame di `plus_one` contiene `x = 5`. I frame sono organizzati in uno **stack** di funzioni chiamate: quando una funzione ritorna, Rust dealloca il suo frame (la deallocazione è detta anche *freeing* o *dropping*). Si chiama stack perché il frame aggiunto più di recente è sempre il prossimo a essere liberato.

Quando un'espressione legge una variabile, il valore è copiato dal suo slot nel frame: in `let mut b = a; b += 1;` il valore di `a` è copiato in `b` e `a` resta invariato anche dopo la modifica di `b`.

I box vivono nell'heap

Copiare dati grandi consuma molta memoria: assegnare un array di un milione di elementi produce un frame con due milioni di elementi. Per trasferire l'accesso ai dati senza copiarli, Rust usa i **puntatori**.

Definition

Puntatore

Un puntatore è un valore che descrive una posizione in memoria; il valore puntato è detto **pointee**. Un modo comune per creare un puntatore è allocare memoria nell'**heap**, la regione di memoria dove i dati possono vivere indefinitamente, slegati da uno specifico frame dello stack.

Rust fornisce il costrutto `Box` per mettere dati nell'heap, avvolgendo ad esempio l'array milionario in `Box::new`:

```rust
let a = Box::new([0; 1_000_000]);
let b = a; // copia il puntatore, non i dati
```

Dopo `let b = a` esiste un solo array: `b` contiene il puntatore, mentre `a` è disattivata perché ne è stato effettuato il *move*. Questo modello di memoria non descrive letteralmente l'implementazione (il compilatore può usare i registri), ma è l'astrazione corretta per ragionare sulla sicurezza.

La deallocazione e l'ownership

La gestione della memoria è il processo di allocare e deallocare memoria; i frame dello stack sono gestiti automaticamente da Rust, ma quando viene deallocata la memoria dell'heap? Immaginiamo che Rust offra una `free()` richiamabile a piacere: dopo `free(b)`, il valore di `b` è un puntatore a memoria invalida, e usarlo leggendo `b[0]` causerebbe comportamento indefinito (crash o dati arbitrari). Rust non permette la deallocazione manuale proprio per evitare questo tipo di comportamento indefinito.

Invece Rust libera automaticamente la memoria heap dei box, secondo un principio che si precisa in due passaggi.

- **Versione quasi corretta**: se una variabile è legata a un box, quando Rust dealloca il frame della variabile dealloca anche la memoria heap del box.
- **Versione corretta**: se una variabile **possiede** un box, quando Rust dealloca il frame della variabile dealloca anche la memoria heap del box.

La seconda versione è necessaria perché due variabili possono essere legate allo stesso box: con `let b = a;` il principio quasi corretto libererebbe la memoria due volte, che è anch'esso comportamento indefinito. Quando `a` è legata a `Box::new(..)`, diciamo che `a` **possiede** il box; l'istruzione `let b = a` **trasferisce** (*move*) la proprietà del box da `a` a `b`, così la deallocazione avviene una sola volta, per conto di `b`.

Le collezioni usano i box

I box sono usati dalle strutture dati di Rust come `Vec`, `String` e `HashMap` per contenere un numero variabile di elementi. Un esempio tipico crea una stringa, la passa a una funzione e la modifica:

```rust
fn add_suffix(mut name: String) -> String {
    name.push_str(" Jr.");
    name
}
```

Il passaggio di `first` ad `add_suffix` muove la proprietà della stringa in `name`, senza copiare i dati ma copiando il puntatore. L'operazione `name.push_str(" Jr.")` ridimensiona l'allocazione heap della stringa: crea un'allocazione più grande, scrive il nuovo contenuto e libera quella originale. Al ritorno, `name` trasferisce la proprietà della stringa alla variabile del chiamante.

Le variabili mosse non sono più usabili

Se `first` fosse usata in `main` dopo la chiamata ad `add_suffix`, leggere `first` accederebbe a memoria deallocata: non è un problema che `first` punti a memoria deallocata, è un problema *usarla* dopo che è diventata invalida. Rifiuta la compilazione con l'errore `E0382` (*borrow of moved value*), perché `first` è di tipo `String`, che non implementa il trait `Copy`.

Info

Principio dei dati heap mossi

Se una variabile `x` muove la proprietà di dati heap a un'altra variabile `y`, allora `x` non può essere usata dopo il move. Muovere la proprietà evita il comportamento indefinito della lettura di memoria deallocata.

Il cloning evita i move

Un modo per evitare il trasferimento è **clonare** i dati con `.clone()`. A differenza di una copia "shallow" del solo puntatore, `first.clone()` esegue una copia "deep" dei dati in una nuova allocazione heap: dopo aver mosso il clone, la variabile originale resta invariata e utilizzabile.

Riferimenti e borrowing

Ownership, box e move forniscono le fondamenta per programmare in sicurezza con l'heap, ma le API solo-move sono scomode. In un esempio con due stringhe passate a `greet`, il passaggio muove i dati nei parametri e le stringhe muoiono a fine funzione: leggerle dopo sarebbe comportamento indefinito, e il compilatore rifiuta il programma. Far restituire le stringhe alla funzione è verboso: Rust offre uno stile conciso per leggere e scrivere senza move, tramite i riferimenti.

I riferimenti sono puntatori non posseduti

Un **riferimento** è un tipo di puntatore. L'espressione `&m1` usa l'operatore `&` per creare un riferimento a (o "prendere in prestito", *borrow*) `m1`, e il parametro diventa di tipo `&String`:

```rust
fn greet(g1: &String, g2: &String) {
    println!("{} {}!", g1, g2);
}
```

Dal riferimento `g1` alla stringa ci sono due passaggi: `g1` punta a `m1` sullo stack, e `m1` è una `String` con un box che punta ai dati sull'heap. Mentre `m1` possiede i dati heap, `g1` non possiede né `m1` né la stringa: quando `greet` termina, nessun dato heap viene deallocato, solo il frame della funzione. I riferimenti sono quindi **puntatori non posseduti**.

Il dereferenziamento

Il meccanismo che "segue" un puntatore fino ai dati è il operatore di **dereferenziazione**, scritto con un asterisco (`*`).

- `*x` a destra di un'assegnazione **legge** il valore puntato, quindi `let a = *x;` con `x: Box<i32>` dà `a = 1`.
- `*x` a sinistra **modifica** il valore puntato, quindi `*x += 1;` fa puntare `x` al valore `2`.
- Con `let r2 = &*x;` il riferimento punta direttamente al valore nell'heap, quindi basta una sola dereferenziazione per leggerlo.

Il dereferenzamento esplicito è raro nel codice idiomatico perché Rust inserisce implicitamente dereferenziazioni e prestiti, ad esempio con l'operatore punto.

- `x.abs()` con `x: Box<i32>` equivale a `i32::abs(*x)`: la sintassi col punto è zucchero sintattico per la chiamata funzionale.
- Su `r: &Box<i32>` la chiamata `r.abs()` inserisce **due** dereferenziazioni implicite.
- Nel verso opposto, `s.len()` con `s: String` inserisce un operatore di borrowing, perché `str::len` si aspetta `&str`.

Aliasing e mutazione

I puntatori sono potenti e pericolosi perché permettono l'**aliasing**: accedere agli stessi dati tramite variabili diverse. Da solo l'aliasing è innocuo, ma combinato con la **mutazione** diventa una ricetta per il disastro, perché una variabile può "tirare il tappeto" sotto l'altra in diversi modi.

- **Deallocando** i dati condivisi, lasciando l'altra variabile a puntare memoria deallocata.
- **Mutando** i dati condivisi, invalidando proprietà attese a runtime dall'altra variabile.
- **Mutando concorrentemente** i dati condivisi, causando una data race con comportamento nondeterministico.

Come esempio si usa il vettore `Vec`: a differenza degli array a lunghezza fissa, memorizza gli elementi nell'heap con lunghezza variabile. La macro `vec!` crea un vettore con gli elementi tra parentesi; il tipo `Vec<i32>` dichiara elementi di tipo `i32`. Il vettore alloca un array heap con una certa **capacità**: un `push` a capacità piena crea un'allocazione più grande, copia tutti gli elementi e dealloca l'array originale. Se esiste un riferimento a un elemento del vettore, il `push`

lo invalida, lasciandolo su memoria invalida: leggerlo causa comportamento indefinito. Il problema è che il vettore è sia aliasato (dal riferimento) che mutato (dal `push`).

Definition

Pointer safety principle

I dati non devono mai essere aliasati e mutati allo stesso tempo. I dati possono essere aliasati, e possono essere mutati, ma non entrambe le cose contemporaneamente. Per i box il principio è rispettato vietando l'aliasing: assegnare un box muove la proprietà e invalida la variabile precedente.

I permessi sulle place

L'idea centrale del borrow checker è che le variabili hanno tre tipi di **permessi** sui propri dati.

- **Read**: i dati possono essere copiati in un'altra posizione.
- **Write**: i dati possono essere mutati.
- **Own**: i dati possono essere mossi o droppati.

I permessi non esistono a runtime, solo nel compilatore. Per default una variabile ha i permessi di lettura e possesso; con `let mut` ha anche il permesso di scrittura. L'idea chiave è che **i riferimenti rimuovono temporaneamente questi permessi**: creare un riferimento a `v` (borrow) rimuove da `v` i permessi di scrittura e possesso finché il riferimento è in uso, poi li restituisce.

Un dettaglio dei diagrammi dei permessi è la distinzione tra `num` e `*num`: accedere ai dati tramite un riferimento non è come manipolare il riferimento stesso. Con `let mut x_ref = &x;` il permesso di scrittura vale per `x_ref` (riassegnabile a un altro riferimento) ma non per `*x_ref` (i dati puntati non sono mutabili). I permessi sono definiti sulle **place**, cioè tutto ciò che può stare a sinistra di un'assegnazione.

- **Variabili**, come `a`.
- **Dereferenziazioni** di place, come `*a`.
- **Accessi ad array** di place, come `a[0]`.
- **Campi** di place, come `a.0` per le tuple o `a.field` per le struct.

Il borrow checker

Lo scopo dei permessi è garantire che i dati aliasati non possano essere mutati: creare un riferimento rende i dati temporaneamente di sola lettura finché il riferimento non è più in uso. Il **borrow checker** cerca le operazioni potenzialmente insicure coinvolgenti riferimenti: nel programma con `num = &v[2]` seguito da `v.push(4)`, l'operazione `push` richiede il permesso di scrittura su `v`, che il borrow ha rimosso.

```Text
1 error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
```

Il messaggio spiega che `v` non può essere mutato mentre il riferimento `num` è in uso; il problema sottostante è che `num` potrebbe essere invalidato dal `push`, e Rust intercetta quella potenziale violazione della memoria-sicurezza.

I riferimenti mutabili

I riferimenti visti finora sono **riferimenti immutabili** (o *shared*): permettono l'aliasing ma vietano la mutazione. Per fornire temporaneamente accesso mutabile senza muovere i dati esistono i **riferimenti mutabili** (o *unique*), creati con l'operatore `&mut` e tipizzati `&mut i32`. Rispetto a quelli immutabili cambiano due cose nei permessi.

- La place prestata `v` **perde tutti i permessi** mentre il riferimento mutabile è in uso (non solo scrittura e possesso): questo li rende sicuri, perché permettono la mutazione ma impediscono l'aliasing.
- La place `*num` **guadagna anche il permesso di scrittura**: questo li rende utili, perché `*num += 1` muta `v[2]`. Il permesso di scrittura vale per `*num` ma non per `num`, che non può essere riassegnato a un altro riferimento mutabile.

I riferimenti mutabili possono anche essere temporaneamente "degradati" a sola lettura: con `let num2 = &*num;` il borrow `&*num` rimuove da `*num` solo il permesso di scrittura, così entrambi i riferimenti restano leggibili.

I lifetime

Un riferimento cambia i permessi finché è "in uso": questo intervallo è il **lifetime** del riferimento, ovvero l'intervallo di codice dalla sua nascita (dove è creato) alla sua morte (l'ultimo uso). Ad esempio il lifetime di `y` inizia con `let y = &x;` e finisce con `let z = *y;`: dopo quella riga, il permesso di scrittura torna a `x`.

Con il controllo di flusso il lifetime non è necessariamente contiguo: in una funzione che capitalizza il primo carattere di un vettore, il ramo `then` usa il riferimento più a lungo, mentre nel ramo `else` la place prestata riguadagna subito il permesso di scrittura.

I dati devono sopravvivere ai propri riferimenti

Come parte del pointer safety principle, il borrow checker impone che **i dati devono sopravvivere a tutti i riferimenti che li puntano**. La prima enforcement usa i permessi già visti: droppare una stringa mentre ne esiste un riferimento fallisce, perché il borrow `&s` ha rimosso il permesso di possesso da `s` mentre `drop` lo richiede.

Per i riferimenti in ingresso o in uscita da una funzione serve un meccanismo diverso: il permesso di **flow**. A differenza di read, write e own, il flow non cambia nel corpo della funzione; un riferimento può "fluire" in un'espressione solo se permesso. Una funzione `first_or(strings, default) -> &String` che ritorna o il primo elemento o `default` non compila: dalla sola firma Rust non sa se l'output è preso da `strings` o `default`, e restituisce l'errore `E0106` (*missing lifetime specifier*). Il problema è reale: se `default` potesse fluire nel valore di ritorno, il chiamante potrebbe droppare `default` e invalidare il risultato. Rust fornisce i **parametri di lifetime** per specificare queste relazioni, trattati nel Capitolo 10.

Correggere gli errori di ownership

Correggere un errore di ownership è una competenza centrale di Rust: quando il borrow checker rifiuta il codice, occorre chiedersi se la funzione sia *davvero* sicura o insicura. Rust rifiuta sempre i programmi insicuri, ma a volte rifiuta anche programmi sicuri, e le strategie di correzione sono diverse nei due casi.

Un programma insicuro con riferimento allo stack

Il primo caso di studio è una funzione che ritorna un riferimento a una stringa locale: il problema è il lifetime dei dati riferiti, perché la stringa muore al termine della funzione. Per estendere la vita della stringa esistono quattro strategie.

- **Muovere la proprietà fuori dalla funzione**: cambiare il tipo di ritorno da `&String` a `String`.
- **Ritornare un letterale**: una stringa letterale vive per sempre (`'static`), senza bisogno di allocazione heap, se la stringa non deve mai cambiare.
- **Usare un puntatore con reference counting**: `Rc<String>` rinvia il borrow-checking al runtime, perché l'`Rc` dealloca i dati solo quando l'ultimo riferimento viene droppato.
- **Fornire uno slot al chiamante**: un parametro `&mut String` in cui la funzione scrive il risultato; è verboso ma più efficiente in memoria se il chiamante controlla le allocazioni.

La strategia migliore dipende dall'applicazione, ma l'idea chiave è riconoscere la causa alla radice: quanto a lungo deve vivere la stringa, e chi è responsabile di deallocarla?

Permessi insufficienti

Un altro errore comune è provare a mutare dati di sola lettura o a droppare dati dietro un riferimento. Una funzione `stringify_name_with_title(name: &Vec<String>)` che chiama `name.push(..)` è rifiutata perché `push` richiede il permesso di scrittura, che un riferimento immutabile non ha; il programma è insicuro perché il `push` potrebbe invalidare altri riferimenti a `name` fuori dalla funzione. Le correzioni possibili hanno compromessi diversi.

- **Cambiare il parametro in `&mut Vec<String>`**: funziona, ma le funzioni non dovrebbero mutare i propri input quando il chiamante non se lo aspetta.
- **Prendere possesso con `Vec<String>`**: funziona, ma è raro che una funzione Rust prenda possesso di strutture che possiedono heap come `Vec` e `String`, perché rende l'input inutilizzabile dal chiamante.
- **Clonare l'input**: permette di mutare la copia locale, ma copia ogni stringa del vettore.
- **Costruire prima il risultato e aggiungere poi il suffisso**: usare `name.join(" ")` e poi `push_str(" Esq.")` è la soluzione più idiomatica, perché `join` copia già i dati in una nuova stringa.

In generale, scrivere funzioni Rust è un bilanciamento del livello *giusto* di permessi richiesti.

Aliasing e mutazione di una struttura dati

Un'altra operazione insicura è usare un riferimento a dati heap che viene deallocato da un altro alias. In `add_big_strings (dst: &mut Vec<String>, src: &[String])`, il riferimento `largest` al massimo di `dst` è creato e poi usato mentre `dst.push(..)` muta il vettore: il borrow di `largest` rimuove la scrittura da `dst`, ma il `push` la richiede; è insicuro perché il `push` potrebbe deallocare i contenuti di `dst`, invalidando `largest`. Tutte le correzioni condividono l'idea di accorciare il lifetime del borrow perché non si sovrapponga alla mutazione.

- **Clonare `largest`**: funziona ma costa l'allocazione e la copia della stringa.
- **Eseguire prima tutti i confronti**: raccogliere in un vettore `to_add` le stringhe da aggiungere, poi mutare `dst`; costa l'allocazione del vettore ausiliario.
- **Copiare solo la lunghezza**: estrarre `largest_len` come `usize`, perché servono solo i contenuti per la lunghezza, è la soluzione più idiomatica e performante.

Copiare o muovere da una collezione

Una confusione comune è copiare dati fuori da una collezione. Copiare un numero da un vettore con `let n = *n_ref;` su `&i32` funziona, perché la dereferenziazione richiede solo il permesso di lettura. Ma con `String` al posto di `i32` il programma non compila più: il vettore possiede la stringa, e la dereferenziazione cercherebbe di prenderne il possesso tramite un riferimento, che invece è un puntatore non posseduto. L'errore `E0507` avverte che non si può muovere fuori da un riferimento condiviso.

Il programma sarebbe insicuro per un **double-free**: dopo `let s = *s_ref;` sia `v` che `s` crederebbero di possedere la stringa, che verrebbe deallocata due volte. La differenza tra i due casi è che `i32` implementa il trait `Copy` mentre `String` no: copiare una `String` copia un puntatore a dati heap, copiare un `i32` no. In sintesi, se un valore non possiede dati heap può essere copiato senza move.

- Un `i32` **non possiede** dati heap, quindi **può** essere copiato senza move.
- Una `String` **possiede** dati heap, quindi **non può** essere copiata senza move.
- Un `&String` **non possiede** dati heap, quindi **può** essere copiato senza move.

Warning

Eccezione dei riferimenti mutabili

Il tipo `&mut i32` non è copiabile: dopo `let b = a;` con `a: &mut i32`, la variabile `a` non può più essere usata. Questo impedisce che due riferimenti mutabili agli stessi dati siano usati contemporaneamente.

Per accedere in sicurezza a un elemento di un vettore di tipi non `Copy` esistono tre strade.

- **Usare un riferimento immutabile**: `&v[0]` senza prendere possesso della stringa.
- **Clonare i dati**: `v[0].clone()` dà il possesso di una copia lasciando intatto il vettore.
- **Muovere fuori con `Vec::remove`**: `v.remove(0)` estrae la stringa, lasciando il vettore più corto.

Programmi sicuri che vengono rifiutati

Rust può anche rifiutare programmi sicuri, perché traccia i permessi a livello fine ma a volte confonde place diverse come la stessa place. Prendere in prestito un campo di una tupla e scrivere l'altro campo funziona: il borrow di `name.0` rimuove scrittura e possesso da `name.0` e da `name` nel suo complesso, ma `name.1` conserva il permesso di scrittura. Se però l'espressione `&name.0` viene spostata in una funzione `get_first(name: &(String, String))`, Rust smette di permettere la mutazione di `name.1`.

La causa è che Rust non guarda l'implementazione di `get_first` ma solo la firma, che dice solo "qualche stringa nell'input viene prestata": Rust assume conservativamente che siano prestati sia `name.0` che `name.1`. Il programma è comunque sicuro, senza comportamento indefinito, e le soluzioni sono due.

- **Inlining**: riportare l'espressione `&name.0` nel corpo, come nel programma originale.
- **Rinvii il borrow-checking al runtime** con i tipi `Cell`, trattati nei capitoli futuri.

Un problema analogo esiste con gli elementi degli array: il borrow checker non distingue le place `a[0]`, `a[1]` e così via, ma usa una sola place `a[_]` che rappresenta tutti gli indici, perché non può sempre determinare il valore di un indice a compile-time. Leggere un indice mentre si scrive un altro viene quindi rifiutato, anche se il programma è sicuro; la standard library fornisce funzioni come `split_at_mut` che aggirano il borrow checker dividendosi l'array in due slice mutabili.

Info

Blocchi unsafe

Funzioni come `split_at_mut` sono implementate con blocchi `unsafe`, che permettono l'uso di puntatori "raw" non controllati dal borrow checker. Quando il borrow checker rifiuta un programma ritenuto sicuro, la strategia generale è cercare funzioni della standard library con blocchi `unsafe` che risolvano il problema, piuttosto che scrivere `unsafe` manualmente.

Le slice

Le **slice** permettono di riferire una sequenza contigua di elementi in una collezione invece della collezione intera; essendo un tipo di riferimento, sono puntatori non posseduti.

Il problema degli indici

Per capire l'utilità delle slice si considera una funzione `first_word` che ritorna la prima parola di una stringa di parole separate da spazi. Senza le slice, la firma può solo ritornare un indice di byte trovato scandendo `s.as_bytes()` con `iter().enumerate()` e cercando lo spazio `b' '`. Il problema è che l'indice ritornato è uno `usize` slegato dalla stringa: dopo `let word = first_word(&s);`, una chiamata `s.clear()` svuota la stringa ma `word` contiene ancora `5`, e usarlo come indice sarebbe un bug. Con una `second_word` servirebbero due indici da tenere sincronizzati manualmente con i dati: Rust risolve con le slice.

Le string slice

Una **string slice** è un riferimento a una porzione di una `String`, specificata con un range tra parentesi quadre: `&s[0..5]` è la porzione che inizia all'indice 0 e finisce prima dell'indice 5, con tipo `&str`.

```rust
let s = String::from("hello world");
let hello: &str = &s[0..5];
let world: &str = &s[6..11];
```

Le slice sono riferimenti speciali perché sono **puntatori "grassi"** (fat pointer): contengono un puntatore all'inizio della sequenza più la lunghezza della sequenza come metadati. Essendo riferimenti, cambiano anche i permessi dei dati riferiti: creare `hello` come slice di `s` rimuove da `s` i permessi di scrittura e possesso finché la slice è in uso. Riscritta con le slice, `first_word` ritorna `&str`, un valore legato ai dati sottostanti, e il bug dell'indice desincronizzato diventa impossibile: dopo `let word = first_word(&s);`, la chiamata `s.clear()` non compila più perché `word` sta ancora prestando `s`, e `clear` richiederebbe un riferimento mutabile.

La sintassi dei range

Con la sintassi dei range di Rust gli estremi ridondanti si possono omettere, in tre modi equivalenti.

- `&s[0..2]` equivale a `&s[..2]`: partendo dall'indice zero, il valore iniziale si può omettere.
- `&s[3..len]` equivale a `&s[3..]`: includendo l'ultimo byte, il valore finale si può omettere.
- `&s[0..len]` equivale a `&s[..]`: si può omettere entrambi per prendere la stringa intera.

Warning

Confini UTF-8

Gli indici dei range delle string slice devono cadere su confini validi di caratteri UTF-8: creare una slice a metà di un carattere multibyte termina il programma con un errore. In questa lezione si assume testo ASCII; la gestione completa di UTF-8 è nel Capitolo 8.

I letterali stringa sono slice

Un letterale stringa è memorizzato dentro il binario del programma: la variabile `let s = "Hello, world!";` ha tipo `&str`, una slice che punta a quel punto specifico del binario. Questo spiega anche perché i letterali sono immutabili: `&str` è un riferimento immutabile.

Le slice come parametri

Definire una funzione con parametro `&str` invece di `&String` la rende più generale e utile senza perdere funzionalità: si può passare direttamente una slice di un letterale, oppure una slice o un riferimento di una `String`, grazie alle **deref coercions**, trattate nel Capitolo 15.

```rust
fn first_word(s: &str) -> &str { /* ... */ }
```

Altri tipi di slice

Le string slice sono specifiche delle stringhe, ma esiste un tipo di slice più generale. Un array come `let a = [1, 2, 3, 4, 5];` ammette una slice `&a[1..3]` di tipo `&[i32]`, che funziona come le string slice memorizzando un riferimento al

primo elemento e una lunghezza; questo tipo di slice si usa per ogni tipo di collezione, ad esempio i vettori del Capitolo 8.

Info

Vantaggio delle slice

Le slice sono riferimenti a sotto-intervalli di una sequenza come una stringa o un vettore. A runtime sono fat pointer con puntatore e lunghezza, e il loro vantaggio sugli indici è che non possono essere invalidati mentre sono in uso.

Ricapitolazione

Ownership e garbage collection

Per contestualizzare l'ownership occorre parlare del **garbage collection**, usato da linguaggi come Python, JavaScript, Java e Go. Un collector a tracciamento scandaglia la memoria a runtime per trovare dati non più raggiungibili e deallocarli. Il beneficio è evitare il comportamento indefinito (come usare memoria liberata in C o C++) senza un type system complesso; gli svantaggi sono due.

- **Prestazioni**: il GC introduce o sovraccarichi piccoli e frequenti (reference counting, come Python e Swift) o sovraccarichi grandi e infrequenti (tracciamento).
- **Imprevedibilità**: è spesso difficile prevedere dal solo codice sorgente quando i dati vengono deallocati.

L'imprevedibilità si vede con un tipo `Document` in Python: copiare il documento in `d2` e chiamare `d2.add_word("world")` muta anche `d`, perché `get_words` restituisce un riferimento mutabile implicito allo stesso array. Riferimenti mutabili pervasivi e impliciti portano a bug imprevedibili quando le strutture dati lasciano "trapelare" i propri interni. La versione Rust rende i puntatori espliciti.

- `new_document` consuma il possesso del vettore: il `Document` **possiede** le parole, deallocate in modo prevedibile quando il proprietario esce dallo scope.
- `add_word` richiede `&mut Document` e consuma il possesso della parola: nessun altro può mutare le parole del documento.
- `get_words` restituisce un riferimento immutabile `&[String]`: creare un nuovo documento richiede una copia profonda con `.to_vec()`, e mutare `d2` non influenza `d`.

Se Rust non è il tuo primo linguaggio, hai già esperienza con memoria e puntatori: Rust li rende solo espliciti, con il doppio beneficio di prestazioni migliori (niente GC) e prevedibilità (nessuna fuga accidentale di dati).

Ownership a runtime

A runtime l'ownership si osserva in tre fatti.

- Rust alloca le variabili locali nei frame dello stack, allocati alla chiamata e deallocati a fine chiamata.
- Le variabili locali contengono **dati** (numeri, booleani, tuple) o **puntatori**.
- I puntatori nascono da box (possiedono dati heap) o da riferimenti (non posseduti).

Le slice sono un tipo speciale di riferimento verso una sequenza contigua di dati, come una porzione di una stringa.

Ownership a compile time

A compile time Rust traccia i permessi di lettura, scrittura e possesso su ogni variabile, e richiede i permessi appropriati per ogni operazione. I permessi cambiano per effetto di due operazioni.

- **Move**: muovere una variabile di tipo non copiabile (come `Box<T>` o `String`) richiede lettura e possesso, e **elimina tutti i permessi** sulla variabile, impedendone l'uso successivo.
- **Borrow**: prendere in prestito una variabile **rimuove temporaneamente** alcuni permessi. Un borrow immutabile disabilita mutazione e move dei dati prestati; un borrow mutabile disabilita anche la lettura, lasciando i dati accessibili solo tramite il riferimento fino alla fine del suo lifetime.

Il collegamento tra permessi e comportamento indefinito

I permessi sono progettati per prevenire specifiche categorie di comportamento indefinito, in particolare due.

- **Use-after-free** (leggere memoria liberata): i borrow immutabili rimuovono il permesso di scrittura, così un `push` che rialloca il vettore non può invalidare un riferimento esistente.
- **Double-free** (liberare memoria due volte): le dereferenziazioni di riferimenti a dati non copiabili non hanno il permesso di possesso, così i dati non possono essere mossi fuori dal riferimento ed essere deallocati due volte.

Questi concetti di memoria, puntatori, comportamento indefinito e permessi sono la fondamenta per capire come le feature successive (struct, enum, trait) interagiscono con l'ownership.

Capitolo 5

Struct

Una *struct* è un tipo di dato personalizzato che permette di raggruppare e nominare più valori correlati che formano un gruppo significativo; per chi conosce un linguaggio orientato agli oggetti, è simile agli attributi dati di un oggetto. Rispetto alle tuple, le struct nominano ogni pezzo di dato, quindi non bisogna affidarsi all'ordine per accedere ai valori. In questa lezione si vedono come definire e istanziare struct, e come definire funzioni associate, in particolare i **metodi**, per specificare il comportamento legato a un tipo. Struct ed enum sono i mattoni per creare nuovi tipi nel dominio del programma e sfruttare appieno il controllo dei tipi a compile-time.

Definire e istanziare struct

Per definire una struct si usa la keyword `struct` seguita dal nome, che dovrebbe descrivere il significato dei dati raggruppati; tra parentesi graffe si definiscono i nomi e i tipi dei pezzi di dato, detti **campi** (*fields*). Per usarla si crea un'**istanza** specificando valori concreti per ogni campo, con coppie `chiave: valore` in qualunque ordine.

```rust
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}
```

Per leggere un valore si usa la notazione puntata, come `user1.email`; se l'istanza è mutabile si può assegnare a un campo particolare, come `user1.email = ...`. L'intera istanza deve essere mutabile: Rust non permette di marcare solo alcuni campi come mutabili. Come ogni espressione, la nuova istanza può essere l'ultima espressione del corpo di una funzione per ritornarla implicitamente.

Field init shorthand

Quando i parametri di una funzione hanno lo stesso nome dei campi della struct, la ripetizione di `email: email` è tediosa e si può usare la *field init shorthand*: se campo e parametro hanno lo stesso nome, basta scrivere una volta solo `email`.

Struct update syntax

Creare una nuova istanza che include quasi tutti i valori di un'altra istanza dello stesso tipo è reso conciso dalla *struct update syntax*: con `..user1` alla fine, i campi non impostati esplicitamente prendono i valori corrispondenti da `user1`, e i campi espliciti si possono elencare in qualunque ordine.

```rust
let user2 = User {
    email: String::from("another@example.com"),
    ..user1
};
```

La sintassi di update usa `=` come un'assegnazione, e infatti **muove** i dati: dopo aver creato `user2`, `user1` è parzialmente invalidata perché la `String` nel campo `username` è stata mossa in `user2`. Se per `email` e `username` fossero state fornite nuove stringhe, usando da `user1` solo i valori `active` e `sign_in_count` (tipi che implementano il trait `Copy`), `user1` resterebbe pienamente valida.

Tuple struct

Rust supporta anche struct simili a tuple, senza nomi di campo ma solo con i tipi, utili quando dare un nome a ogni campo sarebbe verboso o ridondante. Si definiscono con la keyword `struct`, il nome e i tipi tra parentesi tonde; ogni struct definita è un tipo a sé, quindi una funzione che prende un `Color` non può prendere un `Point`, anche se entrambi sono fatti di tre `i32`.

```Rust
struct Color(i32, i32, i32);
struct Point(i32, i32, i32);
```

Le istanze di tuple struct si destrutturano come le tuple e si accede ai singoli valori con punto e indice, ma a differenza delle tuple la destrutturazione richiede di nominare il tipo: `let Point(x, y, z) = origin;`.

Unit-like struct

Esistono anche struct senza alcun campo, dette *unit-like struct* perché si comportano come `()`, il tipo unit. Si dichiarano con `struct`, il nome e un punto e virgola, e sono utili quando serve implementare un trait su un tipo senza che il tipo stesso contenga dati, ad esempio un tipo `AlwaysEqual` le cui istanze siano sempre uguali a ogni altra istanza per scopi di test.

Ownership dei dati dello struct

Nella definizione di `User` si usa il tipo posseduto `String` invece della slice `&str` per una scelta deliberata: ogni istanza possiede tutti i propri dati, che restano validi finché è valida l'intera struct. Una struct può memorizzare riferimenti a dati posseduti da altri, ma ciò richiede i *lifetime*, che garantiscono che i dati riferiti siano validi finché è valida la struct. Senza lifetime specificati il compilatore rifiuta la definizione.

```Text
error[E0106]: missing lifetime specifier
help: consider introducing a named lifetime parameter
```

Fino al Capitolo 10 questi errori si risolvono usando tipi posseduti come `String` invece di riferimenti come `&str`.

Borrow dei campi

Come per i campi di tupla, il borrow checker traccia i permessi sia a livello di struct sia a livello di campo. Se si prende in prestito il campo `p.x` di un `Point`, sia `p` che `p.x` perdono temporaneamente i permessi, ma **non** `p.y`: usare `p` nel suo complesso mentre `p.x` è mutabilmente prestato viene rifiutato, mentre usare `p.y` è permesso.

In generale, davanti a un errore di ownership che coinvolge una struct conviene chiedersi quali campi devono essere presi in prestito e con quali permessi, ricordando però i limiti del borrow checker: a volte Rust assume che siano prestati più campi di quelli realmente usati.

Un programma di esempio con le struct

Per capire quando usare le struct si considera un programma che calcola l'area di un rettangolo a partire da larghezza e altezza. La prima versione con due variabili separate funziona, ma la firma `fn area(width: u32, height: u32)` rivela il problema: la funzione dovrebbe calcolare l'area di *un* rettangolo, ma nulla nel programma dice che i due parametri sono correlati.

Il refactoring con le tuple

Raggruppare i valori in una tupla `(u32, u32)` aggiunge struttura e riduce gli argomenti a uno, ma è meno chiaro: le tuple non nominano i loro elementi, quindi bisogna indicizzare, e la notazione diventa meno ovvia. Scambiare larghezza e altezza non cambierebbe l'area, ma disegnare il rettangolo a schermo richiederebbe di ricordare quale indice è quale.

Il refactoring con le struct

Le struct aggiungono significato etichettando i dati: il tipo si chiama `Rectangle` e i campi `width` e `height` di tipo `u32`. La funzione `area` ora prende un parametro `rectangle: &Rectangle`, un borrow immutabile, perché non si vuole prendere il possesso: `main` lo conserva e può continuare a usare `rect1`. Accedere ai campi di una struct presa in prestito non muove i valori dei campi, ed è per questo che i borrow di struct sono così comuni.

Stampare le struct e i trait derivati

Stampare un'istanza con `println!("{}")` fallisce, perché le graffe usano il formato `Display`, pensato per l'utente finale. Per le struct Rust non indovina il formato voluto: ci sono troppe possibilità (virgole o no, graffe, quali campi mostrare), quindi non esiste un'implementazione fornita di `Display`. Le alternative per stampare richiedono ciascuna una scelta esplicita.

- **Specifica `:?`**: dice a `println!` di usare il formato `Debug`, pensato per gli sviluppatori; richiede l'attributo `#[derive(Debug)]` appena prima della definizione della struct per attivare esplicitamente la funzionalità. La variante `{:#?}` produce un output più leggibile per struct grandi.
- **Macro `dbg!`**: prende il possesso dell'espressione (al contrario di `println!`, che prende un riferimento), stampa file e numero di riga della chiamata insieme al valore risultante e restituisce il possesso del valore. Per non consumare una variabile si passa un riferimento, come `dbg!(&rect1)`.

Info

> **Stream di output**
>
> `dbg!` stampa sullo stream `stderr`, mentre `println!` usa `stdout`; la differenza tra i due stream è trattata nel Capitolo 12.

Oltre a `Debug`, Rust fornisce diversi altri trait usabili con l'attributo `derive` per aggiungere comportamenti utili ai tipi personalizzati; l'implementazione con comportamenti personalizzati e la creazione di trait propri sono argomenti del Capitolo 10.

I metodi

I metodi sono simili alle funzioni: si dichiarano con `fn` e un nome, possono avere parametri e un valore di ritorno, e contengono codice eseguito alla chiamata. A differenza delle funzioni, i metodi sono definiti nel contesto di una struct (o di un enum o trait object), e il loro primo parametro è sempre `self`, che rappresenta l'istanza su cui il metodo è chiamato.

La sintassi dei metodi

Per definire metodi nel contesto di `Rectangle` si apre un blocco `impl` (*implementation*): tutto ciò che sta nel blocco è associato al tipo. Il primo parametro diventa `&self`, abbreviazione di `self: &Self`, dove `Self` è un alias del tipo del blocco `impl`. La chiamata usa la *method syntax* dopo un'istanza: punto, nome del metodo, parentesi e argomenti.

```rust
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
}
```

Come per i parametri normali, un metodo può prendere il possesso di `self`, prenderlo in prestito immutabilmente con `&self` o mutabilmente con `&mut self`; qui si usa `&self` perché il metodo deve solo leggere i dati, non scriverli. Prendere il possesso con solo `self` è raro: serve quando il metodo trasforma `self` in altro e si vuole impedire al chiamante di usare l'istanza originale. Il motivo principale per usare i metodi è l'organizzazione: tutte le operazioni possibili su un'istanza di un tipo stanno in un unico posto, invece che sparse per la libreria.

Un metodo può avere lo stesso nome di un campo della struct: con `rect1.width` e le parentesi Rust intende il metodo, senza parentesi il campo. Quando il metodo omonimo si limita a restituire il valore del campo è detto *getter*: Rust non li genera automaticamente come altri linguaggi, ma permettono di rendere il campo privato e il metodo pubblico per un accesso in sola lettura nella API pubblica del tipo (si veda il Capitolo 7).

I metodi possono avere più parametri oltre a `self`, che funzionano come i parametri delle funzioni: ad esempio `can_hold(&self, other: &Rectangle) -> bool` prende un borrow immutabile di un altro rettangolo e verifica se sta interamente dentro `self`.

Le funzioni associate

Tutte le funzioni definite in un blocco `impl` sono dette **funzioni associate**, perché associate al tipo. Quelle che non hanno `self` come primo parametro non sono metodi, perché non richiedono un'istanza del tipo: un esempio già incontrato è `String::from`.

Le funzioni associate non-metodo sono spesso **costruttori** che ritornano una nuova istanza; il nome `new` è solo una convenzione, non un nome speciale del linguaggio. La chiamata usa la sintassi `::` con il nome del tipo, come `Rectangle::square(3)`, che è anche un modo per namespacing legato ai moduli del Capitolo 7.

Ogni struct può avere più blocchi `impl` equivalenti a uno unico; la separazione in più blocchi diventa utile con generici e trait nel Capitolo 10.

Le chiamate ai metodi sono zucchero sintattico

Ogni chiamata col punto ha una forma funzionale equivalente. Il chiamata `r.area()` diventa `Rectangle::area(&r)`: il nome della funzione è la funzione associata `Rectangle::area`, l'argomento è il parametro `&self`, e Rust inserisce automaticamente l'operatore di borrowing `&`. Analogamente `r.set_width(2)` diventa `Rectangle::set_width(&mut r, 2)`.

Rust non ha un operatore freccia equivalente a `->` di C o C++: inserisce automaticamente tanti riferimenti e dereferenziazioni quanti servono per far combaciare i tipi del ricevitore. Ad esempio chiamare `r.area()` su `r: &mut Box<Rectangle>` equivale a `Rectangle::area(&**r)`, con due dereferenziazioni (riferimento e box) e un borrow immutabile; è anche una situazione in cui un riferimento mutabile viene "degradato" a condiviso. Nel verso opposto non è permesso chiamare `set_width` (che aspetta `&mut self`) su `&Rectangle` o `&Box<Rectangle>`.

Metodi e ownership

Come per i riferimenti, i metodi richiedono che l'istanza abbia i permessi necessari. Il caso di studio usa tre metodi: `area(&self)` legge, `set_width(&mut self)` scrive e `max(self, other: Rectangle)` consuma l'istanza.

- Su un rettangolo posseduto dichiarato senza `mut`, i metodi `area` e `max` sono chiamabili perché servono solo lettura e possesso.
- Chiamare `set_width` fallisce con l'errore `E0596` (*cannot borrow as mutable*) perché manca il permesso di scrittura; lo stesso errore avviene chiamandolo attraverso un riferimento immutabile, anche se il rettangolo sottostante è mutabile.
- Chiamare `max` muove la struct (che non implementa `Copy`): dopo `let max_rect = rect.max(other_rect);`, usare `rect` dà l'errore `E0382` (*borrow of moved value*).

Un caso più sottile è un metodo `set_to_max(&mut self, other)` che assegna `*self = self.max(other)`: chiamare `self.max(..)` cerca di muovere `*self` dietro a un riferimento mutabile, e Rust rifiuta con l'errore `E0507` (*cannot move out of `*self`*). Per un tipo come `Rectangle` il programma sarebbe in realtà sicuro, perché la struct non possiede dati heap: aggiungendo `#[derive(Copy, Clone)]` alla definizione, `*self` non richiede più il permesso di possesso e il codice compila.

Warning

Perché Copy non è automatico

Rust non deriva `Copy` automaticamente per mantenere stabili le API nel tempo: se l'autore di `Rectangle` aggiungesse un campo `name: String`, tutto il codice client che si affida alla copiabilità smetterebbe di compilare. L'attributo `#[derive(Copy)]` è una dichiarazione esplicita dell'autore che la struct resterà sempre `Copy`.

Con un campo `String` il move fuori da `*self` diventerebbe davvero insicuro: il metodo `max` consumerebbe entrambi i rettangoli, deallocando le stringhe nell'heap, e l'assegnazione `*self = max` dropperebbe i dati precedenti in `*self` liberando di nuovo `(*self).name`. Quel doppio libero è comportamento indefinito: quando appare l'errore "cannot move out of `*self`", Rust sta proteggendo da un double-free.

Ricapitolazione

Le struct permettono di creare tipi personalizzati significativi per il dominio, tenendo collegati i pezzi di dati correlati e dando un nome a ciascuno per rendere il codice chiaro. Nei blocchi `impl` si definiscono funzioni associate al tipo, e i metodi sono le funzioni associate che specificano il comportamento delle istanze. Le struct non sono però l'unico modo di creare tipi personalizzati: la feature enum aggiunge un altro strumento.

Capitolo 6

Enum e pattern matching

Le enumerazioni, o *enum*, permettono di definire un tipo elencandone le possibili varianti. In questa lezione si vede come un enum possa codificare significato insieme ai dati, si esplora l'enum `Option` che esprime che un valore può essere qualcosa o nulla, si usa il pattern matching dell'espressione `match` per eseguire codice diverso per valori diversi, e infine si coprono `if let` e `let...else` come idiomi concisi per gestire gli enum.

Definire un enum

Dove le struct raggruppano campi e dati correlati, come un `Rectangle` con `width` e `height`, gli enum permettono di dire che un valore è **uno di** un insieme possibile di valori. L'esempio classico sono gli indirizzi IP: esistono solo due standard, versione quattro e versione sei, quindi se ne possono *enumerare* tutte le varianti, da cui il nome enumerazione. Un indirizzo IP può essere o l'uno o l'altro, mai entrambi, ed entrambi restano fondamentalmente indirizzi IP: questa proprietà rende la struttura enum appropriata, perché un valore enum è una sola delle sue varianti.

```rust
enum IpAddrKind {
    V4,
    V6,
}
```

Le varianti sono namespaced sotto l'identificatore dell'enum, separate dal doppio punto: `IpAddrKind::V4` e `IpAddrKind::V6` sono entrambe di tipo `IpAddrKind`, quindi una funzione può prendere qualunque variante.

I valori di un enum

Oltre al *kind*, serve un modo per memorizzare i dati dell'indirizzo: una prima soluzione è una struct `IpAddr` con un campo `kind: IpAddrKind` e un campo `address: String`. Ma rappresentare lo stesso concetto con il solo enum è più conciso: si possono mettere i dati direttamente dentro ogni variante, come `V4(String)` e `V6(String)`. Da questo emerges un dettaglio del funzionamento degli enum.

- **Il nome di ogni variante diventa anche una funzione** che costruisce un'istanza dell'enum: `IpAddr::V4()` è una chiamata di funzione che prende una `String` e ritorna un'`IpAddr`, definita automaticamente insieme all'enum.
- **Ogni variante può avere tipi e quantità di dati diversi**: la versione quattro può essere `V4(u8, u8, u8, u8)` mentre la sei resta `V6(String)`, cosa che una struct non permetterebbe.

Dentro una variante di enum si può mettere qualunque tipo di dato: stringhe, tipi numerici, struct, persino un altro enum. La libreria standard definisce proprio `IpAddr` in questo modo, con le varianti `V4(Ipv4Addr)` e `V6(Ipv6Addr)` che incapsulano due struct diverse; e poiché la definizione della standard library non è portata nello scope, si può definire la propria senza conflitto (si veda il Capitolo 7).

Un enum `Message` mostra la varietà possibile di forme delle varianti, simili a definizioni di struct diverse raggruppate sotto un unico tipo.

- **`Quit`**: nessun dato associato.
- **`Move`**: campi con nome, come una struct.
- **`Write`**: una singola `String`.
- **`ChangeColor`**: tre valori `i32`.

Con struct separate, ciascuna con il proprio tipo, non si potrebbe definire facilmente una funzione che accetti qualunque tipo di messaggio; con l'enum, `Message` è un tipo unico. Come per le struct, è possibile definire metodi sugli enum con `impl` e `self`.

L'enum `Option`

`Option` è un enum della libreria standard che codifica lo scenario comunissimo in cui un valore può essere qualcosa oppure nulla. Rust **non ha il null** presente in molti altri linguaggi, il valore che significa "non c'è alcun valore": nelle lingue con null, ogni variabile è in uno di due stati, null o non-null, ed è facilissimo usare un valore null come se non lo fosse.

> *Chiamo il mio errore da un miliardo di dollari […] non ho resistito alla tentazione di inserire un riferimento null, semplicemente perché era così facile da implementare. Questo ha portato a innumerevoli errori, vulnerabilità e crash di sistema, che hanno probabilmente causato un miliardo di dollari di dolore e danni negli ultimi quarant'anni.*
>
> – Tony Hoare

Il concetto che null cerca di esprimere rimane utile: un valore attualmente invalido o assente. Il problema non è il concetto ma l'implementazione, quindi Rust lo codifica nell'enum `Option<T>`, definito dalla standard library così:

```rust
enum Option<T> {
    None,
    Some(T),
}
```

`Option<T>` è così utile che è incluso nella prelude: non serve portarlo in scope, e si possono usare `Some` e `None` direttamente senza il prefisso `Option::`. La sintassi `<T>` è un parametro di tipo generico (si veda il Capitolo 10): significa che la variante `Some` può contenere un dato di qualunque tipo, e ogni tipo concreto usato al posto di `T` rende `Option<T>` un tipo diverso, quindi `Some(5)` è `Option<i32>` e `Some('e')` è `Option<char>`; con `None` da solo, il compilatore richiede l'annotazione del tipo, come in `let absent_number: Option<i32> = None;`.

La superiorità su null sta nel sistema dei tipi: `Option<T>` e `T` sono tipi diversi, quindi il compilatore non permette di usare un `Option<T>` come se fosse un valore sicuramente valido. Ad esempio sommare un `i8` a un `Option<i8>` non compila: per eseguire le operazioni su `T` bisogna prima convertire l'`Option<T>` in un `T`, gestendo esplicitamente il caso del valore assente. Ovunque un valore abbia un tipo che **non** è `Option<T>`, si può assumere con sicurezza che non è nullo: questo limite alla pervasività di null è una decisione di design deliberata di Rust. Per estrarre il `T` da un `Some`, `Option<T>` offre molti metodi utili, ma il costrutto generale per gestire ogni variante è `match`.

L'espressione `match`

`match` è un costrutto di controllo di flusso estremamente potente: confronta un valore con una serie di **pattern** ed esegue codice basandosi su quale pattern combacia. I pattern possono essere valori letterali, nomi di variabili, wildcard e altro; la forza di `match` viene dall'espressività dei pattern e dal fatto che il compilatore verifica che tutti i casi possibili siano gestiti. Si può pensare a `match` come a una macchina smistiamonete: ogni valore "cade" nel primo buco in cui combacia.

Un `match` si compone così: la keyword `match`, l'espressione da confrontare (di qualunque tipo, non solo booleano come in `if`), e le **armi** (*arms*). Ogni arma ha due parti, un pattern e il codice separati da `=>`, e le armi sono separate da virgole.

```rust
match coin {
    Coin::Penny => 1,
    Coin::Nickel => 5,
    Coin::Quarter => 25,
}
```

All'esecuzione il valore è confrontato con i pattern delle armi in ordine; al primo combaciamento viene eseguito il codice associato. Il codice di ogni arma è un'espressione, e il suo valore risultante è il valore ritornato dall'intero `match`; per codice su più righe servono le graffe, e in quel caso la virgola finale è opzionale.

I pattern che legano valori

Le armi di match possono legarsi alle parti del valore che combaciano col pattern: è così che si estraggono i valori dalle varianti di un enum. Nell'esempio delle monete, la variante `Quarter` può contenere un valore `UsState`: aggiungendo una variabile `state` al pattern `Coin::Quarter(state)`, quando una moneta combacia la variabile si lega allo stato interno, e il codice dell'arma può usarla.

Il pattern `match` su `Option<T>`

Le regole del `match` si applicano identiche a `Option<T>`, ed è così che si gestisce il caso qualcosa-o-nulla. Una funzione `plus_one` che aggiunge 1 al contenuto di un `Option<i32>` si scrive con due armi: `None => None` e `Some(i) => Some(i + 1)`. Con `plus_one(Some(5))`, il valore `Some(5)` non combacia con `None`, passa alla seconda arma dove combacia con `Some(i)`: la variabile `i` si lega al `5` contenuto in `Some`, l'arma ritorna `Some(6)`. Con `None`, la prima arma combacia e la funzione ritorna `None` senza provare ad aggiungere nulla. Combinare `match` ed enum, legando una variabile ai dati interni, è uno schema che si vedrà spesso nel codice Rust.

L'esaurienza dei match

I pattern delle armi devono coprire **tutte** le possibilità: i match in Rust sono *esaustivi*, e bisogna esaurire ogni ultima possibilità perché il codice sia valido. Una versione di `plus_one` senza l'arma `None` non compila, e il compilatore sa anche dire quale pattern manca. Nel caso di `Option<T>`, impedire di dimenticare il caso `None` protegge dall'assumere di avere un valore quando potrebbe non esserci, rendendo impossibile l'errore da un miliardo di dollari.

Pattern catch-all e `_`

Con gli enum si possono gestire in modo speciale pochi valori particolari e prendere un'azione di default per tutti gli altri, ad esempio in un gioco d'azzardo dove il 3 dà un cappello, il 7 lo toglie e ogni altro numero muove il giocatore.

- **Variabile catch-all**: l'ultima arma usa una variabile come `other`, che si lega a qualunque valore non elencato; soddisfa l'esaurienza e permette di usare il valore, ma deve stare per ultima perché i pattern sono valutati in ordine, e Rust avverte se si aggiungono armi dopo un catch-all.
- **Placeholder `_`**: è il pattern speciale che combacia con qualunque valore **senza legarsi** ad esso; dichiara che il valore non sarà usato, evitando l'avviso di variabile inutilizzata, e anche questa forma soddisfa l'esaurienza perché ignora esplicitamente tutti gli altri valori.
- **Valore unit `()`**: come codice dell'arma `_`, esprime che non deve succedere nulla per i valori non gestiti.

Match e ownership

Se un enum contiene dati non copiabili come una `String`, bisogna fare attenzione a se il match muove o prende in prestito quei dati. Il caso di studio è un `Option<String>`.

- Con il pattern `Some(_)` il valore interno è ignorato, quindi non viene mosso, e `opt` resta usabile dopo il `match`.
- Con una variabile come `Some(s)`, il campo non ignorato viene **mosso**: `opt` perde i permessi di lettura e possesso, e leggerlo dopo il `match` è illegale.
- La soluzione idiomatica per "sbirciare" senza muovere è fare match su un riferimento, `match &opt`: Rust "spinge giù" il riferimento dall'enum esterno al campo interno, quindi `s` ha tipo `&String` e `opt` resta usabile.

Flusso di controllo conciso con `if let` e `let...else`

`if let` combina `if` e `let` in una forma meno verbosa per gestire i valori che combaciano con **un solo** pattern ignorando il resto. Il caso tipico è un `match` che si cura solo della variante `Some`, costretto ad aggiungere il boilerplate `_ => ()`: la sintassi `if let` prende un pattern e un'espressione separati da `=`, e si comporta come un `match` con una sola arma più il caso ignorato.

```rust
let config_max = Some(3u8);
if let Some(max) = config_max {
    println!("Maximum is set to {max}");
}
```

`if let` significa meno digitazione, meno indentazione e meno boilerplate, ma si perde il controllo di esaurienza di `match`: la scelta tra i due dipende dal compromesso tra concisione e sicurezza. Si può pensare a `if let` come allo zucchero sintattico per un `match` che esegue codice per un pattern e ignora tutti gli altri valori. Con `else`, il suo blocco equivale al blocco del caso `_` nel `match` corrispondente.

`let...else`

Il pattern comune di eseguire un calcolo quando il valore è presente e ritornare un default altrimenti ha una sintassi dedicata. Con `if let` il lavoro finisce nel corpo della condizione, annidato e difficile da seguire, e le versioni che producono il valore da un ramo e ritornano anticipatamente dall'altro sono contorte. La sintassi `let...else` prende un pattern a sinistra e un'espressione a destra, come `if let`, ma **senza ramo `if`**: se il pattern combacia, lega il valore nello scope esterno; se non combacia, il flusso entra nel ramo `else`, che **deve** ritornare dalla funzione.

```rust
let Some(state) = coin else {
    return;
};
```

In questo modo il corpo principale della funzione resta sul "happy path", senza flussi di controllo divergenti per i due rami. Quando la logica è troppo verbosa per un `match`, `if let` e `let...else` sono strumenti da tenere presenti.

Ricapitolazione

Gli enum permettono di creare tipi personalizzati che possono essere uno di un insieme di valori enumerati, e il tipo `Option<T>` della libreria standard usa il sistema dei tipi per prevenire gli errori del null. Quando i valori di un enum contengono dati, `match` o `if let` li estraggono e usano, a seconda di quanti casi servono. Con struct ed enum i programmi Rust possono esprimere i concetti del proprio dominio, e i tipi personalizzati nelle API garantiscono type safety: il compilatore assicura che le funzioni ricevano solo i tipi di valori attesi.

Parte III

Modularità, collezioni e gestione degli errori

Capitolo 7

Pacchetti, crate e moduli

Man mano che i programmi crescono, organizzare il codice diventa sempre più importante: raggruppare funzionalità correlate e separare il codice con caratteristiche distinte chiarisce dove trovare il codice che implementa una certa funzionalità e dove andare per cambiarne il funzionamento. I programmi visti finora vivevano in un solo modulo in un solo file; un progetto cresciuto si organizza dividendo il codice in più moduli e più file, estraendo parti in crate separati che diventano dipendenze esterne (per progetti molto grandi, Cargo offre i *workspaces*, trattati nel Capitolo 14). Un concetto correlato è l'incapsulamento dei dettagli di implementazione: una volta implementata un'operazione, altro codice può chiamarla tramite la sua interfaccia pubblica senza sapere come funziona dentro. Il sistema dei moduli comprende quattro feature.

- **Packages**: una feature di Cargo che permette di buildare, testare e condividere i crate.
- **Crates**: un albero di moduli che produce una libreria o un eseguibile.
- **Moduli e `use`**: permettono di controllare organizzazione, scope e privacy dei percorsi.
- **Paths**: un modo di nominare un item, come una struct, una funzione o un modulo.

Pacchetti e crate

Un *crate* è la quantità minima di codice che il compilatore Rust considera alla volta: anche passando un solo file sorgente a `rustc`, il compilatore considera quel file un crate. I crate possono contenere moduli, eventualmente definiti in altri file compilati insieme al crate. Esistono due forme di crate.

- **Crate binari**: programmi compilabili in un eseguibile, come un programma a riga di comando o un server; ciascuno deve avere una funzione `main` che definisce cosa fa l'eseguibile.
- **Crate di libreria**: non hanno `main` e non compilano in un eseguibile; definiscono funzionalità pensate per essere condivise tra progetti, come il crate `rand` usato nel Capitolo 2. Quando i Rustaceani dicono "crate" intendono quasi sempre un crate di libreria.

Il *crate root* è il file sorgente da cui il compilatore parte e che costituisce il modulo radice del crate. Un *package* è un insieme di uno o più crate che fornisce un insieme di funzionalità, descritti da un file `Cargo.toml` che spiega come buildarli; Cargo stesso è un package che contiene il crate binario dello strumento a riga di comando e un crate di libreria da cui il binario dipende.

Un package rispetta due vincoli di struttura.

- Può contenere **quanti crate binari si vuole**, ma **al massimo un crate di libreria**.
- Deve contenere **almeno un crate**, che sia di libreria o binario.

Alla creazione con `cargo new my-project`, la directory contiene il `Cargo.toml` e `src/main.rs`, senza che il file menzioni `main.rs`: Cargo segue una convenzione per cui `src/main.rs` è il crate root di un crate binario con lo stesso nome del package. Analogamente, se la directory contiene `src/lib.rs`, il package contiene un crate di libreria con lo stesso nome, di cui quel file è la radice. Un package può avere più crate binari ponendo i file nella directory `src/bin`: ciascun file è un crate binario separato.

Info

Package con binario e libreria

Un package con sia `src/main.rs` che `src/lib.rs` ha due crate, un binario e una libreria, con lo stesso nome del package. La pratica consigliata è tenere nel binario il minimo codice per avviare un eseguibile che chiama la libreria: il crate binario diventa un utente della libreria, come un crate esterno qualsiasi, e può usare **solo l'API pubblica**. L'albero dei moduli va definito in `src/lib.rs`; così chi scrive è al tempo stesso autore e cliente, il che aiuta a progettare una buona API.

Moduli e controllo di scope e privacy

I *moduli* permettono di organizzare il codice dentro un crate per leggibilità e riuso, e di controllare la privacy degli item, perché il codice in un modulo è privato per default: i dettagli interni non sono disponibili all'uso esterno, e si può scegliere di rendere pubblici i moduli e i loro contenuti.

Si definisce un modulo con la keyword `mod` seguita dal nome, con il corpo tra graffe. Dentro i moduli si possono annidare

altri moduli e definire struct, enum, costanti, trait e funzioni. L'esempio di riferimento è una libreria `restaurant` che organizza le sue funzioni in un modulo `front_of_house` (dove stanno i clienti) con i sottomoduli `hosting` e `serving`, e un `back_of_house` per la cucina.

```rust
mod front_of_house {
    mod hosting {
        fn add_to_waitlist() {}
    }
}
```

I contenuti di `src/main.rs` e `src/lib.rs` formano un modulo implicito chiamato `crate` alla radice della struttura di moduli del crate, detta *albero dei moduli*. Alcuni moduli si annidano dentro altri (`hosting` dentro `front_of_house`); moduli definiti nello stesso modulo sono *fratelli* (*siblings*). Se il modulo A è contenuto in B, A è *figlio* di B e B è *padre* di A. L'albero dei moduli ricorda l'albero delle directory di un filesystem: come le directory organizzano i file, i moduli organizzano il codice.

Riepilogo rapido dei moduli

Prima dei dettagli, un promemoria di come funzionano moduli, percorsi, `use` e `pub` nel compilatore, e di come la maggior parte degli sviluppatori organizza il codice.

- **Partire dalla radice**: compilando un crate, il compilatore cerca prima il codice nel file radice (`src/lib.rs` per una libreria, `src/main.rs` per un binario).
- **Dichiarare moduli**: nella radice si dichiara un nuovo modulo con `mod garden;`; il compilatore cerca il codice del modulo inline tra graffe al posto del punto e virgola, nel file `src/garden.rs` oppure in `src/garden/mod.rs`.
- **Dichiarare sottomoduli**: in un file diverso dalla radice si possono dichiarare sottomoduli, come `mod vegetables;` in `src/garden.rs`; il compilatore cerca il codice nella directory del modulo padre, tra graffe inline, in `src/garden/vegetables.rs` oppure in `src/garden/vegetables/mod.rs`.
- **Percorsi verso il codice**: una volta che un modulo è parte del crate, ci si può riferire al suo codice da qualunque punto dello stesso crate, se le regole di privacy lo permettono, usando il percorso, come `crate::garden::vegetables::Asparagus`.
- **Privato vs pubblico**: il codice in un modulo è privato rispetto ai moduli padri per default; si rende pubblico un modulo con `pub mod` e pubblici i suoi item con `pub` prima delle dichiarazioni.
- **La keyword `use`**: in uno scope, `use` crea scorciatoie verso gli item per ridurre la ripetizione dei percorsi lunghi: dopo `use crate::garden::vegetables::Asparagus;` basta scrivere `Asparagus` in quello scope.

Percorsi per riferire un item nell'albero dei moduli

Per dire a Rust dove trovare un item nell'albero dei moduli si usa un percorso, come nel filesystem. Un percorso può prendere due forme, in entrambi i casi seguite da uno o più identificatori separati da doppio due punti (`::`).

- **Percorso assoluto**: parte dalla radice del crate; per codice del crate corrente inizia con il letterale `crate`, per codice di un crate esterno inizia col nome del crate.
- **Percorso relativo**: parte dal modulo corrente e usa `self`, `super` o un identificatore del modulo corrente.

Per chiamare `add_to_waitlist` da `eat_at_restaurant` definita nella radice, il percorso assoluto è `crate::front_of_house::hosting::add_to_waitlist`, come `/front_of_house/hosting/add_to_waitlist` nella shell; il percorso relativo è `front_of_house::hosting::add_to_waitlist`, che parte dal nome del modulo allo stesso livello della funzione chiamante.

La scelta tra percorso assoluto e relativo dipende da come ci si aspetta di muovere il codice.

- Se il modulo e la funzione chiamante vengono spostati **insieme**, il percorso relativo continua a funzionare mentre quello assoluto va aggiornato.
- Se la funzione chiamante viene spostata **separatamente**, il percorso assoluto resta valido mentre quello relativo va aggiornato.
- In generale conviene preferire i percorsi **assoluti**, perché è più probabile che definizioni e chiamate si muovano indipendentemente.

Privacy di default e la keyword `pub`

I percorsi corretti di per sé non bastano: gli errori del compilatore dicono che il modulo `hosting` è privato. In Rust **tutti gli item** (funzioni, metodi, struct, enum, moduli e costanti) sono privati rispetto ai moduli padri per default;

per rendere privato un item basta metterlo in un modulo. Le regole di visibilità asimmetriche sono come il back office di un ristorante.

- Gli item in un modulo **padre non possono usare** gli item privati dei moduli figli.
- Gli item nei moduli **figli possono usare** gli item dei loro moduli antenati, perché i figli possono vedere il contesto in cui sono definiti.

Rust sceglie la privacy come default perché nascondere i dettagli interni sia la norma: così si sa quali parti del codice interno si possono cambiare senza rompere il codice esterno. La keyword `pub` espone al contrario un item agli antenati.

Rendere pubblico un modulo non basta, perché `pub` su un modulo lascia i suoi **contenuti** comunque privati: `pub` su `mod hosting` permette ai moduli antenati di riferirsi a `hosting`, non di accedere al suo codice interno; occorre marcare con `pub` anche la funzione `add_to_waitlist` perché la chiamata compili. Con entrambi i `pub`, sia il percorso assoluto sia quello relativo diventano validi, verificando a ogni passo che il modulo padre sia accessibile e l'item marcato `pub`.

Se si intende condividere la libreria, l'API pubblica è il contratto con gli utenti del crate su come interagire col codice.

Percorsi relativi con `super`

Un percorso relativo può partire dal modulo padre invece che dal modulo corrente o dalla radice, usando `super` all'inizio: è come usare `..` nei percorsi del filesystem. Permette di riferire un item che si sa stare nel modulo padre ed è utile quando il modulo è strettamente legato al padre ma questo potrebbe un giorno spostarsi altrove nell'albero.

Nell'esempio, la funzione `fix_incorrect_order` nel modulo `back_of_house` chiama `deliver_order` definita nel modulo padre con `super::deliver_order`: se il codice venisse riorganizzato, i due resterebbero insieme e ci sarebbero meno punti da aggiornare.

Struct ed enum pubblici

`pub` si applica anche a struct ed enum, ma con dettagli diversi.

- **Struct**: `pub` rende pubblica la struct, ma i suoi **campi restano privati**; ogni campo si rende pubblico o meno caso per caso. Una struct con un campo privato ha bisogno di una funzione associata **pubblica** che ne costruisca un'istanza, come `Breakfast::summer`, perché altrimenti chi usa non potrebbe impostare il campo privato.
- **Enum**: rendere pubblico un enum rende pubbliche **tutte le sue varianti**, basta `pub` prima di `enum`.

La differenza di default nasce dall'utilità: un enum i cui tipi non sono pubblici servirebbe a poco, mentre una struct è spesso utile anche senza campi pubblici, quindi i suoi campi seguono la regola generale del privato per default.

Portare i percorsi nello scope con `use`

Scrivere i percorsi per intero a ogni chiamata è scomodo e ripetitivo. La keyword `use` crea una scorciatoia verso un percorso una volta sola, e da lì in poi nello scope si usa il nome corto. Aggiungere `use` e un percorso in uno scope è come creare un collegamento simbolico nel filesystem: dopo `use crate::front_of_house::hosting;` nella radice, `hosting` è un nome valido in quello scope, e la chiamata diventa `hosting::add_to_waitlist()`.

Le scorciatoie con `use` rispettano comunque le regole di privacy e valgono **solo nello scope in cui la `use` appare**: spostando `eat_at_restaurant` in un sottomodulo `customer`, la scorciatoia non si applica più nel corpo della funzione e il compilatore lo segnale, insieme all'avviso di `use` non più usata. La correzione è portare la `use` dentro il modulo o riferire la scorciatoia con `super::hosting`.

Percorsi `use` idiomatici

Con le funzioni è idiomatico portare nello scope il **modulo padre** e specificarlo alla chiamata, non la funzione fino in fondo.

```rust
use crate::front_of_house::hosting;
hosting::add_to_waitlist();
```

Così il codice chiarisce che la funzione non è definita localmente, riducendo comunque la ripetizione del percorso completo; con una `use` che arriva direttamente alla funzione non si capirebbe dove è definita. Per contro, con struct, enum e altri item è idiomatico specificare il **percorso completo**, come `use std::collections::HashMap;`. Non c'è una ragione tecnica: è la convenzione a cui la comunità si è abituata.

L'eccezione all'idioma sono due item con lo **stesso nome**: Rust non permette di portare nello scope due omonimi, come `std::fmt::Result` e `std::io::Result`. In quel caso si porta ogni percorso fino al modulo padre e ci si riferisce ai tipi come `fmt::Result` e `io::Result`, oppure si rinomina con la keyword `as`, che crea un **alias** locale.

```rust
use std::fmt::Result;
use std::io::Result as IoResult;
```

Re-export con `pub use`

Un nome portato con `use` è privato dello scope che lo importa. Combinando `pub` e `use` si rende quel nome disponibile anche a codice esterno, come se fosse definito lì: la tecnica è detta *re-exporting*. Per esempio, con `pub use crate::front_of_house::hosting;` nella radice, il codice esterno può chiamare `restaurant::hosting::add_to_waitlist()` invece del percorso interno completo `restaurant::front_of_house::hosting::add_to_waitlist()`.

Il re-export è utile quando la struttura interna del codice differisce da come i chiamanti pensano al dominio: chi gestisce il ristorante pensa in termini di "front of house" e "back of house", ma i clienti no. Con `pub use` si scrive il codice con una struttura e se ne espone un'altra, organizzata bene sia per chi lavora alla libreria sia per chi la chiama.

Pacchetti esterni

Per usare un pacchetto esterno come `rand` si aggiunge la dipendenza nel `Cargo.toml`, così Cargo scarica da crates.io il pacchetto e le sue dipendenze e lo rende disponibile al progetto; poi si porta nello scope con una `use` che parte dal nome del crate, come `use rand::Rng;`. La libreria standard `std` è anch'essa un **crate esterno** al package: non serve dichiararla in `Cargo.toml` perché è distribuita col linguaggio, ma serve una `use` per portarne gli item nello scope, come `use std::collections::HashMap;`, un percorso assoluto che inizia col nome del crate.

Percorsi annidati e operatore glob

Se si usano più item definiti nello stesso crate o modulo, elencare ogni `use` su una riga propria occupa molto spazio. Due forme compatte aiutano a ridurre le righe.

- **Percorsi annidati**: si specifica la parte comune del percorso, poi doppio due punti e graffe con l'elenco delle parti diverse, come `use std::{cmp::Ordering, io};`, e le graffe si possono usare a qualunque livello del percorso; quando un percorso è sotto-percorso dell'altro, `self` unisce le due `use`, come `use std::io::{self, Write};` che porta nello scope `std::io` e `std::io::Write`.
- **Operatore glob**: `use std::collections::*;` porta nello scope tutti gli item pubblici di quel percorso, da usare con cautela perché rende più difficile capire quali nomi sono nello scope e dove sono definiti, e perché un aggiornamento della dipendenza può introdurre conflitti di nome. L'uso tipico è nei test, per portare tutto ciò che si testa nel modulo `tests` (Capitolo 11), e nel pattern del *prelude*.

Separare i moduli in file diversi

Quando i moduli crescono, si possono spostare le definizioni in file separati per navigare il codice più facilmente, e l'albero dei moduli resta invariato. Per estrarre `front_of_house` si lascia nella radice solo la dichiarazione `mod front_of_house;` e si mette il corpo in `src/front_of_house.rs`: il compilatore sa cercare quel file perché ha incontrato la dichiarazione con quel nome nella radice. Un sottomodulo come `hosting` va invece nel file `src/front_of_house/hosting.rs`, dentro la directory che prende il nome dai suoi antenati nell'albero; se lo si mettesse in `src/hosting.rs`, il compilatore si aspetterebbe un modulo dichiarato nella radice.

Warning

mod non è un include

Un file va caricato con una dichiarazione `mod` **una sola volta** nell'albero dei moduli. Una volta che il compilatore sa che il file fa parte del progetto e dove sta nell'albero, gli altri file si riferiscono al suo codice con un percorso alla dichiarazione. `mod` non è l'operazione di "include" di altri linguaggi: `use` non ha alcun effetto su quali file vengono compilati.

Rust supporta anche uno stile di percorso più vecchio, ancora supportato.

- **Stile moderno**: `src/front_of_house.rs` per un modulo e `src/front_of_house/hosting.rs` per un sottomodulo.
- **Stile vecchio**: `src/front_of_house/mod.rs` e `src/front_of_house/hosting/mod.rs`; usare entrambi gli stili per lo **stesso** modulo è un errore di compilazione, mentre mescolarli per moduli diversi è permesso ma confonde chi

naviga il progetto. Lo svantaggio principale dello stile `mod.rs` è il gran numero di file omonimi aperti insieme nell'editor.

Spostato ogni modulo nel suo file, le chiamate in `eat_at_restaurant` funzionano senza modifiche e la `pub use` nella radice resta invariata: la tecnica permette di spostare i moduli in nuovi file quando crescono.

Ricapitolazione

Rust permette di dividere un package in più crate e un crate in più moduli, così da riferire gli item definiti in un modulo da un altro modulo. I riferimenti si esprimono con percorsi assoluti o relativi, che si possono portare nello scope con `use` per usarne una versione più corta. Il codice dei moduli è privato per default, e si rende pubblico con la keyword `pub`, che per i re-export si combina con `use` stessa. Nel prossimo capitolo si vedranno le strutture dati di collezione della libreria standard da usare nel codice così organizzato.

Capitolo 8

Collezioni comuni

La libreria standard di Rust include diverse strutture dati molto utili chiamate **collezioni**: mentre la maggior parte degli altri tipi rappresenta un solo valore specifico, le collezioni possono contenerne più di uno. A differenza di array e tuple incorporati, i dati puntati da queste collezioni sono memorizzati nell'heap, quindi la quantità di dati non deve essere nota a compile-time e può crescere o ridursi durante l'esecuzione. Ogni tipo di collezione ha capacità e costi diversi, e sceglierne uno appropriato è una competenza che si sviluppa col tempo. In questa lezione si vedono le tre collezioni più usate nei programmi Rust.

- Il **vettore** permette di memorizzare un numero variabile di valori l'uno accanto all'altro.
- La **stringa** è una collezione di caratteri; il tipo `String` è già stato menzionato, ma qui se ne parla a fondo.
- La **hash map** permette di associare un valore a una chiave specifica; è una particolare implementazione della struttura dati più generale chiamata *mappa*.

Vettori

Il primo tipo di collezione è `Vec<T>`, detto vettore: memorizza più valori in un'unica struttura dati che li mette tutti adiacenti in memoria, e può contenere solo valori dello **stesso tipo**. È utile quando si ha una lista di item, come le righe di testo di un file o i prezzi del carrello della spesa.

Creare un nuovo vettore

Per creare un vettore vuoto si chiama `Vec::new` e, poiché non si inseriscono valori, Rust non sa quali elementi si intendono memorizzare: serve l'annotazione di tipo, come in `let v: Vec<i32> = Vec::new();`. I vettori sono implementati con i generici (Capitolo 10): il tipo `Vec<T>` della libreria standard può contenere qualunque tipo, specificato tra parentesi angolari.

Più spesso si crea un vettore con valori iniziali, così Rust può inferire il tipo e l'annotazione non serve. La macro `vec!` crea un nuovo vettore con i valori forniti; il tipo intero risultante è `i32` perché è il tipo intero di default.

```rust
let v = vec![1, 2, 3];
```

Aggiornare un vettore

Per aggiungere elementi a un vettore si usa il metodo `push`: come per qualunque variabile, per cambiarne il valore serve la keyword `mut`. I numeri inseriti sono tutti `i32` e Rust lo inferisce dai dati, quindi l'annotazione `Vec<i32>` non è necessaria.

Leggere gli elementi di un vettore

Esistono due modi per riferire un valore memorizzato in un vettore, e la scelta riguarda il comportamento del programma con un indice fuori intervallo.

- **Sintassi di indicizzazione**: `&v[2]` con `&` e `[]` dà un riferimento all'elemento a quell'indice; i vettori sono indicizzati per numero partendo da zero. Se l'indice non esiste, ad esempio 100 in un vettore di cinque elementi, il programma va in **panic**: si usa quando si vuole che il programma crashi al tentativo di accedere oltre la fine.
- **Metodo `get`**: `v.get(2)` passa l'indice come argomento e ritorna un `Option<&T>` da usare col `match`; con un indice fuori intervallo ritorna `None` **senza panic**. Si usa quando l'accesso oltre il vettore può capitare in circostanze normali, ad esempio con un indice inserito da una persona: ricevuto `None`, il programma può comunicare quanti item contiene il vettore e chiedere un valore valido, più amichevole di un crash per un errore di battitura.

Quando il programma ha un riferimento valido, il borrow checker fa rispettare le regole di ownership e borrowing del Capitolo 4: tenere un riferimento immutabile al primo elemento e chiamare `push` sul vettore non compila, perché aggiungere un elemento può richiedere una nuova allocazione e la copia dei vecchi elementi, il che invaliderebbe il riferimento. Le regole di borrowing prevengono esattamente quella situazione.

Iterare sui valori di un vettore

Per accedere a ogni elemento a turno si itera sugli elementi invece di usare gli indici. Un ciclo `for` su `&v` dà riferimenti immutabili a ciascun `i32`; per leggere il numero riferito da `i` serve l'operatore di dereferenziazione `*`. Analogamente si può iterare su riferimenti mutabili di un vettore mutabile per modificare tutti gli elementi: con `for i in &mut v { *i += 50; }` l'operatore `*` raggiunge il valore prima dell'assegnazione.

Usare gli iteratori in sicurezza

Un dettaglio importante è che gli iteratori contengono un puntatore a dati dentro il vettore: desugarando il ciclo `for`, `v.iter()` produce un iteratore e `next` lo avanza restituendo un riferimento opzionale all'elemento, `Some` oppure `None` a fine vettore. Questo ha conseguenze sulla sicurezza: una funzione che duplica il vettore sul posto iterando con `v.iter()` e chiamando `v.push(*n_ref)` non compila, perché `v.iter()` rimuove il permesso di scrittura che il `push` richiede, con l'errore `E0502`.

Warning

Niente aggiunte o rimozioni durante l'iterazione

Il problema di sicurezza sotto l'errore è la lettura di memoria deallocata: appena il `push` rialloca i contenuti del vettore, invalida il puntatore dell'iteratore. Per usare gli iteratori in sicurezza, Rust non permette di aggiungere o rimuovere elementi dal vettore durante l'iterazione. Un'alternativa senza puntatore è iterare su un **range** di indici, come `0..v.len()`, e accedere con `&v[i]`.

Usare un enum per memorizzare più tipi

I vettori possono contenere solo valori dello stesso tipo, ma questo può essere scomodo. Per memorizzare elementi di tipi diversi si può definire un **enum** le cui varianti tengono i diversi tipi di valore: tutte le varianti sono considerate dello stesso tipo, quello dell'enum, quindi il vettore dell'enum contiene di fatto tipi diversi, come una riga di un foglio di calcolo con interi, numeri in virgola mobile e stringhe.

Rust deve sapere quali tipi saranno nel vettore a compile-time per conoscere esattamente quanta memoria heap serve per ogni elemento; usare un enum più un'espressione `match` garantisce a compile-time che ogni caso possibile sia gestito. Se l'insieme esaustivo dei tipi non è noto a runtime, la tecnica dell'enum non funziona e si usano i trait object del Capitolo 18.

Droppare un vettore droppa i suoi elementi

Come qualunque altra `struct`, un vettore viene liberato quando esce dallo scope, e quando viene droppato anche tutti i suoi contenuti vengono droppati. Il borrow checker assicura che i riferimenti ai contenuti di un vettore siano usati solo mentre il vettore stesso è valido.

Stringhe

I nuovi Rustaceani si bloccano spesso sulle stringhe per tre ragioni combinate: la propensione di Rust a esporre i possibili errori, il fatto che le stringhe sono una struttura dati più complicata di quanto molti programmatori pensino, e UTF-8. Le stringhe si trattano come collezioni perché sono implementate come collezione di byte più alcuni metodi che danno funzionalità utili quando quei byte sono interpretati come testo.

Definire le stringhe

Rust ha un solo tipo stringa nel linguaggio base: la string slice `str`, di solito vista nella forma presa in prestito `&str`, un riferimento a dati UTF-8 memorizzati altrove. I letterali stringa, ad esempio, sono memorizzati nel binario del programma e sono quindi string slice.

Il tipo `String`, fornito dalla libreria standard e non dal linguaggio base, è una stringa **crescibile, mutabile, posseduta e codificata in UTF-8**. Quando i Rustaceani parlano di "stringhe" possono riferirsi sia a `String` sia a `&str`, e entrambi i tipi sono pesantemente usati nella libreria standard ed entrambi sono UTF-8.

Creare una nuova stringa

Molte operazioni di `Vec<T>` sono disponibili anche per `String`, perché `String` è implementata come un wrapper attorno a un vettore di byte con garanzie, restrizioni e capacità aggiuntive. La funzione `new` crea una stringa vuota; con dati iniziali si usano due API equivalenti.

- **`to_string`**: metodo disponibile su ogni tipo che implementa il trait `Display`, come i letterali stringa; `"initial contents".to_string()` crea una `String` con quel contenuto.
- **`String::from`**: crea una `String` da un letterale; sceglierla o `to_string` è questione di stile e leggibilità, entrambe hanno il loro posto tra le tante API generiche per le stringhe.

Le stringhe sono UTF-8, quindi possono contenere dati correttamente codificati in qualunque lingua, inclusi saluti in spagnolo, russo, giapponese o cinese: sono tutti valori `String` validi.

Aggiornare una stringa

Una `String` può crescere e i suoi contenuti possono cambiare come quelli di un `Vec<T>`. In più si possono concatenare valori con l'operatore `+` o la macro `format!`.

Per far crescere una `String` si usano due metodi di append.

- **`push_str`**: aggiunge una string slice; prende una slice perché non serve necessariamente prendere il possesso del parametro, così la variabile prestata resta usabile dopo l'append, ad esempio per stamparla.
- **`push`**: prende un singolo carattere come parametro e lo aggiunge alla stringa.

Per combinare due stringhe esistenti si usa spesso l'operatore `+`, ma il suo comportamento dipende dalla firma del metodo `add` che vi sta sotto.

```rust
fn add(self, s: &str) -> String {
```

La firma spiega due comportamenti sorprendenti di `let s3 = s1 + &s2;`.

- **`s2` passa per riferimento**: `add` può solo aggiungere una string slice a una `String`, non due `String`; il tipo di `&s2` è `&String`, ma il compilatore lo **coerce** in `&str` con una deref coercion (`&s2[..]`, si veda il Capitolo 15). Poiché `add` non prende il possesso del parametro, `s2` resta valida dopo l'operazione.
- **`s1` viene mosso**: `self` non ha `&`, quindi `add` prende possesso di `s1`, gli aggiunge una copia dei contenuti di `s2` e ne restituisce il possesso; `s1` non è più valida dopo la somma. Se `s1` ha capacità sufficiente non avvengono allocazioni, altrimenti ne fa una più grande per contenere entrambe le stringhe.

Con più di due stringhe la concatenazione con `+` diventa malagevole per tutti i `+` e i riferimenti. Per combinazioni più complesse si usa la macro `format!`.

```rust
let s = format!("{s1}-{s2}-{s3}");
```

`format!` funziona come `println!` ma invece di stampare ritorna una `String` col contenuto; il codice è molto più leggibile e usa riferimenti, quindi la chiamata non prende il possesso di alcun parametro.

L'indicizzazione delle stringhe

In molti linguaggi accedere ai singoli caratteri di una stringa tramite indice è un'operazione valida e comune, ma in Rust usare la sintassi di indicizzazione su una `String` è un errore di compilazione: le stringhe Rust non supportano l'indicizzazione. La ragione sta in come Rust memorizza le stringhe.

Una `String` è un wrapper su un `Vec<u8>`: la lunghezza di `"Hola"` è 4 byte, uno per lettera, ma la stringa russa "Здравствуйте" ha lunghezza **24**, perché ogni valore scalare Unicode che contiene occupa 2 byte in UTF-8. Un indice sui byte quindi non corrisponde sempre a un valore scalare Unicode valido: il primo byte di "З" è `208`, ma `208` non è un carattere valido da solo, e restituirlo non è ciò che un utente si aspetta dal primo carattere; succede anche con le lettere latine, dato che `&"hi"[0]` restituirebbe `104`, non `h`. Per evitare valori inattesi e bug scoperti tardi, Rust non compila affatto quel codice e previene i fraintendimenti presto nello sviluppo.

Byte, valori scalari e cluster di grafemi

Con UTF-8 esistono tre modi rilevanti di guardare le stringhe dalla prospettiva di Rust.

- **Byte**: come il computer memorizza i dati; la parola hindi "नमस्ते" in alfabeto devanagari è un vettore di 18 byte.
- **Valori scalari**: i `char` di Rust; quei byte sono sei valori scalari, ma il quarto e il sesto non sono lettere, sono segni diacritici senza senso da soli.
- **Cluster di grafemi**: la cosa più vicina a ciò che una persona chiamerebbe *lettere*; la parola è fatta di quattro lettere percepite dall'umano.

Rust offre modi diversi di interpretare i dati grezzi perché ogni programma scelga l'interpretazione che serve, in qualunque lingua umana sia il dato. Un'ultima ragione del divieto di indicizzazione è prestazionale: le operazioni di indicizzazione sono attese a tempo costante O(1), ma con una `String` non è garantibile, perché Rust dovrebbe scorrere i contenuti dall'inizio fino all'indice per determinare quanti caratteri validi ci sono.

Slicing di stringhe

Indicizzare una stringa è spesso una cattiva idea perché non è chiaro cosa dovrebbe ritornare: un byte, un carattere, un cluster o una slice. Se servono davvero gli indici, Rust chiede di essere più specifici: invece di `[]` con un numero singolo si usa `[]` con un **range** per creare una string slice di byte particolari, come `&hello[0..4]` che per la stringa russa contiene `Зд`, i primi 4 byte. Tagliare i byte a metà di un carattere, come `&hello[0..1]`, causa un panic a runtime come un indice invalido in un vettore, quindi le slice con range vanno create con cautela.

Iterare sulle stringhe

Il modo migliore di operare su pezzi di stringa è dichiarare esplicitamente se servono caratteri o byte. Due metodi lo permettono.

- **`chars`**: separa e ritorna i valori scalari Unicode, quindi `"Зд".chars()` produce due `char` iterabili.
- **`bytes`**: ritorna ogni byte grezzo, adatto ad alcuni domini; bisogna ricordare però che un valore scalare Unicode valido può essere composto da più di un byte.

I cluster di grafemi, necessari per alfabeti complessi come il devanagari, non sono forniti dalla libreria standard: l'operazione è complessa e esistono crate su crates.io che la forniscono.

Gestire la complessità delle stringhe

Le stringhe sono complicate, e i linguaggi presentano questa complessità al programmatore in modi diversi. Rust sceglie come comportamento di default la gestione **corretta** dei dati `String`, chiedendo più attenzione ai dati UTF-8 in anticipo: questo espone più complessità rispetto ad altri linguaggi, ma previene errori con caratteri non-ASCII più avanti nel ciclo di sviluppo. La libreria standard offre molte funzionalità costruite su `String` e `&str` per gestire correttamente queste situazioni, come `contains` per cercare in una stringa e `replace` per sostituire parti di essa.

Hash map

L'ultima collezione comune è la hash map: il tipo `HashMap<K, V>` memorizza una mappatura di chiavi di tipo `K` a valori di tipo `V` usando una **funzione di hash**, che determina come collocare chiavi e valori in memoria. Molti linguaggi supportano questa struttura con nomi diversi: *hash*, *map*, *object*, *hash table*, *dizionario* o *array associativo*.

Le hash map sono utili quando si vogliono cercare dati non tramite indice come nei vettori, ma tramite una chiave di qualunque tipo: in un gioco, i punteggi di ogni squadra si tengono in una hash map dove ogni chiave è il nome della squadra e i valori sono i punteggi.

Creare una nuova hash map

Un modo di crearne una vuota è `HashMap::new` e aggiungere elementi con `insert`, avendo prima portato `HashMap` nello scope con `use std::collections::HashMap`: delle tre collezioni comuni è la meno usata, quindi **non è inclusa nella prelude**, e ha anche meno supporto dalla libreria standard, senza una macro per costruirla. Come i vettori, memorizza i dati nell'heap ed è **omogenea**: tutte le chiavi hanno lo stesso tipo e tutti i valori hanno lo stesso tipo.

```rust
use std::collections::HashMap;
let mut scores = HashMap::new();
scores.insert(String::from("Blue"), 10);
```

Accedere ai valori in una hash map

Per leggere un valore si passa la chiave al metodo `get`, che ritorna un `Option<&V>`: se non c'è un valore per quella chiave ritorna `None`. Il risultato tipico si gestisce chiamando `copied` per ottenere un `Option<i32>` invece di `Option<&i32>` e `unwrap_or` per impostare un default, come zero, se la chiave non è presente.

L'iterazione su ogni coppia chiave-valore avviene con un ciclo `for` come per i vettori, ma l'ordine di stampa è **arbitrario**: non c'è garanzia sull'ordine di attraversamento.

Ownership nelle hash map

Per i tipi che implementano il trait `Copy`, come `i32`, i valori sono **copiati** nella hash map. Per i valori posseduti come `String`, i valori vengono **mossi** e la hash map ne diventa il proprietario: dopo `insert(field_name, field_value)` le due variabili non sono più usabili. Inserendo riferimenti, i valori non vengono mossi, ma i valori puntati devono essere validi almeno finché è valida la hash map: sono questioni di lifetime del Capitolo 10.

Aggiornare una hash map

Il numero di coppie può crescere, ma ogni chiave unica può avere **un solo valore** alla volta (non viceversa: più chiavi possono condividere lo stesso valore). Per cambiare i dati occorre decidere come gestire il caso in cui la chiave ha già un valore, e le strategie sono tre.

- **Sovrascrivere il valore**: un nuovo `insert` con la stessa chiave sostituisce il vecchio valore, che viene scartato; due `insert` consecutivi per la chiave "Blue" lasciano una sola coppia, con l'ultimo valore.
- **Aggiungere solo se la chiave non è presente**: è comune controllare se una chiave esiste già e agire di conseguenza, e l'API dedicata è `entry`. Prende la chiave da controllare e ritorna un enum `Entry` che rappresenta un valore che potrebbe o meno esistere: il metodo `or_insert` su `Entry` ritorna un riferimento mutabile al valore se la chiave esiste, altrimenti inserisce il parametro come nuovo valore e ne ritorna un riferimento mutabile. È molto più pulito che scrivere la logica a mano e collabora meglio col borrow checker.
- **Aggiornare in base al valore vecchio**: un caso d'uso comune è leggere il valore di una chiave e aggiornarlo in base al vecchio, come contare quante volte appare ogni parola in un testo. Con le parole come chiavi, `text.split_whitespace()` itera sulle sotto-slice separate da spazi, `entry(word).or_insert(0)` inserisce `0` alla prima occorrenza e ritorna il riferimento mutabile al contatore, che si incrementa dereferenziando con `*`.

```rust
for word in text.split_whitespace() {
    let count = counts.entry(word).or_insert(0);
    *count += 1;
}
```

Il riferimento mutabile esce dallo scope alla fine del ciclo `for`, quindi tutte le modifiche sono sicure e permesse dalle regole di borrowing.

Funzioni di hash

Per default `HashMap` usa la funzione di hash **SipHash**, che fornisce resistenza agli attacchi di denial-of-service che sfruttano le tabelle hash. Non è l'algoritmo più veloce disponibile, ma il calo di prestazioni è compensato dalla sicurezza. Profilando il codice, se la funzione di default risulta troppo lenta si può passare a un'altra specificando un *hasher*, un tipo che implementa il trait `BuildHasher` (Capitolo 10): non serve implementarlo da zero, su crates.io esistono librerie con hasher per molti algoritmi comuni.

Ricapitolazione

Vettori, stringhe e hash map forniscono gran parte della funzionalità necessaria per memorizzare, accedere e modificare dati nei programmi. I programmi si facciano più complessi e le operazioni possono fallire, quindi il prossimo capitolo è il momento perfetto per discutere la gestione degli errori.

Capitolo 9

Gestione degli errori

Gli errori sono un fatto della vita nel software, quindi Rust ha diverse feature per gestire le situazioni in cui qualcosa va storto. In molti casi Rust richiede di riconoscere la possibilità di un errore e di agire di conseguenza prima che il codice compili, e questo rende il programma più robusto assicurando che gli errori siano scoperti e gestiti prima di rilasciare il codice in produzione.

Rust divide gli errori in due categorie principali: **riparabili** e **non riparabili**. Per un errore riparabile, come un file non trovato, molto probabilmente si vuole solo riportare il problema all'utente e riprovare l'operazione; gli errori non riparabili sono sempre sintomi di bug, come accedere a una posizione oltre la fine di un array, e impongono di fermare subito il programma. Molti linguaggi non distinguono le due categorie e usano lo stesso meccanismo, come le eccezioni: Rust **non ha eccezioni**; ha invece il tipo `Result<T, E>` per gli errori riparabili e la macro `panic!`, che ferma l'esecuzione, per quelli non riparabili.

Errori non riparabili con `panic!`

A volte accadono cose brutte nel codice su cui non si può far nulla, e in questi casi esiste la macro `panic!`. In pratica un panic si causa in due modi.

- **Con un'azione che lo provoca**, come accedere a un array oltre la sua fine.
- **Chiamando esplicitamente `panic!`**, ad esempio `panic!("crash and burn");`.

Per default un panic stampa un messaggio di fallimento, fa lo *unwinding*, ripulisce lo stack ed esce. Il messaggio mostra il testo del panic e la posizione nel codice sorgente, come `src/main.rs:2:5`; la riga indicata può anche trovarsi nel codice di una libreria chiamata dal nostro, non nella nostra riga che ha innescato la catena.

Unwinding o abort

Quando avviene un panic, per default il programma inizia l'*unwinding*: Rust risale lo stack e ripulisce i dati di ogni funzione incontrata. Risalire e pulire è tanto lavoro, quindi Rust permette l'alternativa dell'*aborting* immediato, che termina il programma senza pulire: la memoria usata verrà poi ripulita dal sistema operativo. Se serve un binario il più piccolo possibile, si passa da unwinding ad abort aggiungendo `panic = 'abort'` alla sezione `[profile]` appropriata del `Cargo.toml`, ad esempio `[profile.release]`.

Il backtrace di un panic

L'accesso a un indice oltre la fine di un vettore causa un panic: con `[]` Rust dovrebbe restituire un elemento, ma con un indice invalido non c'è nessun elemento corretto da restituire. In C leggere oltre la fine di una struttura dati è comportamento indefinito e può restituire qualunque cosa sia in quella posizione di memoria: è la *buffer overread*, che porta a vulnerabilità di sicurezza se un attaccante manipola l'indice per leggere dati che non gli appartengono. Per proteggere il programma, Rust si rifiuta di continuare.

La riga `note:` dell'errore suggerisce di impostare la variabile d'ambiente `RUST_BACKTRACE` per ottenere il **backtrace**, la lista di tutte le funzioni chiamate fino a quel punto. Per leggerlo si parte dall'alto e si scende fino ai file scritti da noi: quello è il punto dove ha origine il problema; le righe sopra sono codice chiamato dal nostro codice, quelle sotto codice che chiama il nostro. I backtrace funzionano come negli altri linguaggi e richiedono i simboli di debug, attivi di default con `cargo build` e `cargo run` senza `--release`.

Errori riparabili con `Result`

Molti errori non sono abbastanza seri da fermare del tutto il programma: se aprire un file fallisce perché non esiste, si può volerlo creare invece di terminare il processo. L'enum `Result` ha due varianti, con due parametri di tipo generico.

```rust
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

`T` è il tipo del valore restituito nel caso di successo dentro `Ok`, ed `E` il tipo dell'errore restituito nel caso di fallimento dentro `Err`; grazie a questi parametri generici, `Result` si usa in molte situazioni diverse. L'esempio classico è `File::open`, che ritorna `Result<std::fs::File, std::io::Error>`: la chiamata può riuscire restituendo un file handle leggibile e scrivibile, oppure fallire perché il file non esiste o manca il permesso di accesso. In caso di successo la variabile contiene un'istanza di `Ok` col file handle, in caso di fallimento un'istanza di `Err` con informazioni sul tipo di errore.

Il modo base per gestire il risultato è il `match` del Capitolo 6: `Ok` restituisce il valore interno da assegnare a una variabile, `Err` in questo esempio chiama `panic!` col messaggio d'errore. `Result` e le sue varianti sono portate nello scope dalla prelude, come `Option`.

Match su errori diversi

Fare `panic!` qualunque sia la ragione del fallimento è grossolano: se `File::open` fallisce perché il file non esiste, si vuole creare il file; per altre ragioni, come un permesso mancante, si vuole comunque il panic. Dentro `Err`, il valore è di tipo `io::Error`, una struct della libreria standard il cui metodo `kind` ritorna un valore `io::ErrorKind`, un enum le cui varianti rappresentano i tipi di errore possibili delle operazioni di I/O.

Il pattern è un `match` interno su `error.kind()`: se la variante è `ErrorKind::NotFound` si prova a creare il file con `File::create`, che a sua volta può fallire e richiede una seconda arma del match interno; le altre armi gestiscono gli altri tipi di errore.

Info

Alternative al match con Result

Il `match` è utile ma è un costrutto primitivo: gestire errori richiede spesso molti `match` annidati. Nel Capitolo 13 si vedranno le closure, usate da molti metodi definiti su `Result<T, E>` più concisi del `match`, come `unwrap_or_else`; lo stesso esempio di match su `ErrorKind` si scrive con `File::open("hello.txt").unwrap_or_else(|error| ..)` senza alcuna espressione `match`, e molti altri metodi ripuliscono i `match` annidati nella gestione degli errori.

Scorciatoie per il panic in caso di errore

Il `match` funziona ma è verboso e non sempre comunica l'intento. `Result<T, E>` ha molti metodi helper per compiti specifici.

- **`unwrap`**: se il valore è `Ok`, restituisce il valore contenuto; se è `Err`, chiama `panic!` per noi, con un messaggio di errore di default che riporta il valore dell'errore.
- **`expect`**: funziona come `unwrap`, ma il messaggio usato nella chiamata a `panic!` è il parametro che gli si passa, invece del messaggio generico. Scegliere `expect` con un buon messaggio comunica l'intento e rende più facile risalire all'origine del panic.

Nel codice di qualità di produzione la maggior parte dei Rustaceani sceglie `expect` invece di `unwrap`, con un contesto che spiega perché l'operazione è attesa riuscire sempre: se l'assunzione si rivela falsa, resta più informazione per il debugging.

Propagare gli errori

Quando l'implementazione di una funzione chiama qualcosa che può fallire, invece di gestire l'errore dentro la funzione stessa si può **restituirlo al codice chiamante**, che decide cosa fare: questo dà più controllo al chiamante, dove ci può essere più informazione o logica per gestire l'errore di quanta ne sia disponibile nel contesto della funzione.

La funzione di riferimento `read_username_from_file` legge un username da un file e ritorna `Result<String, io::Error>`: in caso di successo il chiamante riceve `Ok` con la `String` letta, in caso di problema `Err` con un'istanza di `io::Error`. Si sceglie quel tipo d'errore perché è il tipo restituito da entrambe le operazioni che possono fallire, `File::open` e `read_to_string`. Il corpo gestisce ogni `Result` con un `match`: nel caso `Err` usa `return` per uscire anticipatamente e passare l'errore al chiamante; nel caso di successo di entrambe le operazioni ritorna `Ok(username)`. Il chiamante può allora scegliere di fare `panic!`, usare un username di default o cercarlo altrove: la funzione non ha abbastanza informazioni per decidere, quindi propaga tutto verso l'alto.

L'operatore ?

Questo schema di propagazione è così comune che Rust fornisce l'operatore punto interrogativo `?` per semplificarlo. Il `?` dopo un valore `Result` funziona quasi come i `match` visti: se il valore è `Ok`, il valore interno viene restituito dall'espressione e il programma continua; se è `Err`, l'errore viene restituito dall'intera funzione come con `return`, propagandolo al codice chiamante.

```rust
let mut username_file = File::open("hello.txt")?;
```

C'è una differenza dai `match` scritti a mano: i valori di errore su cui si chiama `?` passano dalla funzione `from`, definita nel trait `From` della libreria standard, che converte i valori da un tipo all'altro. L'errore ricevuto viene **convertito** nel tipo di errore del tipo di ritorno della funzione corrente: questo permette a una funzione di usare un unico tipo d'errore per tutte le sue modalità di fallimento, ad esempio un `OurError` personalizzato con `impl From<io::Error> for OurError`, senza codice aggiuntivo.

Il `?` elimina molto boilerplate: si possono persino concatenare le chiamate di metodo subito dopo il `?`, come `File::open("hello.txt")?.read_to_string(&mut username)?;`, e per l'operazione comune di leggere un file in una stringa la libreria standard offre `fs::read_to_string`, che apre il file, crea la `String` e ne ritorna il contenuto.

Dove usare l'operatore ?

L'operatore `?` si può usare solo in funzioni il cui tipo di ritorno è **compatibile** col valore su cui si usa, perché è definito come un ritorno anticipato. Usarlo in una `main` che ritorna `()` non compila: il messaggio d'errore spiega che è permesso solo in funzioni che ritornano `Result`, `Option` o un altro tipo che implementa `FromResidual`. Le correzioni sono due.

- **Cambiare il tipo di ritorno** della funzione per renderlo compatibile, se non ci sono vincoli contrari.
- **Gestire il `Result`** con un `match` o uno dei metodi di `Result<T, E>` nel modo appropriato.

`?` si usa anche sui valori `Option<T>`, con un comportamento analogo: se il valore è `None`, viene restituito anticipatamente dalla funzione; se è `Some`, il valore interno è il risultato dell'espressione e la funzione continua. Una funzione che trova l'ultimo carattere della prima riga di un testo ritorna `Option<char>` e si scrive in una riga combinando `text.lines().next()?` e `chars().last()?`. Non si possono mescolare i tipi: `?` converte automaticamente solo `Result` in `Result` e `Option` in `Option`; per la conversione esplicita si usano metodi come `ok` su `Result` o `ok_or` su `Option`.

`main` è speciale perché è punto d'ingresso e di uscita di un eseguibile, ma può anche ritornare `Result<(), E>`. Con il tipo `Result<(), Box<dyn Error>>` (un trait object, si veda il Capitolo 18; qui si legge "qualunque tipo di errore") la `main` può usare `?`, e la firma resta corretta anche aggiungendo codice che ritorna altri tipi di errore. Quando `main` ritorna `Ok(())` l'eseguibile esce col valore `0`, con `Err` esce con un valore diverso da zero, per compatibilità con la convenzione degli eseguibili C. In generale `main` può ritornare qualunque tipo implementi il trait `std::process::Termination`, che ha una funzione `report` che ritorna un `ExitCode`.

Quando usare `panic!` e quando `Result`

Come decidere tra chiamare `panic!` e ritornare `Result`? Quando il codice va in panic non c'è modo di riprendersi: chiamare `panic!` per qualunque errore significa decidere a nome del chiamante che una situazione è non riparabile. Ritornare `Result` dà invece opzioni al chiamante, che può provare a riprendersi nel modo appropriato oppure decidere che quel caso è non riparabile e chiamare `panic!` a sua volta, trasformando l'errore riparabile in uno non riparabile. Per questo **ritornare `Result` è la scelta di default** quando si definisce una funzione che può fallire.

Ci sono situazioni in cui è più appropriato scrivere codice che fa panic.

- **Esempi**: in un esempio che illustra un concetto, un robusto codice di gestione degli errori rende l'esempio meno chiaro; una chiamata come `unwrap` si intende come segnaposto della gestione che l'applicazione reale vorrà.
- **Prototipi**: `unwrap` e `expect` sono comodi quando non si è ancora deciso come gestire gli errori, e lasciano marcatori chiari nel codice per quando si vorrà rendere il programma più robusto.
- **Test**: se una chiamata fallisce in un test, tutto il test deve fallire, anche se quel metodo non è la funzionalità sotto test; poiché `panic!` è il modo in cui un test viene marcato come fallito, chiamare `unwrap` o `expect` è esattamente ciò che serve.

Quando si ha più informazione del compilatore

Chiamare `expect` è appropriato anche quando una logica esterna assicura che il `Result` conterrà un `Ok`, ma quella logica non è visibile al compilatore. L'operazione resta comunque un `Result` da gestire, perché in generale può fallire

anche se nella situazione particolare è logicamente impossibile: se ispezionando il codice si è certi che la variante `Err` non si verificherà mai, è perfettamente accettabile chiamare `expect` e **documentare nell'argomento** il motivo per cui non ci sarà mai un `Err`, come fare il parse della stringa hardcoded `127.0.0.1` in un `IpAddr`. Il compilatore non è abbastanza intelligente da vedere che la stringa è sempre valida, e se un giorno l'indirizzo arrivasse da un'altra fonte, quel commento ricorderà di sostituire `expect` con una gestione più robusta.

Linee guida per la gestione degli errori

È consigliabile far fare panic al codice quando è possibile che finisca in uno **stato corrotto**, cioè quando un'assunzione, una garanzia, un contratto o un invariante è stato violato, come valori invalidi, contraddittori o mancanti passati al codice, **in più** di una delle condizioni seguenti.

- Lo stato corrotto è **inaspettato**, al contrario di qualcosa che capita ogni tanto, come un utente che inserisce dati nel formato sbagliato.
- Il codice dopo quel punto deve poter **contare sull'assenza** di quello stato corrotto, invece di verificare il problema a ogni passo.
- **Non c'è un buon modo di codificare** quell'informazione nei tipi usati (si veda il Capitolo 18).

Altre linee guida distinguono i casi per responsabilità e prevedibilità.

- **Input privi di senso**: se chi chiama il codice passa valori insensati, meglio restituire un errore quando possibile, così l'utente della libreria decide cosa fare; se continuare fosse insicuro o dannoso, `panic!` segnala il bug nel codice chiamante durante lo sviluppo.
- **Fallimenti attesi**: quando il fallimento è prevedibile è più appropriato ritornare `Result`, come un parser che riceve dati malformati o una richiesta HTTP che incontra un rate limit: il chiamante sa che quel fallimento è possibile e deve decidere come gestirlo.
- **Sicurezza**: se l'operazione con valori invalidi può mettere a rischio l'utente, il codice deve verificare la validità **prima** di procedere e fare panic altrimenti; è il motivo per cui la libreria standard fa `panic!` sugli accessi fuori dai limiti, un classico problema di sicurezza.
- **Contratti**: le funzioni hanno contratti, garanti solo se gli input soddisfano certi requisiti; violare il contratto è sempre un bug lato chiamante, non un errore da far gestire esplicitamente, perché non c'è modo ragionevole di riprendersi: sono i programmatori chiamanti a dover sistemare il codice. Le condizioni del contratto vanno documentate nella API della funzione.

Avere molti controlli di errore in ogni funzione sarebbe verboso: il sistema dei tipi di Rust può fare molti controlli al posto nostro. Se una funzione ha un certo tipo come parametro, si può procedere sapendo che il compilatore ha già assicurato un valore valido: con un tipo invece di `Option` il programma si aspetta *qualcosa* e non deve gestire `Some` e `None`; il codice che prova a passare nulla non compila nemmeno. Analogamente, un tipo senza segno come `u32` assicura che il parametro non sia mai negativo.

Tipi personalizzati per la validazione

L'uso del sistema dei tipi si spinge oltre con la creazione di un **tipo personalizzato per la validazione**. Nel gioco dell'indovina del Capitolo 2 non si validava mai che il numero fosse tra 1 e 100; un primo miglioramento è fare il parse come `i32` e aggiungere un controllo `if` che avvisi l'utente e riparta col ciclo. Ma se il programma avesse **molte** funzioni con il requisito 1–100, quel controllo in ogni funzione sarebbe tedioso e potrebbe pesare sulle prestazioni.

La soluzione è un nuovo tipo in un modulo dedicato, con le validazioni in una funzione che crea le istanze invece di ripeterle ovunque. Un tipo `Guess` incapsula queste garanzie.

- **Funzione associata `new`**: prende un `value: i32`, verifica che sia tra 1 e 100 e, se il test fallisce, chiama `panic!`, avvisando il programmatore chiamante del bug da sistemare, perché creare un `Guess` fuori range violerebbe il contratto; se passa, crea e restituisce il `Guess`. Le condizioni di possibile panic vanno descritte nella documentazione pubblica della API.
- **Campo `value` privato**: il codice esterno al modulo non può impostare `value` direttamente e deve passare per `Guess::new`, così non esiste modo di creare un `Guess` con un valore non verificato.
- **Metodo `value`**: prende `&self` e restituisce l'`i32`, un *getter* pubblico necessario proprio perché il campo è privato.

Una funzione che accetta o restituisce solo numeri tra 1 e 100 può dichiarare nella firma un `Guess` invece di un `i32`, e il suo corpo non ha bisogno di controlli aggiuntivi.

Ricapitolazione

Le feature di gestione degli errori di Rust aiutano a scrivere codice più robusto. La macro `panic!` segnala che il programma è in uno stato che non può gestire e dice al processo di fermarsi invece di procedere con valori invalidi o

scorretti. L'enum `Result` usa il sistema dei tipi per indicare operazioni che potrebbero fallire in modo riparabile, e comunica al codice chiamante che deve gestire potenziale successo o fallimento. Usare `panic!` e `Result` nelle situazioni appropriate rende il codice più affidabile di fronte ai problemi inevitabili. Dopo aver visto come la libreria standard usa i generici con gli enum `Option` e `Result`, il prossimo capitolo parla di come funzionano i generici e di come usarli nel proprio codice.

Parte IV

Astrazioni e test

Capitolo 10

Tipi generici, trait e lifetime

Ogni linguaggio di programmazione offre strumenti per gestire la duplicazione di concetti. In Rust uno di questi strumenti sono i **generici**, ovvero segnaposto astratti per tipi concreti o altre proprietà. Li abbiamo già usati senza saperlo: `Option<T>` nel capitolo sulle enum, `Vec<T>` e `HashMap<K, V>` nel capitolo sulle collezioni, `Result<T, E>` nella gestione degli errori. Questa lezione mostra come definire i propri tipi, funzioni e metodi generici, come usare i **trait** per definire comportamento in forma astratta e come usare i **lifetime**, una varietà di generici che danno al compilatore informazioni su come le reference si rapportano tra loro.

Dalla duplicazione all'astrazione

I generici permettono di sostituire tipi specifici con un segnaposto che rappresenta più tipi, eliminando la duplicazione. Prima di introdurne la sintassi conviene però esercitarsi su una forma di astrazione più semplice: estrarre una funzione che sostituisce valori specifici con un parametro che rappresenta più valori. Riconoscendo il codice duplicato estraibile in una funzione si impara anche a riconoscere il codice duplicato generalizzabile con i generici.

Il caso di partenza è la ricerca del numero più grande in una lista. Se due funzioni fanno la stessa cosa su liste di `i32` e su liste di `char`, differendo solo nei tipi dei parametri, la stessa tecnica di estrazione si applica ai tipi. I passi per trasformare codice duplicato in astrazione sono i seguenti.

- **Identificare** il codice duplicato.
- **Estrarre** il codice duplicato nel corpo di una funzione, specificandone input e valori di ritorno nella firma.
- **Aggiornare** le occorrenze duplicate perché chiamino la funzione.

Applicando questi passi ai tipi, il corpo della funzione può operare su un tipo astratto invece che su tipi concreti, esattamente come opera su valori astratti invece che su valori concreti.

Tipi generici

Usiamo i generici per creare definizioni di item come firme di funzioni o struct, riutilizzabili con molti tipi concreti diversi.

Nelle definizioni di funzione

I generici si collocano nella firma, dove normalmente specifichiamo i tipi dei parametri e del valore di ritorno:

```rust
fn largest<T>(list: &[T]) -> &T
```

Come per i parametri di valore, anche il nome del parametro di tipo va dichiarato prima dell'uso, tra parentesi angolari tra il nome della funzione e la lista dei parametri. Si legge: la funzione `largest` è generica su un qualche tipo `T`. Le convenzioni di nomenclatura prevedono quanto segue.

- **Breve**: i nomi dei parametri di tipo in Rust sono corti, spesso di una sola lettera.
- **UpperCamelCase**: la convenzione di nomenclatura dei tipi si applica anche ai parametri di tipo.
- **T**: è il nome di default scelto dalla maggior parte dei programmatori Rust, abbreviazione di *type*.

La firma `fn largest<T>(list: &[T]) -> &T` da sola però **non compila**. Il motivo è il cuore della filosofia dei generici in Rust.

Warning

Un tipo generico senza vincoli non ha capacità

Quando `largest` riceve uno slice `&[T]`, la funzione non può assumere **nulla** sul tipo `T`: potrebbe essere `i32`, `String` o `File`. Ma `largest` richiede che `T` sia confrontabile con `>`, cioè che implementi il trait `PartialOrd`. A differenza dei *template* del C++, dove il compilatore lamenta solo la chiamata su un tipo incompatibile, Rust esige che le capacità attese dei tipi generici siano dichiarate in anticipo: se `T` dev'essere confrontabile, `largest` deve dirlo. A differenza di Java, dove ogni oggetto ha metodi di base come `toString()`, in Rust non esistono metodi di base: senza vincoli un

tipo generico `T` non può essere stampato, clonato o mutato (anche se può essere droppato).

Nelle definizioni di struct

Una struct può dichiarare parametri di tipo generico dopo il proprio nome e usarli nei campi al posto dei tipi concreti. Con un solo parametro, la definizione impone che **tutti** i campi siano dello stesso tipo. Con più parametri, i campi possono avere tipi diversi:

```rust
struct Point<T, U> {
    x: T,
    y: U,
}
```

Assegnare `5` a `x` e `4.0` a `y` in una `Point<T>` con un solo parametro è un errore di tipo: la prima assegnazione fissa `T` come intero e la seconda lo contraddice. Si possono usare quanti parametri di tipo si vogliono, ma più di pochi rendono il codice difficile da leggere; se servono molti generici, probabilmente il codice va ristrutturato in pezzi più piccoli.

Nelle definizioni di enum

Anche le enum possono trattenere tipi generici nelle loro varianti. Le definizioni già incontrate di `Option<T>` e `Result<T, E>` ora si leggono naturalmente.

```rust
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

`Result` è generica su due tipi e ha due varianti: `Ok` trattiene un valore di tipo `T` (il successo) ed `Err` un valore di tipo `E` (l'errore). L'astrazione è riutilizzabile ovunque un'operazione possa riuscire o fallire con tipi diversi. Quando riconosciamo più definizioni di struct o enum che differiscono solo nei tipi dei valori trattenuti, possiamo evitare la duplicazione usando i generici.

Nelle definizioni di metodo

I metodi si implementano su struct ed enum generiche come al solito, dentro blocchi `impl`. Le regole da ricordare sono le seguenti.

- **Dichiarazione**: `T` va dichiarato subito dopo `impl` (`impl<T> Point<T>`), così Rust sa che l'identificatore tra parentesi angolari è un tipo generico e non concreto.
- **Scoping**: un metodo definito in un `impl` generico è disponibile su ogni istanza, qualunque tipo concreto sostituisca il generico.
- **Implementazioni specifiche**: si può implementare un metodo solo su `Point<f32>` scrivendo `impl Point<f32>` senza dichiarare tipi dopo `impl`; le altre istanze `Point<T>` non avranno quel metodo.
- **Metodi misti**: i parametri generici della firma di un metodo possono essere diversi da quelli della struct: i primi si dichiarano dopo `fn`, i secondi dopo `impl`.

Warning

Niente specializzazione dei metodi

Non si può implementare simultaneamente un metodo omonimo in versione generica e in versione specifica: con `distance_from_origin` sia per ogni `T` sia per `f32`, il compilatore rifiuterebbe il programma perché non saprebbe quale implementazione scegliere per `Point<f32>`. Più in generale Rust non ha meccanismi di specializzazione dei metodi in stile orientato agli oggetti, con una sola eccezione: i metodi di default dei trait.

Le prestazioni del codice generico

Usare tipi generici non rende il programma più lento rispetto ai tipi concreti. Rust lo ottiene con la **monomorfizzazione**, il processo di trasformazione del codice generico in codice specifico riempiendo i tipi concreti usati nelle chiamate, eseguito a compile time.

Definition

Monomorfizzazione

Il compilatore esamina tutti i punti in cui il codice generico è chiamato e genera codice per i tipi concreti con cui è chiamato. Un uso di `Option<T>` con `i32` e `f64` produce due definizioni specializzate, come se avessimo duplicato a mano ogni definizione.

Poiché il codice generico viene compilato in codice che specifica il tipo in ogni istanza, non si paga alcun costo a runtime: la monomorfizzazione rende i generici di Rust estremamente efficienti.

Definire comportamento condiviso con i trait

Definition

Trait

Un trait definisce la funzionalità che un particolare tipo ha e può condividere con altri tipi. I trait raggruppano firme di metodi per definire un insieme di comportamenti necessario a uno scopo, permettendo di esprimere comportamento condiviso in astratto. Sono simili alle *interfacce* di altri linguaggi, con alcune differenze.

Il comportamento di un tipo consiste nei metodi che possiamo chiamare su quel tipo: tipi diversi condividono lo stesso comportamento se supportano le stesse chiamate. Ad esempio, un trait `Summary` per una libreria di aggregazione mediale dichiara una firma di metodo senza corpo, terminata da punto e virgola.

```rust
pub trait Summary {
    fn summarize(&self) -> String;
}
```

Il compilatore imporrà che ogni tipo con il trait `Summary` abbia il metodo `summarize` definito esattamente con quella firma. Un trait può avere più metodi nel proprio corpo, elencati uno per riga e ciascuno terminato da punto e virgola.

Implementare un trait su un tipo

L'implementazione è simile a quella dei metodi ordinari, ma dopo `impl` si scrive il nome del trait, poi la parola chiave `for`, poi il tipo:

```rust
impl Summary for NewsArticle {
    fn summarize(&self) -> String { /* ... */ }
}
```

Gli utenti del crate chiamano i metodi del trait sulle istanze come metodi normali, con una differenza: devono portare in scope anche il trait, oltre ai tipi. Un'altra restrizione regola chi può implementare cosa.

Warning

La regola dell'orfano

Si può implementare un trait su un tipo solo se il trait o il tipo, o entrambi, sono locali al nostro crate. Implementare `Display` (della libreria standard) sul nostro tipo è lecito; implementare il nostro `Summary` su `Vec<T>` è lecito; implementare `Display` su `Vec<T>` no, perché entrambi sono esterni. La restrizione è parte della proprietà di **coerenza**, detta regola dell'orfano perché il tipo genitore non è presente: senza di essa, due crate potrebbero implementare lo stesso trait per lo stesso tipo e Rust non saprebbe quale implementazione usare.

Implementazioni di default

Invece di richiedere l'implementazione di tutti i metodi su ogni tipo, un trait può fornire un comportamento di default per alcuni o tutti i metodi. Chi implementa il trait può mantenere o sovrascrivere ogni default: un blocco `impl Summary for NewsArticle {}` vuoto basta a ottenere il comportamento di default, e la sintassi per sovrascrivere è identica a quella per implementare un metodo senza default. Due proprietà caratterizzano i default.

- **Chiamate incrociate**: un'implementazione di default può chiamare altri metodi dello stesso trait anche se questi non hanno un default; così un trait può offrire molta funzionalità chiedendo all'implementatore di specificarne solo una parte (ad esempio `summarize` di default che chiama un obbligatorio `summarize_author`).

- **Niente risalita**: da un'implementazione che sovrascrive un metodo non è possibile chiamare l'implementazione di default di quel medesimo metodo.

Usare i trait come parametri

Per scrivere funzioni che accettano molti tipi diversi si usa la sintassi `impl Trait`: al posto di un tipo concreto, il parametro riceve la parola chiave `impl` seguita dal nome del trait.

```rust
pub fn notify(item: &impl Summary) {
```

Il parametro accetta qualsiasi tipo che implementi il trait specificato e nel corpo si possono chiamare i metodi del trait; passare un tipo che non lo implementa (come `String` o `i32`) non compila. La sintassi `impl Trait` è però zucchero per una forma più lunga e più espressiva.

Definition

Trait bound

La forma completa dichiara il vincolo dopo i due punti, dentro le parentesi angolari: `pub fn notify<T: Summary>(item: &T)`. Con due parametri `&impl Summary` indipendenti i tipi possono differire; per forzare lo **stesso** tipo per `item1` e `item2` serve il trait bound con un unico `T`, cosa che la sintassi breve non può esprimere.

I vincoli si combinano in due modi complementari.

- **Sintassi `+`**: specifica più trait per lo stesso parametro, come `impl Summary + Display` oppure `T: Summary + Display`.
- **Clausola `where`**: quando i vincoli tra nome della funzione e lista dei parametri rendono la firma illeggibile, si spostano dopo la firma con `where T: Display + Clone, U: Clone + Debug`, riavvicinando nome, parametri e tipo di ritorno.

Ritornare tipi che implementano trait

La sintassi `impl Trait` si usa anche in posizione di ritorno, per dichiarare che una funzione restituisce un tipo che implementa un trait senza nominare il tipo concreto. Questo è particolarmente utile per closure e iteratori (capitolo 13), che creano tipi noti solo al compilatore o molto lunghi da scrivere. Esiste però una limitazione importante: `impl Trait` in posizione di ritorno funziona solo se la funzione restituisce **un solo** tipo. Un ritorno che alterna tra `NewsArticle` e `SocialPost` con `impl Summary` non compila; quel comportamento richiede i trait object, trattati nel capitolo sulle feature orientate agli oggetti.

Implementare metodi condizionalmente con i trait bound

Un blocco `impl` con parametri generici e trait bound implementa metodi solo per i tipi che soddisfano i vincoli. Una `Pair<T>` può avere `cmp_display` solo quando `T` implementa `PartialOrd` (confronto) **e** `Display` (stampa). Si può persino implementare un trait per ogni tipo che ne implementa un altro.

Definition

Blanket implementation

Le implementazioni di un trait su qualsiasi tipo che soddisfa certi trait bound si chiamano *blanket implementation* e sono usate estesamente nella libreria standard: `impl<T: Display> ToString for T` rende disponibile `to_string` su ogni tipo che implementa `Display`, ad esempio `3.to_string()`. Appaiono nella documentazione del trait, sezione "Implementors".

Trait e trait bound permettono di ridurre la duplicazione specificando al compilatore il comportamento atteso dai tipi generici. Nei linguaggi dinamicamente tipizzati un metodo mancante si scoprirebbe a runtime; Rust sposta questi errori a compile time, costringendoci a correggerli prima che il codice possa girare, e non serve codice di controllo a runtime perché la verifica è già avvenuta: flessibilità dei generici senza costo in prestazioni.

Validare le reference con i lifetime

Definition

Lifetime

I lifetime sono un altro tipo di generici che stiamo già usando senza saperlo: anziché garantire che un tipo abbia il comportamento desiderato, garantiscono che le reference siano valide finché ne abbiamo bisogno. Il lifetime di una reference è lo scope per cui quella reference è valida.

Come i tipi, i lifetime sono quasi sempre impliciti e dedotti: dobbiamo annotare i tipi solo quando più tipi sono possibili, e analogamente dobbiamo annotare i lifetime solo quando le reference potrebbero relazionarsi in più modi. Rust richiede di esprimere queste relazioni con parametri di lifetime generici, così che le reference concrete usate a runtime siano sicuramente valide. È un concetto che la maggior parte degli altri linguaggi non ha, quindi risulta poco familiare.

Reference dangling

L'obiettivo principale dei lifetime è prevenire le reference dangling: se fossero permesse, un programma potrebbe riferirsi a dati diversi da quelli che intende riferire. Il caso classico è una variabile `r` dichiarata in uno scope esterno che viene fatta puntare a una variabile `x` dichiarata in uno scope interno: quando lo scope interno termina, `x` non esiste più e l'uso di `r` non compila. Il messaggio d'errore dice che `x` "non vive abbastanza": `r` è ancora valido nello scope esterno, quindi "vive più a lungo" del dato che referenzia. Se Rust lo permettesse, `r` referenzerebbe memoria deallocata e qualsiasi operazione su di essa fallirebbe. Nota sul contesto: dichiarare una variabile senza valore iniziale non è in conflitto con l'assenza di null in Rust, perché usarla prima dell'inizializzazione è un errore a compile time.

Il borrow checker

Definition

Borrow checker

Il compilatore Rust ha un *borrow checker* che confronta gli scope per stabilire se tutti i borrow sono validi. Annotando idealmente i lifetime delle variabili (`'a` per `r`, `'b` per `x`), il confronto è banale: se `r` ha lifetime `'a` ma referenzia memoria con lifetime `'b` più corto, il programma è rifiutato perché il referente non vive quanto la reference. La regola è che il dato referenziato deve sempre vivere almeno quanto la reference che lo referenzia.

Lifetime generici nelle funzioni

Consideriamo una funzione `longest` che restituisce la più lunga tra due string slice. Prendere slice (reference) e non `String` evita che la funzione si prenda l'ownership dei parametri. L'implementazione ovvia non compila, perché Rust non può dire se la reference restituita punta a `x` o a `y`: nemmeno noi lo sappiamo scrivendo la funzione, dato che il ramo `if` restituisce una reference a `x` e il ramo `else` una a `y`. Definendo la funzione non conosciamo né i valori concreti né i lifetime concreti delle reference ricevute, quindi non possiamo esaminare gli scope come nel caso precedente; il borrow checker non può stabilire come i lifetime di `x` e `y` si relazionano con quello del valore di ritorno. La soluzione sono parametri di lifetime generici che dichiarano la relazione.

La sintassi delle annotazioni ha tre caratteristiche.

- **Apostrofo**: i nomi dei parametri di lifetime devono iniziare con `'`, sono di solito tutti minuscoli e molto corti.
- **Posizione**: l'annotazione si colloca dopo la `&` della reference, separata dal tipo con uno spazio: `&i32`, `&'a i32`, `&'a mut i32`.
- **Relazionalità**: un'annotazione di lifetime da sola non significa molto, perché serve a dire a Rust come si rapportano tra loro i parametri di lifetime di **più** reference.

Nelle firme di funzione

I parametri di lifetime si dichiarano dentro le parentesi angolari tra nome della funzione e lista dei parametri, come i tipi generici. La firma deve esprimere il vincolo: la reference restituita sarà valida finché entrambi i parametri sono validi.

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str
```

La firma ora dice che, per un qualche lifetime `'a`, la funzione riceve due slice che vivono almeno quanto `'a` e restituisce uno slice che vive almeno quanto `'a`. In pratica il lifetime della reference restituita è il **minore** dei lifetime dei valori referenziati dagli argomenti. Tre precisazioni completeano il quadro.

- **Nessuna modifica**: le annotazioni non cambiano la durata di alcun valore passato o restituito; descrivono solo le relazioni tra i lifetime, e il borrow checker rifiuta i valori che non rispettano i vincoli.

- **Contratto nella firma**: le annotazioni vanno nella firma, non nel corpo; diventano parte del contratto della funzione come i tipi, e così gli errori del compilatore indicano con precisione il punto del codice e il vincolo violato.
- **Sostituzione concreta**: la funzione non deve sapere quanto vivono `x` e `y`, solo che esiste uno scope sostituibile per `'a` che soddisfa la firma.

Un esperimento mentale conferma il meccanismo: se `result` viene usato dopo la fine dello scope di `string2`, il codice non compila anche se noi vediamo che `result` punta a `string1`, ancora valida. Abbiamo detto a Rust che il lifetime del ritorno è il minore dei due, quindi il borrow checker respinge l'uso: non può vedere la validità caso per caso, lavora sul contratto dichiarato.

Le relazioni tra lifetime

Quanto si debba annotare dipende da ciò che la funzione fa. Se `longest` restituisse sempre il primo parametro, il parametro `y` non avrebbe bisogno di annotazione, perché il suo lifetime non ha relazioni con il ritorno: si può scrivere `fn longest<'a>(x: &'a str, y: &str) -> &'a str`. Ne discendono due regole generali.

- **Origine del ritorno**: quando una funzione restituisce una reference, il lifetime del tipo di ritorno deve abbinarsi a quello di uno dei parametri; se la reference restituita non si riferisce a un parametro, si riferisce a un valore creato dentro la funzione, che muore alla fine di essa: sarebbe una reference dangling che nessuna annotazione può salvare.
- **Rimedio pratico**: in quel caso la soluzione giusta è restituire un tipo con ownership del dato, spostando sul chiamante la responsabilità della pulizia.

In ultima analisi la sintassi dei lifetime serve a **collegare** i lifetime di parametri e valori di ritorno: una volta collegati, Rust ha abbastanza informazioni per permettere le operazioni memory-safe e vietare quelle che creerebbero dangling pointer.

Nelle definizioni di struct

Finora le nostre struct trattenevano solo tipi con ownership. Una struct può trattenere reference, ma allora ogni reference nella definizione richiede un'annotazione di lifetime, dichiarata tra parentesi angolari dopo il nome della struct come per i tipi generici:

```rust
struct ImportantExcerpt<'a> {
    part: &'a str,
}
```

L'annotazione significa che un'istanza di `ImportantExcerpt` non può vivere più a lungo della reference che trattiene nel campo `part`. Perché sia valido, il dato referenziato deve esistere prima che l'istanza venga creata e restare in scope almeno fino a quando l'istanza esce di scope.

Lifetime elision

Abbiamo già scritto funzioni con reference in ingresso e in uscita, come `first_word`, che compilano senza annotazioni. Il motivo è storico: nelle versioni pre-1.0 ogni reference richiedeva un lifetime esplicito (`fn first_word<'a>(s: &'a str) -> &'a str`), ma scrivendo tanto codice Rust il team notò che i programmatori ripetevano le stesse annotazioni in situazioni prevedibili e deterministiche, e programmò quei pattern nel compilatore. La storia è rilevante perché altri pattern potrebbero emergere in futuro, riducendo ulteriormente le annotazioni richieste.

Definition

Regole di elisione dei lifetime

Le *lifetime elision rules* non sono regole per il programmatore: sono un insieme di casi particolari che il compilatore considera, e se il codice vi rientra le annotazioni non servono. Non forniscono inferenza completa: se dopo averle applicate resta ambiguità, il compilatore non tira a indovinare ma emette un errore che si risolve aggiungendo le annotazioni. I lifetime sui parametri si chiamano *input lifetime*, quelli sui valori di ritorno *output lifetime*.

Le regole sono tre e si applicano sia alle definizioni `fn` sia ai blocchi `impl`.

- **Prima regola (input)**: il compilatore assegna un parametro di lifetime diverso a ogni lifetime in ogni tipo di input; ogni reference (`&'_ i32`) e ogni struttura con lifetime (`ImportantExcerpt<'_>`) riceve il proprio parametro, quindi `fn foo(x: &i32, y: &i32)` diventa `fn foo<'a, 'b>(x: &'a i32, y: &'b i32)`.
- **Seconda regola (output)**: se c'è esattamente **un** parametro di lifetime in input, quel lifetime viene assegnato a tutti gli output: `fn foo<'a>(x: &'a i32) -> &'a i32`.

- **Terza regola (metodi)**: se ci sono più input ma uno di essi è `&self` o `&mut self` (quindi siamo in un metodo), il lifetime di `self` viene assegnato a tutti gli output; questa regola rende i metodi molto più leggibili.

Applicandole come farebbe il compilatore: la firma di `first_word` riceve un lifetime per la prima regola e lo propaga all'output per la seconda, quindi non serve annotare nulla. Per `longest` la prima regola produce due lifetime distinti, ma né la seconda (due input, non uno) né la terza (funzione, non metodo) si applicano: il lifetime del ritorno resta sconosciuto ed ecco perché Listing 10-20 non compilava.

Nelle definizioni di metodo

Implementando metodi su una struct con lifetime si usa la stessa sintassi dei tipi generici. I nomi di lifetime dei **campi** della struct vanno sempre dichiarati dopo `impl` e usati dopo il nome della struct, perché fanno parte del tipo della struct. Le reference nelle firme dei metodi dentro il blocco possono invece essere legate ai lifetime dei campi o esserne indipendenti, e le regole di elisione evitano spesso le annotazioni. Due esempi con `ImportantExcerpt` chiariscono il meccanismo.

- **Elisione sul self**: un metodo `level` con solo `&self` e ritorno `i32` non richiede annotazioni, grazie alla prima regola; la dichiarazione di `'a` dopo `impl` resta però obbligatoria, perché appartiene al tipo.
- **Terza regola in azione**: un metodo `announce_and_return_part` con `&self` e un ulteriore parametro `announcement` riceve due input lifetime per la prima regola; poiché uno è `&self`, il tipo di ritorno prende il lifetime di `self` e tutti i lifetime sono determinati.

Il lifetime statico

Un lifetime speciale merita attenzione.

Definition

Lifetime statico

`'static` indica che la reference interessata **può** vivere per l'intera durata del programma. Tutti i literal di stringa hanno lifetime `'static`, perché il loro testo è memorizzato direttamente nel binario del programma, sempre disponibile: si può scrivere `let s: &'static str = "...";`.

Warning

Usare 'static con cautela

I messaggi d'errore a volte suggeriscono di usare il lifetime `'static`, ma prima di farlo occorre chiedersi se la reference vive davvero per l'intero programma e se lo si desidera. Nella maggior parte dei casi il suggerimento nasce da un tentativo di creare una reference dangling o da un disallineamento di lifetime disponibili: la soluzione è correggere quei problemi, non dichiarare `'static`.

Tipi generici, trait bound e lifetime insieme

I tre strumenti della lezione si combinano nella stessa firma. La funzione `longest` con un parametro aggiuntivo `ann` di tipo generico `T` vincolato da una clausola `where` unisce tutto:

```rust
use std::fmt::Display;

fn longest_with_announcement<'a, T>(
    x: &'a str, y: &'a str, ann: T,
) -> &'a str
where T: Display {
```

Poiché i lifetime sono un tipo di generici, la dichiarazione del parametro di lifetime `'a` e quella del parametro di tipo `T` vanno nella **stessa** lista dentro le parentesi angolari dopo il nome della funzione. Il quadro complessivo della lezione è il seguente.

- **Tipi generici**: permettono di applicare il codice a tipi diversi senza duplicazione.
- **Trait e trait bound**: garantiscono che i tipi generici abbiano il comportamento di cui il codice ha bisogno.
- **Lifetime**: assicurano che il codice flessibile non produca reference dangling.

Tutta questa analisi avviene a compile time e non ha alcun effetto sulle prestazioni a runtime. C'è molto di più su questi temi: i trait object come altro modo di usare i trait nel capitolo sulle feature orientate agli oggetti, e scenari di lifetime più complessi trattati nel Rust Reference; ma il passo successivo è imparare a scrivere test per verificare che il

codice funzioni come previsto.

Capitolo 11

Test automatizzati

> *"Il test dei programmi può essere un modo molto efficace di mostrare la presenza di bug, ma è disperatamente inadeguato per mostrarne l'assenza." (saggio* The Humble Programmer*, 1972)*
>
> – Edsger W. Dijkstra

La **correttezza** di un programma è la misura in cui il codice fa ciò che intendiamo fare. Rust è progettato con una forte attenzione alla correttezza, ma la correttezza è complessa e non facile da dimostrare: il sistema di tipi porta una parte enorme di questo peso, però non può rilevare tutto. Consideriamo una funzione `add_two` che somma 2 al parametro ricevuto: il type checking e il borrow checking garantiscono che non si passi una `String` o una reference invalida, ma non possono garantire che la funzione restituisca il parametro più 2 anziché, diciamo, il parametro più 10 o meno 50. I test colmano esattamente questo divario: possiamo scrivere test che asseriscono che passando `3` il valore restituito sia `5`, ed eseguirli a ogni modifica del codice per assicurarci che il comportamento corretto esistente non sia cambiato. Scrivere buoni test è una competenza complessa; questa lezione tratta la meccanica delle strutture di test di Rust: annotazioni, macro, comportamento di default delle esecuzioni e organizzazione in unit test e integration test.

Come scrivere i test

Definition

Test

I test sono funzioni Rust che verificano che il codice non di test funzioni nel modo previsto. Il corpo di una funzione di test tipicamente compie tre azioni: preparare i dati o lo stato necessari, eseguire il codice da testare e asserire che i risultati siano quelli attesi.

La struttura delle funzioni di test

Nella sua forma più semplice, un test in Rust è una funzione annotata con l'attributo `test`: gli attributi sono metadati su pezzi di codice Rust, come l'attributo `derive` già usato con le struct. Per trasformare una funzione in un test si aggiunge `#[test]` sulla riga prima di `fn`:

```rust
#[test]
fn it_works() {
    assert_eq!(add(2, 2), 4);
}
```

Con `cargo test`, Rust compila un binario *test runner* che esegue le funzioni annotate e riporta per ciascuna se passa o fallisce. Ogni nuovo progetto libreria Cargo genera automaticamente un modulo di test con una funzione di test dentro, come modello: si possono aggiungere quante funzioni e quanti moduli di test si vogliono. Serve sempre l'annotazione perché nel modulo `tests` possono esserci anche funzioni non di test, di supporto.

Un test fallisce quando qualcosa nel corpo della funzione va in **panic**: ogni test gira in un thread nuovo e quando il thread principale vede che il thread del test è morto, il test è segnato come fallito. L'output di `cargo test` riporta il nome di ogni test con esito `ok` o `FAILED`, poi una sezione con la ragione dettagliata di ogni fallimento, poi l'elenco dei soli nomi dei test falliti (utile con molti test), e infine la riga di riepilogo. Le statistiche del riepilogo comprendono anche i test ignorati, quelli filtrati e i *benchmark* (misurazioni di prestazioni, al momento disponibili solo nella toolchain nightly); una sezione `Doc-tests` riguarda invece i test della documentazione, trattati nel capitolo su Cargo e crates.io.

Verificare i risultati con assert

La libreria standard fornisce tre macro di asserzione con ruoli complementari.

- `assert!`: riceve un argomento che valuta a Boolean; se è `true` non succede nulla e il test passa, se è `false` la macro chiama `panic!` e il test fallisce. È l'ideale per funzioni che restituiscono un Boolean, come il metodo `can_hold` di `Rectangle`: si afferma che un rettangolo 8×7 contiene uno 5×1, e che uno più piccolo non ne contiene uno più grande negando il risultato.

- `assert_eq!`: confronta due argomenti per uguaglianza e, in caso di fallimento, stampa **entrambi** i valori, facilitando il debug: il messaggio dice che l'asserzione `left == right` è fallita mostrando i due valori.
- `assert_ne!`: passa se i due valori sono diversi e fallisce se sono uguali; è utile quando non si sa che valore arriverà ma si sa quale **non** deve essere (ad esempio una funzione che modifica l'input in modo dipendente dal giorno della settimana).

Due dettagli completano il quadro delle macro di uguaglianza.

- **Nomenclatura**: in altri linguaggi e framework i parametri si chiamano `expected` e `actual` e l'ordine conta; in Rust si chiamano `left` e `right` e l'ordine è irrilevante, quindi `assert_eq!(4, result)` produce lo stesso fallimento di `assert_eq!(result, 4)`.
- **Trait richiesti**: sotto la superficie le macro usano gli operatori `==` e `!=`, e al fallimento stampano gli argomenti con la formattazione di debug; i valori confrontati devono quindi implementare `PartialEq` e `Debug`. Tutti i tipi primitivi e la maggior parte dei tipi della libreria standard li implementano; per le proprie struct ed enum basta l'annotazione `#[derive(PartialEq, Debug)]`.

Ogni macro di asserzione accetta infine **messaggi di fallimento personalizzati** come argomenti opzionali dopo quelli obbligatori: vengono passati alla macro `format!`, quindi si può usare una format string con segnaposto `{}`. Il caso tipico è verificare che l'output di una funzione `greeting` **contenga** il nome passato invece dell'uguaglianza esatta (i requisiti cambieranno e il test non va riscritto); il messaggio personalizzato riporta il valore effettivamente ricevuto e documenta cosa significa l'asserzione.

Verificare i panic con should_panic

Oltre ai valori di ritorno è importante verificare che il codice gestisca le condizioni di errore come previsto. L'attributo `#[should_panic]`, posto dopo `#[test]` e prima della funzione, fa passare il test se il codice nella funzione va in panic e fallire se non lo fa. L'esempio di riferimento è il tipo `Guess` del capitolo sugli errori: il resto del codice dipende dalla garanzia che le istanze contengano solo valori tra 1 e 100, quindi un test deve garantire che creare `Guess` con 200 vada in panic.

Un test `should_panic` da solo però è **impreciso**: passerebbe anche se il panic avvenisse per una ragione diversa da quella attesa. Per renderlo preciso si aggiunge il parametro opzionale `expected` all'attributo: il test harness verifica che il messaggio di fallimento contenga il testo fornito.

Warning

Precisione di expected

Il parametro `expected` riceve una sottostringa del messaggio di panic atteso: quanto specificare dipende da quanto il messaggio è unico o dinamico e dalla precisione desiderata. Se il codice scambia i corpi dei rami `if value < 1` e `else if value > 100`, il test fallisce mostrando che il panic è avvenuto ma il messaggio non conteneva la stringa attesa — ed è da lì che si inizia a cercare il bug.

Usare Result T E nei test

Tutti i test visti finora vanno in panic quando falliscono, ma si possono scrivere test che restituiscono `Result<T, E>`, restituendo `Ok(())` quando passano ed `Err` con una `String` dentro quando falliscono:

```rust
#[test]
fn it_works() -> Result<(), String> {
    if 2 + 2 == 4 { Ok(()) }
    else { Err(String::from("two plus two...")) }
}
```

Scrivere test che restituiscono `Result<T, E>` permette di usare l'operatore punto interrogativo nel corpo, un modo comodo per far fallire il test se una qualunque operazione al suo interno restituisce `Err`. Il contratto da ricordare è il seguente.

- **Incompatibilità**: l'annotazione `#[should_panic]` non si può usare su test che restituiscono `Result<T, E>`.
- **Alternativa**: per asserire che un'operazione restituisce una variante `Err` non si usa il punto interrogativo sul valore, ma `assert!(value.is_err())`.

Controllare l'esecuzione dei test

Come `cargo run` compila il codice e poi esegue il binario risultante, `cargo test` compila il codice in **modalità test** ed esegue il binario di test risultante. Il comportamento di default è eseguire tutti i test **in parallelo** e **catturare** l'output prodotto durante l'esecuzione, per rendere più leggibile l'output relativo ai risultati. Le opzioni a riga di comando si dividono in due gruppi: alcune vanno a `cargo test`, altre al binario di test; per separarle si elencano gli argomenti per `cargo test` seguiti dal separatore `--` e poi quelli per il binario. `cargo test --help` mostra le prime, `cargo test -- --help` le seconde.

Test in parallelo o in sequenza

Con più test, il default è l'esecuzione parallela su thread: i test finiscono prima e il feedback arriva prima. Ma proprio perché girano contemporaneamente, i test **non devono dipendere** l'uno dall'altro né da stato condiviso, incluso l'ambiente condiviso come la directory di lavoro corrente o le variabili d'ambiente. L'esempio del libro è emblematico.

- **Il problema**: se ogni test scrive un file `test-output.txt` e poi ne rilegge il contenuto per asserire un valore, un test può sovrascrivere il file tra la scrittura e la lettura di un altro; il secondo test fallisce non perché il codice è sbagliato ma perché i test si sono interferiti.
- **Le soluzioni**: far scrivere a ogni test un file diverso, oppure eseguire i test uno alla volta.

Per rinunciare al parallelismo o controllare il numero di thread si passa il flag `--test-threads` con il numero desiderato al binario di test: con `cargo test -- --test-threads=1` non si usa alcun parallelismo. L'esecuzione è più lenta, ma i test non si interferiscono se condividono stato.

Mostrare l'output delle funzioni

Di default la libreria di test cattura tutto ciò che viene stampato su standard output durante un test **passato**: una `println!` nel test non appare nel terminale, resta solo la riga che indica il superamento. Per un test fallito invece l'output stampato appare insieme al resto del messaggio di fallimento, che è prezioso per il debug. Per vedere i valori stampati anche dai test superati si usa il flag `--show-output`: `cargo test -- --show-output`.

Eseguire un sottoinsieme di test per nome

Eseguire la suite completa può richiedere tempo; lavorando su un'area specifica del codice conviene eseguire solo i test pertinenti. Le possibilità di selezione sono le seguenti.

- **Test singolo**: si passa il nome della funzione di test come argomento a `cargo test`; l'output mostra `2 filtered out` per indicare quanti test non sono stati eseguiti.
- **Filtro**: si può specificare **parte** di un nome: qualsiasi test il cui nome contiene quel valore viene eseguito (ad esempio `cargo test add` esegue tutti i test con `add` nel nome).
- **Un solo valore**: non si possono specificare più nomi separati in questo modo; viene usato solo il primo valore dato a `cargo test`.
- **Moduli**: il modulo in cui appare un test diventa parte del suo nome, quindi si possono eseguire tutti i test di un modulo filtrando sul nome del modulo.

Ignorare test a meno che non richiesti

Alcuni test possono essere molto costosi in termini di tempo e si vorrebbe escluderli dalla maggior parte delle esecuzioni. Invece di elencare come argomenti tutti i test da eseguire, si annotano i test costosi con l'attributo `ignore`:

```rust
#[test]
#[ignore]
fn expensive_test() { /* ... */ }
```

Con un test annotato così, `cargo test` esegue gli altri e riporta `expensive_test` come `ignored`. Le opzioni di esecuzione correlate sono due.

- `--ignored`: esegue **solo** i test ignorati, per quando arriva il momento di controllarne i risultati.
- `--include-ignored`: esegue tutti i test, ignorati e non.

Controllando quali test girano si ottengono risultati rapidi di `cargo test`, mantenendo la possibilità di eseguire i test lenti quando serve.

Organizzazione dei test

Il testing è una disciplina complessa e comunità diverse usano terminologia e organizzazione diverse; la comunità Rust pensa ai test in due categorie principali. Scriverle entrambe è importante per assicurarsi che i pezzi della libreria funzionino come attesi, separatamente e insieme.

- **Unit test**: piccoli e mirati, testano un modulo alla volta in isolamento e possono testare interfacce private.
- **Integration test**: interamente esterni alla libreria, la usano come farebbe qualsiasi altro codice esterno, sfruttando solo l'interfaccia pubblica e potenzialmente esercitando più moduli per test.

Unit test

Lo scopo degli unit test è testare ogni unità di codice isolata dal resto, per individuare rapidamente dove il codice funziona e dove no. Gli unit test vivono nella directory `src`, in ogni file insieme al codice che testano; la convenzione è creare un modulo `tests` in ogni file per contenere le funzioni di test.

Definition

Attributo cfg test

L'annotazione `#[cfg(test)]` sul modulo `tests` dice a Rust di compilare ed eseguire il codice di test solo quando si lancia `cargo test`, non con `cargo build`: risparmia tempo di compilazione e spazio nell'artefatto compilato. L'attributo `cfg` sta per *configuration* e indica che l'item va incluso solo in presenza di una data opzione di configurazione — in questo caso `test`, fornita da Rust per compilare ed eseguire i test. Vale per tutto il modulo, incluse le funzioni helper, non solo per le funzioni annotate `#[test]`.

Testare funzioni private

Nella comunità del testing c'è dibattito sul testare o meno direttamente le funzioni private, e altri linguaggi lo rendono difficile o impossibile; qualunque ideologia si segua, le regole di privacy di Rust **permettono** di testare le funzioni private. Il motivo è strutturale.

Info

I test sono solo codice Rust

I test sono codice Rust come qualsiasi altro e il modulo `tests` è un modulo come gli altri: come visto nel capitolo su pacchetti e moduli, gli item nei moduli figli possono usare gli item dei moduli antenati. Con `use super::*;` si porta in scope tutto ciò che appartiene al genitore del modulo `tests` e il test può chiamare la funzione non `pub` (ad esempio `internal_adder`). Nulla in Rust obbliga a testare le funzioni private, ma nulla lo impedisce.

Integration test

In Rust gli integration test sono interamente esterni alla libreria: la usano come qualsiasi altro codice, quindi possono chiamare solo le funzioni dell'**API pubblica**. Il loro scopo è verificare che molte parti della libreria funzionino insieme correttamente: unità di codice corrette singolarmente possono avere problemi una volta integrate, quindi la copertura del codice integrato conta quanto quella delle singole unità.

La directory tests

Gli integration test richiedono una directory `tests` al livello superiore del progetto, accanto a `src`: Cargo vi cerca i file di integration test e compila **ogni file come crate individuale**. Ne derivano alcune conseguenze pratiche.

- **Import esplicito**: ogni file della directory è una crate separata, quindi bisogna portare la libreria in scope con `use adder::add_two;` — cosa non necessaria negli unit test.
- **Nessuna annotazione**: in `tests/integration_test.rs` non serve `#[cfg(test)]`; Cargo tratta la directory in modo speciale e compila i suoi file solo quando si esegue `cargo test`.
- **Output a sezioni**: l'output ha tre sezioni — unit test, integration test, doc test — e ogni file di integration test ha la propria sezione; se un test in una sezione fallisce, le sezioni successive **non** vengono eseguite.
- **Selezioni**: si può eseguire una singola funzione di integration test per nome, oppure tutti i test di un file con `cargo test --test integration_test`.

Sottoinsiemi condivisi tra integration test

Ogni file della directory `tests` è una crate separata, il che è utile per creare scope distinti che imitano il modo in cui gli utenti finali useranno la nostra crate. Ma significa anche che i file in `tests` **non** condividono il comportamento dei file in `src`, dove il meccanismo dei moduli permette di condividere codice. Il caso tipico è un insieme di funzioni helper da usare in più file di integration test.

- **Il problema**: estraendo le helper in `tests/common.rs` (con una funzione `setup`), il file appare nell'output dei test con una propria sezione `running 0 tests`, pur non contenendo alcun test né essendo richiamato da nessuna parte

— non è ciò che volevamo.

- **La soluzione**: invece di `tests/common.rs` si crea `tests/common/mod.rs`; con questa convenzione di nomi Rust non tratta `common` come file di integration test, e i file nelle sottodirectory della directory `tests` non vengono compilati come crate separate né appaiono nell'output. Dai file di test si dichiara `mod common;` e si chiama `common::setup()`.

Integration test per crate binarie

Se un progetto è una crate binaria con solo `src/main.rs` e senza `src/lib.rs`, non si possono creare integration test che portino in scope le funzioni definite in `main.rs` con `use`: solo le crate di libreria espongono funzioni usabili da altre crate, le crate binarie sono pensate per essere eseguite da sole.

Info

Perché separare main dalla logica

Per questo i progetti Rust che forniscono un binario adottano la struttura con un `src/main.rs` snello che chiama la logica che vive in `src/lib.rs`: così gli integration test possono testare la crate di libreria con `use`, rendendo disponibile la funzionalità importante. Se quella funzionalità funziona, anche il poco codice in `main.rs` funzionerà — e quel poco codice non ha bisogno di test.

Sintesi della lezione

Le funzionalità di test di Rust forniscono un modo per specificare come il codice dovrebbe funzionare, così da assicurare che continui a lavorare come atteso anche mentre lo si modifica. Il quadro d'insieme è il seguente.

- **Unit test**: esercitano separatamente le diverse parti di una libreria e possono testare dettagli implementativi privati.
- **Integration test**: verificano che molte parti della libreria funzionino insieme correttamente, usando l'API pubblica come farà il codice esterno.
- **Ruolo dei test**: anche se il sistema di tipi e le regole di ownership prevengono alcune categorie di bug, i test restano importanti per ridurre i bug logici legati a come il codice dovrebbe comportarsi.

Capitolo 12

Un progetto di I/O

Questa lezione è un progetto guidato: costruire una versione semplificata del classico strumento di ricerca `grep` (***g**lobally search a **r**egular **e**xpression and **p**rint*), chiamata `minigrep`, che riceve un percorso di file e una stringa, legge il file e stampa le righe che contengono la stringa. La velocità, la safety, il singolo binario prodotto e il supporto multipiattaforma rendono Rust un linguaggio ideale per gli strumenti a riga di comando. Il progetto ricapitola molte competenze già acquisite e ne introduce di nuove: argomenti da riga di comando, lettura di file, variabili d'ambiente e scrittura degli errori su *standard error*. In particolare combina l'organizzazione del codice, vettori e stringhe, la gestione degli errori, trait e lifetime dove appropriato, e i test; introduce inoltre in anteprima closures, iteratori e trait object.

Accettare argomenti da riga di comando

Per far leggere a `minigrep` i valori degli argomenti passati da riga di comando serve la funzione `std::env::args` della libreria standard. Due dettagli bastano per usarla: gli iteratori producono una serie di valori, e chiamando il metodo `collect` su un iteratore lo si trasforma in una collezione, come un vettore, contenente tutti gli elementi prodotti:

```rust
use std::env;

let args: Vec<String> = env::args().collect();
```

Questa piccola firma esemplifica tre principi generali.

- **Import idiomatico**: `args` è annidata in due livelli di moduli (`std::env::args`), quindi si porta in scope il modulo genitore `std::env` anziché la funzione: così sono facilmente usabili anche le altre funzioni del modulo, e non c'è ambiguità su dove `args` sia definita.
- **Annotazione di tipo**: `collect` può creare molti tipi di collezioni, quindi è uno dei pochi casi in Rust in cui l'annotazione di tipo serve spesso, perché il compilatore non può inferire quale collezione si vuole.
- **Nome del programma**: il primo valore del vettore (indice 0) è il nome del binario stesso, come nella lista degli argomenti del C: ciò permette ai programmi di usare il nome con cui sono stati invocati. Gli argomenti utente iniziano dall'indice 1.

Info

Unicode non valido negli argomenti

`std::env::args` va in panic se un argomento contiene Unicode non valido; in tal caso si usa `std::env::args_os`, che restituisce un iteratore di valori `OsString` invece di `String`. Qui si usa `args` per semplicità, perché `OsString` varia per piattaforma ed è più complessa da gestire di `String`.

Leggere un file

La lettura del file specificato come argomento richiede il modulo `std::fs`: la funzione `fs::read_to_string` apre il file indicato dal percorso e restituisce un valore di tipo `std::io::Result<String>` che contiene il contenuto del file. Il tipo è un `Result` perché la lettura può fallire, e qui si annida il seme del problema di design che il resto della lezione risolve: la funzione `main` accumula responsabilità (analizza argomenti **e** legge file), e più responsabilità una funzione accumula, più è difficile ragionarci, testarla e modificarla senza romperne una parte. La buona pratica è rifattorizzare presto durante lo sviluppo, perché è molto più facile rifattorizzare piccole quantità di codice.

Rifattorizzare per modularità e gestione degli errori

La rifattorizzazione affronta quattro problemi concreti che è utile riconoscere in ogni progetto binario.

- **Responsabilità multiple**: `main` esegue due compiti; ogni funzione dev'essere responsabile di un solo compito.
- **Variabili confuse**: `query` e `file_path` sono variabili di **configurazione**, mentre `contents` serve alla **logica** del programma; più `main` cresce, più variabili entrano in scope e più è difficile seguirne lo scopo. Le variabili di

configurazione vanno raggruppate in una struttura.

- **Messaggi inutili**: l'uso di `expect` stampa lo stesso messaggio (`Should have been able to read the file`) per ogni modalità di fallimento della lettura (file mancante, permessi negati), senza dare informazioni all'utente.
- **Errori per programmatori**: con argomenti insufficienti l'utente riceve un `index out of bounds` da Rust, che non spiega il problema; tutto il codice di gestione degli errori dovrebbe stare in un solo posto, così i manutentori futuri abbiano un unico punto da consultare e i messaggi siano significativi per gli utenti finali.

Separazione delle responsabilità nei progetti binari

Il problema di allocare più compiti a `main` è comune a molti progetti binari; il processo di separazione delle responsabilità seguire questi passi.

- **Divisione**: separare il programma in un file `main.rs` e un file `lib.rs`, spostando la logica del programma in `lib.rs`.
- **Parsing piccolo**: finché la logica di analisi degli argomenti è piccola, può restare in `main`.
- **Parsing complicato**: quando cresce, estrarla da `main` in funzioni o tipi dedicati.

Le responsabilità che restano in `main` dopo il processo si limitano a quattro cose.

- **Chiamare** la logica di analisi degli argomenti con i valori ricevuti.
- **Configurare** qualsiasi altro aspetto.
- **Chiamare** una funzione `run` in `lib.rs`.
- **Gestire** l'errore se `run` ne restituisce uno.

Il pattern separa le preoccupazioni: `main.rs` gestisce l'esecuzione del programma, `lib.rs` gestisce tutta la logica del compito. Poiché non si può testare direttamente `main`, questa struttura permette di testare tutta la logica del programma spostandola fuori da `main`; il codice che resta in `main` è abbastanza piccolo da verificarne la correttezza leggendolo.

Raggruppare i valori di configurazione

L'estrazione di una funzione `parse_config` rivela che le due variabili restituite sono correlate e fanno parte di un'unica configurazione: restituire una tupla e poi subito scomporla è il segnale che l'astrazione non è ancora quella giusta. La soluzione è una struct `Config` con campi dal nome significativo, che comunica come i valori si relazionano e a cosa servono. Poiché `args` in `main` è l'owner dei valori e li presta solo in prestito alla funzione, `Config` non può prenderne l'ownership senza violare le regole di borrow: la via più semplice è chiamare `clone` sui valori, facendo una copia completa che `Config` possiede.

Info

Il trade-off di clone

Molti Rustaceani tendono a evitare `clone` per i suoi costi a runtime; tuttavia qui copiare poche stringhe una sola volta rende il codice molto più semplice, senza dover gestire i lifetime delle reference. Meglio un programma funzionante e un po' inefficiente che tentare di iper-ottimizzare al primo giro: l'esperienza renderà naturale partire dalla soluzione più efficiente.

Un costruttore per Config

Poiché lo scopo della funzione di parsing è creare un'istanza di `Config`, la forma più idiomatica è trasformarla in una funzione associata `new` dentro un blocco `impl`, come `String::new`: così si creano istanze con `Config::new`. Il passo successivo riguarda però la gestione degli errori: la tecnica del `panic!` (il controllo della lunghezza di `args`, come per il tipo `Guess` del capitolo sugli errori) dà un messaggio ragionevole ma accompagnato da informazioni superflue.

Warning

panic per i problemi di programmazione, Result per quelli d'uso

Una chiamata a `panic!` è appropriata per un problema di **programmazione**, non per un problema di **uso** del programma: l'utente che non passa abbastanza argomenti non merita un backtrace con `thread 'main'` e `RUST_BACKTRACE`. Per i problemi d'uso si restituisce un `Result` che indica successo o errore. Dato che molti programmatori si aspettano che le funzioni `new` non falliscano mai, la funzione viene anche rinominata `build`.

`Config::build` restituisce quindi un `Result` con un'istanza di `Config` nel caso di successo e una stringa letterale (con lifetime `'static`) nel caso d'errore: invece di chiamare `panic!` per argomenti insufficienti si restituisce un `Err`, e il `Config` viene avvolto in un `Ok`. La gestione lato `main` introduce due nuovi strumenti.

- `unwrap_or_else`: definito su `Result<T, E>` dalla libreria standard, permette una gestione degli errori personalizzata senza `panic!`; se il valore è `Ok` si comporta come `unwrap` restituendo il valore interno, se è `Err` chiama il codice della **closure** passata come argomento — una funzione anonima che riceve il valore interno di `Err` nell'argomento tra le barre verticali.

- `process::exit`: la closure stampa l'errore e chiama `process::exit(1)`, che ferma il programma immediatamente e restituisce il numero passato come **codice di stato di uscita**; uno stato di uscita diverso da zero è la convenzione per segnalare al processo chiamante che il programma è uscito con una condizione di errore.

Estrarre la logica in run e restituire Result

Tutta la logica rimanente (dalla lettura del file in poi) si estrae in una funzione `run` che riceve l'istanza di `Config`. Anche `run` migliora la gestione degli errori restituendo `Result<(), Box<dyn Error>>` al posto del panic di `expect`, con tre cambiamenti significativi.

- **Trait object**: il tipo di errore è `Box<dyn Error>`, dove `dyn` sta per *dynamic*: significa che la funzione restituisce un tipo che implementa il trait `Error` senza dover specificare quale tipo particolare, dando la flessibilità di restituire errori di tipi diversi in casi diversi.
- **Operatore `?`**: al posto di `expect`, l'operatore punto interrogativo restituisce il valore di errore dalla funzione corrente al chiamante, senza `panic!`.
- `Ok(())`: il tipo di successo è il tipo unit `()`, il modo idiomatico per indicare che si chiama `run` solo per i suoi effetti collaterali, senza un valore da restituire.

Se `main` ignorasse il `Result` di `run`, il compilatore emette un warning che ricorda il valore probabilmente segnala un errore non controllato. La gestione in `main` usa `if let` anziché `unwrap_or_else`, perché `run` non restituisce un valore da spacchettare: interessa solo rilevare l'errore, stamparlo e chiamare `process::exit(1)`. Questo è il secondo vantaggio della centralizzazione: i due posti che stampano errori si trovano entrambi in `main`, ed è lì che si decide come presentarli.

Dividere il codice in una crate di libreria

L'ultimo passo della modularizzazione sposta la logica di ricerca in `src/lib.rs`, marcando `pub` la funzione `search` per designarla parte dell'API pubblica della crate di libreria. La crate binaria la porta in scope con `use minigrep::search` e la chiama da `run`, che stampa ogni riga restituita con un ciclo `for`. Un dettaglio di design merita attenzione: la funzione raccoglie tutti i risultati in un vettore e lo restituisce **prima** di stampare, quindi con file grandi i risultati non appaiono man mano che vengono trovati — un limite che gli iteratori del capitolo successivo permetteranno di superare.

Aggiungere funzionalità con lo sviluppo guidato dai test

Definition

Sviluppo guidato dai test (TDD)

Il processo TDD applica ripetutamente questi passi: (1) scrivere un test che fallisce ed eseguirlo per verificare che fallisca per la ragione attesa; (2) scrivere o modificare il minimo codice necessario per farlo passare; (3) rifattorizzare il codice aggiunto o modificato verificando che i test continuino a passare. Scrivere il test prima del codice aiuta a mantenere un'alta copertura dei test lungo tutto il processo e guida il design del codice.

La teoria degna di nota in questa sezione riguarda la firma della funzione `search` e i suoi lifetime:

```rust
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str>
```

Il lifetime esplicito `'a` collega il parametro `contents` al valore di ritorno: il vettore restituito contiene string slice che referenziano porzioni di `contents`, **non** di `query`.

Warning

Non collegare tutti i parametri al ritorno

Senza annotazioni, il compilatore non può sapere quale dei due parametri serve per l'output, e il testo d'aiuto dell'errore suggerisce di specificare lo stesso parametro di lifetime per **tutti** i parametri e il tipo di ritorno — ma sarebbe sbagliato: `contents` è l'unico parametro che contiene il testo da cui si restituiscono le parti corrispondenti, quindi è l'unico che dev'essere collegato al valore di ritorno. Connettere argomenti e valori di ritorno nella firma non è richiesto dagli altri linguaggi, ma diventa naturale con la pratica.

Gli strumenti di implementazione sono minimi ed eleganti: il metodo `lines` delle stringhe restituisce un iteratore riga per riga; il metodo `contains` verifica se una riga contiene la query; un vettore mutabile con `push` accumula le righe corrispondenti, che vengono restituite alla fine.

Lavorare con le variabili d'ambiente

La funzionalità di ricerca senza distinzione maiuscole/minuscole si attiva con una variabile d'ambiente anziché con un'opzione da riga di comando: così l'utente la imposta una volta e tutte le ricerche di quella sessione di terminale ne beneficiano. La teoria sottostante copre tre punti.

- **Normalizzazione**: `search_case_insensitive` usa `to_lowercase` su query e righe prima del confronto; `to_lowercase` **crea dati nuovi** (una `String` allocata, non una slice esistente — `"rUsT"` non contiene una `u` minuscola da riusare), e la variabile va quindi ombregliata con shadowing; passandola a `contains` serve la `&` perché la firma del metodo accetta una string slice. La gestione Unicode di `to_lowercase` è basilare, non accurata al 100%.
- **Lettura della variabile**: la funzione `env::var` del modulo `std::env` restituisce un `Result`: variante `Ok` con il valore se la variabile d'ambiente è impostata a qualsiasi valore, variante `Err` se non è impostata. Con il metodo `is_ok` si controlla se la variabile è impostata: non interessa il **valore**, solo se è impostata o meno, quindi `is_ok` è più appropriato di `unwrap` o `expect`.
- **Configurazione**: il booleano `ignore_case` entra nella struct `Config` e la funzione `run` lo usa per scegliere tra `search` e `search_case_insensitive`.

Quando un programma accetta la stessa configurazione sia da argomento sia da variabile d'ambiente, decide quale delle due ha la precedenza; il modulo `std::env` contiene molte altre funzionalità per le variabili d'ambiente.

Reindirizzare gli errori sullo standard error

Definition

Standard output e standard error

Nei terminali esistono due tipi di output: lo *standard output* (`stdout`) per le informazioni generali e lo *standard error* (`stderr`) per i messaggi d'errore. La distinzione permette all'utente di dirigere l'output di successo di un programma verso un file mantenendo i messaggi d'errore sullo schermo.

La macro `println!` stampa solo su `stdout`, quindi per stampare su `stderr` serve la macro `eprintln!` della libreria standard. Il test comportamentale è istruttivo: ridirezionando lo standard output su un file con la sintassi `cargo run > output.txt` e provocando un errore (nessun argomento), il messaggio finisce **nel file** — il programma non è ben comportato. Sostituendo le due stampe d'errore con `eprintln!`, il messaggio appare sullo schermo e il file resta vuoto; con argomenti validi, i risultati finiscono nel file e il terminale non mostra nulla. È il comportamento atteso dagli strumenti a riga di comando: `stdout` per l'output di successo, `stderr` per gli errori — ed è la rifattorizzazione di questa lezione a renderlo banale, perché tutto il codice che stampa errori è concentrato in `main`.

Sintesi della lezione

Il capitolo ha ricapitolato i concetti principali già appresi e mostrato come eseguire le operazioni di I/O comuni in Rust. Il quadro finale è il seguente.

- **Struttura**: `main.rs` snello con la sola esecuzione, `lib.rs` con tutta la logica testabile.
- **Errori**: `Result` e `?` per i problemi d'uso, `process::exit` con codice diverso da zero, messaggi centralizzati e stampati su `stderr`.
- **Input**: argomenti da riga di comando con `env::args` e `collect`, file con `fs::read_to_string`, configurazione raggruppata in una struct con costruttore che restituisce `Result`.
- **Qualità**: TDD per guidare l'implementazione, lifetime espliciti per collegare i dati letti ai risultati restituiti.

Combinato con i capitoli precedenti, il codice risultante è ben organizzato, memorizza i dati nelle strutture appropriate, gestisce bene gli errori ed è ben testato.

Capitolo 13

Closure e iteratori

Il design di Rust attinge a molti linguaggi e tecniche esistenti, e una influenza significativa è la **programmazione funzionale**: programmare in stile funzionale include usare le funzioni come valori, passandole come argomenti, restituendole da altre funzioni, assegnandole a variabili per un'esecuzione successiva. Questa lezione non dibatte cosa sia o non sia la programmazione funzionale, ma tratta le feature di Rust simili a quelle dei linguaggi comunemente definiti funzionali: le **closure**, un costrutto simile a una funzione che si può memorizzare in una variabile, e gli **iteratori**, un modo di elaborare serie di elementi; segue il loro uso per migliorare il progetto di I/O e l'analisi delle prestazioni (che sono migliori di quanto si possa pensare). Anche il pattern matching e le enum, già viste, sono influenzate dallo stile funzionale; padroneggiare closure e iteratori è una parte importante dello scrivere codice Rust veloce e idiomatico.

Closure

Definition

Closure

Le closure di Rust sono funzioni anonime che si possono salvare in una variabile o passare come argomenti ad altre funzioni. Si possono creare in un posto e chiamare in un altro per valutarle in un contesto diverso. A differenza delle funzioni, le closure possono **catturare** valori dallo scope in cui sono definite, il che permette riuso di codice e personalizzazione del comportamento.

Catturare l'ambiente

L'esempio di riferimento è una azienda di magliette che regala una maglietta limitata a un membro della lista di distribuzione: se il vincitore ha un colore preferito nel profilo riceve quello, altrimenti riceve il colore più presente in magazzino. Il metodo `giveaway` riceve la preferenza come `Option<ShirtColor>` e chiama su di essa il metodo `unwrap_or_else` definito dalla libreria standard su `Option<T>`, che prende un argomento: una closure senza parametri che restituisce un valore `T`. Il comportamento è semplice.

- **Some**: `unwrap_or_else` restituisce il valore dentro la variante, senza valutare la closure.
- **None**: `unwrap_or_else` chiama la closure e restituisce il valore che essa produce.

L'aspetto interessante è la closure passata, `|| self.most_stocked()`: tra le barre verticali non ci sono parametri (se ne avesse, comparirebbero lì), e il corpo chiama il metodo sull'istanza `Inventory` corrente. La libreria standard non sa nulla dei tipi `Inventory` o `ShirtColor` né della logica dello scenario: la closure cattura un reference immutabile a `self` e lo passa insieme al codice specificato. Le funzioni, al contrario, **non** possono catturare l'ambiente in questo modo.

Inferire e annotare i tipi delle closure

Le closure di solito non richiedono l'annotazione dei tipi dei parametri o del valore di ritorno come le funzioni `fn`. La differenza è di ruoli: i tipi sono richiesti sulle funzioni perché fanno parte di un'interfaccia esplicita esposta agli utenti, e definirla rigidamente garantisce l'accordo sui tipi usati e restituiti; le closure invece non sono usate in un'interfaccia esposta, sono memorizzate in variabili e usate senza nomi. Sono tipicamente brevi e rilevanti solo in un contesto ristretto, e in quel contesto limitato il compilatore può dedurre i tipi come fa per le variabili. Le quattro forme equivalenti mostrano quanta sintesi è possibile:

```rust
fn  add_one_v1   (x: u32) -> u32 { x + 1 }
let add_one_v2 = |x: u32| -> u32 { x + 1 };
let add_one_v3 = |x|             { x + 1 };
let add_one_v4 = |x|               x + 1  ;
```

La prima riga è una funzione, la seconda una closure pienamente annotata, la terza senza annotazioni di tipo, la quarta senza parentesi graffi (facoltative perché il corpo è una sola espressione). Due precisazioni completano il quadro dell'inferenza.

- **Valutazione necessaria**: le versioni non annotate richiedono che la closure sia valutata per compilare, perché i

tipi vengono dedotti dall'uso — come `Vec::new()` che ha bisogno di annotazioni o di valori inseriti perché il tipo sia dedotto.
- **Tipi bloccati**: il compilatore deduce **un solo tipo concreto** per ogni parametro e per il valore di ritorno; una closure non annotata chiamata una prima volta con una `String` blocca quei tipi, e chiamarla poi con un intero è un errore di tipo.

Catturare reference o trasferire ownership

Le closure catturano valori dall'ambiente in tre modi, che mappano direttamente i tre modi in cui una funzione può ricevere un parametro.

- **Prestito immutabile**: una closure che si limita a stampare un vettore cattura un reference immutabile; altri prestiti immutabili restano possibili prima e dopo la sua definizione e chiamata.
- **Prestito mutabile**: una closure che aggiunge un elemento al vettore cattura un reference mutabile; tra la definizione e la chiamata nessun altro prestito è permesso (il prestito mutabile esclude gli altri), quindi un `println!` in mezzo non compilerebbe — il prestito termina quando la closure viene chiamata e non più usata.
- **Ownership**: con la parola chiave `move` prima della lista dei parametri si forza la closure a prendere l'ownership dei valori che usa, anche quando il corpo non la richiederebbe strettamente.

Info

Quando serve move

La tecnica è utile soprattutto nel passare una closure a un **nuovo thread**, per spostare i dati così che siano posseduti dal thread nuovo. Se la closure del thread catturasse solo un reference immutabile e il thread principale terminasse per primo droppando il dato, il reference nel thread diventerebbe invalido; il compilatore quindi **richiede** che il dato sia spostato dentro la closure passata al thread, perché il reference resti valido. La scelta del modo di cattura è comunque guidata dal corpo: la closure usa il minimo accesso necessario, salvo forzature esplicite.

Spostare valori catturati fuori dalle closure

Dopo la cattura (che determina cosa entra nella closure), è il corpo a stabilire cosa succede a reference e valori quando la closure viene valutata (cioè cosa ne esce): il corpo può spostare un valore catturato fuori dalla closure, mutarlo, né spostarlo né mutarlo, o non catturare nulla. Il modo in cui una closure cattura e gestisce i valori determina quali trait implementa — e i trait sono il meccanismo con cui funzioni e struct specificano che tipo di closure possono usare. Ogni closure implementa automaticamente uno, due o tutti e tre i trait `Fn`, in modo additivo.

- **`FnOnce`**: si applica alle closure che possono essere chiamate **una volta**; tutte le closure implementano almeno questo trait, perché tutte possono essere chiamate. Una closure che **sposta** valori catturati fuori dal proprio corpo implementa **solo** `FnOnce`, perché dopo lo spostamento non può essere chiamata di nuovo.
- **`FnMut`**: si applica alle closure che non spostano valori catturati fuori dal corpo ma potrebbero **mutarli**; possono essere chiamate più di una volta.
- **`Fn`**: si applica alle closure che né spostano né mutano i valori catturati, incluse quelle che non catturano nulla; possono essere chiamate più volte senza mutare l'ambiente, il che è importante ad esempio per chiamate concorrenti multiple.

I due esempi della libreria standard mostrano come i trait bound esprimono vincoli comportamentali.

- **`unwrap_or_else`**: la firma è `pub fn unwrap_or_else<F>(self, f: F) -> T where F: FnOnce() -> T`, con `F` tipo generico per la closure; il bound `FnOnce() -> T` esprime il vincolo che la closure dev'essere chiamabile una volta senza argomenti e restituire un `T` — e che `unwrap_or_else` **non la chiamerà più di una volta** (in corpo: con `Some` non la chiama, con `None` la chiama una volta sola). Poiché tutte le closure implementano `FnOnce`, il metodo accetta tutti e tre i tipi di closure: la massima flessibilità.
- **`sort_by_key`**: definito sugli slice, prende una closure che riceve un reference all'item corrente e restituisce un valore `K` ordinabile (ad esempio ordinare rettangoli per `width`); chiama la closure **una volta per ogni elemento**, quindi il suo bound è `FnMut`, non `FnOnce`.

Da qui la conseguenza pratica: una closure che sposta un valore fuori dall'ambiente (come quella che fa `push` di una `String` catturata dentro un vettore) implementa solo `FnOnce` e **non compila** con `sort_by_key` — può essere chiamata una sola volta, perché alla seconda chiamata il valore non è più nell'ambiente. Il modo corretto di contare le chiamate è tenere un **contatore** nell'ambiente e incrementarlo nel corpo: così la closure cattura solo un reference mutabile e implementa `FnMut`, compatibile con il bound.

Info

Funzioni al posto delle closure

Se ciò che si vuole fare non richiede catturare valori dall'ambiente, si può usare il **nome di una funzione** al posto di una closure dove serve qualcosa che implementi uno dei trait `Fn`: su un `Option<Vec<T>>` si può scrivere `unwrap_or_else(Vec::new)`, perché il compilatore implementa automaticamente per ogni definizione di funzione tutti i trait `Fn` ad essa applicabili.

Elaborare serie di elementi con gli iteratori

Il pattern **iterator** permette di eseguire un compito su una sequenza di elementi a turno: l'iteratore è responsabile della logica dell'iterazione su ogni elemento e della determinazione di quando la sequenza è finita, così non si deve reimplementare quella logica.

Definition

Pigrizia degli iteratori

Gli iteratori in Rust sono **pigri** (*lazy*): non hanno alcun effetto finché non si chiamano metodi che li consumano. La chiamata al metodo `iter` su un vettore, da sola, non fa nulla di utile; crea solo l'iteratore. Il ciclo `for` visto nel capitolo sul flusso di controllo creava e consumava implicitamente un iteratore sotto il cofano.

In linguaggi senza iteratori nella libreria standard, la stessa funzionalità richiederebbe una variabile indice da 0, l'indicizzazione nel vettore e l'incremento in un ciclo fino al totale degli elementi; gli iteratori gestiscono tutta questa logica, riducendo il codice ripetitivo in cui si può sbagliare e dando la flessibilità di usare la stessa logica con molti tipi di sequenza, non solo strutture indicizzabili come i vettori.

Il trait Iterator e il metodo next

Tutti gli iteratori implementano il trait `Iterator` della libreria standard, la cui definizione introduce una sintassi nuova:

```rust
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}
```

La sintassi `type Item` e `Self::Item` definisce un **tipo associato** al trait (trattato nel capitolo sulle feature avanzate): implementare `Iterator` richiede di definire anche un tipo `Item`, che è il tipo restituito dall'iteratore. Il trait richiede l'implementazione di un solo metodo, `next`, che restituisce un elemento per volta avvolto in `Some` e `None` quando l'iterazione è terminata. Tre osservazioni derivano dall'uso diretto di `next`.

- **Mutabilità**: l'iteratore va dichiarato `mut`, perché chiamare `next` cambia lo stato interno con cui tiene traccia della posizione nella sequenza; ogni chiamata **consuma** un elemento. Con il ciclo `for` la mutabilità non serve, perché il ciclo prende l'ownership dell'iteratore e lo rende mutabile dietro le quinte.
- **Tre varianti**: `iter` produce un iteratore su reference immutabili; `into_iter` prende l'ownership e restituisce valori posseduti; `iter_mut` itera su reference mutabili.
- **Consumo**: `next` restituisce reference immutabili ai valori del vettore creato con `iter`, e gli elementi non tornano: l'iteratore si esaurisce.

Metodi che consumano l'iteratore

Il trait `Iterator` ha molti metodi con implementazioni di default fornite dalla libreria standard; alcuni chiamano `next` nella loro definizione, ed è per questo che l'implementazione del trait richiede proprio `next`. I metodi che chiamano `next` si chiamano **consuming adapters**, perché chiamarli esaurisce l'iteratore. L'esempio è `sum`, che prende l'ownership dell'iteratore, accumula ogni elemento in un totale chiamando ripetutamente `next` e restituisce il totale al termine: dopo la chiamata a `sum`, l'iteratore non è più utilizzabile.

Metodi che producono altri iteratori

Gli **iterator adapter** sono metodi definiti sul trait `Iterator` che **non** consumano l'iteratore: ne producono uno diverso cambiando qualche aspetto dell'originale. L'esempio è `map`, che prende una closure da chiamare su ogni elemento e restituisce un nuovo iteratore che produce gli elementi modificati:

```rust
let v2: Vec<i32> = v1.iter().map(|x| x + 1).collect();
```

Chiamare solo `map` però produce un warning e non fa nulla: la closure specificata non viene mai chiamata, perché gli adapter sono pigri e serve consumare l'iteratore. Il metodo che chiude il cerchio è `collect`, già incontrato con `env::args`: consuma l'iteratore e raccoglie i valori risultanti in una collezione. Poiché `map` prende una closure, si può

specificare qualsiasi operazione su ogni elemento: le closure permettono di personalizzare il comportamento riusando la logica di iterazione del trait `Iterator`. Si possono anche **concatenare** più chiamate a iterator adapter per eseguire azioni complesse in modo leggibile, ma essendo tutti pigri, serve sempre una chiamata a un consuming adapter per ottenere i risultati.

Closure che catturano il loro ambiente

Molti iterator adapter prendono closure come argomenti, e comunemente quelle passate sono closure che catturano l'ambiente. L'esempio è `filter`, che prende una closure che riceve un elemento e restituisce un Boolean: se restituisce `true`, il valore è incluso nell'iterazione prodotta da `filter`; se `false`, è escluso. Una funzione `shoes_in_size` usa `into_iter` per prendere l'ownership del vettore di scarpe, `filter` con una closure che cattura dal proprio ambiente il parametro `shoe_size` e confronta con la misura di ogni scarpa, e `collect` per raccogliere i risultati in un vettore da restituire: la sequenza completa — creazione, adattamento, consumo — in tre chiamate.

Migliorare il progetto di I/O

Con le nuove conoscenze sugli iteratori si possono rendere più chiari e concisi punti del progetto `minigrep`.

Rimuovere un clone con un iteratore

La funzione `Config::build` del capitolo precedente prendeva in prestito uno slice di `String` e clonava i valori per farne possedere i campi all'istanza di `Config`; il `clone` serviva proprio perché `build` non possedeva `args`. Con gli iteratori la firma cambia radicalmente: `build` prende l'**ownership di un iteratore** come argomento, il tipo del parametro diventa `mut args: impl Iterator<Item = String>` — la sintassi `impl Trait` già vista, qui con l'annotazione del tipo associato `Item = String` — e il `mut` serve perché iterare muta l'iteratore. In `main` si passa direttamente il valore di ritorno di `env::args` (che **è** un iteratore) senza raccoglierlo in un vettore. Nei corpo, il metodo `next` sostituisce i controlli di lunghezza e l'indicizzazione.

- **Primo valore ignorato**: il primo elemento restituito da `env::args` è il nome del programma; si chiama `next` e si scarta il risultato.
- **Estrazione con match**: per `query` e `file_path` si chiama `next` e si fa un `match` sul `Result`: `Some` estrae il valore da spostare in `Config`, `None` significa argomenti insufficienti e si esce subito con un `Err`.
- **Niente più cloni**: prendendo l'ownership dell'iteratore invece di prendere in prestito lo slice, i valori `String` si possono **spostare** dall'iteratore dentro `Config`, eliminando `clone` e le nuove allocazioni.

Chiarire il codice con gli iterator adapter

Anche la funzione `search` si riscrive con gli adapter: la versione con ciclo `for` esplicito e vettore intermedio mutabile diventa una concatenazione di `filter` (che tiene solo le righe per cui `line.contains(query)` restituisce `true`) e `collect` (che raccoglie le righe corrispondenti in un vettore). I benefici di stile e di design sono tre.

- **Stato mutabile minimo**: lo stile funzionale preferisce minimizzare lo stato mutabile, e togliere il vettore `results` mutabile rende il codice più chiaro.
- **Possibilità future**: eliminare lo stato condiviso aprirebbe la strada a una ricerca **parallela**, senza dover gestire accessi concorrenti al vettore dei risultati.
- **Pigrizia sfruttata**: un ulteriore miglioramento è restituire un iteratore da `search`, cambiando il tipo di ritorno in `impl Iterator<Item = &'a str>` e togliendo `collect`: la funzione diventa essa stessa un iterator adapter, e il ciclo `for` in `run` sfrutta la pigrizia dell'iteratore stampando ogni riga **appena trovata**, anziché raccogliere tutto prima di stampare — una differenza visibile su file grandi.

Scegliere tra cicli e iteratori

Quale stile preferire tra il ciclo `for` esplicito e la versione con iteratori? La maggior parte dei programmatori Rust preferisce lo stile con iteratori: è più difficile da padroneggiare all'inizio, ma una volta familiari gli adapter, il codice si concentra sull'**obiettivo di alto livello** del ciclo anziché sui dettagli dell'indicizzazione e della costruzione di vettori — l'astrazione mette in risalto i concetti specifici del codice, come la condizione di filtraggio.

Prestazioni nei cicli vs negli iteratori

L'intuizione suggerisce che il ciclo di basso livello sia più veloce; il benchmark la smentisce. Cercando la parola *the* nell'intero testo di *The Adventures of Sherlock Holmes* caricato in una `String`, la versione con ciclo `for` e quella con iteratori mostrano prestazioni sostanzialmente equivalenti, con la versione a iteratori leggermente più rapida.

Definition

Astrazioni a costo zero

Gli iteratori sono una delle **zero-cost abstractions** di Rust: usare l'astrazione non impone alcun overhead a runtime. In molti casi il codice con iteratori compila nello **stesso assembly** che si scriverebbe a mano, perché si applicano ottimizzazioni come il loop unrolling e l'eliminazione del controllo dei limiti sugli accessi agli array.

> *"In generale, le implementazioni C++ obbediscono al principio dell'overhead zero: ciò che non usi, non lo paghi. E inoltre: ciò che usi, non potresti scriverlo meglio a mano."* (Foundations of C++, *keynote ETAPS 2012)*
>
> – Bjarne Stroustrup

Le closure e gli iteratori sono feature di Rust ispirate alle idee dei linguaggi funzionali e contribuiscono alla capacità di Rust di esprimere chiaramente idee di alto livello con prestazioni di basso livello: le loro implementazioni non influiscono sulle prestazioni a runtime, parte dell'obiettivo di Rust di fornire astrazioni a costo zero.

Parte V

Ecosistema e smart pointers

Capitolo 14

Di più su Cargo e crates.io

Finora abbiamo usato solo le funzionalità di base di Cargo per compilare, eseguire e testare il codice, ma può fare molto di più. Questa lezione tratta le funzionalità avanzate: personalizzare la compilazione con i *release profile*, pubblicare librerie su crates.io, organizzare progetti grandi con i *workspace*, installare binari da crates.io ed estendere Cargo con comandi personalizzati. Il quadro d'insieme è importante per capire il ruolo di Cargo nell'ecosistema Rust.

Personalizzare la compilazione con i release profile

Definition

Release profile

I *release profile* sono profili predefiniti e personalizzabili con configurazioni diverse, che danno al programmatore più controllo sulle opzioni di compilazione del codice. Ogni profilo è configurato indipendentemente dagli altri.

Cargo ha due profili principali, già incontrati nell'output delle compilazioni.

- `dev`: usato da `cargo build`, con default pensati per lo sviluppo (output `unoptimized + debuginfo`).
- `release`: usato da `cargo build --release`, con default pensati per il rilascio (output `optimized`).

Ogni profilo ha impostazioni di default che si applicano quando nel file *Cargo.toml* del progetto non ci sono sezioni `[profile.*]`. Aggiungendo una sezione `[profile.*]` si **soprascrive un sottoinsieme** qualsiasi dei default, senza ridefinire tutto. L'impostazione di riferimento è `opt-level`, che controlla il numero di ottimizzazioni applicate al codice su una scala da 0 a 3: il meccanismo illustra il trade-off generale della compilazione.

- **Default di `dev`**: `opt-level = 0`; durante lo sviluppo si compila spesso e serve velocità di compilazione, accettando un codice che gira più lentamente.
- **Default di `release`**: `opt-level = 3`; in rilascio conviene spendere più tempo a compilare perché la compilazione avviene una volta sola, mentre il programma verrà eseguito molte volte: si scambia tempo di compilazione con codice più veloce.

Qualunque default si può soprascrivere aggiungendo un valore diverso in *Cargo.toml*, ad esempio `[profile.dev]` con `opt-level = 1` per applicare in sviluppo più ottimizzazioni del default ma meno di un build di rilascio.

Pubblicare una crate su crates.io

Abbiamo usato pacchetti di crates.io come dipendenze dei nostri progetti, ma si può anche condividere il proprio codice pubblicando pacchetti propri. Il registro delle crate distribuisce il **codice sorgente** dei pacchetti, quindi ospita essenzialmente codice open source. Rust e Cargo hanno funzionalità che rendono i pacchetti pubblicati più facili da trovare e usare.

Commenti di documentazione utili

Documentare accuratamente i pacchetti aiuta gli altri utenti a sapere come e quando usarli. Oltre ai commenti con `//` già visti, Rust ha un tipo particolare di commento per la documentazione che genera documentazione HTML, mostrando il contenuto per gli item dell'API pubblica: interessa a chi vuole **usare** la crate, non a chi vuole studiarne l'implementazione. Le caratteristiche dei *documentation comment* sono le seguenti.

- **Sintassi**: tre slash `///` invece di due, con supporto alla notazione Markdown per formattare il testo.
- **Posizione**: vanno collocati immediatamente prima dell'item che documentano.
- **Generazione**: `cargo doc` esegue lo strumento `rustdoc` distribuito con Rust e mette l'HTML nella directory *target/doc*; `cargo doc --open` compila anche la documentazione di tutte le dipendenze e apre il risultato nel browser.

Gli autori di crate usano comunemente alcune sezioni nella documentazione, che funzionano da checklist degli aspetti che interessano gli utenti.

- **Panics**: gli scenari in cui la funzione potrebbe andare in panic; i chiamanti che non vogliono panic devono evitare di chiamarla in quelle situazioni.

- **Errors**: se la funzione restituisce un `Result`, descrivere i tipi di errore possibili e le condizioni che li causano, così i chiamanti possono gestirli in modi diversi.
- **Safety**: se la funzione è `unsafe` da chiamare, una sezione che spieghi perché è insicura e gli invarianti che il chiamante deve mantenere.

I commenti di documentazione come test

Aggiungere blocchi di codice di esempio nei commenti di documentazione ha un bonus: `cargo test` **esegue gli esempi come test** (la sezione *Doc-tests* dell'output già vista nella lezione sui test). Niente è meglio di una documentazione con esempi, e niente è peggio di esempi che non funzionano perché il codice è cambiato da quando la documentazione è stata scritta: se la funzione o l'esempio cambiano fino a far fallire un `assert_eq!`, i doc test rilevano che esempio e codice sono fuori sincrono.

Commenti sull'item che li contiene

Lo stile di commento `//!` aggiunge documentazione **all'item che contiene** i commenti, anziché all'item che li segue. Si usa tipicamente nel file radice della crate (*src/lib.rs* per convenzione) o dentro un modulo, per documentare la crate o il modulo nel suo insieme: i commenti `//!` all'inizio di *src/lib.rs* descrivono l'intera crate e appaiono nella pagina iniziale della documentazione, sopra l'elenco degli item pubblici. Questi commenti sono utili per spiegare lo scopo complessivo del contenitore e aiutare gli utenti a capire l'organizzazione della crate.

Esportare un'API pubblica conveniente

La struttura dell'API pubblica è una considerazione importante quando si pubblica: chi usa la crate conosce la struttura meno di chi l'ha scritta e può avere difficoltà a trovare i pezzi che cerca se la gerarchia dei moduli è profonda. La struttura che ha senso durante lo sviluppo può non essere conveniente per gli utenti, costretti a `use my_crate::some_module::another_module::UsefulType;` anziché `use my_crate::UsefulType;`.

Definition

Re-export con pub use

Il re-export con `pub use` prende un item pubblico in una posizione e lo rende pubblico in un'altra, come se fosse definito lì. Permette di creare una struttura pubblica diversa da quella interna **senza riorganizzare il codice**: si aggiungono `pub use` che riportano i tipi al livello superiore, dove compaiono nella pagina iniziale della documentazione. Gli utenti possono ancora usare la struttura interna se preferiscono.

Due usi complementari completano il quadro.

- **Dipendenze**: un uso comune di `pub use` è ri-esportare le definizioni di una dipendenza nella crate corrente, rendendole parte della propria API pubblica.
- **Iterazione**: creare una struttura di API pubblica utile è più un'arte che una scienza; `pub use` dà flessibilità su come strutturare la crate internamente e **disaccoppia** la struttura interna da quella presentata agli utenti.

Configurare un account crates.io

Prima di pubblicare occorre creare un account su crates.io e ottenere un API token, effettuando l'accesso con un account GitHub (al momento è un requisito). La procedura tecnica è minimale: si visita la pagina delle impostazioni dell'account, si recupera la chiave e si esegue `cargo login` incollando il token quando richiesto. Il comando informa Cargo del token e lo memorizza localmente in *~/.cargo/credentials.toml*.

Warning

Il token è un segreto

Il token va tenuto segreto e non condiviso con nessuno; se per qualsiasi motivo è stato condiviso, va revocato immediatamente e rigenerato su crates.io.

Aggiungere metadati a una nuova crate

Prima di pubblicare serve aggiungere metadati nella sezione `[package]` di *Cargo.toml*. I punti chiave sono i seguenti.

- **Nome unico**: localmente una crate si può chiamare come si vuole, ma i nomi su crates.io sono assegnati al primo che arriva (*first-come, first-served*): il nome occupato non è più pubblicabile da altri, quindi conviene cercare il nome desiderato prima di tentare la pubblicazione.
- **Campi obbligatori**: `cargo publish` senza `description` e `license` fallisce con un errore del registro; la descrizione è una o due frasi perché appare nei risultati di ricerca, la licenza dichiara i termini d'uso.
- **Identificatori di licenza**: il campo `license` richiede un *license identifier value* secondo gli identificatori SPDX della Software Package Data Exchange della Linux Foundation, ad esempio `MIT`; per una licenza non presente nella lista SPDX si usa `license-file` col nome di un file contenente il testo.
- **Licenze multiple**: si possono specificare più identificatori separati da `OR`, come in `MIT OR Apache-2.0` — la doppia licenza usata dallo stesso Rust, pratica comune nella comunità.

Pubblicare su crates.io

Con account, token, nome unico e metadati completi, `cargo publish` carica una versione specifica della crate sul registro perché altri la usino come dipendenza.

Warning

La pubblicazione è permanente

Una versione pubblicata **non può mai essere sovrascritta** e il codice non può essere cancellato se non in circostanze particolari. L'obiettivo di crates.io è fare da archivio permanente, così le build di tutti i progetti che dipendono da essa continuano a funzionare: permetterne la cancellazione renderebbe impossibile questo obiettivo. Il numero di versioni pubblicabili però non ha limiti.

Per pubblicare una nuova versione di una crate esistente si cambia il valore `version` in *Cargo.toml* usando le regole del *Semantic Versioning* per decidere il numero appropriato in base ai tipi di cambiamenti fatti, e si esegue di nuovo `cargo publish`.

Deprecare versioni con cargo yank

Benché le versioni precedenti non si possano rimuovere, si può impedire ai progetti futuri di aggiungerle come nuove dipendenze: è utile quando una versione è rotta per qualche motivo. Il comando è `cargo yank --vers`, con le semantica seguenti.

- **Effetto**: lo *yank* impedisce ai **nuovi** progetti di dipendere da quella versione mentre tutti i progetti **esistenti** che la usano continuano a funzionare; in pratica i progetti con un *Cargo.lock* non si rompono, e i *Cargo.lock* generati in futuro non useranno la versione ritirata.
- **Annullamento**: aggiungendo `--undo` al comando si annulla lo yank e i progetti possono tornare a dipendere da quella versione.
- **Limiti**: uno yank **non cancella alcun codice**; non può ad esempio eliminare segreti caricati per errore, che in quel caso vanno resettati immediatamente.

Workspace con Cargo

Quando il progetto cresce e la crate di libreria diventa grande, si può voler dividere ulteriormente il pacchetto in più crate di libreria; la funzionalità di Cargo che aiuta a gestire più pacchetti correlati sviluppati insieme è il *workspace*.

Definition

Workspace

Un workspace è un insieme di pacchetti che condividono lo stesso *Cargo.lock* e la stessa directory di output. I pacchetti sono detti membri e la struttura tipica prevede una crate binaria che dipende da più crate di libreria, sviluppate in tandem.

La struttura di riferimento si costruisce così.

- **Cargo.toml radice**: nel workspace creato ad hoc la directory contiene un *Cargo.toml* che configura l'intero workspace: **senza** sezione `[package]**, inizia con una sezione`[workspace]`dove si aggiungono i membri; è buona pratica impostare il`resolver' all'algoritmo più recente.
- **Membri automatici**: eseguendo `cargo new` dentro la directory del workspace, il pacchetto creato viene **automaticamente aggiunto** alla lista `members` nel *Cargo.toml* radice.
- **Target condiviso**: il workspace ha una sola directory *target* al livello superiore dove finiscono gli artefatti compilati; anche eseguendo `cargo build` da dentro la directory di un membro, gli artefatti vanno nella *target* del workspace e non in una propria. La condivisione esiste perché le crate del workspace sono pensate per dipendere l'una dall'altra: se ogni crate avesse la propria *target*, dovrebbe ricompilare tutte le altre; condividendone una si evitano ricostruzioni inutili.

Dipendenze tra i membri e da pacchetti esterni

Le regole di dipendenza nel workspace sono esplicite e precise.

- **Dipendenze interne**: Cargo **non presuppone** che le crate del workspace dipendano l'una dall'altra; la relazione va dichiarata esplicitamente con una *path dependency* nel *Cargo.toml* del membro che usa l'altra (ad esempio `adder` che dipende da `add_one`).
- **Esecuzione mirata**: dal livello superiore si sceglie quale pacchetto eseguire con `cargo run -p adder` o testare con `cargo test -p add_one`, usando il flag `-p` col nome del membro.

- **Un solo Cargo.lock**: il workspace ha un solo *Cargo.lock* al livello superiore anziché uno per crate: aggiungendo `rand` ai *Cargo.toml* di due membri, Cargo risolve entrambi a **una sola versione** e la registra nel lock unico, garantendo che le crate usino sempre versioni compatibili tra loro.
- **Nessuna propagazione automatica**: anche se `rand` è usato da qualche parte del workspace, un altro membro **non può** usarlo se non lo dichiara nel proprio *Cargo.toml*: aggiungere `use rand;` senza la dipendenza dichiarata è un errore di import non risolto.
- **Regole di compatibilità**: Cargo risolve a una versione unica solo entro le regole del Semantic Versioning; due membri che dipendono da `rand 0.8.0` e `rand 0.8.1` useranno entrambi la 0.8.1 (compatibili per semver), mentre `rand 0.7.0` e `rand 0.8.0` sono incompatibili e Cargo userà una versione diversa per ciascuna, cercando comunque di minimizzare il numero di versioni.

I test nel workspace seguono la stessa logica: `cargo test` al livello superiore esegue i test di **tutte** le crate del workspace (con una sezione di output per ciascuna, più i doc test), e il flag `-p` restringe l'esecuzione a una crate sola. Anche la pubblicazione segue il modello a membro: ogni crate del workspace va **pubblicata separatamente**, di nuovo col flag `-p`.

Man mano che il progetto cresce, usare un workspace permette di lavorare con componenti più piccole e facili da capire rispetto a un unico blocco di codice, e semplifica il coordinamento tra crate che cambiano spesso insieme.

Installare binari con cargo install

Il comando `cargo install` permette di installare e usare crate binarie in locale: non è pensato per sostituire i pacchetti di sistema, ma come modo comodo per gli sviluppatori Rust di installare strumenti condivisi su crates.io. Tre dettagli lo delimitano.

- **Solo target binari**: si possono installare solo pacchetti con un *binary target*, cioè con un *src/main.rs* o un altro file specificato come binario, in contrapposizione ai target di libreria che non sono eseguibili da soli; il README della crate di solito dice se è una libreria, un binario o entrambi.
- **Percorso di installazione**: i binari installati finiscono nella cartella *bin* della root di installazione, tipicamente $*HOME/.cargo/bin * conrustupenessunaconfigurazionepersonalizzata; quelladirectoryvanel$'PATH' per poter eseguire i programmi installati.
- **Esempio**: `cargo install ripgrep` scarica, compila in profilo `release` e installa l'eseguibile `rg`, l'implementazione Rust del tool `grep` per cercare nei file, eseguibile con `rg --help`.

Estendere Cargo con comandi personalizzati

Cargo è progettato per essere estendibile con nuovi sotto-comandi senza doverlo modificare: se un binario nel `$PATH` si chiama `cargo-something`, lo si può eseguire **come se fosse un sotto-comando di Cargo** con `cargo something`. I comandi personalizzati compaiono anche nell'elenco di `cargo --list`. Poter usare `cargo install` per installare estensioni ed eseguirle come gli strumenti nativi di Cargo è uno dei vantaggi più convenienti del suo design.

Sintesi della lezione

Condividere codice con Cargo e crates.io è parte di ciò che rende l'ecosistema Rust utile per tanti compiti diversi. L'idea di fondo merita di essere ricordata.

- **Libreria standard piccola e stabile**: Rust offre una stdlib volutamente limitata.
- **Crates come estensione**: le crate sono facili da condividere, usare e migliorare su una linea temporale **diversa da quella del linguaggio**, che evolve lentamente e con garanzie di stabilità.
- **Cultura della condivisione**: non serve timidezza nel condividere su crates.io codice utile a noi; è probabile che sia utile anche a qualcun altro.

Capitolo 15

Smart pointers

Un **puntatore** è un concetto generale per una variabile che contiene un indirizzo in memoria, il quale si riferisce ("punta a") ad altri dati. Il tipo di puntatore più comune in Rust è la reference, indicata dal simbolo `&`, che prende in prestito il valore a cui punta: non ha capacità speciali oltre al riferire dati e non ha overhead.

Definition

Smart pointer

Gli *smart pointer* sono strutture dati che agiscono come un puntatore ma hanno anche metadati e capacità aggiuntive. Il concetto non è esclusivo di Rust (origine in C++), e rispetto alle reference c'è una differenza in più nel sistema di ownership: le reference **prendono in prestito** i dati, mentre in molti casi gli smart pointer **possiedono** i dati che puntano.

Gli smart pointer sono di solito implementati come struct che, a differenza di una struct ordinaria, implementano i trait `Deref` e `Drop`: `Deref` permette a un'istanza di comportarsi come una reference così da scrivere codice che funzioni con entrambe, `Drop` permette di personalizzare il codice eseguito quando l'istanza esce dallo scope. Gli smart pointer più comuni della libreria standard sono tre.

- `Box<T>`: per allocare valori sullo heap.
- `Rc<T>`: un tipo a *reference counting* che abilita ownership multipla.
- `Ref<T>` e `RefMut<T>`: accessi attraverso `RefCell<T>`, un tipo che fa rispettare le regole di borrowing a runtime invece che a compile time.

La lezione copre inoltre il pattern di *interior mutability*, dove un tipo immutabile espone un'API per mutare un valore interno, e i cicli di reference: come possono causare memory leak e come prevenirli.

Usare Box T per puntare a dati sullo heap

Lo smart pointer più semplice è il *box*, di tipo `Box<T>`: permette di memorizzare dati sullo **heap** anziché sullo stack, lasciando sullo stack solo il puntatore ai dati. Non ha overhead di prestazioni oltre a memorizzare i dati sullo heap, ma nemmeno molte capacità extra; si usa più spesso in tre situazioni.

- **Dimensione sconosciuta a compile time**: quando un tipo ha una dimensione non calcolabile a compile time e serve in un contesto che richiede una dimensione esatta.
- **Trasferimento di grandi quantità di dati**: quando si vuole trasferire l'ownership senza che i dati vengano copiati; trasferire la ownership di grandi dati è lento perché vengono copiati sullo stack, mentre con un box sullo heap si copia solo la piccola quantità di dati del puntatore.
- **Trait object**: quando si vuole possedere un valore e interessa solo che sia di un tipo che implementa un particolare trait, non di un tipo specifico.

Mettere un singolo valore sullo heap non è molto utile: per valori come un `i32` lo stack, dove sono memorizzati di default, è la scelta appropriata nella maggior parte dei casi. Come ogni valore posseduto, quando un box esce dallo scope viene deallocato, sia per il box (sullo stack) sia per i dati a cui punta (sullo heap).

Abilitare i tipi ricorsivi con i box

Definition

Tipo ricorsivo

Un valore di un *tipo ricorsivo* può avere un altro valore dello stesso tipo come parte di sé. I tipi ricorsivi pongono un problema perché Rust deve sapere a compile time quanto spazio occupa un tipo: l'annidamento di valori di un tipo ricorsivo potrebbe teoricamente continuare all'infinito, quindi Rust non può calcolare lo spazio necessario. Poiché i box hanno una dimensione nota, si abilitano i tipi ricorsivi inserendo un box nella definizione.

Il tipo ricorsivo di riferimento è la *cons list*, la struttura dati del Lisp costruita di coppie annidate (la versione Lisp della linked list, dal nome della funzione `cons` che costruisce una coppia dai suoi due argomenti): una lista `1, 2, 3` è la coppia `(1, (2, (3, Nil)))`, dove ogni elemento contiene il valore corrente e il successivo, e l'ultimo contiene solo `Nil`, il caso base della ricorsione — non il concetto di "null"/"nil" del capitolo sulle enum, cioè un valore assente o invalido. La cons list non è una struttura usata comunemente in Rust: per una lista di item `Vec<T>` è una scelta migliore; la si usa come esempio perché è semplice a parte la ricorsione.

La definizione enum con una variante `Cons(i32, List)` che contiene direttamente un altro valore `List` **non compila**:

l'errore dice che il tipo "ha dimensione infinita". Il ragionamento del compilatore merita di essere ricostruito.

- **Tipo non ricorsivo**: per la enum `Message` del capitolo 6, Rust esamina ogni variante e alloca lo spazio della più grande, perché solo una variante sarà usata per valore.
- **Tipo ricorsivo**: partendo da `Cons`, il compilatore vede che serve lo spazio di un `i32` più lo spazio di un `List`; per calcolare lo spazio di `List` riparte da `Cons`, e il processo continua all'infinito.

Il suggerimento del compilatore introduce il concetto chiave: inserire **un'indirezione** (ad esempio un `Box`) per rompere il ciclo. *Indirection* significa memorizzare il valore **indirettamente**, cioè memorizzare un puntatore al valore invece del valore stesso. Poiché `Box<T>` è un puntatore, Rust sa sempre quanto spazio serve: la dimensione di un puntatore non cambia in base alla quantità di dati puntata. Con `Cons(i32, Box<List>)` il valore `List` occupa lo spazio di un `i32` più quello del puntatore del box, la catena infinita è rotta e il compilatore calcola la dimensione; concettualmente la lista esiste ancora, ma gli item sono affiancati tramite puntatori anziché annidati uno dentro l'altro. I box offrono solo indirezione e allocazione sullo heap, senza le capacità speciali degli altri smart pointer né il loro overhead di prestazioni: utili quando l'indirezione è l'unica feature necessaria. Il tipo `Box<T>` è uno smart pointer perché implementa `Deref` (permette di trattarlo come una reference) e `Drop` (pulizia dei dati sullo heap).

Trattare gli smart pointer come reference normali col trait Deref

Implementare il trait `Deref` permette di personalizzare il comportamento dell'operatore di dereferenziazione `*` (da non confondere con la moltiplicazione). Implementandolo in modo che uno smart pointer sia trattabile come una reference normale, si può scrivere codice che opera sulle reference e usarlo anche con gli smart pointer. Il percorso didattico parte dalle reference ordinarie: con `y` uguale a una reference a `x`, un'asserzione sul valore di `y` richiede `*y` per seguire la reference fino al valore puntato; scrivere `assert_eq!(5, y)` senza dereferenziare è un errore di compilazione, perché confrontare un numero e una reference a un numero non è permesso. Lo stesso codice riscritto con `Box<i32>` funziona identicamente: l'operatore `*` segue il puntatore del box come seguiva la reference.

Definire il proprio smart pointer

Per capire cosa rende speciale `Box<T>`, si costruisce un tipo wrapper `MyBox<T>`: una tuple struct con un elemento di tipo `T` e una funzione associata `new`, come il vero `Box<T>` (che è definito esattamente così). Usando `*` su un valore `MyBox<T>` però il codice **non compila**: Rust non sa come dereferenziare `MyBox`, perché il tipo non implementa quell'abilità. Il trait `Deref` della libreria standard richiede un metodo solo, `deref`, che prende `self` in prestito e restituisce una reference ai dati interni; la sintassi `type Target = T;` definisce il tipo associato del trait. Con l'implementazione che restituisce `&self.0`, il codice con `*y` compila e le asserzioni passano. Tre aspetti del meccanismo meritano attenzione.

- **Sostituzione**: quando si scrive `*y`, dietro le quinte Rust esegue `*(y.deref())`: il compilatore sa dereferenziare le `&` reference, e `deref` gli dà la capacità di prendere qualunque tipo che implementa `Deref`, chiamare `deref` e ottenere una reference che sa dereferenziare.
- **Ownership**: `deref` restituisce una reference al valore, non il valore: se restituisse il valore direttamente lo sposterebbe fuori da `self`, e in genere usare l'operatore `*` non deve prendere ownership del valore interno.
- **Non ricorsivo**: la sostituzione dell'operatore `*` con la chiamata a `deref` e una dereferenziazione avviene **una volta sola** per ogni `*` scritto, quindi non ricorre all'infinito e il risultato è il dato concreto.

Usare la deref coercion in funzioni e metodi

Definition

Deref coercion

La *deref coercion* converte una reference a un tipo che implementa `Deref` in una reference a un altro tipo: ad esempio `&String` in `&str`, perché `String` implementa `Deref` in modo da restituire una string slice. È una comodità che Rust applica automaticamente sugli argomenti di funzioni e metodi quando il tipo passato non corrisponde al tipo del parametro: una sequenza di chiamate a `deref` converte il tipo fornito in quello richiesto.

La deref coercion esiste perché chi scrive le chiamate non debba aggiungere tante reference e dereferenziazioni esplicite con `&` e `*`, e perché più codice funzioni sia con le reference sia con gli smart pointer. L'esempio è una funzione `hello(name: &str)` chiamata con `&MyBox<String>`: Rust converte `&MyBox<String>` in `&String` con la `deref` implementata da noi, e `&String` in `&str` con quella fornita dalla libreria standard, fino a far combaciare il tipo col parametro. Senza deref coercion servirebbero `&(*m)[..]`: più difficile da leggere, scrivere e capire. Il numero di chiamate a `deref` necessarie è risolto a compile time, quindi **non c'è penalità a runtime**.

La deref coercion con reference mutabili

Analogamente a `Deref` per l'operatore `*` sulle reference immutabili, il trait `DerefMut` lo personalizza sulle reference mutabili. Rust applica la coercion in tre casi, con una asimmetria cruciale.

- **Da `&T` a `&U`**: quando `T: Deref<Target=U>`, cioè la coercion già vista, valida per le reference immutabili.
- **Da `&mut T` a `&mut U`**: quando `T: DerefMut<Target=U>`, la stessa coercion per le reference mutabili.
- **Da `&mut T` a `&U`**: quando `T: Deref<Target=U>`, cioè una reference mutabile può coercersi in una immutabile.

Il caso inverso — da `&T` a `&mut T` — **non è mai possibile**: le reference immutabili non si coerceranno mai in mutabili. Il motivo è nelle regole di borrowing: una reference mutabile dev'essere l'unica reference a quel dato (altrimenti il programma non compilerebbe), quindi convertirla in una immutabile non può violare le regole; convertire una reference immutabile in una mutabile richiederebbe che quella immutabile fosse l'unica al dato, ma le regole di borrowing non lo garantiscono, e Rust non può fare quell'assunzione.

Eseguire codice di pulizia col trait Drop

Il secondo trait importante per il pattern degli smart pointer è `Drop`, che permette di personalizzare cosa succede quando un valore sta per uscire dallo scope: si può implementare su qualunque tipo, per rilasciare risorse come file o connessioni di rete. Si introduce nel contesto degli smart pointer perché la sua funzionalità è usata quasi sempre nella loro implementazione: ad esempio quando un `Box<T>` viene droppato, dealloca lo spazio sullo heap che il box punta.

In alcuni linguaggi, per alcuni tipi, il programmatore deve chiamare esplicitamente il codice di liberazione della memoria o delle risorse ogni volta che finisce di usare un'istanza (file handle, socket, lock): se dimentica, il sistema può sovraccaricarsi e crashare. In Rust si specifica che un particolare codice venga eseguito quando un valore esce dallo scope e il compilatore lo inserisce automaticamente: non serve attenzione nel piazzare codice di pulizia ovunque e non si perdono risorse. Il trait richiede un metodo solo, `drop`, che prende una reference mutabile a `self`, e il cui corpo è dove si mette la logica di pulizia. Due osservazioni sul comportamento automatico sono rilevanti.

- **Ordine inverso**: le variabili vengono droppate in ordine inverso rispetto alla loro creazione.
- **Nessuna chiamata esplicita**: Rust chiama `drop` automaticamente quando le istanze escono dallo scope, senza che sia necessario (né possibile) invocarlo a mano.

Warning

Non si può chiamare drop manualmente

Disabilitare la funzionalità automatica di `drop` non è semplice, e in genere non serve: lo scopo del trait è proprio che sia gestito da solo. Talvolta però serve pulire un valore **in anticipo**, ad esempio con smart pointer che gestiscono lock: si può volere che il `drop` che rilascia il lock venga forzato così che altro codice nello stesso scope possa acquisirlo. Chiamare il metodo `drop` del trait manualmente **non compila**: Rust lo chiamerebbe comunque alla fine dello scope, causando un errore di *double free* (pulizia dello stesso valore due volte). L'errore usa il termine *destructor*, il termine generale di programmazione per una funzione che pulisce un'istanza (analogamente al *constructor* che la crea).

La soluzione è la funzione `std::mem::drop` della libreria standard, diversa dal metodo `drop` del trait: la si chiama passando come argomento il valore da forzare al drop, e fa sì che il codice di pulizia venga eseguito in quel punto anziché alla fine dello scope. Con `Drop` e il sistema di ownership non serve ricordarsi di pulire (lo fa Rust) e non ci sono problemi di pulizia di valori ancora in uso: il sistema che garantisce la validità delle reference garantisce anche che `drop` sia chiamato una sola volta, quando il valore non è più in uso.

Rc T lo smart pointer a reference counting

Nella maggior parte dei casi l'ownership è chiara: si sa esattamente quale variabile possiede un dato valore. Ci sono però casi in cui un singolo valore può avere **più owner**: nelle strutture dati a grafo, più archi puntano allo stesso nodo, che è concettualmente posseduto da tutti gli archi che lo puntano, e non va pulizzato finché ha archi che puntano a lui.

Definition

Rc T

`Rc<T>` è l'abbreviazione di *reference counting*: il tipo tiene traccio del numero di reference a un valore per determinare se è ancora in uso. Se le reference a un valore diventano zero, il valore può essere pulito senza che nessuna reference diventi invalida. L'immagine giusta è un televisore in salotto: la prima persona che entra lo accende, altri possono entrare a guardarlo, e l'**ultima** persona che esce lo spegne perché non è più usato — spegnerlo mentre altri lo guardano causerebbe proteste.

`Rc<T>` si usa quando si vogliono allocare dati sullo heap che **più parti del programma** leggono e **non si può determinare a compile time** quale parte finirà per ultima: se si sapesse, quella parte sarebbe semplicemente l'owner

e varrebbero le regole di ownership normali verificate a compile time. Due limiti da ricordare: `Rc<T>` è solo per scenari **single-threaded** (il reference counting multithread è nel capitolo sulla concorrenza), e permette di condividere dati solo in **lettura** (reference immutabili).

Condividere dati

L'esempio estende la cons list: due liste `b` e `c` che condividono l'ownership di una terza lista `a`. Con la definizione basata su `Box<T>` non funziona: le varianti `Cons` possiedono i dati che contengono, quindi creando `b` la lista `a` viene **spostata** dentro `b`, e usarla di nuovo per creare `c` è vietato perché è già stata mossa. Un'alternativa sarebbe cambiare `Cons` per trattenere reference, ma richiederebbe parametri di lifetime che imporrebbero a ogni elemento di vivere almeno quanto l'intera lista — vero in questo scenario, ma non in tutti.

La soluzione è sostituire `Box<T>` con `Rc<T>` nel tipo `List`: ogni variante `Cons` trattiene un valore e un `Rc<T>` che punta a un `List`. Creando `b` e `c` si **clona** l'`Rc<List>` in `a` con `Rc::clone`, aumentando il numero di reference da uno a due e poi a tre: `a`, `b` e `c` condividono l'ownership dei dati, e i dati non vengono puliti finché le reference non sono zero.

Info

La convenzione Rc::clone

Si potrebbe chiamare `a.clone()` invece di `Rc::clone(&a)`, ma la convenzione di Rust è usare `Rc::clone`: l'implementazione **non fa una deep copy** dei dati come il `clone` della maggior parte dei tipi, incrementa solo il reference count, che richiede poco tempo. Usare la forma esplicita `Rc::clone` distingue visivamente i cloni deep-copy (da considerare quando si cercano problemi di prestazioni) dai cloni che aumentano solo il contatore (da ignorare).

Clonare per aumentare il contatore

La funzione `Rc::strong_count` restituisce il numero corrente di reference forti (si chiama *strong* perché esiste anche un `weak_count`, trattato a fine lezione). Osservando il contatore in un programma con uno scope interno si vede il ciclo di vita completo.

- **Creazione**: l'`Rc<List>` in `a` parte con contatore 1; ogni chiamata a `Rc::clone` lo aumenta di 1.
- **Uscita dallo scope**: quando `c` esce dallo scope il contatore scende di 1, e **non serve chiamare alcuna funzione** per diminuirlo: è l'implementazione del trait `Drop` a diminuire automaticamente il reference count quando un valore `Rc<T>` esce dallo scope.
- **Pulizia**: quando anche `b` e `a` escono dallo scope il contatore arriva a 0 e l'`Rc<List>` viene pulito completamente.

`Rc<T>` permette a un singolo valore di avere più owner, e il contatore garantisce che il valore resti valido finché esiste almeno uno degli owner — ma solo via reference immutabili: se permettesse anche più reference mutabili, violerebbe le regole di borrowing del capitolo sull'ownership, dove più prestiti mutabili dello stesso dato causano data race e incoerenze.

RefCell T e il pattern dell'interior mutability

Definition

Interior mutability

L'*interior mutability* è un design pattern di Rust che permette di mutare dati anche quando ci sono reference immutabili a quei dati — cosa normalmente vietata dalle regole di borrowing. Il pattern usa codice `unsafe` dentro una struttura dati per piegare le regole usuali di mutazione e borrowing: codice *unsafe* indica al compilatore che stiamo verificando le regole manualmente invece di affidarci a lui. I tipi che seguono il pattern si possono usare solo quando **possiamo garantire** che le regole di borrowing saranno rispettate a runtime pur non potendolo garantire a compile time; il codice unsafe è avvolto in un'API sicura e il tipo esterno resta immutabile.

Far rispettare le regole di borrowing a runtime

A differenza di `Rc<T>`, il tipo `RefCell<T>` rappresenta **ownership singola** sui dati che contiene; la differenza da `Box<T>` sta in **quando** vengono verificate le regole di borrowing (una reference mutabile alla volta, o qualsiasi numero di immutabili; le reference sempre valide).

- **`Box<T>` e reference**: gli invarianti sono verificati a **compile time**; violarli è un errore di compilazione.
- **`RefCell<T>`**: gli invarianti sono verificati a **runtime**; violarli fa andare il programma in **panic** ed uscire.

Verificare a compile time è la scelta migliore nella maggior parte dei casi, ed è il default di Rust: gli errori vengono scoperti prima nello sviluppo e non c'è impatto sulle prestazioni a runtime perché tutta l'analisi è fatta in anticipo. Il vantaggio del controllo a runtime è che rende possibili certi scenari **memory-safe** che i controlli statici vieterebbero: l'analisi statica è intrinsecamente **conservativa**, perché alcune proprietà del codice sono impossibili da rilevare

analizzandolo (l'esempio celebre è il problema dell'arresto, fuori dallo scopo del libro). Poiché Rust non può essere sicuro che il codice rispetti le regole di ownership, può rifiutare un programma **corretto**: se accettasse un programma scorretto, gli utenti non potrebbero fidarsi delle garanzie di Rust, mentre rifiutarne uno corretto è solo un inconvenience. `RefCell<T>` è utile quando siamo sicuri che il codice segue le regole di borrowing ma il compilatore non capisce e garantisce che sia così. Come `Rc<T>`, è solo per scenari single-threaded.

Il riepilogo delle scelte tra i tre tipi è il seguente.

- **Owner**: `Rc<T>` abilita owner multipi degli stessi dati; `Box<T>` e `RefCell<T>` hanno owner singolo.
- **Prestiti**: `Box<T>` permette prestiti immutabili o mutabili verificati a compile time; `Rc<T>` solo prestiti immutabili verificati a compile time; `RefCell<T>` prestiti immutabili o mutabili verificati a runtime.
- **Mutazione**: proprio perché permette prestiti mutabili a runtime, `RefCell<T>` permette di mutare il valore interno anche quando il `RefCell<T>` stesso è immutabile — la definizione di interior mutability.

Usare l'interior mutability nei mock object

Una conseguenza delle regole di borrowing è che un valore immutabile non si può prendere in prestito in modo mutabile: un metodo che riceve `&self` non può mutare il valore. Ma esistono situazioni in cui è utile che un valore **muti se stesso nei propri metodi** pur apparendo immutabile al resto del codice. Il caso pratico è il testing.

Definition

Test double e mock object

Talvolta, durante i test, si usa un tipo al posto di un altro per osservare un comportamento particolare e asserire che sia implementato correttamente: questo segnaposto si chiama *test double*, nel senso della controfigura (*stunt double*) nel cinema. I *mock object* sono tipi specifici di test double che **registrano** ciò che accade durante il test, così da poter asserire che le azioni corrette siano avvenute. Rust non ha oggetti nello stesso senso di altri linguaggi né funzionalità di mock object nella libreria standard, ma una struct può servire allo stesso scopo.

Lo scenario è una libreria che traccia un valore rispetto a un massimo e manda messaggi in base alla vicinanza al massimo (ad esempio la quota di chiamate API di un utente): la libreria fornisce solo la logica di tracciamento, e le applicazioni forniscono il meccanismo di invio implementando un trait `Messenger` col metodo `send(&self, msg)`. Il mock `MockMessenger` deve registrare i messaggi in un `Vec<String>` dentro `sent_messages`, ma c'è il problema: `send` riceve un reference immutabile a `self`, quindi non può fare `push` — e non si può cambiare il trait in `&mut self` solo per comodità del test, perché non si vuole modificare il design esistente. La soluzione è l'interior mutability.

- **Campo**: `sent_messages` diventa di tipo `RefCell<Vec<String>>` invece di `Vec<String>`.
- **Scrittura**: dentro `send` (che resta `&self`, conforme al trait) si chiama `borrow_mut` sul `RefCell` per ottenere un reference mutabile al vettore interno e chiamare `push`.
- **Lettura**: nell'asserzione si chiama `borrow` per ottenere un reference immutabile al vettore e contarne gli elementi.

Tenere traccia dei borrow a runtime

Con le reference si usano `&` e `&mut`; con `RefCell<T>` si usano i metodi `borrow` e `borrow_mut`, parte dell'API sicura del tipo. `borrow` restituisce lo smart pointer `Ref<T>` e `borrow_mut` restituisce `RefMut<T>`; entrambi implementano `Deref`, quindi si trattano come reference normali. Il meccanismo di conteggio funziona così.

- **Conteggio**: `RefCell<T>` tiene traccia di quanti smart pointer `Ref<T>` e `RefMut<T>` sono attivi; ogni `borrow` aumenta il conteggio dei prestiti immutabili, e quando un `Ref<T>` esce dallo scope il conteggio scende di 1.
- **Regole**: come le regole di compile time, `RefCell<T>` permette molti prestiti immutabili **oppure** un prestito mutabile alla volta.
- **Violazioni**: cercare un secondo prestito mutabile attivo nello stesso scope (due `RefMut<T>` vive insieme) non è un errore di compilazione ma un **panic a runtime** col messaggio `already borrowed: BorrowMutError`.

Scegliere di intercettare gli errori di borrowing a runtime anziché a compile time comporta dei trade-off: gli errori si scoprono più tardi nel processo di sviluppo, potenzialmente solo in produzione, e c'è una piccola penalità di prestazioni per il conteggio a runtime. In cambio, `RefCell<T>` rende possibile scrivere mock object che mutano se stessi in contesti dove sono ammessi solo valori immutabili — più funzionalità di quelle che le reference normali forniscono.

Owner multipli di dati mutabili con Rc e RefCell insieme

Un uso comune di `RefCell<T>` è in combinazione con `Rc<T>`: `Rc<T>` dà owner multipli ma solo accesso immutabile; un `Rc<T>` che contiene un `RefCell<T>` dà un valore che può avere owner multipli **e** essere mutato. Nella cons list, con `Cons(Rc<RefCell<i32>>, Rc<List>)` si può chiamare `borrow_mut` sul valore condiviso per cambiarlo, e la modifica è visibile da **tutti** gli owner che condividono quel valore. Si ottiene così un valore `List` immutabile all'esterno ma con dati interni modificabili; i controlli a runtime delle regole di borrowing proteggono dai data race, e a volte vale la pena scambiare un po' di velocità con questa flessibilità nelle strutture dati. Per codice multithread non va bene `RefCell<T>`: la versione thread-safe è `Mutex<T>`, trattata nel capitolo sulla concorrenza.

I cicli di reference possono causare memory leak

Le garanzie di memory safety di Rust rendono difficile, ma **non impossibile**, creare memoria mai pulita — il *memory leak*. Prevenire completamente i memory leak **non** è una delle garanzie di Rust: i memory leak sono memory safe. Lo si vede usando `Rc<T>` e `RefCell<T>` insieme: si possono creare reference in cui gli item si riferiscono l'uno all'altro **in un ciclo**, e il reference count di ogni item nel ciclo non arriva mai a 0, quindi i valori non vengono mai droppati.

Creare un ciclo

Con `Cons` che contiene un `RefCell<Rc<List>>` (per modificare a cosa punta una variante, non solo il valore che contiene) e un metodo `tail` per accedere al secondo elemento, il ciclo si crea in due mosse: la lista `a` contiene `5, Nil`, la lista `b` contiene `10` e punta ad `a`, poi con `borrow_mut` si modifica `a` perché punti a `b` invece che a `Nil`. Dopo la modifica il contatore di **entrambe** le istanze è 2; alla fine di `main`, droppare `b` lo porta da 2 a 1 e droppare `a` da 2 a 1: la memoria sullo heap non viene mai rilasciata perché ogni istanza ha ancora un contatore a 1 — l'una referenzia l'altra. Se si prova a stampare la lista, Rust cerca di seguire il ciclo finché non va in **stack overflow**. Le conseguenze in questo esempio sono modeste perché il programma finisce subito, ma un programma più complesso che allocasse molta memoria in un ciclo e la trattenesse a lungo userebbe più memoria del necessario e potrebbe esaurire quella disponibile.

Warning

I cicli sono un bug logico, non un errore del compilatore

Creare cicli di reference non è facile ma nemmeno impossibile: con valori `RefCell<T>` che contengono `Rc<T>`, o combinazioni annidate di interior mutability e reference counting, spetta a noi assicurarsi di non creare cicli — **non ci si può affidare a Rust per catturarli**. Creare un ciclo è un bug di logica del programma, da minimizzare con test automatizzati, code review e altre pratiche di sviluppo.

Prevenire i cicli con Weak T

La soluzione strutturale è riorganizzare le strutture dati così che alcune reference esprimano ownership e altre no: solo le relazioni di ownership incidono sulla possibilità di droppare un valore. Lo strumento sono i **reference deboli**.

Definition

Reference forti e deboli

Le **strong reference** sono il modo di condividere l'ownership di un'istanza `Rc<T>`: `Rc::clone` aumenta lo `strong_count`, e l'istanza viene pulita solo se arriva a 0. Le **weak reference**, create con `Rc::downgrade`, **non esprimono una relazione di ownership**: il loro conteggio (`weak_count`) non influisce sulla pulizia, e non causano cicli, perché un ciclo che coinvolge reference deboli si rompe appena lo strong count dei valori coinvolti arriva a 0.

Poiché il valore puntato da un `Weak<T>` potrebbe essere già stato droppato, per usarlo bisogna verificare che esista ancora: il metodo `upgrade` su un `Weak<T>` restituisce un `Option<Rc<T>>` — `Some` se il valore non è ancora droppato, `None` se lo è. Restituendo un `Option`, Rust garantisce che entrambi i casi siano gestiti e che non esistano puntatori invalidi. L'esempio strutturale è un **albero** i cui nodi conoscono sia i figli che il genitore, con la struct `Node` che contiene il proprio valore, i riferimenti ai figli e un campo `parent`. Il ragionamento sulle relazioni guida la scelta dei tipi.

- **Padre → figli**: un nodo padre deve possedere i figli, quindi `children` è un `RefCell<Vec<Rc<Node>>>` (condiviso con le variabili per accedere a ogni nodo, e con interior mutability per aggiungere figli).
- **Figlio → padre**: un figlio **non deve possedere** il proprio padre (se si droppa un figlio, il padre deve restare), quindi il campo `parent` è un `RefCell<Weak<Node>>`: il nodo può riferirsi al padre senza possederlo. Mettere `Rc<T>` creerebbe un ciclo tra `leaf.parent` e `branch.children` con strong count mai a zero.

La dinamica dei contatori conferma il comportamento. Creata la foglia `leaf` e poi il ramo `branch` (che la possiede tra i figli e a cui `leaf` si collega col `Weak`), `branch` ha strong count 1 e weak count 1, mentre `leaf` ha strong count 2 e weak count 0. Quando lo scope interno termina e `branch` esce dallo scope, il suo strong count scende a 0 e il suo `Node` viene droppato: il weak count di 1 da `leaf.parent` **non ha alcun effetto** sulla pulizia, quindi nessun memory leak. Accedere al padre di `leaf` dopo quello scope restituisce `None`. Tutta la logica che gestisce i contatori e il drop è costruita dentro `Rc<T>` e `Weak<T>` con le loro implementazioni di `Drop`: specificando che la relazione figlio→genitore è una `Weak<T>` nella definizione di `Node`, i nodi padre possono puntare ai figli e viceversa senza creare cicli di reference né memory leak.

Sintesi della lezione

Gli smart pointer permettono di creare garanzie e trade-off diversi da quelli che Rust fa di default con le reference normali. Il quadro finale è il seguente.

- **`Box<T>`**: dimensione nota, punta a dati allocati sullo heap; abilita i tipi ricorsivi via indirection.
- **`Rc<T>`**: tiene il conto delle reference ai dati sullo heap, così i dati possono avere owner multipli.
- **`RefCell<T>`**: con la sua interior mutability dà un tipo usabile quando serve un tipo immutabile che deve mutare un valore interno, verificando le regole di borrowing a runtime.
- **`Deref` e `Drop`**: i due trait che abilitano gran parte della funzionalità degli smart pointer, rispettivamente il trattamento come reference e la pulizia personalizzata.
- **`Weak<T>`**: reference deboli per riferimenti senza ownership, che prevengono i cicli di reference causa di memory leak.

Chi volesse implementare i propri smart pointer può approfondire nel *Rustonomicon*, la guida del codice unsafe. Il passo successivo è la concorrenza in Rust, che introduce anche qualche nuovo smart pointer.

Parte VI

Concorrenza e async

Capitolo 16

Concorrenza senza paura

Gestire la programmazione concorrente in modo sicuro ed efficiente è un altro degli obiettivi principali di Rust. La **programmazione concorrente**, in cui parti diverse di un programma vengono eseguite indipendentemente, e la **programmazione parallela**, in cui parti diverse vengono eseguite *allo stesso tempo*, sono sempre più importanti man mano che più computer sfruttano processori multipli; storicamente la programmazione in questi contesti è stata difficile e soggetta a errori, e Rust spera di cambiarlo.

All'inizio il team di Rust pensava che garantire la memory safety e prevenire i problemi di concorrenza fossero due sfide separate da risolvere con metodi diversi; col tempo ha scoperto che il sistema di ownership e il sistema di tipi sono un insieme di strumenti potente per gestire **entrambe**. Sfruttando ownership e type checking, molti errori di concorrenza in Rust sono errori a compile time anziché a runtime: invece di passare tempo a riprodurre le circostanze esatte in cui un bug di concorrenza si manifesta, il codice scorretto si rifiuta di compilare presentando un errore che spiega il problema, così da correggerlo mentre si lavora al codice e non dopo la pubblicazione in produzione.

Definition

Concorrenza senza paura

Fearless concurrency è il soprannome di questo aspetto di Rust: la concorrenza senza paura permette di scrivere codice privo di bug sottili e facile da rifattorizzare senza introdurne di nuovi. Per semplicità il capitolo usa *concorrente* per indicare *concorrente e/o parallelo*; nel capitolo successivo, dove la distinzione conta di più, si è più specifici.

Molti linguaggi sono dogmatici nelle soluzioni offerte per la concorrenza: ad esempio Erlang ha una funzionalità elegante per la concorrenza a scambio di messaggi ma solo modi oscuri per condividere stato tra thread. Supportare solo un sottoinsieme delle soluzioni possibili è una strategia ragionevole per i linguaggi di alto livello, che promettono benefici in cambio del controllo; i linguaggi di basso livello però devono offrire la soluzione con le migliori prestazioni in ogni situazione, con meno astrazioni sull'hardware. Rust offre quindi una varietà di strumenti per modellare i problemi nel modo appropriato alle proprie esigenze: thread, concorrenza a scambio di messaggi con i canali, concorrenza a stato condiviso, e i trait `Sync` e `Send` che estendono le garanzie di Rust ai tipi definiti dall'utente.

Usare i thread per eseguire codice simultaneamente

Nella maggior parte dei sistemi operativi attuali il codice di un programma eseguito gira in un **processo**, e il sistema operativo gestisce più processi alla volta; dentro un programma ci possono anche essere parti indipendenti eseguite simultaneamente, chiamate **thread** (ad esempio un web server con più thread per rispondere a più richieste contemporaneamente).

Dividere il calcolo in più thread può migliorare le prestazioni ma aggiunge complessità: poiché i thread girano simultaneamente, **non c'è garanzia intrinseca sull'ordine** in cui le parti di codice su thread diversi verranno eseguite. I problemi tipici sono i seguenti.

- **Race condition**: thread che accedono a dati o risorse in un ordine inconsistente.
- **Deadlock**: due thread che aspettano l'un l'altro, impedendo a entrambi di continuare.
- **Bug intermittenti**: bug che accadono solo in certe situazioni e sono difficili da riprodurre e correggere in modo affidabile.

Rust cerca di mitigare gli effetti negativi, ma programmare in contesto multithread richiede comunque attenzione e una struttura del codice diversa da quella dei programmi single-thread. Sul piano dell'implementazione, la libreria standard di Rust usa un modello **1:1**: un thread del linguaggio corrisponde a un thread del sistema operativo; esistono crate che implementano altri modelli con trade-off diversi (e il sistema async del capitolo successivo offre un altro approccio alla concorrenza).

Creare un nuovo thread con spawn

Si crea un nuovo thread chiamando `thread::spawn` e passandogli una closure che contiene il codice da eseguire nel thread nuovo. Due comportamenti del modello vanno tenuti presente.

- **Spegnimento automatico**: quando il thread principale di un programma Rust termina, **tutti** gli thread generati vengono spenti, che abbiano finito o no di girare.
- **Nondeterminismo**: l'output di un programma con più thread può variare a ogni esecuzione, perché dipende da come il sistema operativo schedula i thread: le chiamate a `thread::sleep` fermano un thread per una breve

durata permettendo a un altro di girare, ma i thread "probabilmente" si alternano senza che sia garantito — e anche se al thread generato si chiede di stampare fino a `i` uguale a 9, può darsi che arrivi solo a 5 prima che il principale lo spenga.

Aspettare che tutti i thread finiscano

Il problema del thread generato che finisce presto (o non gira affatto) si risolve salvando il valore di ritorno di `thread::spawn`: il tipo è `JoinHandle<T>`, un valore che quando chiama il proprio metodo `join` aspetta che il proprio thread finisca. La semantica è la seguente.

- **Blocking**: chiamare `join` sul handle **blocca** il thread attualmente in esecuzione finché il thread rappresentato dal handle non termina; bloccare un thread significa impedirgli di svolgere lavoro o di uscire.
- **Posizione**: dove si chiama `join` fa la differenza. Se la chiamata sta dopo il ciclo del thread principale, i due thread si alternano e il principale aspetta la fine del generato prima di uscire; se si sposta la chiamata **prima** del ciclo, il principale aspetta prima tutto il thread generato e poi esegue il proprio ciclo, quindi l'output non è più intervallato.

Dettagli piccoli come la posizione di `join` possono determinare se i thread girano o meno allo stesso momento.

Closure move con i thread

La parola chiave `move` si usa spesso con le closure passate a `thread::spawn` perché in quel caso la closure prende ownership dei valori che usa dall'ambiente, trasferendo ownership da un thread a un altro (il capitolo 13 l'ha trattata per le closure in generale; qui interessa l'interazione con `thread::spawn`). Il caso problematico è la closure che usa un vettore creato dal thread principale.

- **Prestito inferito**: la closure usa `v` e quindi lo cattura, e Rust **inferisce** come catturarlo; poiché `println!` ha bisogno solo di una reference, la closure prova a prendere `v` in prestito. Ma Rust non può sapere quanto a lungo girerà il thread generato, quindi non sa se la reference a `v` sarà sempre valida.
- **Scenario invalido**: il thread principale può droppare `v` (con `std::mem::drop`) subito dopo lo spawn; se Rust permettesse il codice, il thread generato si ritroverebbe con una reference invalida.
- **Correzione**: aggiungendo `move` prima della closure si forza quest'ultima a prendere ownership dei valori usati invece di lasciare che Rust inferisca il prestito; ora il thread generato possiede `v` e il codice compila.

Warning

move non sospende le regole di ownership

Applicare `move` allo scenario del drop **non funziona**: sposterebbe `v` nell'ambiente della closure e il thread principale non potrebbe più chiamare `drop` su di esso — un errore di compilazione diverso. `move` **sovrascrive il default conservativo di Rust** (prendere in prestito), ma non permette di violare le regole di ownership: con l'errore originale Rust stava proteggendoci, garantendo che il thread principale non potesse invalidare la reference del thread generato; dire a Rust di spostare ownership al thread generato è la promessa che il thread principale non userà più `v`.

Trasferire dati tra thread con lo scambio di messaggi

Un approccio alla concorrenza sicura sempre più popolare è lo **scambio di messaggi**, dove thread o attori comunicano inviandosi messaggi che contengono dati.

> *"Non comunicare condividendo memoria; invece, condividete memoria comunicando."*
>
> – Documentazione del linguaggio Go

Per realizzare la concorrenza a messaggi la libreria standard fornisce un'implementazione dei **canali**. Un canale è un concetto generale di programmazione per cui i dati vengono inviati da un thread a un altro; l'immagine è un canale d'acqua direzionale come un torrente o un fiume: qualcosa messo nel fiume (una paperella di gomma) viaggia a valle fino alla fine del corso d'acqua.

Definition

Canale

Un canale ha due metà: un **trasmettitore** (upstream, dove si mette la paperella nel fiume) e un **ricevitore** (a valle, dove la paperella finisce). Una parte del codice chiama metodi sul trasmettitore con i dati da inviare, un'altra controlla l'estremità ricevente per i messaggi in arrivo. Un canale è detto **chiuso** se una delle due metà viene droppata.

Info

mpsc

La funzione `mpsc::channel` crea un nuovo canale; `mpsc` sta per *multiple producer, single consumer*: l'implementazione della libreria standard permette a un canale di avere più estremi **trasmittenti** che producono valori ma un solo estremo **ricevente** che li consuma — come più torrenti che confluiscono in un unico grande fiume. La funzione restituisce una tupla con trasmittitore e ricevente; le abbreviazioni tradizionali `tx` e `rx` nominano le variabili.

Il ruolo dell'ownership nel messaggio

Le regole di ownership svolgono un ruolo vitale nell'invio di messaggi e aiutano a scrivere codice concorrente sicuro, perché **prevenire gli errori è il vantaggio di pensare in termini di ownership** in tutto il programma. Il meccanismo si vede in un esperimento: usare un valore `val` nel thread generato *dopo* averlo mandato giù per il canale. Il codice **non compila**, e la ragione è la seguente.

- **send prende ownership**: la funzione `send` prende ownership del suo parametro, e quando il valore viene spostato è il ricevente a prenderne ownership; questo impedisce di usare di nuovo per sbaglio il valore dopo l'invio.
- **Razionale**: permetterlo sarebbe una cattiva idea — una volta che il valore è in un altro thread, quel thread potrebbe modificarlo o dropparlo prima che il valore sia riusato, causando errori o risultati inattesi per dati inconsistenti o inesistenti.
- **send restituisce Result**: se il ricevente è già stato droppato e non c'è dove inviare il valore, l'operazione restituisce un errore; negli esempi si chiama `unwrap` per semplicità, in un'applicazione reale si gestisce l'errore propriamente.

Sul lato ricevente, due metodi sono rilevanti.

- `recv`: abbreviazione di *receive*, **blocca** l'esecuzione del thread e aspetta finché arriva un valore, che restituisce in un `Result<T, E>`; quando il trasmettitore chiude, restituisce un errore per segnalare che non arriveranno altri valori.
- `try_recv`: **non blocca**, restituisce subito un `Result<T, E>` con `Ok` se c'è un messaggio disponibile e `Err` se non ce ne sono; è utile quando il thread ha altro lavoro da fare mentre aspetta messaggi, ad esempio un ciclo che chiama `try_recv` periodicamente.

Per ricevere **più valori** si tratta `rx` come un iteratore: per ogni valore ricevuto si stampa, e quando il canale si chiude l'iterazione termina — non serve chiamare `recv` esplicitamente. Che il thread principale stia aspettando si vede dal fatto che non ha codice di pausa eppure i valori arrivano intervallati dalle pause del thread generato. Per creare **più produttori** si sfrutta il *multiple producer* di `mpsc`: si chiama `clone` sul trasmettitore prima di creare il primo thread generato, si passa il clone al primo e l'originale al secondo, così due thread inviano messaggi allo stesso ricevente; l'ordine di ricezione può variare da esecuzione a esecuzione — è questo che rende la concorrenza interessante e difficile.

Concorrenza a stato condiviso

Lo scambio di messaggi è un buon modo di gestire la concorrenza ma non è l'unico: un altro metodo è l'accesso di più thread agli **stessi dati condivisi** — la parte del motto di Go che dice "non comunicare condividendo memoria". I canali, in qualunque linguaggio, sono simili alla **ownership singola**: una volta trasferito un valore giù per il canale non si deve più usare; la concorrenza a memoria condivisa è invece come la **ownership multipla**: più thread possono accedere allo stesso luogo di memoria contemporaneamente. Come visto col capitolo sugli smart pointer, l'ownership multipla aggiunge complessità perché i diversi owner vanno gestiti, e il sistema di tipi e le regole di ownership di Rust assistono molto nel farlo correttamente.

Controllare l'accesso con i mutex

Definition

Mutex

Mutex è l'abbreviazione di *mutual exclusion*: un mutex permette a un solo thread alla volta di accedere a certi dati. Per accedere ai dati un thread deve prima segnalare di volere l'accesso chiedendo di **acquisire il lock** del mutex — la struttura dati che fa parte del mutex e tiene traccio di chi ha l'accesso esclusivo. Per questa ragione si dice che il mutex **fa da guardia** ai dati che contiene. La metafora è il microfono unico in una tavola rotonda: chi vuole parlare deve chiederlo, può trattenerlo quanto vuole, e deve passarlo al prossimo quando finisce; se lo dimentica, nessun altro può parlare.

I mutex hanno la reputazione di essere difficili da usare perché richiedono di ricordare due regole.

- **Acquisire il lock** prima di usare i dati.
- **Rilasciare il lock** quando si è finito, così che altri thread possano acquisirlo.

La gestione dei mutex è notoriamente difficile da fare bene — è per questo che molti sono entusiasti dei canali. Grazie al sistema di tipi e alle regole di ownership di Rust, però, **non si possono sbagliare lock e unlock**.

L'API di Mutex T

Anche in un contesto single-threaded l'API illustra il punto centrale del tipo. La creazione usa la funzione associata `new`; per accedere ai dati si usa il metodo `lock` per acquisire il lock, che **blocca** il thread corrente finché non è il suo turno. Tre proprietà rendono il tipo sicuro da usare.

- **Il tipo forza il lock**: la variabile è di tipo `Mutex<i32>`, non `i32`, quindi si **deve** chiamare `lock` per usare il valore interno: il sistema di tipi non lascia dimenticarlo, perché non permette l'accesso all'`i32` interno altrimenti.
- **MutexGuard**: `lock` restituisce (avvolto in un `LockResult`, gestito con `unwrap`) un tipo chiamato `MutexGuard`, che implementa `Deref` per puntare ai dati interni — si usa come reference mutabile — e ha un'implementazione di `Drop` che **rilascia il lock automaticamente** quando esce dallo scope. Non si rischia quindi di dimenticare il rilascio bloccando il mutex per gli altri thread.
- **Lock avvelenato**: la chiamata a `lock` fallisce se il thread che detiene il lock va in panic: nessuno potrebbe mai ottenerlo, quindi negli esempi si usa `unwrap` per propagare il panic.

Accesso condiviso a Mutex T tra più thread

L'esempio di riferimento è un contatore incrementato da 10 thread. La costruzione del caso d'uso procede per errori di compilazione, ciascuno dei quali insegna un concetto.

- **Ownership singola insufficiente**: passare a tutti gli thread la stessa closure che sposta il contatore dentro il thread non compila — il messaggio dice che `counter` è stato **spostato nell'iterazione precedente** del ciclo: Rust vieta di spostare ownership del contatore in più thread.
- **Rc non è thread-safe**: avvolgere il `Mutex<T>` in `Rc<T>` e clonarlo per ogni thread produce un errore diverso: `` `Rc<Mutex<i32>>` non può essere inviato tra thread in sicurezza `` perché il trait `Send` non è implementato. Il problema concreto è che quando `Rc<T>` gestisce il conteggio aggiunge a ogni `clone` e sottrae a ogni drop, **senza primitive di concorrenza** a garantire che gli aggiornamenti del contatore non siano interrotti da un altro thread — il che porterebbe a conteggi sbagliati, bug sottili, memory leak o valori droppati troppo presto.
- **Arc**: serve un tipo esattamente come `Rc<T>` ma che aggiorni il reference count in modo **thread-safe**: è `Arc<T>`, dove la *a* sta per *atomic* (tipo a reference counting atomico). Gli atomici sono primitive di concorrenza aggiuntive che si comportano come i tipi primitivi ma sono sicure da condividere tra thread; `Arc<T>` ha la **stessa API** di `Rc<T>`, quindi bastano il `use`, la `new` e la `clone` per far compilare l'esempio.

Due considerazioni completano la sezione.

- **Costo della thread safety**: ci si può chiedere perché non tutti i tipi primitivi siano atomici e perché i tipi della libreria standard non usino `Arc<T>` di default: la thread safety ha una penalità di prestazioni che conviene pagare **solo quando serve**; operando su valori in un solo thread, il codice gira più veloce senza le garanzie degli atomici.
- **Tipi atomici semplici**: per semplici operazioni numeriche esistono tipi più semplici di `Mutex<T>` nel modulo `std::sync::atomic` della libreria standard, che forniscono accesso atomico, concorrente e sicuro ai tipi primitivi.

Il confronto tra RefCell Rc e Mutex Arc

Il parallelismo tra le due coppie di tipi è strutturale e merita di essere ricordato. Il contatore è immutabile ma se n'è ottenuta una reference mutabile al valore interno: `Mutex<T>` fornisce **interior mutability**, come la famiglia `Cell` — nello stesso modo in cui nel capitolo 15 si usava `RefCell<T>` per mutare i contenuti dentro un `Rc<T>`, qui si usa `Mutex<T>` per mutare i contenuti dentro un `Arc<T>`.

Warning

I deadlock sono possibili

Rust **non** protegge da ogni tipo di errore logico nell'uso di `Mutex<T>`. Come `Rc<T>` comportava il rischio di cicli di reference che causano memory leak, `Mutex<T>` comporta il rischio di **deadlock**: un'operazione che deve bloccare due risorse mentre due thread hanno acquisito ciascuno un lock e si aspettano l'un l'altro per sempre. Come i cicli, anche i deadlock sono bug di logica che spetta al programmatore prevenire.

Concorrenza estendibile con i trait Sync e Send

Un fatto interessante è che quasi ogni funzionalità di concorrenza vista finora è parte della **libreria standard, non del linguaggio**: le opzioni per gestire la concorrenza non si limitano a linguaggio e libreria standard, si possono

scrivere funzionalità proprie o usare quelle scritte da altri. I concetti chiave incorporati **nel linguaggio** sono però due *marker trait* del modulo `std::marker`: `Send` e `Sync`.

Definition

Send

Il marker trait `Send` indica che l'ownership dei valori del tipo che lo implementa può essere **trasferita tra thread**. Quasi tutti i tipi Rust sono `Send`, con eccezioni notevoli. Un tipo composto interamente di tipi `Send` è automaticamente `Send` esso stesso; quasi tutti i tipi primitivi lo sono, a parte i *raw pointer* (trattati nel capitolo sulle feature avanzate).

- **L'eccezione Rc**: `Rc<T>` non può implementare `Send` perché clonando un valore e trasferendone ownership a un altro thread, i due thread potrebbero aggiornare il reference count **contemporaneamente**; per questo è implementato per scenari single-thread, dove non si vuole pagare la penalità di prestazioni della thread safety.
- **Garanzia automatica**: il sistema di tipi e i trait bounds assicurano che non si possa mai inviare per sbaglio un `Rc<T>` tra thread in modo insicuro — l'errore `il trait `Send` non è implementato per `Rc<Mutex<i32>>`` è esattamente quello ricevuto; passando ad `Arc<T>`, che `Send` lo implementa, il codice compila.

Definition

Sync

Il marker trait `Sync` indica che il tipo che lo implementa può essere **referenziato da più thread** in sicurezza. In altre parole, un tipo `T` implementa `Sync` se `&T` (una reference immutabile a `T`) implementa `Send`, cioè se la reference può essere inviata a un altro thread senza problemi. I tipi primitivi sono tutti `Sync`, e un tipo composto interamente di tipi `Sync` è anch'esso `Sync`.

`Sync` è il concetto Rust più vicino al significato colloquiale di *thread-safe* (un dato usabile in sicurezza da più thread concorrenti). La ragione di avere **due** trait separati è che un tipo può essere l'uno, o entrambi, o nessuno; gli esempi del libro illustrano le quattro combinazioni.

- **Né Send né Sync**: `Rc<T>`, per le ragioni viste.
- **Send ma non Sync**: `RefCell<T>` (e la famiglia dei `Cell<T>`) sono `Send` se `T` è `Send`, ma **non** sono `Sync`: un `RefCell` può attraversare il confine di un thread, ma non essere acceduto in concorrenza, perché l'implementazione del borrow checking a runtime di `RefCell<T>` non è thread-safe.
- **Send e Sync**: `Mutex<T>` è `Send` e `Sync`, e può condividere l'accesso tra più thread.
- **Sync ma non Send**: il tipo `MutexGuard<'a, T>` restituito da `Mutex::lock` è `Sync` (se `T` è `Sync`) ma **non** `Send`, perché alcune piattaforme richiedono che un mutex sia sbloccato **dallo stesso thread** che l'ha bloccato.

Implementare Send e Sync manualmente è unsafe

Poiché i tipi composti interamente di tipi `Send` e `Sync` li implementano automaticamente, quasi mai serve implementarli a mano; come marker trait non hanno **alcun metodo** da implementare, servono solo a far rispettare invarianti legate alla concorrenza. Implementarli manualmente richiede codice Rust *unsafe*, trattato nel capitolo sulle feature avanzate: l'informazione importante ora è che costruire nuovi tipi concorrenti non composti da parti `Send` e `Sync` richiede un'attenzione particolare per mantenere le garanzie di sicurezza (approfondite nel *Rustonomicon*).

Sintesi della lezione

La concorrenza non finisce qui: il capitolo successivo si concentra sulla programmazione asincrona e il progetto del capitolo 21 userà questi concetti in una situazione più realistica. Il quadro finale è il seguente.

- **Nel linguaggio, non nella libreria**: poiché pochissimo della gestione della concorrenza fa parte del linguaggio, molte soluzioni sono implementate come crate, che evolvono più in fretta della libreria standard — conviene cercare le crate aggiornate per le situazioni multithread.
- **Strumenti della libreria standard**: canali per lo scambio di messaggi e smart pointer come `Mutex<T>` e `Arc<T>` sicuri nei contesti concorrenti.
- **La garanzia**: il sistema di tipi e il borrow checker assicurano che il codice che usa queste soluzioni non finisca in data race o reference invalide; una volta che il codice compila, si può essere certi che girerà su più thread senza i bug difficili da rintracciare comuni in altri linguaggi.

La programmazione concorrente non è più un concetto da temere: si può procedere senza paura, rendendo i propri programmi concorrenti.

Capitolo 17

Programmazione asincrona

Molte operazioni che chiediamo al computer possono richiedere tempo, e sarebbe bello fare altro mentre aspettiamo che i processi lunghi finiscano. La logica dei nostri programmi però è scritta in modo per lo più lineare: vorremmo specificare le operazioni da eseguire e i punti in cui una funzione può mettersi in pausa per lasciare girare un'altra parte del programma, senza dover fissare in anticipo ordine e modalità di esecuzione. La **programmazione asincrona** è un'astrazione che permette di esprimere il codice in termini di punti di pausa potenziali e risultati futuri, occupandosi per noi dei dettagli del coordinamento. Questo capitolo introduce l'approccio alternativo ai thread del capitolo 16: le **futures** di Rust, gli **stream** e la sintassi `async` e `await`, insieme alle crate di terze parti che implementano i **runtime** asincroni — il codice che gestisce e coordina l'esecuzione delle operazioni asincrone.

Parallelismo e concorrenza

Finora abbiamo trattato parallelismo e concorrenza come per lo più intercambiabili; ora serve distinguerli con precisione, perché le differenze si manifesteranno lavorando con async. Il caso di riferimento è l'export di un video (operazione che può durare da minuti a ore e usa tutta la CPU e la GPU disponibili) rispetto al download di un video (che dipende dalla velocità della rete).

- **CPU-bound**: l'export è un'operazione limitata dalla potenza di elaborazione del computer (*compute-bound*); con un solo core e un sistema operativo che non la interrompesse — cioè eseguita *sincronamente* — non si potrebbe fare altro mentre gira.
- **I/O-bound**: il download è limitato dalla velocità di input/output del computer; la CPU deve aspettare che i dati arrivino dalla rete, e anche un solo secondo è un tempo lunghissimo per un processore moderno che esegue miliardi di operazioni al secondo.

In entrambi i casi il sistema operativo fornisce una forma di concorrenza con interruzioni invisibili, ma solo al livello dell'intero programma. Spesso noi comprendiamo i nostri programmi a un livello molto più granulare del sistema operativo e possiamo vedere opportunità di concorrenza che esso non vede — ad esempio in un gestore di download, dove partire con un download non deve bloccare l'interfaccia e l'utente deve poterne avviare più di uno. Il problema è che molte API del sistema operativo per la rete sono **bloccanti**: fermano il progresso del programma finché i dati non sono pronti (il termine *blocking* è di solito riservato alle chiamate che interagiscono con file, rete o altre risorse, perché sono i casi in cui un singolo programma beneficerebbe di un'operazione *non* bloccante). Si potrebbe evitare il blocco generando un thread dedicato per ogni download, ma l'overhead delle risorse di sistema usate da quei thread diventerebbe alla lunga un problema; sarebbe preferibile definire un insieme di *task* e lasciare che il runtime scelga ordine e modalità di esecuzione. È esattamente ciò che dà l'astrazione **async** di Rust.

Le due dinamiche di lavoro

La distinzione tra concorrenza e parallelismo si comprende con le modalità con cui un team può dividersi il lavoro su un progetto software.

- **Concorrenza**: quando una persona lavora su più compiti diversi prima che uno sia completato — come avere due progetti aperti sul computer e passare dall'uno all'altro quando ci si annoia o ci si blocca. Essendo una persona sola, non si può progredire su entrambi *esattamente allo stesso momento*, ma si può fare multitasking alternandosi.
- **Parallelismo**: quando il team divide un gruppo di compiti assegnando a ciascun membro un compito da svolgere da solo; ogni persona progredisce esattamente allo stesso tempo delle altre.
- **Serialità**: in entrambi i flussi può emergere lavoro *seriale* — un compito che dipende dal completamento di un altro può avvenire solo in serie, uno dopo l'altro; e la concorrenza e il parallelismo possono intersecarsi, ad esempio quando un collega è bloccato fino al nostro compito e concentriamo tutti gli sforzi per "sbloccarlo".

Le stesse dinamiche valgono per hardware e software: su una macchina con un solo core la CPU esegue una sola operazione alla volta ma può lavorare **in concorrenza**, mettendo in pausa un'attività per passare ad altre (con thread, processi o async) prima di tornare alla prima; su una macchina multi-core può anche lavorare **in parallelo**, con un core su un compito e un altro su un compito del tutto indipendente, davvero simultaneamente. L'esecuzione di codice async in Rust avviene di solito in concorrenza; a seconda di hardware, sistema operativo e runtime, quella concorrenza può usare il parallelismo sotto il cofano.

Futures e sintassi async

Gli elementi chiave della programmazione asincrona in Rust sono le futures e le parole chiave `async` e `await`.

Definition

Future

Una *future* è un valore che potrebbe non essere pronto ora ma lo diventerà in un punto del futuro (lo stesso concetto appare in molti linguaggi, a volte con nomi come *task* o *promise*). Rust fornisce un trait `Future` come blocco costruttivo, così che operazioni async diverse possano essere implementate con strutture dati diverse ma con un'interfaccia comune. Ogni future trattiene le proprie informazioni sul progresso compiuto e su cosa significhi essere "pronta".

La parola chiave `async` si applica a blocchi e funzioni per specificare che possono essere interrotte e riprese; dentro un blocco o una funzione async si usa `await` per *aspettare una future* (aspettare che diventi pronta). Ogni punto in cui si attende una future è un punto potenziale di pausa e ripresa del blocco o della funzione; il processo di chiedere a una future se il suo valore è disponibile si chiama **polling**. Alcuni linguaggi (come C# e JavaScript) usano parole chiave simili, ma Rust gestisce la sintassi in modo significativamente diverso — per buone ragioni. Nella pratica quotidiana si usano quasi sempre `async` e `await`, che il compilatore traduce in codice equivalente basato sul trait `Future`, così come traduce i cicli `for` in codice equivalente basato sul trait `Iterator`; ma poiché il trait esiste, si può anche implementare `Future` per i propri tipi quando serve.

La pigrizia delle futures

Il primo programma del capitolo (un piccolo web scraper che recupera due URL in concorrenza e stampa il titolo di chi finisce prima) rivela subito la proprietà fondamentale: le futures in Rust sono **pigre** — non fanno nulla finché non glielo si chiede con `await` (il compilatore mostra un warning per una future non usata). Il parallelismo con gli iteratori del capitolo 13 è esatto: come gli iteratori non fanno nulla finché non si chiama `next` (direttamente o con `for` e `map`), le futures non fanno nulla finché non lo si chiede esplicitamente. La pigrizia permette a Rust di evitare di eseguire codice async finché non serve davvero; è un comportamento diverso da `thread::spawn`, dove la closure passa a girare immediatamente, e da come molti altri linguaggi trattano l'async — ma è importante perché Rust possa offrire le sue garanzie di prestazioni.

Tre dettagli di sintassi caratterizzano l'uso quotidiano.

- **await è postfix**: la parola chiave va **dopo** l'espressione attesa, non prima; questo rende molto più leggibili le catene di metodi, dove `await` si frappone tra le chiamate.
- **Ogni attesa va esplicitata**: nell'esempio, tanto la richiesta HTTP quanto il metodo `text` della risposta sono asincroni e vanno attesi separatamente — la risposta può arrivare in parti (header, cookie) e il corpo può richiedere tempo, quindi anche ottenere il testo completo è un'operazione asincrona.
- **async fn è zucchero**: quando Rust vede un blocco `async` lo compila in un tipo di dati unico e anonimo che implementa il trait `Future`; quando vede una funzione `async`, la compila in una funzione non async il cui corpo è un blocco async. Scrivere `async fn` equivale quindi a scrivere una funzione che restituisce `impl Future<Output = T>` con la sintassi `impl Trait` del capitolo 10; l'intero corpo della funzione originale viene avvolto in un blocco `async move`, che è l'espressione restituita.

Eseguire codice async con un runtime

Il posto in cui chiamare una funzione async è la `main` — ma `main` **non può** essere `async`: il compilatore lo rifiuta esplicitamente.

Definition

Runtime

Il codice async ha bisogno di un *runtime*: una crate Rust che gestisce i dettagli dell'esecuzione del codice asincrono. La `main` di un programma può **inizializzare** un runtime, ma non è essa stessa un runtime. Ogni programma Rust che esegue codice async ha almeno un punto in cui imposta un runtime che esegue le futures. A differenza della maggior parte dei linguaggi, che includono un runtime, Rust non ne fornisce uno: esistono molti runtime diversi, ciascuno con trade-off adatti al proprio caso d'uso (un web server ad alto throughput con molti core e molta RAM ha esigenze molto diverse da un microcontrollore con un core, poca RAM e nessuna allocazione sullo heap). Le crate che forniscono i runtime spesso offrono anche versioni async di funzionalità comuni come I/O su file o rete. Alcuni runtime forniscono macro che permettono una `main` async, riscrivendola in una `fn main` normale che avvia il runtime.

Negli esempi si usa la funzione `block_on` della crate didattica `trpl`: prende una future come argomento, imposta un runtime e **blocca il thread corrente** finché la future non gira fino al completamento, restituendo il valore prodotto — è il punto scelto per passare dal codice sincrono a quello asincrono. La scelta delle futures da passare a `block_on` è una questione di design: negli esempi si passa un blocco `async` che attende esplicitamente la chiamata a `page_title`,

perché nella maggior parte del codice reale si fa più di una chiamata async.

La macchina a stati invisibile

Ogni **punto di await** rappresenta un punto in cui il controllo torna al runtime. Per farlo funzionare, Rust tiene traccia dello stato coinvolto nel blocco async, così che il runtime possa avviare altro lavoro e tornare quando è pronto a far avanzare il primo. È una **macchina a stati invisibile**, come se si scrivesse a mano un enum per salvare lo stato corrente a ogni punto di await; scrivere a mano le transizioni tra stati sarebbe tedioso e soggetto a errori, ma il compilatore crea e gestisce automaticamente le strutture dati della macchina a stati per il codice async, e le normali regole di borrowing e ownership continuano ad applicarsi (il compilatore le verifica, con messaggi d'errore utili). Alla fine qualcosa deve eseguire la macchina a stati, e quel qualcosa è il runtime — la parte di un runtime responsabile dell'esecuzione del codice async si chiama **executor**. È per questo che `main` non può essere async: qualcun altro dovrebbe gestire la macchina a stati della future che restituirebbe, ma `main` è il punto di partenza del programma.

Far competere due futures con select

Per recuperare due URL in concorrenza si chiamano le funzioni e si salvano le futures risultanti — che ancora non fanno nulla, essendo pigre — per poi passarle a `trpl::select`, che restituisce un valore che indica quale delle futures passate finisce prima. Poiché entrambe possono legittimamente "vincere", non avrebbe senso restituire un `Result`: `select` restituisce invece un tipo nuovo, `Either`, simile a `Result` per la presenza di due casi ma **senza** nozioni di successo o fallimento.

```rust
enum Either<A, B> {
    Left(A),
    Right(B),
}
```

`Left` contiene l'output della prima future se vince il primo argomento, `Right` quello della seconda — in ordine di posizione degli argomenti. Costruire un piccolo web scraper funzionante dimostra i fondamenti del lavoro con le futures.

Applicare la concorrenza con async

Applicando async alle stesse sfide di concorrenza del capitolo 16, le API somigliano spesso a quelle dei thread — ma anche quando sembrano simili, hanno spesso comportamenti diversi e quasi sempre caratteristiche di prestazioni diverse.

Creare un nuovo task con spawn_task

La versione async del conteggio su due thread usa `trpl::spawn_task`, simile a `thread::spawn`, e una `sleep` asincrona. Le corrispondenze e le differenze con i thread sono sistematiche.

- **Spegnimento**: come per i thread, quando il blocco async principale finisce, il task generato viene spento; per farlo girare fino in fondo si usa un join handle.
- **join diventa await**: con i thread si chiamava `join` per bloccare fino alla fine; qui si attende il handle stesso, perché **il handle del task è una future** (il suo `Output` è un `Result`, che si spacchetta dopo l'attesa).
- **Niente thread**: la differenza maggiore è che non serve generare un thread del sistema operativo — e nemmeno un task: poiché i blocchi async compilano in futures anonime, si possono mettere i due cicli in blocchi async separati e usare `trpl::join` per farli girare entrambi fino al completamento.

Definition

join e fairness

La funzione `trpl::join` riceve due futures e produce una singola future nuova il cui output è una tupla con gli output di entrambe una volta che **entrambe** sono completate; si attende la future prodotta da `join`, non le due originali in sequenza (altrimenti si tornerebbe a un flusso sequenziale). `join` è **fair**: controlla ogni future con la stessa frequenza, alternandosi, senza mai lasciarne correre una avanti se l'altra è pronta. Con i thread è il sistema operativo a decidere quale thread controllare; con l'async è il runtime a decidere. I runtime non sono obbligati a garantire fairness per ogni operazione, e offrono spesso API per scegliere.

L'esecuzione con `join` produce lo stesso identico ordine a ogni esecuzione, molto diversamente dai thread — la fairness è una proprietà che il runtime può garantire ma che ai thread non è data.

Scambio di messaggi tra task

Si usa di nuovo lo scambio di messaggi, con versioni async dei tipi e funzioni: `trpl::channel` è una versione asincrona del canale multiple-producer single-consumer del capitolo 16. Le differenze sono concentrate in tre punti.

- **Ricevente mutabile**: la versione async del canale usa un ricevente `rx` mutabile anziché immutabile.
- **recv non blocca**: il metodo `recv` produce una future da attendere invece del valore direttamente. Il `Receiver::recv` sincrono blocca finché non arriva un messaggio; quello async **non blocca**: restituisce il controllo al runtime finché non arriva un messaggio o la parte trasmittente chiude.
- **send non attende**: la chiamata `send` non viene attesa perché non blocca — il canale è illimitato (*unbounded*).

Per ricevere un numero **sconosciuto** di messaggi serve un costrutto nuovo, perché Rust non ha ancora un modo di usare il ciclo `for` su una serie prodotta *asincronamente*: il ciclo `while let`, versione ciclica di `if let`, continua a eseguire finché il pattern specificato continua a combaciare col valore. Il meccanismo è il seguente: `rx.recv().await` restituisce `Some(message)` ogni volta che arriva un messaggio e `None` quando il canale chiude, indipendentemente da quanti messaggi siano arrivati; ogni completamento del ciclo tocca di nuovo il punto di await, e il runtime lo mette in pausa finché non arriva un altro messaggio.

Warning

Il codice dentro un solo blocco async è lineare

Se invio e ricezione stanno in un unico blocco async, **non c'è concorrenza**: dentro un blocco async, l'ordine in cui appaiono le parole chiave `await` è anche l'ordine in cui vengono eseguite. Tutte le `send` accadono, intervallate dalle `sleep` e dai loro punti di await, e solo dopo il ciclo `while let` attraversa gli await delle `recv` — i messaggi arrivano tutti insieme dopo l'intero ritardo. Per il comportamento voluto (ritardo tra un messaggio e l'altro) le operazioni di `tx` e `rx` vanno in **blocchi async separati**, uniti da `trpl::join`: è il runtime a eseguirli separatamente.

L'altro problema dell'esempio è che il programma **non esce mai**, e la catena di cause lo spiega per intero: la future di `join` completa solo quando entrambe le futures hanno completato; quella di ricezione completa solo quando il ciclo `while let` finisce; il ciclo finisce solo quando `rx.recv().await` produce `None`; ciò accade solo quando l'altro estremo del canale chiude; il canale chiude solo chiamando `rx.close` o quando il trasmittente `tx` viene droppato; e `tx` non viene droppato finché il blocco async più esterno non finisce — che non può finire perché è bloccato su `join`. La soluzione è l'ownership: il blocco che invia prende `tx` solo **in prestito** (inviare non richiede ownership), ma dichiarandolo `async move` il trasmittente viene **droppato** alla fine del blocco, il canale si chiude e il programma termina. La parola chiave `move` funziona con i blocchi async come con le closure, con le stesse dinamiche. Per **più produttori** si clona `tx` come nel capitolo 16 e si usa la macro `trpl::join!`, che attende un numero arbitrario di futures note a compile time; l'ordine in cui le futures vengono **create** non conta, conta l'ordine in cui vengono **attese**.

Cedere il controllo al runtime

Vale anche l'inverso: Rust mette in pausa i blocchi async e restituisce il controllo al runtime **solo** ai punti di await — tutto ciò che sta tra un punto di await e l'altro è sincrono. Ne deriva un rischio concreto.

Warning

Una future può far morire di fame le altre

Fare molto lavoro in un blocco async senza punti di await **blocca** le altre futures: si dice che una future *affama* le altre (in inglese si parla di *starving*). Se una future esegue un lavoro costoso o indefinito, occorre decidere quando e dove restituire il controllo al runtime. L'esempio simula il problema con una funzione `slow` basata su `std::thread::sleep` (bloccante): due futures che chiamano `slow` più volte eseguono **tutto** il loro lavoro ciascuna prima dell'altro, senza mai alternarsi.

Le soluzioni progressive dell'esempio illustrano lo strumento giusto.

- **Punti di await aggiuntivi**: interporre chiamate a `trpl::sleep` (con await) tra le chiamate lente rende l'esecuzione delle due futures intervallata.
- **yield_now**: ma non si vuole *dormire*, si vuole progredire il più in fretta possibile, restituendo solo il controllo; la funzione `trpl::yield_now` lo fa direttamente, è più chiara nell'intento e può essere molto più veloce di `sleep`, perché i timer hanno limiti di granularità (la `sleep` usata dorme sempre almeno un millisecondo, anche con una durata di un nanosecondo — e i computer moderni in un millisecondo fanno molto).

Questo significa che l'async può essere utile anche per compiti CPU-bound, come strumento per strutturare le relazioni tra parti del programma (al costo dell'overhead della macchina a stati): è una forma di **multitasking cooperativo**, dove ogni future ha il potere di determinare quando cede il controllo tramite i punti di await — e quindi la responsabilità di non bloccare troppo a lungo (in alcuni sistemi operativi embedded basati su Rust è l'**unico** tipo di multitasking). Cedere il controllo non è gratuito: a volte spezzare un compito CPU-bound lo rallenta molto, e per la performance complessiva può essere meglio lasciare che un'operazione blocchi brevemente; bisogna sempre misurare i colli di bottiglia reali.

Costruire astrazioni async proprie

Le futures si **compongono**: si può costruire ad esempio una funzione `timeout` combinando i blocchi già visti. Il disegno dell'API illustra il ragionamento di design.

- **Funzione async**: per poterla attendere.
- **Primo parametro generico**: una future qualsiasi, resa generica per funzionare con tutte.
- **Secondo parametro Duration**: il tempo massimo, facilmente passato a `trpl::sleep`.
- **Ritorno Result**: `Ok` col valore prodotto dalla future se completa in tempo, `Err` con la durata se il timeout scade prima.

L'implementazione fa competere la future con un timer costruito con `select`, e qui emerge un dettaglio di fairness: l'implementazione di `trpl::select` **non è fair** — polla sempre gli argomenti nell'ordine in cui sono passati (altre implementazioni scelgono a caso), quindi si passa prima `future_to_try` così ha una chance di completare anche con durate minime; dal match su `Either` si restituisce `Ok(output)` o `Err(max_time)`. Poiché le futures si compongono con altre futures, si possono costruire strumenti potenti da blocchi piccoli — timeout con retry, combinati con chiamate di rete. Nella pratica si lavora per lo più con `async` e `await`, e in seconda battuta con funzioni come `select` e macro come `join!` per controllare l'esecuzione delle futures più esterne.

Stream, futures in sequenza

Il metodo `recv` del canale async produce una sequenza di elementi nel tempo: è un'istanza di un pattern molto più generale noto come **stream**. Molti concetti si rappresentano naturalmente come stream: elementi che diventano disponibili in una coda, pezzi di dati letti incrementalmente dal filesystem quando l'insieme è troppo grande per la memoria, dati che arrivano dalla rete nel tempo. Poiché gli stream sono futures, si combinano con ogni altro tipo di future — per raggruppare eventi ed evitare troppe chiamate di rete, impostare timeout su sequenze di operazioni lunghe, frenare gli eventi dell'interfaccia utente per evitare lavoro inutile.

Rispetto agli iteratori del capitolo 13, due sono le differenze: il **tempo** (gli iteratori sono sincroni, il ricevente del canale è asincrono) e l'**API** (si chiama il metodo sincrono `next` dell'iteratore, mentre il ricevente ha un metodo `recv` asincrono). La somiglianza non è una coincidenza: lo stream è una forma asincrona di iterazione, e l'API generale fornisce l'elemento successivo come `Iterator`, ma in modo asincrono — al punto che si può creare uno stream da qualunque iteratore.

Info

Il pattern Ext

Usare il metodo `next` su uno stream richiede di portare in scope il trait giusto — che **non** è `Stream` ma `StreamExt`. `Ext` sta per *extension*: è un pattern comune nella comunità Rust per estendere un trait con un altro. `Stream` definisce l'interfaccia di basso livello che combina `Iterator` e `Future`; `StreamExt` fornisce API di livello più sopra, incluso `next` e altri metodi di utilità simili a quelli del trait `Iterator`. Né l'uno né l'altro sono ancora nella libreria standard di Rust, ma la maggior parte delle crate dell'ecosistema usa definizioni simili.

Uno sguardo più da vicino ai trait per async

Per il lavoro quotidiano i dettagli bastano, ma in alcune situazioni servono il trait `Future`, il tipo `Pin` e il trait `Unpin`.

Il trait Future

La definizione del trait introduce sintassi e tipi nuovi:

```rust
use std::pin::Pin;
use std::task::{Context, Poll};

pub trait Future {
    type Output;
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}
```

Il tipo associato `Output` dice a cosa la future risolve (analogamente a `Item` per `Iterator`). Il metodo `poll` riceve un riferimento speciale `Pin` per `self` e un riferimento mutabile a un `Context`, e restituisce un `Poll<Self::Output>`:

```rust
pub enum Poll<T> {
    Ready(T),
    Pending,
}
```

`Poll` è simile a un `Option` — una variante con valore e una senza — ma significa qualcosa di **molto diverso**: `Pending` indica che la future ha ancora lavoro da fare e il chiamante dovrà ricontrollare più tardi; `Ready` indica che la future ha finito e il valore `T` è disponibile. Tre osservazioni completano il quadro.

- **Non pollare dopo Ready**: è raro dover chiamare `poll` direttamente, ma se accade, con la maggior parte delle futures il chiamante **non** deve richiamare `poll` dopo un `Ready` — molte futures vanno in panic se pollate di nuovo una volta pronte, e quelle sicure da re-pollare lo dichiarano nella documentazione; è simile al comportamento di `Iterator::next`.
- **await compila in poll**: quando si vede `await`, Rust lo compila in chiamate a `poll`; servirebbe un ciclo che riprova finché la future non è pronta, ma un ciclo letterale sarebbe bloccante — l'esatto opposto dello scopo. Rust garantisce invece che il ciclo possa cedere il controllo a chi può mettere in pausa questa future per lavorare su altre e ricontrollarla più tardi: il runtime, e questo scheduling è uno dei suoi compiti principali.
- **Il ruolo del runtime**: il runtime **polla** ogni future di cui è responsabile, rimettendola a dormire quando risponde `Poll::Pending` e facendola avanzare quando risponde `Poll::Ready`.

Il tipo Pin e il trait Unpin

Il caso che spinge a questi tipi è comune: una collezione (come un vettore) contenente un numero di futures noto solo a runtime, da attendere con `trpl::join_all`. Si mette ogni future in un `Box` per trasformarla in **trait object**, così che le futures anonime prodotte dai tipi possano essere trattate come lo stesso tipo (tutte implementano `Future`) — sorprendentemente necessario anche quando tutti i blocchi restituiscono `()`, perché il compilatore crea un tipo enumerato **unico** per ogni blocco async, e come non si possono mescolare due struct scritte a mano in un `Vec`, non si possono mescolare le enum generate dal compilatore.

Il codice però non compila: `` `dyn Future<Output = ()>` cannot be unpinned `` — la future non implementa il trait `Unpin`. Il suggerimento è usare la macro `pin!` o `Box::pin`. Per capire perché, occorre guardare dentro la macchina a stati.

- **Self-referenzialità**: tra un punto di await e il successivo, Rust crea una variante della macchina a stati con l'accesso ai dati usati in quella sezione (per ownership o per riferimento). A differenza della maggior parte dei tipi Rust, le futures dei blocchi async possono finire con **reference a se stesse** nei campi di una variante.
- **Il pericolo del movimento**: qualunque oggetto con una reference a se stesso è insicuro da **spostare**, perché le reference puntano all'indirizzo di memoria effettivo: spostare la struttura lascerebbe le reference interne puntare al vecchio luogo, ormai invalido e riusabile dal computer per altri scopi. Il borrow checker previene lo spostamento di item con reference attive — ma muovere la future stessa (per metterla in un `Vec` o restituirla da una funzione) significherebbe muovere la macchina a stati.
- **La garanzia di Pin**: invece di aggiornare ogni reference a ogni spostamento (overhead enorme), si garantisce che la struttura **non si muova**. `Pin` è un involucro per tipi simili a puntatori (`&`, `&mut`, `Box`, `Rc` — tecnicamente i tipi che implementano `Deref`/`DerefMut`); non è esso stesso un puntatore e non ha comportamenti propri come il reference counting: è puramente uno strumento con cui il compilatore fa rispettare vincoli sull'uso dei puntatori. Pinnare un valore avvolgendo in `Pin` un puntatore a quel valore lo rende non più spostabile: con `Pin<Box<SomeType>>` si pina il valore `SomeType`, **non** il puntatore `Box` — il puntatore può muoversi liberamente, purché i dati puntati restino al loro posto.
- **Unpin**: la maggior parte dei tipi è perfettamente sicura da muovere anche dietro un puntatore pinnato — servirebbe pensare al pinning solo quando ci sono reference interne, e primitive come numeri e Boolean ovviamente non ne hanno, come la maggior parte dei tipi d'uso comune (un `Vec` si muove senza pensieri). `Unpin` è un **marker trait** come `Send` e `Sync` del capitolo 16, senza funzionalità propria: dice al compilatore che un dato tipo **non** deve mantenere garanzie sul movimento. Il compilatore lo implementa automaticamente per tutti i tipi dove può provare che è sicuro; il caso speciale è la notazione `!Unpin`, per i tipi che **devono** mantenere quelle garanzie. Due cose da ricordare: `Unpin` è il caso "normale" e `!Unpin` quello speciale; e implementa `Unpin` o `!Unpin` conta **solo** quando si usa un puntatore pinnato a quel tipo. Una `String` avvolta in `Pin` può essere sostituita in memoria con un'altra `String` senza violare il contratto, proprio perché non ha reference interne.

La combinazione di `Pin` e `Unpin` rende possibile implementare in sicurezza una intera classe di tipi complessi altrimenti problematici perché **self-referenziali**: oggi appaiono per lo più nell'async, e contano soprattutto per costruire librerie di basso livello o runtime, non per il codice quotidiano — ma quando appaiono nei messaggi d'errore, ora se ne conosce

la soluzione (la macro `pin!` o `Box::pin`).

Il trait Stream

La definizione del trait `Stream` fonde visivamente `Iterator` e `Future`: da `Iterator` l'idea di sequenza (`next` che restituisce `Option<Self::Item>`), da `Future` l'idea di prontezza nel tempo (`poll` che restituisce `Poll<Self::Output>`):

```rust
trait Stream {
    type Item;
    fn poll_next(
        self: Pin<&mut Self>,
        cx: &mut Context<'_>
    ) -> Poll<Option<Self::Item>>;
}
```

Il tipo associato `Item` è come quello di `Iterator`, dove possono esserci da zero a molti elementi — a differenza di `Future`, dove c'è sempre un singolo `Output`, anche se di tipo unit. Il metodo `poll_next` polla come `Future::poll` e produce una sequenza come `Iterator::next`: il tipo di ritorno combina `Poll` **esterno** (va controllata la prontezza, come una future) con `Option` **interno** (segnala se ci sono altri messaggi, come un iteratore). Una definizione molto simile finirà probabilmente nella libreria standard; nel frattempo è parte del kit della maggior parte dei runtime.

Nel lavoro quotidiano non si usa `poll_next` né `Stream` direttamente, ma `next` e `StreamExt`: si potrebbe scrivere a mano la macchina a stati come si potrebbe usare `poll` direttamente, ma `await` è molto più comodo, e `StreamExt` fornisce `next` con un'implementazione di default che gestisce correttamente i dettagli delle chiamate a `Stream::poll_next`. `StreamExt` è implementato automaticamente per ogni tipo che implementa `Stream`, ma i due trait sono definiti separatamente, così che la comunità possa iterare sulle API di convenienza senza toccare il trait fondamentale — chi implementa un proprio tipo di stream deve implementare **solo** `Stream`, e chi lo usa ottiene automaticamente `StreamExt` con i suoi metodi.

Futures task e thread insieme

Quando scegliere i thread e quando l'async? Dipende — e spesso la scelta non è thread **o** async, ma thread **e** async. I trade-off delle due alternative si completano.

- **Limiti dei thread**: i modelli basati sui thread esistono da decenni in molti sistemi operativi e molti linguaggi li supportano, ma su molti sistemi usano una quantità considerevole di memoria per ogni thread; e i thread sono un'opzione solo quando sistema operativo e hardware li supportano — alcuni sistemi embedded non hanno un sistema operativo, quindi niente thread.
- **Le task**: nel modello async le operazioni concorrenti non richiedono thread propri, ma girano su **task** (come con `spawn_task`). Una task è simile a un thread, ma invece di essere gestita dal sistema operativo è gestita da codice a livello di libreria: il runtime.

Definition

Task

La somiglianza delle API di spawn non è casuale, e la gerarchia è precisa. I **thread** fanno da confine per insiemi di operazioni *sincrone*: la concorrenza è possibile **tra** thread. Le **task** fanno da confine per insiemi di operazioni *asincrone*: la concorrenza è possibile sia **tra** task che **dentro** una task, perché una task può alternare futures nel proprio corpo. Le **futures** sono l'unità di concorrenza più granulare di Rust, e ogni future può rappresentare un albero di altre futures. L'executor del runtime gestisce le task, e le task gestiscono le futures: le task sono come thread leggeri gestiti dal runtime, con capacità aggiuntive che derivano proprio da questa gestione.

Due precisazioni bilanciano il quadro.

- **Nessuna superiorità automatica**: la concorrenza con i thread è in certi modi un modello di programmazione più **semplice** di quella con async — può essere un pregio o un difetto; i thread sono per certi versi "fire and forget", senza equivalente nativo delle future, e corrono fino al completamento senza interruzioni se non dal sistema operativo stesso.
- **Cooperazione reale**: thread e task collaborano bene, perché le task possono (in molti runtime) essere spostate tra thread. Il runtime usato nel capitolo è **multithread di default**, e molti runtime usano il *work stealing* per spostare le task tra thread in modo trasparente in base all'utilizzo, migliorando le prestazioni complessive — un approccio che richiede thread **e** task, e quindi futures.

Le regole pratiche per scegliere sono due.

- **Molto parallelizzabile (CPU-bound)**: elaborare tanti dati dove ogni parte si può processare separatamente — i thread sono la scelta migliore.
- **Molto concorrente (I/O-bound)**: gestire messaggi da fonti diverse che arrivano a intervalli e ritmi diversi — l'async è la scelta migliore.

Se servono sia parallelismo che concorrenza non bisogna scegliere: si usano insieme, ciascuno nella parte che gli riesce meglio. L'esempio tipico mescola un thread che possiede il trasmittente di un canale async e invia numeri con pause di un secondo, mentre un blocco async li attende con `block_on`; tornando allo scenario d'apertura del capitolo, un set di codifiche video gira su un thread dedicato (essendo compute-bound) e notifica l'interfaccia via canale async — combinazioni di questo tipo sono innumerevoli nel mondo reale.

Sintesi della lezione

Non è l'ultima volta che si incontra la concorrenza in questo libro: il progetto del capitolo 21 applicherà questi concetti in una situazione più realistica, confrontando più direttamente la risoluzione di problemi con thread e con task/futures. Il messaggio finale è il seguente.

- **Strumenti complementari**: futures (incluse le stream), task e thread si incastrano insieme, con trade-off complementari.
- **Sicurezza in ogni approccio**: qualunque approccio si scelga, Rust fornisce gli strumenti per scrivere codice concorrente sicuro e veloce, da un web server ad alto throughput a un sistema operativo embedded.

Il passo successivo del libro riguarda i modi idiomatici di modellare problemi e strutturare soluzioni man mano che i programmi crescono, e come gli idiomi di Rust si rapportano a quelli della programmazione orientata agli oggetti.

Parte VII

OOP, pattern e feature avanzate

Capitolo 18

Rust e la programmazione orientata agli oggetti

La programmazione orientata agli oggetti (OOP) è un modo di modellare i programmi. Gli oggetti come concetto programmatico furono introdotti dal linguaggio Simula negli anni Sessanta e influenzarono l'architettura di Alan Kay, in cui gli oggetti si passano messaggi; per descriverla coniò il termine *object-oriented programming* nel 1967. Esistono molte definizioni concorrenti di cosa sia l'OOP: secondo alcune Rust è orientato agli oggetti, secondo altre no. Questa lezione esplora le caratteristiche comunemente considerate orientate agli oggetti e come si traducono in Rust idiomatico, poi implementa un pattern di design orientato agli oggetti confrontandolo con una soluzione che sfrutta invece i punti di forza di Rust.

Caratteristiche dei linguaggi orientati agli oggetti

Non c'è consenso nella comunità sulla features che un linguaggio deve avere per essere considerato orientato agli oggetti. Rust è influenzato da molti paradigmi, OOP incluso (come la programmazione funzionale ha influenzato il capitolo sulle closure). Tre caratteristiche sono comunemente condivise dai linguaggi OOP, e per ciascuna vale la pena chiedersi se Rust la supporta.

Gli oggetti contengono dati e comportamento

Il libro *Design Patterns: Elements of Reusable Object-Oriented Software* (la cosiddetta *Gang of Four*) definisce l'OOP così: i programmi orientati agli oggetti sono fatti di oggetti, e un **oggetto** impacchetta insieme dati e procedure che operano su quei dati, tipicamente chiamate **metodi** o **operazioni**. Con questa definizione Rust è orientato agli oggetti: struct ed enum hanno dati, e i blocchi `impl` forniscono metodi su struct ed enum. Anche se struct ed enum con metodi non si chiamano "oggetti", forniscono la stessa funzionalità secondo la definizione.

Incapsulamento che nasconde i dettagli implementativi

L'**incapsulamento** significa che i dettagli implementativi di un oggetto non sono accessibili al codice che lo usa: l'unico modo di interagire con l'oggetto è attraverso la sua API pubblica, e il codice che lo usa non può raggiungerne gli interni per cambiare dati o comportamento direttamente. Questo permette di modificare e rifattorizzare gli interni senza cambiare il codice che usa l'oggetto.

Rust supporta l'incapsulamento attraverso quanto visto nel capitolo sulla modularità: la parola chiave `pub` decide quali moduli, tipi, funzioni e metodi sono pubblici, e tutto il resto è privato di default. Il caso di studio è la struct `AveragedCollection`, che mantiene una lista di interi e **cacha** la loro media in un campo separato, così che non vada ricalcolata a ogni richiesta.

- **Struttura**: la struct è `pub` così che altro codice possa usarla, ma i campi (`list` e `average`) restano privati.
- **API pubblica**: i metodi `add`, `remove` e `average` sono gli unici modi per accedere o modificare i dati, e ogni aggiunta o rimozione chiama internamente il metodo privato `update_average` che tiene la media sincronizzata.
- **Razionale**: se i campi fossero pubblici, il codice esterno potrebbe modificare `list` senza aggiornare `average`, lasciandole fuori sincronia; l'incapsulamento impedisce che la media diventi incoerente.
- **Flessibilità**: potremmo sostituire il `Vec<i32>` con un `HashSet<i32>` senza toccare il codice che usa la struct, fintanto che le firme dei metodi pubblici restano le stesse — con i campi pubblici non sarebbe così, perché i due tipi hanno metodi diversi per aggiungere e rimuovere elementi.

Se l'incapsulamento è un aspetto richiesto per essere considerati orientati agli oggetti, Rust soddisfa il requisito.

Ereditarietà come sistema di tipi e come condivisione di codice

L'**ereditarietà** è il meccanismo per cui un oggetto può ereditare elementi dalla definizione di un altro, ottenendo dati e comportamento del genitore senza ridefinirli. Se un linguaggio deve avere l'ereditarietà per essere orientato agli oggetti, Rust **non** è tale: non c'è modo di definire una struct che eredita i campi e le implementazioni di metodi della struct genitore senza usare una macro. Chi è abituato all'ereditarietà può però usare altre soluzioni, a seconda del motivo per cui la cercherebbe.

- **Riuso di codice**: si può implementare un comportamento per un tipo e riusarlo per un altro in modo limitato con le **implementazioni di default dei trait**, già viste col metodo `summarize` del trait `Summary`: ogni tipo che

implementa il trait ottiene il metodo senza codice ulteriore, e può sovrascrivere il default — simile a una classe genitore con un'implementazione e una classe figlia che la eredita o la sovrascrive.
- **Polimorfismo**: l'altro motivo è di sistema di tipi, permettere che un tipo figlio sia usato negli stessi posti del genitore.

Definition

Polimorfismo

Per molti il polimorfismo è sinonimo di ereditarietà, ma è un concetto più generale: si riferisce a codice che può lavorare con dati di **tipi multipli** (per l'ereditarietà, tipicamente sottoclassi). Rust usa invece i **generics** per astrarre sui tipi possibili e i **trait bound** per imporre vincoli su cosa quei tipi devono fornire — un approccio talvolta chiamato *bounded parametric polymorphism*.

Rust ha scelto trade-off diversi non offrendo l'ereditarietà, per ragioni di design.

- **Condivisione eccessiva**: l'ereditarietà rischia di condividere più codice del necessario; le sottoclassi non dovrebbero sempre condividere tutte le caratteristiche del genitore, ma con l'ereditarietà lo fanno, rendendo il design meno flessibile.
- **Metodi fuori posto**: introduce la possibilità di chiamare metodi sulle sottoclassi che non hanno senso o causano errori perché non si applicano a quella sottoclasse.
- **Ereditarietà singola**: alcuni linguaggi permettono di ereditare da una sola classe, restringendo ulteriormente la flessibilità del design.

Per queste ragioni Rust usa i **trait object** invece dell'ereditarietà per ottenere il polimorfismo a runtime.

Trait objects per astrarre su comportamenti condivisi

Nel capitolo sulle collezioni avevamo visto che un limite dei vettori è contenere elementi di un solo tipo, e avevamo creato la enum `SpreadsheetCell` con varianti per interi, float e testo: una soluzione perfettamente buona quando gli item intercambiabili sono un **insieme fisso di tipi noti a compile time**. A volte però vogliamo che l'utente della nostra libreria possa **estendere** l'insieme dei tipi validi.

Il caso di studio è una libreria GUI che itera su una lista di componenti chiamando su ciascuno un metodo `draw`: al momento di scriverla non possiamo conoscere tutti i tipi che altri programmatori vorranno creare (uno aggiungerà `Image`, un altro `SelectBox`), ma sappiamo che la libreria deve trattenere molti valori di tipi diversi e chiamare `draw` su ciascuno, senza sapere cosa farà quel metodo — solo che sarà disponibile. In un linguaggio con ereditarietà si definirebbe una classe `Component` con `draw`, da cui `Button` e `SelectBox` erediterebbero; senza ereditarietà serve un'altra struttura.

Definition

Trait object

Un trait object punta a un'istanza di un tipo che implementa il trait specificato **e** a una tabella usata per cercare i metodi del trait su quel tipo a runtime. Si crea specificando un puntatore (una reference o uno smart pointer come `Box<T>`), poi la parola chiave `dyn`, poi il trait rilevante: ad esempio `Box<dyn Draw>`. Si può usare ovunque al posto di un tipo generico o concreto, e il sistema di tipi garantisce a compile time che ogni valore usato in quel contesto implementi il trait — non serve conoscere tutti i tipi possibili a compile time.

Il caso GUI si implementa così: si definisce il trait `Draw` col metodo `draw`, e la struct `Screen` con un campo `components` di tipo `Vec<Box<dyn Draw>>`; il metodo `run` di `Screen` itera chiamando `draw` su ogni componente.

Trait objects o generics con trait bound

La soluzione con trait objects funziona in modo diverso da una struct generica con trait bound, e la differenza è il criterio di scelta tra le due.

- **Parametro generico**: può essere sostituito da **un solo** tipo concreto alla volta; una `Screen<T>` dove `T: Draw` ammette liste omogenee (tutti `Button` o tutti `TextField`). Se servono solo collezioni omogenee, generics e trait bound sono preferibili perché le definizioni vengono monomorfizzate a compile time sui tipi concreti.
- **Trait object**: permette a **tipi concreti multipli** di comparire a runtime; una sola istanza di `Screen` può trattenere un vettore che contiene `Box<Button>` insieme a `Box<TextField>`.

Un utente della libreria definisce i propri tipi (`SelectBox` con campi `width`, `height`, `options`) e implementa `Draw` su di essi; la nostra `Screen`, scritta senza conoscere `SelectBox`, può operare sul tipo nuovo e disegnarlo perché implementa il trait. Il concetto — preoccuparsi solo dei messaggi a cui un valore risponde anziché del suo tipo concreto — è simile al **duck typing** dei linguaggi dinamicamente tipizzati ("se cammina come un'anatra e starnazza come un'anatra, è un'anatra"): il metodo `run` non controlla il tipo concreto di nessun componente, chiama solo `draw`. Il vantaggio rispetto al duck typing è che non serve verificare a runtime che un valore implementi il metodo né preoccuparsi di errori: **se**

il valore non implementa il trait, il codice non compila (usare una `String` come componente è un errore di compilazione, non un errore a runtime).

Info

Trait objects e inferenza di tipo

Un aspetto negativo dei trait objects è l'interazione con l'inferenza di tipo. Per un `Vec<T>` normale basta il tipo di un solo elemento per dedurre `T`; con i trait object, fattorizzare la lista di componenti in una variabile separata fa **fallire la compilazione**: il compilatore perde l'informazione del tipo atteso della struct `Screen` nel punto dove la variabile è definita, e deduce `Box<SelectBox>` dal primo elemento, rifiutando poi il `Button` ("mismatched types"). La correzione richiede un suggerimento all'algoritmo: un cast esplicito `as Box<dyn Draw>` su un qualunque elemento, o un'annotazione di tipo sulla variabile. In generale i trait objects peggiorano l'esperienza di sviluppo per i client di un'API quanto a inferenza di tipo.

Dynamic dispatch

Nel capitolo 10 avevamo visto la monomorfizzazione dei generici: il compilatore genera implementazioni non generiche per ogni tipo concreto usato. Il codice risultante fa **static dispatch**: il compilatore sa quale metodo si chiama a compile time. L'opposto è il **dynamic dispatch**: il compilatore non può dire a compile time quale metodo verrà chiamato, ed emette codice che a runtime saprà quale chiamare.

Con i trait objects Rust deve usare il dynamic dispatch: il compilatore non conosce tutti i tipi che potranno essere usati, quindi non sa quale metodo di quale tipo chiamare; a runtime usa i puntatori dentro il trait object per sapere quale metodo chiamare. I costi del trade-off sono i seguenti.

- **Lookup a runtime**: la ricerca del metodo ha un costo che non esiste col static dispatch.
- **Niente inlining**: il dynamic dispatch impedisce al compilatore di scegliere di inline-are il codice del metodo, il che a sua volta impedisce alcune ottimizzazioni.
- **Regole di compatibilità**: Rust ha regole su dove il dynamic dispatch è o non è usabile, chiamate *dyn compatibility* (dette anche *object safety*), fuori dallo scopo della lezione.

In cambio si ottiene la flessibilità di collezioni eterogenee estendibili dagli utenti: è un trade-off da considerare.

Implementare un pattern di design orientato agli oggetti

Definition

State pattern

Lo *state pattern* è un pattern di design orientato agli oggetti in cui si definisce un insieme di stati che un valore può avere internamente. Gli stati sono rappresentati da **state object**, e il comportamento del valore cambia in base al proprio stato. Ogni state object è responsabile del proprio comportamento e di governare quando deve cambiare in un altro stato; il valore che trattiene uno state object **non sa nulla** dei comportamenti diversi degli stati né di quando avviene la transizione. Il vantaggio è che quando i requisiti cambiano non serve cambiare il codice del valore che trattiene lo stato né quello che lo usa: basta aggiornare il codice dentro uno degli state object, o aggiungerne di nuovi.

Il caso di studio è il workflow di un post di un blog, con stati "draft", "review" e "published": il post nasce come draft vuoto, quando il draft è pronto si richiede una review, quando il post è approvato viene pubblicato, e **solo** i post pubblicati restituiscono il contenuto da stampare — così che un post non approvato non possa essere pubblicato per sbaglio. Qualsiasi altra modifica tentata non deve avere effetto (approvare un draft prima della review lascia il post un draft non pubblicato).

L'implementazione in stile tradizionale

Si implementa prima il pattern nel modo più tradizionalmente orientato agli oggetti, poi in uno più naturale per Rust. Il tipo visibile all'utente è solo `Post`, che usa il pattern trattenendo uno state object in un campo; le transizioni sono gestite internamente, così gli utenti non le gestiscono direttamente e **non possono commettere errori** con gli stati, come pubblicare un post non revisionato. La struttura ha tre scelte architetturali.

- **Tipo privato del campo stato**: `state: Option<Box<dyn State>>`, dove `State` è un trait privato che definisce il comportamento condiviso dagli stati; gli state object sono le struct `Draft`, `PendingReview` e `Published`, che implementano `State`. Il campo privato garantisce che ogni nuovo `Post` nasca come `Draft` (in `Post::new` il campo viene impostato su `Some(Box::new(Draft {}))`): non c'è modo di creare un `Post` in un altro stato.
- **Metodi che non dipendono dallo stato**: `add_text` prende `&mut self` e fa `push_str` sul `content`; non interagisce col campo `state`, quindi non è parte del pattern ma è parte del comportamento da supportare.

- **Contenuto inizialmente vuoto**: il primo `content` restituisce sempre una string slice vuota fintanto che il post è in draft (con `assert_eq!` vuoto come test ideale).

Le transizioni introducono la sintassi chiave del pattern.

```rust
fn request_review(self: Box<Self>) -> Option<Box<dyn State>>;
```

Invece di `self`, `&self` o `&mut self` il metodo riceve `self: Box<Self>`: la sintassi significa che il metodo è valido solo se chiamato su un `Box` che trattiene il tipo, e prende ownership del `Box<Self>`, **invalidando il vecchio stato** così che il valore di stato del `Post` possa trasformarsi in un nuovo stato. Il meccanismo richiede due mosse collegate.

- **Option::take**: per consumare il vecchio stato occorre prenderne ownership, ed è qui che serve l'`Option` del campo: `take` estrae il valore `Some` lasciando un `None` al suo posto (Rust non permette campi non popolati nelle struct), permettendo di **muovere** il valore di stato fuori dal `Post` anziché prenderlo in prestito; impostare temporaneamente `None` garantisce che `Post` non possa usare il vecchio stato dopo la trasformazione.
- **Transizioni come valori di ritorno**: `request_review` su `Draft` restituisce un nuovo `PendingReview` boxed; `PendingReview::request_review` restituisce se stesso (richiedere una review di un post già in review non cambia nulla); `approve` funziona allo stesso modo, con `Draft::approve` che restituisce `self` (nessun effetto), `PendingReview::approve` che restituisce un `Published`, e `Published` che restituisce se stesso per entrambi.

Il vantaggio emerge qui: il metodo `request_review` su `Post` è lo **stesso qualunque sia lo stato**, perché ogni stato è responsabile delle proprie regole. Infine `content` su `Post` **delega** allo stato: chiama un metodo `content` sul valore in `state` passando il post come argomento. Tre dettagli tecnici completano l'implementazione.

- **as_ref**: serve una reference al valore dentro l'`Option` (`Option<&Box<dyn State>>`) e non ownership, perché non si può muovere `state` fuori dal `&self` del parametro della funzione.
- **unwrap giustificato**: sappiamo che `state` conterrà sempre un `Some` quando i metodi hanno finito, anche se il compilatore non può capirlo — uno dei casi del capitolo sugli errori in cui abbiamo più informazioni del compilatore.
- **Default del trait**: il metodo `content` su `State` ha un'implementazione di default che restituisce una stringa vuota, così `Draft` e `PendingReview` non implementano nulla; solo `Published` la sovrascrive restituendo `post.content`. Il metodo richiede annotazioni di lifetime (reference a un `post` in ingresso e reference a una parte di esso in uscita). Questo però **sfa i confini** di responsabilità tra `State` e `Post`: è una convenienza a costo di un compromesso di design.

Info

Perché non una enum

Un'alternativa sarebbe una enum con gli stati come varianti: certamente possibile, e vale la pena provarla per confronto. Lo svantaggio è che ogni posto che controlla il valore dell'enum avrebbe bisogno di un'espressione `match` (o simile) per gestire ogni variante possibile, il che può diventare più ripetitivo della soluzione a trait object — dove la logica delle regole vive negli state object anziché essere sparsa.

I trade-off del state pattern

Il funzionamento è completo, ma l'organizzazione del codice ha costi che vale la pena conoscere.

- **Località delle regole**: per capire i diversi modi in cui un post pubblicato può comportarsi basta guardare un solo posto, l'implementazione di `State` su `Published`; senza il pattern servirebbero espressioni `match` sparse nei metodi di `Post` o nel codice che lo usa, e per capire le implicazioni di uno stato bisognerebbe guardare più posti.
- **Estensibilità**: aggiungere un nuovo stato richiede solo una nuova struct che implementa i metodi del trait in un solo posto.
- **Accoppiamento**: siccome sono gli stati a implementare le transizioni, alcuni stati sono **accoppiati tra loro**: aggiungere uno stato `Scheduled` tra `PendingReview` e `Published` richiede modificare `PendingReview` perché transiti verso `Scheduled`; meno lavoro sarebbe se `PendingReview` non dovesse cambiare, ma richiederebbe un pattern diverso.
- **Duplicazione**: parte della logica è duplicata. Eliminarla con implementazioni di default di `request_review` e `approve` che restituiscono `self` **non funziona**: con `State` come trait object il compilatore non conosce il `self` concreto, quindi il tipo di ritorno non è noto a compile time (una delle regole di dyn compatibility). L'altra duplicazione è nei metodi di `Post`, che usano tutti `Option::take` e delegano: con molti metodi di questo tipo, si potrebbe definire una macro per eliminare la ripetizione.

Codificare gli stati e il comportamento come tipi

Implementando il pattern esattamente come definito per i linguaggi orientati agli oggetti non sfruttiamo a pieno i punti di forza di Rust. L'alternativa **codifica gli stati in tipi diversi**, così che il sistema di tipi impedisca gli stati e le transizioni **invalide** con errori a compile time. Il ridisegno lavora su tre punti.

- **Metodo assente anziché vuoto**: invece di un metodo `content` su un post draft che restituisce una stringa vuota, il post draft **non ha il metodo `content` affatto**: chiamarlo è un errore di compilazione che dice che il metodo non esiste. È impossibile mostrare per sbaglio il contenuto di un draft in produzione, perché quel codice non compila.
- **Tipi distinti**: `Post::new` restituisce un `DraftPost` (non un `Post`); la struct `Post` rappresenta un post **pubblicato** ed è l'unica con `content`; `DraftPost` ha `add_text` ma non `content`, e siccome `content` è privato e nessuna funzione restituisce `Post`, non c'è modo di crearne un'istanza direttamente.
- **Transizioni che consumano**: `request_review` su `DraftPost` restituisce un `PendingReviewPost`, e `approve` su `PendingReviewPost` restituisce un `Post`; entrambe prendono ownership di `self`, consumando l'istanza precedente e trasformandola, quindi non restano istanze `DraftPost` in giro dopo la transizione. L'unico modo per ottenere un `Post` pubblicato (con `content`) è chiamare `approve` su un `PendingReviewPost`, e l'unico modo per ottenere un `PendingReviewPost` è `request_review` su un `DraftPost`: **il workflow del blog post è codificato nel sistema di tipi**.

Questo secondo approccio non segue più fedelmente il pattern a oggetti (le transizioni non sono incapsulate dentro l'implementazione di `Post`, e in `main` servono riassegnazioni con shadowing dei valori restituiti), ma il guadagno è che gli stati invalidi sono **impossibili** grazie al type checking a compile time — certi bug, come mostrare il contenuto di un post non pubblicato, vengono scoperti prima di arrivare in produzione. Sebbene Rust sia capace di implementare i pattern orientati agli oggetti, altri pattern, come codificare lo stato nel sistema di tipi, hanno trade-off diversi: ripensare il problema per sfruttare le feature di Rust — come l'ownership, che i linguaggi OOP non hanno — può fornire benefici che un pattern a oggetti non ottiene.

I trade-off di design

Un ultimo tema trasversale del capitolo merita menzione: per essere un ingegnere Rust efficace non basta sapere come funziona il linguaggio, ma bisogna decidere **quali** dei suoi molti strumenti sono appropriati per un dato compito. Il caso emblematico è una configurazione globale da passare come reference immutabile in tutto il programma, con tre design candidati.

- **Reference**: una struct che trattiene `&'a Config`.
- **Reference counting**: una struct che trattiene `Rc<Config>`.
- **Atomic reference counting**: una struct che trattiene `Arc<Config>`.

Dato solo il contesto, tutti e tre sono candidati potenziali: serve una **nuova esigenza** per decidere. Se la reference alla configurazione dev'essere condivisibile tra più thread, la risposta è dettata dai marker trait del capitolo sulla concorrenza: assumendo `Config: Send + Sync`, sia `&Config` sia `Arc<Config>` soddisfano l'esigenza (implementano `Send` e `Sync`), mentre `Rc` no, perché i puntatori non atomici non sono thread-safe. Il metodo generale è esattamente questo: enunciare le esigenze del sistema e usare il sistema di tipi per **derivare** i design che le soddisfano, invece di scegliere per abitudine.

Sintesi della lezione

Che si consideri Rust orientato agli oggetti o no, ora sappiamo come ottenere alcune feature orientate agli oggetti con i trait object. Il quadro finale è il seguente.

- **Trait objects**: permettono collezioni eterogenee di tipi che condividono un comportamento, estendibili dagli utenti della libreria, con la garanzia statica che il trait sia implementato.
- **Dynamic dispatch**: dà flessibilità in cambio di un po' di prestazioni a runtime (lookup del metodo, niente inlining).
- **Pattern OOP**: sono implementabili (state pattern con `Box<dyn State>`, `Option::take`, `self: Box<Self>`), ma i punti di forza di Rust — ownership, sistema di tipi — offrono alternative come gli stati codificati come tipi, che rendono gli stati invalidi errori di compilazione.
- **Design guidato dalle esigenze**: la scelta tra reference, `Rc`, `Arc`, generici o trait object va derivata dai requisiti (thread safety, omogeneità, estensibilità), non dall'abitudine.

Il prossimo capitolo tratta i pattern, un'altra feature di Rust che offre molta flessibilità: finora li abbiamo usati senza vederne l'intera potenza.

Capitolo 19

Pattern e matching

Definition

Pattern

I pattern sono una sintassi speciale di Rust per confrontare valori con la **struttura dei tipi**, sia complessi sia semplici. Un pattern consiste di una combinazione di letterali, array/enum/struct/tuple destrutturati, variabili, wildcard e segnaposto. Esempi sono `x`, `(a, 3)` e `Some(Color::Red)`. Nei contesti in cui sono validi, queste componenti descrivono la **forma** dei dati: il programma confronta i valori con i pattern per determinare se hanno la forma giusta per far girare un particolare pezzo di codice, e se il valore si adatta alla forma del pattern, le parti nominate possono essere usate nel codice associato.

Usare i pattern insieme alle espressioni `match` e agli altri costrutti dà più controllo sul flusso del programma — come già visto con le espressioni `match` del capitolo sulle enum (l'esempio della macchina smistamonete). Questo capitolo è un riferimento completo: copre tutti i posti validi in cui usare i pattern, la differenza tra pattern refutabile e irrefutabile, e tutta la sintassi dei pattern.

Tutti i posti dove si possono usare i pattern

I pattern compaiono in molti posti in Rust, e li abbiamo usati a lungo senza accorgercene. Vale la pena passarli in rassegna tutti, con il costrutto formale di ciascuno.

I bracci di match

Formalmente, un'espressione `match` è composta dalla parola chiave `match`, un valore su cui fare il matching, e uno o più **bracci** (*match arm*), ciascuno consistente di un pattern e un'espressione da eseguire se il valore combacia col pattern del braccio. I pattern sono le parti a sinistra della freccia, come `None` e `Some(i)` in un match su un `Option<i32>`.

I requisiti e gli strumenti di completezza sono i seguenti.

- **Esauribilità**: le espressioni `match` devono essere **esaurienti**: tutte le possibilità del valore devono essere considerate, altrimenti il programma non compila.
- **Catch-all**: un modo per coprire ogni possibilità è un pattern finale che combacia con qualsiasi valore, ad esempio un nome di variabile, che non può mai fallire e quindi copre tutti i casi rimanenti.
- **Underscore**: il pattern `_` combacia con qualsiasi cosa ma **non si lega mai a una variabile**, quindi è spesso usato nell'ultimo braccio per ignorare i valori non specificati.

Le istruzioni let

Ogni volta che abbiamo scritto `let x = 5;` abbiamo usato un pattern senza saperlo. La forma formale è `let PATTERN = EXPRESSION;`: il nome di variabile nella posizione del pattern è una forma di pattern particolarmente semplice, che significa "lega tutto alla variabile, qualunque sia il valore". L'aspetto di pattern matching di `let` si vede nella destrutturazione delle tuple:

```rust
let (x, y, z) = (1, 2, 3);
```

Rust confronta il valore `(1, 2, 3)` col pattern `(x, y, z)`, vede che combaciano — il numero di elementi è lo stesso in entrambi — e lega `1` a `x`, `2` a `y` e `3` a `z`. Il pattern tupla si può pensare come tre pattern di variabile annidati. Se il numero di elementi del pattern non combacia con quello della tupla, il tipo complessivo non combacia e si ottiene un errore di compilazione: destrutturare una tupla di tre elementi in due variabili non funziona; la correzione è ignorare uno o più valori con `_` o `..`, o togliere variabili finché il numero combacia.

Le espressioni condizionali if let

Oltre a usare `if let` come forma abbreviata di un `match` con un solo caso, si possono **mescolare** `if let`, `else if` ed `else if let`, ottenendo più flessibilità di un `match` (che può esprimere un solo valore da confrontare con i pattern); inoltre Rust non richiede che le condizioni della serie si rapportino tra loro. L'esempio del colore di sfondo decide in

base a: il colore preferito dell'utente se specificato; altrimenti verde se oggi è martedì; altrimenti viola o arancio se l'età specificata come stringa si può fare il parsing in un numero (a seconda del valore); altrimenti blu. La struttura condizionale permette di supportare requisiti complessi.

Un dettaglio importante riguarda lo shadowing: `if let Ok(age) = age` introduce una **nuova** variabile `age` col valore dentro la variante `Ok`, ombreggiando quella esistente — proprio come possono fare i bracci di un `match`. Questo significa che la condizione `if age > 30` va **dentro** quel blocco: non si può combinare tutto in `if let Ok(age) = age && age > 30`, perché la nuova `age` non è valida finché il nuovo scope non inizia con la parentesi graffa. Il lato negativo di `if let` è che il compilatore **non verifica l'esauribilità**: omettendo l'ultimo `else` e mancando così dei casi, nessun compilatore segnala il possibile bug di logica — cosa che con `match` avverrebbe.

I cicli condizionali while let

Simile a `if let` per costruzione, il ciclo condizionale `while let` fa girare un ciclo `while` **finché il pattern continua a combaciare**. L'esempio attende messaggi tra thread controllando un `Result` invece di un `Option`: il metodo `recv` estrae il primo messaggio dal ricevente del canale e restituisce un `Ok(value)` a ogni messaggio in arrivo (finché esiste il trasmittente), producendo un `Err` quando la parte trasmittente si disconnette — quindi il ciclo `while let` stampa i valori finché `rx.recv()` continua a restituire `Ok`.

I cicli for

In un ciclo `for`, il valore che segue direttamente la parola chiave `for` è un pattern. Con l'adattatore `enumerate` l'iteratore produce una tupla con valore e indice, e il pattern `(index, value)` la destruttura: al primo giro la tupla `(0, 'a')` lega `index` a 0 e `value` a `'a'`.

I parametri delle funzioni

Anche i parametri delle funzioni sono pattern: in `fn foo(x: i32)` la parte `x` è un pattern, la forma semplice come in `let`. Come con `let`, si può far combaciare una tupla negli argomenti col pattern: passando `&(3, 5)` a una funzione il cui parametro ha il pattern `&(x, y)`, `x` lega 3 e `y` lega 5. Le liste di parametri delle **closure** supportano gli stessi pattern, dato che le closure sono simili alle funzioni.

La refutabilità

I pattern esistono in due forme, e la distinzione non va memorizzata per uso quotidiano, ma serve per saperla riconoscere nei messaggi d'errore.

Definition

Pattern irrefutabile e refutabile

I pattern che combaciano per **qualsiasi** valore passato sono **irrefutabili**: ad esempio la variabile in `let x = 5;`, che combacia con qualunque cosa e non può fallire. I pattern che **possono fallire** per qualche possibile valore sono **refutabili**: in `if let Some(x) = a_value`, `Some(x)` è refutabile perché se il valore è `None` non combacia; lo stesso vale per `&[x, ..]` su uno slice con zero elementi.

I costrutti si dividono esattamente secondo la refutabilità.

- **Solo irrefutabili**: i parametri delle funzioni, le istruzioni `let` e i cicli `for` accettano solo pattern irrefutabili, perché il programma non può fare nulla di sensato quando i valori non combaciano. Usare `let Some(x) = some_option_value;` non compila: se il valore fosse `None` il pattern fallirebbe, e il `let` non ha niente di valido da fare con un `None` — Rust segnala correttamente l'errore.
- **Entrambi**: `if let`, `while let` e l'istruzione `let...else` accettano pattern refutabili **e** irrefutabili, ma il compilatore mette in guardia contro gli irrefutabili perché, per definizione, quei costrutti servono a gestire il possibile fallimento: la loro funzionalità sta nel comportarsi diversamente a seconda di successo o fallimento.

La correzione tipica passa per cambiare il costrutto, non solo il pattern: con un pattern refutabile dove serve un irrefutabile si passa da `let` a `let...else`, che con un blocco gestisce il caso di non combaciamento — "abbiamo dato al codice una via d'uscita". Viceversa, dare a `let...else` un pattern che combacia sempre (come un nome di variabile) genera un warning: non ha senso usare un costrutto condizionale con un pattern irrefutabile. Ne segue la regola per i `match`: tutti i bracci possono (e in genere devono) usare pattern refutabili, tranne l'**ultimo**, che deve coprire i valori rimanenti con un pattern irrefutabile; un `match` con un solo braccio irrefutabile è permesso ma inutile, sostituibile da un semplice `let`.

La sintassi dei pattern

Questa sezione raccoglie tutta la sintassi valida nei pattern, con il motivo e il contesto d'uso di ciascuna.

Letterali e variabili nominate

I pattern possono combaciare direttamente con i **letterali**, utile quando il codice deve fare qualcosa a un valore concreto particolare. Le **variabili nominate** sono pattern irrefutabili che combaciano con qualsiasi valore — ma dentro `match`, `if let` e `while let` c'è una complicazione: poiché ognuna di queste espressioni **inizia un nuovo scope**, le variabili dichiarate come parte di un pattern ombreggiano quelle omonime fuori dal costrutto. L'esempio classico va ricostruito per intero, perché il risultato sorprende.

- **Setup**: `x` uguale a `Some(5)` e `y` uguale a `10`, poi un `match` su `x`.
- **Primo braccio**: `Some(50)` non combacia con `Some(5)`, si prosegue.
- **Secondo braccio**: il pattern introduce una **nuova** variabile `y` che combacia con qualsiasi valore dentro un `Some`: essendo in un nuovo scope, **non** è la `y` esterna col valore 10; questa nuova `y` si lega al valore interno di `Some`, cioè 5, e il braccio stampa "Matched, y = 5".
- **Ipotesi `None`**: con `x` uguale a `None`, i primi due bracci non combacerebbero e vincerebbe l'underscore; il braccio non introduce una `x` nuova, quindi la `x` usata lì sarebbe quella esterna (non ombreggiata).
- **Fine scope**: terminato il `match` finisce anche lo scope della `y` interna: la stampa finale mostra `x = Some(5), y = 10`.

Per **confrontare** i valori delle variabili esterne anziché introdurre nuove variabili che le ombreggiano servono i match guard, trattati più avanti.

Pattern multipli con la barra verticale

L'operatore `|` è l'operatore *or* dei pattern: in un `match`, un braccio con `1 | 2` esegue il proprio codice se il valore combacia con uno dei due.

Intervalli di valori con ..=

La sintassi `..=` permette di combaciare con un intervallo **inclusivo** di valori: con `1..=5` il primo braccio vince per `x` uguale a 1, 2, 3, 4 o 5. È più comodo dell'operatore `|` (che richiederebbe `1 | 2 | 3 | 4 | 5`), soprattutto per intervalli ampi come qualunque numero tra 1 e 1000. Due vincoli caratterizzano questa sintassi.

- **Solo numerici o char**: il compilatore controlla a compile time che l'intervallo **non sia vuoto**, e i soli tipi per cui Rust può stabilirlo sono i valori numerici e i `char`; gli intervalli sono quindi permessi solo su questi tipi.
- **Esempio char**: con `'a'..='j'` e `'k'..='z'`, il valore `'c'` combacia col primo intervallo.

Destrukturare i valori

I pattern permettono di destrutturare struct, enum e tuple per usare le parti diverse di questi valori separatamente. La destrutturazione funziona su più livelli.

- **Struct**: il pattern `let Point { x: a, y: b } = p;` crea le variabili `a` e `b` dai campi `x` e `y`; i nomi delle variabili non devono per forza combaciare coi campi, ma è consuetudine farlo — e siccome `let Point { x: x, y: y } = p;` contiene molta duplicazione, Rust offre lo **shorthand dei campi di struct**: elencare solo il nome del campo crea variabili con lo stesso nome. In un `match` si possono anche usare **letterali** dentro il pattern struct anziché variabili: il pattern `Point { x: 0, y }` combacia solo con i punti sull'asse y (campo `x` uguale a zero) e destruttura il resto in variabili — test e binding nello stesso pattern. Il `match` si ferma al primo pattern che combacia: un punto sull'origine combacia sia col braccio dell'asse x sia con quello dell'asse y, ma solo il primo viene eseguito.
- **Enum**: il pattern per destrutturare un enum rispecchia il modo in cui i dati sono definiti nella variante: per una variante senza dati (come `Message::Quit`) si può solo combaciare col valore letterale, senza destrutturare oltre; per varianti simili a struct si usano le parentesi graffe coi campi (con shorthand); per varianti simili a tuple il numero di variabili nel pattern deve combaciare col numero di elementi.
- **Annidamento**: il matching funziona anche su item annidati: un pattern può combaciare una variante `Message::ChangeColor` che contiene una variante interna `Color::Rgb`, legando i tre `i32` interni — condizioni complesse su **due** enum in un'unica espressione `match`. Struct e tuple si possono mescolare e annidare in modi arbitrari, destrutturando tutti i valori primitivi da una combinazione complessa.

Ignorare i valori in un pattern

Ci sono quattro modi di ignorare valori interi o parti di essi, ciascuno con una semantica precisa.

- **Underscore per un valore intero**: `_` è la wildcard che combacia con qualsiasi valore ma **non si lega**; oltre che nell'ultimo braccio di un `match`, si può usare ovunque, inclusi i parametri delle funzioni. Ignorare un parametro è utile ad esempio nell'implementare un trait con una firma richiesta quando il corpo non usa uno dei parametri: con `_` si evita il warning di parametro non usato.
- **Underscore annidato per parti di un valore**: dentro un altro pattern, `_` ignora solo una parte: nel caso dell'impostazione che non va sovrascritta se già personalizzata, il braccio `Some(_)` combacia con `Some` senza usare il valore interno, mentre il braccio `_` copre tutti gli altri casi. Si possono usare underscore in più punti dello stesso pattern per ignorare, ad esempio, il secondo e il quarto valore di una tupla di cinque.
- **Nomi che iniziano con underscore**: Rust emette un warning per le variabili non usate (potrebbero essere bug), ma a volte si creano variabili che non si useranno subito, ad esempio nel prototipazione; prefissare il nome con `_` dice a Rust di non avvisare.

Warning

La differenza tra `__` e `_x`

C'è una differenza sottile tra `_` da solo e un nome che inizia con underscore: `_x` **lega** il valore alla variabile, mentre `_` non si lega affatto. Conseguenza concreta con l'ownership: in un `match` su una `String`, il braccio `_s` **sposta** il valore dentro `_s` e impedisce di usare `s` di nuovo (errore "value used after move"), mentre il braccio `_` non sposta mai nulla e il codice compila. La stessa distinzione vale per i parametri di funzione e per ogni altro contesto.

- **`..` per le parti restanti**: con valori con molte parti, la sintassi `..` ignora qualsiasi parte non esplicitamente abbinata nel resto del pattern — più rapido che elencare `y: _` e `z: _` in una struct con molti campi di cui uno o due sono rilevanti, e si espande a quanti valori serve, ad esempio in una tupla per abbinare solo il primo e l'ultimo valore. L'uso però dev'essere **non ambiguo**: con `..` usato due volte, Rust non può determinare quanti valori ignorare prima di legare `second` e quanti dopo — il nome della variabile non significa nulla di speciale per il compilatore, e si ottiene un errore di compilazione.

Condizioni aggiuntive con i match guard

Definition

Match guard

Un match guard è una condizione `if` aggiuntiva, specificata **dopo** il pattern in un braccio di `match`, che deve essere vera perché quel braccio sia scelto. Permette di esprimere idee più complesse di quanto un pattern da solo consenta, e la condizione può usare le variabili create nel pattern. Sono disponibili **solo** nelle espressioni `match`, non in `if let` o `while let`.

L'esempio canonico è la condizione di parità, non esprimibile in un pattern: il braccio `Some(x) if x % 2 == 0` combacia `Some(4)` e poi verifica che il resto della divisione per 2 sia zero. Se il guard è falso (ad esempio con `Some(5)`), Rust passa al braccio successivo, che senza guard combacia con qualsiasi `Some`. Due proprietà completeano il quadro.

- **Costo dell'espressività**: il compilatore **non verifica l'esauribilità** quando sono coinvolti i match guard.
- **Soluzione dello shadowing**: il guard risolve il problema delle variabili esterne ombreggiate: invece del pattern `Some(y)` (che introdurrebbe una nuova `y` ombreggiante), si scrive `Some(n) if n == y` — il guard **non è un pattern** e non introduce variabili, quindi la sua `y` è quella esterna, e si può confrontare il valore interno con la variabile esterna.

Info

Precedenza del guard rispetto alla barra verticale

Combinando l'operatore `|` con un match guard, la condizione si applica a **tutti** i pattern del braccio: in `4 | 5 | 6 if y` il guard si applica a 4, 5 **e** 6, anche se potrebbe sembrare applicato solo all'ultimo valore. La precedenza è quella di `(4 | 5 | 6) if y`, non di `4 | 5 | (6 if y)`: con `x` uguale a 4 e `y` falso, il primo braccio non viene scelto e il match prosegue al braccio successivo.

Binding con la chiocciola

L'operatore **at** `@` permette di creare una variabile che trattiene un valore **allo stesso tempo** in cui lo si testa per il combaciamento col pattern: specificando `id @ 3..=7` si **cattura** in una variabile `id` qualunque valore combaci con l'intervallo, oltre a verificare che il valore sia nell'intervallo. Il confronto con i bracci alternativi chiarisce il ruolo di `@`.

- **Solo intervallo**: un braccio con solo `3..=7` non offre al codice nessuna variabile col valore effettivo: potrebbe essere 10, 11 o 12, ma il codice del braccio non sa quale.
- **Solo variabile**: un braccio con la sola variabile (shorthand del campo) ha il valore disponibile, ma **nessun test**: qualsiasi valore combacerebbe.
- **`@`**: unisce i due mondi — testare un valore e salvarlo in una variabile dentro un unico pattern.

Sintesi della lezione

I pattern di Rust sono molto utili per distinguere tipi diversi di dati. Il quadro finale è il seguente.

- **In `match`**: Rust garantisce che i pattern coprano ogni valore possibile, altrimenti il programma non compila.
- **In `let` e nei parametri**: i pattern rendono questi costrutti più utili, abilitando la destrutturazione dei valori in parti più piccole assegnate a variabili.
- **Refutabilità**: i costrutti condizionali (`if let`, `while let`, `let...else`) accettano pattern refutabili; `let`, `for` e le funzioni solo irrefutabili.
- **Strumenti**: multi-pattern con `|`, intervalli `..=`, destrutturazione annidata, ignoramento con `_`, `_x` e `..`, match guard e binding `@` — semplici o complessi a seconda delle esigenze.

Per il capitolo successivo — il penultimo del libro — si passa ad alcuni aspetti avanzati di una varietà di feature di Rust.

Capitolo 20

Feature avanzate

Questo capitolo raccoglie gli aspetti del linguaggio che si incontrano solo occasionalmente: sono utili in situazioni molto specifiche e conviene conoscerli come riferimento, perché compaiono nei messaggi di errore del compilatore e nel codice altrui. Le aree trattate sono cinque: Rust unsafe, i trait avanzati, i tipi avanzati, le funzioni e le closure avanzate e le macro.

Rust unsafe

Tutto il codice visto finora era sottoposto in fase di compilazione alle garanzie di sicurezza della memoria di Rust. Il linguaggio però nasconde un secondo linguaggio, chiamato Rust unsafe, che funziona come il Rust regolare ma non fa rispettare quelle garanzie e dà accesso a capacità aggiuntive.

Perché esiste Rust unsafe

La static analysis per natura è conservativa: quando il compilatore non ha informazioni sufficienti per essere certo che il codice rispetti le garanzie, rifiuta il programma, anche se sarebbe valido, perché è meglio rifiutare alcuni programmi validi che accettarne alcuni non validi. Il blocco `unsafe` serve a dire al compilatore che sappiamo cosa stiamo facendo, assumendocene la responsabilità. Una seconda ragione è che l'hardware sottostante è intrinsecamente insicuro: senza operazioni unsafe non si potrebbero fare la programmazione di sistemi a basso livello, come interagire direttamente con il sistema operativo o scriverne uno, che è uno degli obiettivi del linguaggio.

Warning

Cosa NON disattiva unsafe

Il blocco `unsafe` **non** disattiva il borrow checker né le altre verifiche di sicurezza: un riferimento usato dentro un blocco unsafe viene comunque controllato. `unsafe` dà accesso solo alle cinque operazioni non verificate e non significa che il codice nel blocco sia necessariamente pericoloso o errato: è il programmatore che deve garantire che l'accesso alla memoria sia valido. Gli errori di sicurezza della memoria possono quindi verificarsi solo dentro un blocco `unsafe`, il che li rende rintracciabili: conviene tenere i blocchi `unsafe` il più piccoli possibile e, quando serve, isolarli dentro un'astrazione sicura con un'API sicura, come fa la libreria standard, così che l'unsafe non si propaghi a tutti i punti d'uso.

Le cinque azioni unsafe

Solo all'interno di un blocco `unsafe` si possono compiere cinque azioni, spesso chiamate superpoteri unsafe. Vediamole in dettaglio.

- **Dereferenziare un raw pointer**: accesso alla memoria tramite i puntatori grezzi.
- **Chiamare una funzione o un metodo unsafe**: assunzione dei contratti documentati dalla funzione.
- **Accedere o modificare una variabile static mutabile**: gestione manuale della concorrenza sui globali.
- **Implementare un trait unsafe**: promessa di rispettare invarianti non verificabili dal compilatore.
- **Accedere ai campi di una `union`**: lettura di memoria interpretata con più tipi possibili.

I raw pointer

I raw pointer sono due nuovi tipi simili ai riferimenti: `*const T` immutabile e `*mut T` mutabile (l'asterisk fa parte del nome del tipo, non è l'operatore di dereferenziazione; qui immutabile significa che non si può assegnare direttamente dopo la dereferenziazione). Rispetto a riferimenti e smart pointer hanno quattro proprietà che li distinguono.

- **Regole di borrowing ignorate**: possono coesistere più puntatori mutabili o immutabili sullo stesso dato.
- **Validità non garantita**: non è garantito che puntino a memoria valida.
- **Annullabilità**: possono essere null.
- **Nessuna pulizia automatica**: non implementano alcuna deallocazione.

Si creano con gli operatori di raw borrow `&raw const x` e `&raw mut x`, oppure con un cast `as`, ed è possibile crearli anche in codice sicuro: è la dereferenziazione con `*` che richiede il blocco `unsafe`. Un puntatore a un indirizzo arbitrario (per esempio ottenuto castando un intero) è comportamento indefinito: potrebbe non esserci nulla, il compilatore potrebbe ottimizzare via l'accesso o il programma potrebbe terminare con un segmentation fault. Creare il puntatore non fa danno; è l'accesso al valore puntato che può essere invalido. Con i raw pointer si possono anche creare un puntatore

mutabile e uno immutabile alla stessa posizione e modificarla, creando potenzialmente una data race: i principali casi d'uso sono l'interfacciamento con codice C e la costruzione di astrazioni sicure che il borrow checker non sa capire.

Funzioni e metodi unsafe

Una funzione unsafe si dichiara anteponendo `unsafe` alla definizione; il significato è che la funzione ha requisiti che il chiamante deve rispettare e che Rust non può garantire. Chiamarla dentro un blocco `unsafe` equivale ad affermare di averne letto la documentazione e di assumersi la responsabilità del contratto. Anche dentro il corpo di una funzione unsafe le operazioni unsafe richiedono un proprio blocco `unsafe`: così i blocchi restano piccoli e localizzati.

L'esempio canonico di astrazione sicura sopra codice unsafe è il metodo `split_at_mut` della libreria standard, che divide una slice mutabile in due all'indice dato. In Rust sicuro non è implementabile: il borrow checker non capisce che si prendono in prestito due parti diverse e non sovrapposte della stessa slice, e segnala due borrow simultanei. L'implementazione reale usa `as_mut_ptr` per ottenere il raw pointer della slice e la funzione unsafe `slice::from_raw_parts_mut` per costruire le due slice, con un'asserzione preventiva che l'indice sia dentro i limiti. La funzione risultante però non è `unsafe`: è un'astrazione sicura che usa `unsafe` solo internamente, perché genera solo puntatori validi ai dati a cui ha accesso. Al contrario, chiamare `slice::from_raw_parts_mut` su un indirizzo arbitrario produce quasi certamente un crash, perché non si possiede quella memoria e non c'è garanzia che contenga valori validi.

`extern` e le FFI

A volte Rust deve interagire con codice scritto in un altro linguaggio: la keyword `extern` serve a costruire una Foreign Function Interface (FFI), il modo in cui un linguaggio definisce funzioni richiamabili da un linguaggio diverso.

- **Chiamare funzioni esterne**: dentro un blocco `unsafe extern "C"` si dichiarano i nomi e le firme delle funzioni esterne da chiamare; `"C"` indica l'application binary interface (ABI), cioè come la funzione va chiamata a livello di assembly. Gli elementi dichiarati nel blocco sono implicitamente unsafe perché gli altri linguaggi non rispettano le regole di Rust; le funzioni effettivamente sicure possono essere marcate `safe` per poterle chiamare senza blocco `unsafe`, ma è una promessa del programmatore, non una verifica.
- **Chiamare Rust da altri linguaggi**: si aggiunge `extern "C"` prima del `fn` e l'annotazione `#[unsafe(no_mangle)]`, che impedisce al compilatore di alterare il nome della funzione (il name mangling). Serve perché ogni compilatore altera i nomi in modo diverso ed è compito del programmatore evitare collisioni di nomi tra librerie.

```rust
#[unsafe(no_mangle)]
pub extern "C" fn call_from_c() {
    println!("Chiamata da C!");
}
```

Variabili statiche mutabili

Le variabili globali di Rust si chiamano variabili statiche. Quelle immutabili sono sicure da leggere: seguono la convenzione `SCREAMING_SNAKE_CASE`, possono contenere solo riferimenti con lifetime `'static` e, a differenza delle costanti i cui valori possono essere duplicati a ogni uso, occupano un indirizzo fisso in memoria. Una variabile statica può anche essere mutabile, ma leggerla o modificarla è unsafe: con più thread che vi accedono si avrebbero data race, un comportamento indefinito. Il compilatore nega per default anche la creazione di riferimenti a statiche mutabili tramite una lint: occorre l'annotazione `#[allow(static_mut_refs)]` oppure l'accesso via raw pointer. Quando è possibile conviene usare le tecniche di concorrenza e gli smart pointer thread-safe del Capitolo 16, che fanno verificare dal compilatore la sicurezza degli accessi. Per ogni funzione o operazione unsafe è idiomatico scrivere un commento che inizia con `SAFETY` e spiega come si rispettano le regole o cosa deve fare il chiamante.

Trait unsafe e union

Un trait è unsafe quando almeno uno dei suoi metodi ha un'invariante che il compilatore non può verificare: si dichiara con `unsafe trait` e l'implementazione con `unsafe impl`, promettendo di rispettare quelle invarianti. È il caso dei marker trait `Send` e `Sync`: il compilatore li implementa automaticamente solo se il tipo è composto interamente da tipi che li implementano; se un tipo contiene per esempio raw pointer e lo si vuole marcare `Send`, bisogna farlo manualmente con `unsafe`. L'ultima azione riservata a `unsafe` è accedere ai campi di una `union`, simile a una `struct` ma in cui una sola istanza usa uno dei campi dichiarati: servono principalmente per interfacciarsi con il codice C e l'accesso è unsafe perché Rust non può garantire il tipo del dato attualmente immagazzinato.

Miri

Miri è uno strumento ufficiale di Rust che rileva comportamenti indefiniti: mentre il borrow checker è statico e agisce in compilazione, Miri è dinamico e agisce a runtime, eseguendo il programma o la suite di test e segnalando le violazioni delle regole del linguaggio. Richiede una build nightly (`rustup +nightly component add miri`) e si lancia con `cargo +nightly miri run` o `cargo +nightly miri test`. Due limiti sono essenziali.

- **Copertura solo del codice eseguito**: essendo un'analisi dinamica, trova problemi solo nel codice effettivamente eseguito, quindi va usato insieme a un buon testing.
- **Non onniscienza**: se Miri segnala un problema c'è sicuramente un bug, ma il contrario non vale; inoltre non copre ogni forma di unsoundness.

Usare una delle cinque azioni unsafe non è sbagliato né sconsigliato in sé: è solo più difficile scriverlo correttamente perché il compilatore non può aiutare. L'annotazione esplicita `unsafe` rende più facile risalire all'origine dei problemi; per un'esplorazione approfondita la guida ufficiale è il Rustonomicon.

Trait avanzati

I trait visti nel Capitolo 10 hanno dettagli più avanzati che conviene ora esaminare: tipi associati, parametri di tipo di default, overloading degli operatori, supertrait e il pattern newtype applicato ai trait.

Tipi associati

I tipi associati collegano un segnaposto di tipo a un trait, così che le firme dei metodi possano usare quei tipi; chi implementa il trait specifica il tipo concreto che sostituisce il segnaposto. In questo modo si può definire un trait che usa certi tipi senza sapere quali saranno finché il trait non viene implementato.

```rust
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}
```

L'esempio canonico è `Iterator`, il cui tipo associato `Item` rappresenta il tipo dei valori su cui si itera: `next` restituisce `Option<Self::Item>`. La differenza rispetto ai generici è fondamentale: con un parametro generico un trait può essere implementato più volte sullo stesso tipo cambiando i tipi concreti (per esempio `Iterator<u32> for Counter` e `Iterator<String> for Counter`), e ogni chiamata di `next` richiederebbe allora un'annotazione di tipo. Con i tipi associati esiste una sola implementazione per tipo (un solo `impl Iterator for Counter`): il tipo di `Item` si sceglie una volta sola, non servono annotazioni alle chiamate. Inoltre il tipo associato diventa parte del contratto del trait: ogni implementatore deve fornirlo, e documentarlo nell'API è una buona pratica.

Parametri di tipo di default e overloading degli operatori

Un parametro generico può avere un tipo concreto di default, con la sintassi `<TipoSegnaposto=TipoConcreto>`: chi implementa il trait non deve specificarlo se il default va bene. Il caso d'uso principale è l'overloading degli operatori.

- **Overloading consentito**: Rust non permette di creare operatori propri o di overloadare operatori arbitrari, ma permette di implementare i trait elencati in `std::ops` associati agli operatori, come `Add` per `+`.
- **Il default di `Add`**: il trait `Add<Rhs=Self>` ha il tipo associato `Output` e il metodo `add(self, rhs: Rhs) -> Self::Output`; se non si specifica `Rhs`, il tipo dell'operando destro è `Self`, cioè due valori dello stesso tipo.
- **Personalizzare `Rhs`**: implementando `Add<Meters>` per un tipo `Millimeters` (due newtype che incapsulano `u32`) si può sommare millimetri a metri convertendo correttamente, specificando il tipo del parametro destro invece del default.

```rust
trait Add<Rhs=Self> {
    type Output;
    fn add(self, rhs: Rhs) -> Self::Output;
}
```

I parametri di tipo di default servono in due modi: estendere un tipo esistente senza rompere il codice già scritto (aggiungendo un nuovo parametro con un default) e permettere personalizzazioni che la maggior parte degli utenti non

usa, riducendo il boilerplate.

Disambiguare metodi con lo stesso nome

Nulla impedisce a due trait di avere metodi con lo stesso nome, né di implementarli entrambi sullo stesso tipo, insieme a un metodo intrinseco con lo stesso nome. La risoluzione segue due regole.

- **Metodi**: chiamando `person.fly()` il compilatore preferisce il metodo implementato direttamente sul tipo; per chiamare quello di un trait si scrive `Pilot::fly(&person)`, anteponendo il nome del trait. Se il metodo ha `self`, Rust capisce quale implementazione usare dal tipo del ricevente.
- **Funzioni associate non metodi**: senza `self` Rust non sempre può dedurre, e serve la sintassi pienamente qualificata, indicando dentro le parentesi angolari che si vuole trattare un tipo come implementazione di quel trait.

```rust
fn main() {
    println!("{}", <Dog as Animal>::baby_name());
}
```

La sintassi generale è `<Tipo as Trait>::funzione(ricevente_se_metodo, altri_argomenti)`. Si può omettere ogni parte che Rust riesce a dedurre, e serve la forma verbosa solo quando esistono più implementazioni con lo stesso nome.

Supertrait

Un trait può dipendere da un altro trait: per implementare il primo, il tipo deve implementare anche il secondo, così che la definizione del primo possa usare gli elementi associati del secondo. Il trait da cui si dipende si chiama supertrait e si dichiara come un trait bound sul trait stesso, per esempio `trait OutlinePrint: Display`, che permette al metodo `outline_print` di usare `to_string`, disponibile per ogni tipo che implementa `Display`. Implementare `OutlinePrint` su un tipo senza `Display` è un errore di compilazione: occorre prima implementare `Display`.

Il pattern newtype per la orphan rule

La orphan rule permette di implementare un trait su un tipo solo se il trait, il tipo o entrambi sono locali alla crate. La si aggira con il pattern newtype: creare una tuple struct con un solo campo che incapsula il tipo esterno. Il wrapper è locale alla crate, quindi il trait vi si può implementare; il termine newtype viene da Haskell e non c'è penalità a runtime perché il wrapper viene eliminato in compilazione. Per esempio, per implementare `Display` su `Vec<T>` si crea una struct `Wrapper` che contiene il `Vec<T>` e si implementa `Display` sul wrapper. Lo svantaggio è che il nuovo tipo non ha i metodi del valore incapsulato: occorre implementarli uno a uno delegando a `self.0`, oppure implementare `Deref` per averli tutti, oppure implementare solo quelli voluti per limitarne il comportamento.

Tipi avanzati

Il sistema dei tipi di Rust ha alcune funzionalità rimaste ai margini: il pattern newtype per la sicurezza dei tipi, gli alias di tipo, il never type e i tipi a dimensione dinamica.

Il newtype per sicurezza e astrazione

Oltre ad aggirare la orphan rule, il pattern newtype serve a far rispettare staticamente che certi valori non vengano confusi e a indicare l'unità di misura: una funzione con parametro `Millimeters` non compila se chiamata con un `Meters` o con un semplice `u32`. Inoltre può nascondere i dettagli implementativi: un tipo `People` che incapsula una `HashMap<i32, String>` può esporre un'API pubblica che non rivela l'uso di ID interni. È un modo leggero di ottenere incapsulamento, del tutto analogo a quanto visto per l'OOP nel Capitolo 18.

Alias di tipo

La keyword `type` dichiara un alias che dà un altro nome a un tipo esistente: per esempio `type Kilometers = i32`. A differenza del newtype, l'alias è un puro sinonimo, non un nuovo tipo: `Kilometers` e `i32` sono lo stesso tipo, si possono sommare valori dei due e mescolarli senza che il compilatore segnali errori, perdendo i benefici del controllo dei tipi.

- **Ridurre la ripetizione**: il caso d'uso principale è accorciare tipi verbosi come `Box<dyn Fn() + Send + 'static>`, sostituiti ovunque da un alias come `Thunk`; un nome significativo comunica anche l'intento (thunk indica codice valutato più tardi, appropriato per una closure memorizzata).
- **Interfacce coerenti**: la libreria standard dichiara `type Result<T> = std::result::Result<T, std::io::Error>` nel modulo `std::io`, così tutte le funzioni di I/O condividono la stessa forma `std::io::Result<T>`; essendo un alias, restano utilizzabili i metodi di `Result` e l'operatore `?`.

Il never type

Rust ha un tipo speciale `!`, chiamato empty type in teoria dei tipi perché non ha valori, ma noto come never type perché sta al posto del tipo di ritorno di una funzione che non ritorna mai. Le funzioni che ritornano never si chiamano funzioni divergenti: non potendo esistere valori di tipo `!`, non possono mai restituire. La sua utilità nasce nelle espressioni `match`, i cui rami devono avere tutti lo stesso tipo: espressioni di tipo `!` vengono coerce a qualunque altro tipo. Per questo un ramo che termina con `continue` (che non produce un valore, ma riporta il controllo in cima al loop) convive con un ramo che produce un `u32`, e il tipo complessivo è `u32`. Lo stesso avviene con `panic!`, che non produce un valore perché termina il programma: è così che in `unwrap` il ramo `panic!` (tipo `!`) e il ramo con `T` danno un risultato di tipo `T`. Anche un loop senza `break` ha valore `!`, perché non termina mai.

Tipi a dimensione dinamica e il trait `Sized`

Rust deve conoscere quanto spazio allocare per i valori di un tipo; esistono però i tipi a dimensione dinamica (DST, o unsized types), i cui valori hanno una dimensione conoscibile solo a runtime. Il caso centrale è `str` da solo (non `&str`): non si può creare una variabile di tipo `str` né passarla come argomento, perché due stringhe di lunghezze diverse occuperebbero spazi diversi mentre tutti i valori di un tipo devono avere la stessa dimensione. La soluzione è la regola d'oro dei DST.

Definition

Regola d'oro dei DST

I valori di un tipo a dimensione dinamica devono sempre stare dietro a un puntatore di qualche tipo, che porta con sé un po' di metadati in più che memorizza la dimensione dell'informazione dinamica. Una string slice `&str` è per questo due valori (l'indirizzo della `str` e la sua lunghezza), la cui dimensione è nota in compilazione; lo stesso vale per `Box<str>`, `Rc<str>` e per i trait, che sono tutti DST da usare dietro un puntatore come `&dyn Trait` o `Box<dyn Trait>`.

Per gestire i DST esiste il trait `Sized`, implementato automaticamente per ogni tipo di dimensione nota in compilazione. Due conseguenze sono importanti da ricordare.

- **Bound implicito**: ogni funzione generica riceve implicitamente un bound `Sized` (la definizione `fn f<T>(t: T)` è trattata come `fn f<T: Sized>(t: T)`), quindi per default funziona solo su tipi di dimensione nota.
- **`?Sized`**: la sintassi speciale `?Sized` su un bound significa che il tipo può essere o non essere `Sized`, e rilassa il default; la notazione `?Trait` con questo significato esiste solo per `Sized`. Quando il tipo può non essere sized, il parametro va usato dietro un puntatore, per esempio `&T` invece di `T`.

Funzioni e closure avanzate

Le funzioni ordinarie si possono passare come argomenti ad altre funzioni: le funzioni vengono coerte al tipo `fn` (con la f minuscola, da non confondere col trait `Fn`), chiamato puntatore a funzione. Il tipo `fn` è un tipo vero e proprio, non un trait, quindi si specifica direttamente come tipo del parametro senza vincoli generici.

```rust
fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {
    f(arg) + f(arg)
}
```

I puntatori a funzione implementano tutti e tre i trait delle closure (`Fn`, `FnMut` e `FnOnce`), quindi un puntatore a funzione si può sempre passare dove è attesa una closure: conviene scrivere funzioni generiche con i trait delle closure, così accettano entrambe. Il caso opposto è l'interfacciamento con codice esterno che non ha closure, come C, dove serve accettare solo `fn`. Anche gli inizializzatori delle varianti di enum (i cui nomi sono funzioni) si usano come puntatori a funzione, per esempio `map(Status::Value)`.

Ritornare closure

Le closure sono rappresentate da trait, quindi non si possono ritornare direttamente e non hanno un tipo concreto restituibile (il puntatore `fn` non è ammesso come tipo di ritorno se la closure cattura valori). In generale si usa la sintassi `impl Trait` per ritornare una closure: `fn make_adder() -> impl Fn(i32) -> i32`. Però ogni closure ha il suo tipo distinto: ritornare `impl Fn` da più funzioni genera tipi opachi diversi anche a firma identica, quindi una collezione `Vec` di closure provenienti da funzioni diverse con `impl Fn` non compila. La soluzione, vista più volte, è il trait object: `Box<dyn Fn(i32) -> i32>` dà a tutte lo stesso tipo.

Macro

Il termine macro indica una famiglia di funzionalità: le macro dichiarative con `macro_rules!` e tre tipi di macro procedurali, che esaminiamo dopo aver capito perché servono, visto che esistono già le funzioni.

Macro e funzioni a confronto

Le macro sono un modo di scrivere codice che scrive altro codice, ossia metaprogramming: si espandono producendo più codice di quello scritto a mano, come fa `derive` per i trait o `println!` e `vec!` usati in tutto il libro. Rispetto alle funzioni hanno due poteri aggiuntivi.

- **Numero variabile di parametri**: la firma di una funzione dichiara quanti e quali parametri ha, mentre `println!` si può chiamare con uno o più argomenti.
- **Espansione prima dell'interpretazione**: le macro si espandono prima che il compilatore interpreti il codice, quindi possono per esempio implementare un trait su un tipo, cosa impossibile per una funzione che viene chiamata a runtime.

Gli svantaggi sono due: le definizioni sono più complesse di quelle delle funzioni, perché si scrive Rust che genera Rust, e quindi più difficili da leggere e mantenere; inoltre una macro deve essere definita o portata in scope prima della chiamata nel file, mentre le funzioni si definiscono e chiamano ovunque.

Macro dichiarative con `macro_rules!`

La forma più diffusa è la macro dichiarativa, detta anche macro by example o macro `macro_rules!`. Funziona come un'espressione `match`: il valore confrontato è il codice sorgente letterale passato alla macro, i pattern si confrontano con la struttura di quel codice e il codice associato al pattern che combacia sostituisce la chiamata, tutto in fase di compilazione. La definizione dell'equivalente semplificato di `vec!` mostra la struttura.

```rust
#[macro_export]
macro_rules! vec {
    ($($x:expr),* $(,)?) => {
        {
            let mut temp_vec = Vec::new();
            $(temp_vec.push($x);)*
            temp_vec
        }
    };
}
```

L'annotazione `#[macro_export]` rende la macro disponibile ogni volta che la crate che la definisce è portata in scope. Il nome va dichiarato senza punto esclamativo, e il corpo contiene bracci pattern `=>` codice come un match. La sintassi dei pattern macro è diversa da quella del Capitolo 19 perché confronta struttura di codice, non valori: `$` dichiara una variabile del sistema di macro, `$()` cattura il codice che combacia, `$x:expr` combacia qualunque espressione Rust dandole il nome `$x`, la virgola dopo `$()` è il separatore letterale tra le ripetizioni e `*` indica zero o più ripetizioni. Nella parte di sostituzione, il codice dentro `$()*` viene generato una volta per ogni combaciamento. Con `vec![1, 2, 3]` il pattern combacia tre volte e genera il codice con tre `push`.

Macro procedurali

Le macro procedurali agiscono più come funzioni: ricevono codice in ingresso, lo trasformano e producono codice in uscita, invece di confrontare pattern e sostituire. Devono risiedere in una propria crate con tipo speciale `proc-macro` (per ragioni tecniche che si vorrebbero eliminare) e condividono tutte la stessa struttura: una funzione annotata che

riceve un `TokenStream` e produce un `TokenStream`. Il tipo `TokenStream`, definito dalla crate `proc_macro` inclusa in Rust, rappresenta una sequenza di token: il codice sorgente su cui la macro opera è l'input, il codice prodotto è l'output. Esistono tre tipi.

- **Custom `derive`**: aggiungono codice tramite l'attributo `#[derive(Nome)]` su struct ed enum.
- **Attribute-like**: definiscono attributi personalizzati applicabili a qualunque item, anche funzioni, e ricevono due `TokenStream`, il contenuto dell'attributo e l'item annotato.
- **Function-like**: sembrano chiamate di funzione, prendono un numero arbitrario di argomenti e manipolano i token dentro le parentesi, permettendo analisi più complesse di `macro_rules!`.

Un esempio di `derive` e l'ecosistema

Un custom derive tipico si costruisce con due crate strettamente correlate: una con il trait (per esempio `hello_macro` con il trait `HelloMacro`) e una con il suffisso convenzionale `_derive` per la macro (per esempio `hello_macro_derive`, pubblicata separatamente). La funzione della macro va annotata con `#[proc_macro_derive(HelloMacro)]`, dove il nome combacia con quello del trait. Due crate dell'ecosistema rendono il lavoro fattibile.

- **`syn`**: converte il `TokenStream` in una struttura dati (un `DeriveInput`) che rappresenta il codice Rust analizzato, con campi come `ident` per il nome del tipo.
- **`quote`**: riconverte le strutture di `syn` in codice Rust, con meccanismi di templating: la macro `quote!` accetta `#nome` e lo sostituisce con il valore della variabile, e il risultato va convertito in `TokenStream` con il metodo `into`.

La separazione in due funzioni è idiomatica: la funzione esterna (annotata) si occupa di analizzare il `TokenStream` ed è quasi identica in ogni crate di macro procedurali; la funzione interna trasforma l'albero sintattico ed è specifica allo scopo. Se l'analisi fallisce la funzione deve fare panic (per esempio con `unwrap`), perché l'API di `proc_macro_derive` richiede un `TokenStream` e non un `Result`. In un esempio del genere è utile anche la macro incorporata `stringify!`, che a compile time trasforma un'espressione in un letterale stringa senza valutarla, a differenza di `format!`, e risparmia un'allocazione. Nell'esempio `#[derive(HelloMacro)]` sulla struct `Pancakes` genera l'implementazione del trait che stampa il nome del tipo; Rust non ha reflection, quindi il nome può essere conosciuto solo in compilazione, generando codice.

Sintesi del capitolo

Queste funzionalità non si usano quotidianamente, ma conviene sapere che esistono: quando compaiono nei suggerimenti del compilatore o nel codice altrui, saperle riconoscere guida verso la soluzione. Il capitolo funziona da riferimento per le circostanze particolari in cui servono.

Capitolo 21

Un web server multithread

Il progetto finale del libro è un web server che risponde ai browser, costruito a mano senza librerie: essendo Rust un linguaggio di sistemi, si può scegliere il livello di astrazione e scendere più in basso di quanto sia pratico in altri linguaggi. Le scelte didattiche sono due: non si usano le crate pronte per la produzione presenti su crates.io, perché l'obiettivo è imparare le idee generali dietro quelle crate, e non si usano async e await, perché costruire un thread pool è già una sfida sufficiente senza aggiungere un runtime asincrono (il Capitolo 17 ha del resto mostrato che molti runtime async usano internamente thread pool). Il piano del progetto procede per passi: capire TCP e HTTP, ascoltare connessioni TCP su un socket, analizzare richieste HTTP, creare risposte HTTP ben formate e migliorare il throughput con un thread pool.

I protocolli

I due protocolli coinvolti in un web server sono l'Hypertext Transfer Protocol (HTTP) e il Transmission Control Protocol (TCP). Entrambi sono protocolli di richiesta e risposta: un client avvia richieste, un server le ascolta e risponde. Le rispettive responsabilità sono complementari.

- **TCP**: è il protocollo di livello inferiore, descrive come l'informazione arriva da un server all'altro ma non specifica quale sia quell'informazione.
- **HTTP**: è costruito sopra TCP e ne definisce il contenuto delle richieste e delle risposte; tecnicamente può viaggiare su altri protocolli, ma nella pratica quasi sempre invia i dati tramite TCP.

Il server single-threaded

Un server basilare ascolta una connessione TCP, legge la richiesta e scrive la risposta, una connessione alla volta. La libreria standard offre tutto il necessario nel modulo `std::net`.

Ascoltare la connessione TCP

Il tipo `TcpListener` rappresenta un listener in ascolto su un indirizzo: il metodo `bind` lo collega all'indirizzo `127.0.0.1:7878` (IP e porta; in rete collegarsi a una porta per ascoltarla si dice binding), e come `new` restituisce un `Result<T, E>` perché il binding può fallire, per esempio se due istanze del programma ascoltano la stessa porta. Il metodo `incoming` restituisce un iteratore di stream di tipo `TcpStream`: un singolo stream rappresenta una connessione aperta tra client e server, dove connessione è il nome dell'intero processo di richiesta e risposta, dalla connessione del client alla chiusura da parte del server. Due aspettano attenzione.

- **Tentativi, non connessioni**: l'iterazione è su tentativi di connessione, non su connessioni riuscite; un tentativo può fallire per ragioni spesso specifiche del sistema operativo, come il limite al numero di connessioni simultanee aperte.
- **Chiusura e ripetizioni**: quando lo stream esce dallo scope e viene droppato, la connessione si chiude nell'implementazione di `drop`; i browser a volte rispondono alla chiusura riprovando, e a volte aprono più connessioni senza inviare richieste per accorciare i tempi di richieste future, quindi una singola richiesta del browser può produrre più messaggi lato server.

Leggere la richiesta

La lettura dello stream usa `BufReader`, che avvolge un riferimento allo stream e gestisce le chiamate ai metodi del trait `std::io::Read` aggiungendo il buffering. `BufReader` implementa il trait `std::io::BufRead`, che fornisce il metodo `lines`: questo divide il flusso di dati a ogni byte di newline e restituisce un iteratore di `Result<String, std::io::Error>`, perché i dati potrebbero non essere UTF-8 validi o la lettura potrebbe fallire. Il browser segnala la fine di una richiesta HTTP inviando due newline di fila, quindi per ottenere una richiesta intera si raccolgono le righe fino alla riga vuota.

Il formato di richiesta e risposta

HTTP è un protocollo testuale con formati precisi per richieste e risposte, definiti da una riga iniziale, gli header e il corpo.

- **Richiesta**: la prima riga è la riga di richiesta (request line), con metodo (come `GET` o `POST`, che descrive come il client fa la richiesta), URI richiesto (uniform resource identifier, quasi sinonimo di URL) e versione di HTTP, conclusa dalla sequenza CRLF (`\r\n`, carriage return e line feed); seguono gli header e infine l'eventuale corpo, che le richieste `GET` non hanno.
- **Risposta**: la prima riga è la riga di stato (status line), con versione di HTTP, codice di stato numerico che riassume l'esito (200 è la risposta standard di successo, 404 segnala contenuto non trovato) e reason phrase che lo descrive; dopo il CRLF seguono gli header, un altro CRLF e il corpo.

Per scrivere la risposta si converte la stringa in byte con `as_bytes` e si usa il metodo `write_all` sullo stream, che accetta un `&[u8]` e invia i byte direttamente sulla connessione. Un header come `Content-Length`, impostato alla dimensione del corpo, assicura una risposta valida; un confronto sul contenuto della riga di richiesta permette di rispondere con contenuti diversi a percorsi diversi.

I limiti del modello a thread singolo

Il server così costruito elabora le connessioni in serie: una richiesta lenta blocca tutte le successive, anche se potrebbero essere gestite rapidamente. Le tecniche per evitare l'intasamento sono diverse, e l'alternativa async del Capitolo 17 è una di esse; il progetto usa un thread pool.

Il thread pool

Un thread pool è un gruppo di thread già creati e in attesa di un compito: quando arriva un nuovo compito, il pool lo assegna a un thread, che lo elabora mentre gli altri restano disponibili; finite le elaborazioni, il thread torna nel gruppo dei thread in attesa. Il numero di thread è fisso e volutamente piccolo, per due ragioni.

- **Protezione dai DoS**: creare un thread per ogni richiesta consentirebbe a chi invia milioni di richieste di esaurire le risorse del server; con un pool fisso chi attacca non può spingere il sistema fuori controllo.
- **Throughput limitato ma prevedibile**: le richieste in eccesso si accumulano in una coda e ogni thread preleva un compito dalla coda, lo elabora e ne chiede un altro; si elaborano fino a N richieste in contemporanea, dove N è il numero di thread.

Altre strade per il throughput esistono (modello fork/join, async I/O single-threaded e multithreaded), ma con un linguaggio di basso livello come Rust tutte sono percorribili.

Sviluppo guidato dal compilatore

Come nel progetto di I/O si è usato lo sviluppo guidato dai test, qui si usa lo sviluppo guidato dal compilatore: prima si scrive il codice chiamante con l'interfaccia desiderata (un `ThreadPool::new(size)` e un metodo `execute` che prende una closure, con la stessa forma di `thread::spawn`), poi si implementa finché gli errori del compilatore non terminano. Scrivere prima l'interfaccia del cliente guida la struttura dell'implementazione. Inoltre la crate passa da binaria a libreria, così il thread pool è riusabile per qualunque lavoro e non solo per servire richieste web.

La firma di `execute`

La scelta dei bound della closure accettata da `execute` segue la firma di `thread::spawn` della libreria standard.

```rust
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T,
    F: Send + 'static,
    T: Send + 'static,
```

Il tipo `F` della closure richiede `FnOnce` perché l'esecuzione avviene una sola volta (il Once di FnOnce), `Send` per trasferire la closure da un thread all'altro e `'static` perché non si sa quanto a lungo il thread la eseguirà. Il numero di thread si dichiara come `usize` (un numero negativo di thread non ha senso) e `new` verifica con `assert!` che sia maggiore di zero, documentando nella documentazione generata anche i panic possibili; un'alternativa più flessibile sarebbe una funzione `build` che restituisce un `Result`, ma creare un pool senza thread è trattato come errore irrecuperabile. La rigenerazione

della documentazione con `cargo doc --open` permette di verificare le buone pratiche di documentazione del Capitolo 14.

I Worker

`thread::spawn` esegue il codice subito dopo la creazione del thread, ma il pool ha l'esigenza opposta: creare i thread e farli aspettare codice che arriverà più tardi. La libreria standard non lo permette, quindi serve una struttura intermedia, il Worker, termine comune nelle implementazioni di pool: ogni Worker preleva il codice da eseguire e lo esegue nel proprio thread, come i cuochi di un ristorante che aspettano gli ordini e poi li evadono. Ogni Worker ha un identificativo per distinguerlo nei log e nel debug e un `JoinHandle<()>`; il pool tiene un vettore di Worker, creati con `Worker::new`. Due dettagli di robustezza sono degni di nota.

- **Fallimenti di spawn**: se il sistema operativo non ha risorse per creare il thread, `thread::spawn` fa panic e trascina con sé l'intero server anche se altri thread sono stati creati; un'implementazione di produzione userebbe `std::thread::Builder` e il suo `spawn` che restituisce un `Result`.
- **Incapsulamento**: il Worker e il suo costruttore restano privati, perché il codice esterno non deve conoscere i dettagli implementativi del pool.

Il canale come coda di lavori

Il ponte tra pool e Worker è un canale mpsc come quelli del Capitolo 16, usato come coda di lavori.

- `Job`: è il tipo degli elementi inviati sul canale, un alias di tipo per il trait object che incapsula la closure, `Box<dyn FnOnce() + Send + 'static>`, secondo la tecnica degli alias del Capitolo 20.
- **Invio**: il pool mantiene il sender e il metodo `execute` incapsula la closure in un `Job` e la invia attraverso il canale.
- **Ricezione**: ogni Worker possiede il receiver e il suo thread lo scorre in un loop infinito, eseguendo la closure di ogni lavoro ricevuto.

Un problema di ownership va risolto: il canale della libreria standard è multi-produttore e mono-consumatore, quindi non si può clonare il receiver per più Worker, e prelevare un lavoro comporta una mutazione, per cui serve condividerlo in modo sicuro. La soluzione combina i due smart pointer thread-safe del Capitolo 16.

Definition

Arc<Mutex<T» sul receiver

Il receiver viene incapsulato in `Arc<Mutex<T>>`: **`Arc`** permette a più Worker di esserne comproprietari, clonando il conteggio di riferimenti per ciascuno; **`Mutex`** garantisce che un solo Worker alla volta prelevi un lavoro dal receiver. Nel thread del Worker si chiama `lock` per acquisire il mutex (con `unwrap` o `expect` per il caso, qui corretto, di mutex avvelenato perché un altro thread ha fatto panic tenendo il lock) e poi `recv` per ricevere il lavoro; `recv` blocca, quindi se non ci sono lavori il thread attende.

Perché `let` e non `while let`

Dopo i capitoli 17 e 19 verrebbe naturale scrivere il loop del Worker come `while let Ok(job) = receiver.lock().unwrap().recv().unwrap()`, e il codice compila, ma il comportamento è sbagliato: una richiesta lenta fa ancora attendere le altre. La ragione è sottile e riguarda la durata dei valori temporanei.

- **`let`**: in `let job = receiver.lock().unwrap().recv().unwrap();` i valori temporanei dell'espressione a destra, incluso il `MutexGuard`, vengono droppati appena l'istruzione termina, quindi il lock viene rilasciato prima di eseguire il lavoro.
- **`while let`**: `while let`, `if let` e `match` non droppano i temporanei fino alla fine del blocco associato, quindi il lock resta tenuto per tutta la durata di `job()` e gli altri Worker non possono ricevere lavori.

La struttura `Mutex` non ha un metodo `unlock` pubblico: il possesso del lock è legato alla lifetime del `MutexGuard` restituito da `lock`, ed è così che il borrow checker fa rispettare la regola che una risorsa protetta da `Mutex` non è accessibile senza tenere il lock; ma questo legame implica anche che un `MutexGuard` longevo estende il lock oltre il necessario.

Spegnimento pulito

Il pool funzionante ha ancora un difetto: i campi `workers`, `id` e `thread` non usati direttamente segnalano che nessuna pulizia avviene, e fermare il server con l'interruzione da terminale arresta tutti i thread immediatamente, anche a metà di una richiesta. Uno shutdown pulito richiede due pezzi: implementare `Drop` per fare `join` su ogni thread, e trovare un modo per dire ai thread di smettere di accettare richieste.

`Drop` e `Vec::drain`

L'implementazione di `Drop` sul pool scorre i Worker e chiama `join` sul loro thread, così i thread finiscono le richieste in corso prima della chiusura. Sorgono però due questioni di ownership, risolte diversamente a seconda del campo.

- **Il thread nel Worker**: `join` consuma il suo argomento, mentre dal loop si ha solo un borrow mutabile; la via idiomatica per estrarre il valore è il metodo `Vec::drain` sul vettore dei Worker, che accetta un range (con `..` rimuove tutti i valori) e restituisce un iteratore che ne prende possesso.
- **L'`Option` come workaround**: l'alternativa vista nel Capitolo 18 era incapsulare il thread in un `Option<JoinHandle<()>>` e usare `Option::take`, sostituendo `Some` con `None`; ma per un valore che si sa essere sempre presente avvolgerlo in `Option` solo come espediente non è idiomatico, e quando capita conviene cercare alternative più pulite. Il sender invece ha davvero bisogno dell'`Option::take`, perché `drop` lo deve estrarre dal pool per dropparlo prima dei join.

Un avvertimento riguarda i panic annidati: siccome `drop` può essere chiamato durante un panic, un `unwrap` che fallisca lì dentro causerebbe un double panic, che arresta immediatamente il programma e interrompe ogni pulizia in corso; accettabile in un esempio, sconsigliato in produzione.

Chiudere il canale

Fare `join` non basta a spegnere i thread, perché i loro loop cercano lavori per sempre: il pool si bloccherebbe attendendo il primo thread. Il segnale di arresto usa la semantica del canale stesso: droppare esplicitamente il sender chiude il canale, il che indica che non arriveranno più messaggi; a quel punto tutte le chiamate di `recv` nei loop dei Worker restituiscono un errore, e il loop gestisce quel caso uscendo in modo pulito. Il protocollo di shutdown completo segue quindi questo ordine: il pool droppa il sender, i Worker ricevono l'errore e terminano il loop, e solo allora il `drop` del pool completa i `join` attendendo la fine di ogni thread.

Sintesi del capitolo

Il progetto conclude il percorso: un server web basilare risponde alle richieste tramite un thread pool, con spegnimento pulito che libera tutti i thread del pool. Tutti i concetti dei capitoli finali del libro (trait object, alias di tipo, `Arc` e `Mutex`, canali, `Drop`, ownership e borrow checker) convergono in poche decine di righe di Rust, e le stesse idee sottostanno alle crate di produzione su crates.io.